

**AIoT AutoCar로 배우는
온디바이스 AI 프로그래밍**

한백전자 기술연구소

목차

목차.....	5
1. 인공지능 사례	8
1.1. 인식	8
1.2. 모방	9
1.3. 창작	10
내용정리	13
2. 실습 환경	15
2.1. AIoT AutoCar	15
2.2. PC 와 AIoT AutoCAR 의 RNDIS 네트워크 연결	18
2.3. 개발 환경	19
내용정리	23
3. AIoT Autocar 제어	25
3.1. 차량 제어	25
3.2. 카메라 제어	29
3.3. 6 축 센서	30
내용정리	34
4. 카메라 활용	37
4.1. OpenCV	38
내용정리	64
연습문제	66
5. 오디오 활용	69
5.1. 오디오 하위 시스템	69
5.2. PyAudio	75
5.3. 텍스트 음성 변환	109
5.4. 구글 어시스턴트	115
내용정리	161
연습문제	163
6. 데이터 사이언스	167
6.1. Numpy	167
6.2. Pandas	209
6.3. Matplotlib	248
7. 인공지능	284
7.1. 머신러닝	285

7.2.	딥러닝	326
7.3.	강화 학습	381
7.4.	Tensorflow	401
7.5.	Keras	426
8.	온디바이스 AI	444
8.1.	휠 얼라인먼트	445
8.2.	차량 자세 제어	453
8.3.	사진 분류	459
8.4.	음성 분류	467
8.5.	장애물 회피	476
8.6.	목표물 추적	485
8.7.	트랙 주행	492
8.8.	트래픽 콘 주행	503
8.9.	라인 트레이서	510
부록		518
A.	OpenCV 설치	518
B.	OpenCV 비디오 속성	521
C.	MFCC	521
D.	One Hot	523

1. 인공지능 사례

인공지능은 인간의 지적 능력을 모방한 컴퓨터 시스템입니다. 새로운 지식 또는 정보가 주어지면 학습, 패턴 인식, 특징 분석 등의 알고리즘으로 기존 지식을 개선합니다. 과거 인공지능은 고성능 컴퓨터가 필요했기에 스마트폰, 차량 등 단말 장치에서 인공지능을 이용하려면 AI 서버를 따로 만들어야했습니다. 그러나 소형 고성능 하드웨어와 고효율 알고리즘이 개발되면서 소형 단말 장치에서도 인공지능을 구현할 수 있게 되었고, AI 서버에 접속할 필요 없이 인공지능을 자체 내장하고 있는 온디바이스 AI On-Device AI가 등장했습니다. 이에 대해서는 뒤에서 더 자세히 설명하겠습니다.

초창기 인공지능은 다양한 이론들에 비해 컴퓨터 성능은 부족했으므로 대량의 데이터를 사람이 직접 입력하고 소규모의 의사결정을 하는 수준에 그쳤지만, 현대에 들어서 컴퓨터 하드웨어 기술이 발전하고 여러 인공지능 이론이 많은 개선을 거치면서 빠르게 성장하고 있습니다.

1.1. 인식

사물 인식은 대표적인 인공지능 응용 사례입니다. 대용량 데이터 학습에 특화된 인공지능 기술을 이용하여 프레임 당 약 8,294,400 개의 데이터를 학습합니다. 이러한 사물 인식 기술을 응용하면 다음 그림처럼 시각적 정보를 자막으로 바꿔주는 인공지능을 구현할 수 있습니다.

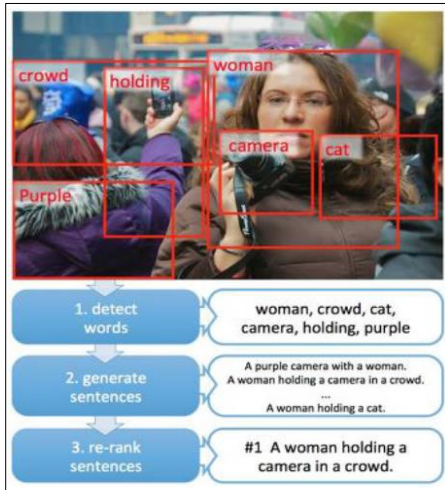


그림 1 사물 인식을 응용한 자막 생성 과정

1.2. 모방

진짜 데이터를 기반으로 가짜 데이터를 생성하도록 학습하는 알고리즘을 이용한 인공지능입니다. 현실 데이터를 모방하여 가상의 데이터를 만들어내는 기술은 사람의 얼굴 사진을 학습해 가상의 인물을 만들어내거나, 특정 인물의 글씨체를 학습해 필기를 만들어내는 등 다양한 방면에 활용할 수 있는 기술입니다. 그러나 위조 데이터를 만들어내는 특성 때문에 악용될 가능성이 매우 큰 기술이며 실제로 동영상에 특정 인물을 합성해내는 등 악용사례가 나타나고 있습니다. 이러한 악용사례를 대면하고 윤리적 문제를 해결하려는 태도는 개발자로서 필수 덕목입니다.

그림의 Stage-1 은 학습이 부족한 상태에서 미숙하게 그려낸 그림이며, Stage-2 는 충분히 학습한 상태에서 그려낸 그림입니다.

Text description	This bird is blue with white and has a very short beak	This bird has wings that are brown and has a yellow belly	A white bird with a black crown and yellow beak	This bird is white, black, and brown in color, with a brown beak	The bird has small beak, with reddish brown crown and gray belly	This is a small, black bird with a white breast and white on the wingbars.	This bird is white black and yellow in color, with a short black beak
Stage-1							
Stage-2							

그림 2 문장에서 표현하는 새를 그려내는 인공지능

1.3. 창작

여러 데이터의 상호관계와 순서를 반영하여 학습하는 알고리즘을 이용한 인공지능입니다. 수많은 창작물 데이터에서 색과 색의 관계를 학습하여 조화로운 색 조합을 결정하고, 인기 있는 노래 데이터를 학습해 사람이 듣기 좋은 노래를 작곡하는 등 논리적으로 분석하기 힘든 영역까지 활용할 수 있는 기술입니다. 다음 그림의 인공지능은 직접 체험해볼 수 있도록 공개되어 있습니다. 아래 주소에서 스케치를 업로드하거나 샘플 이미지를 이용해 인공지능 컬러링을 체험할 수 있습니다.

- <https://paintschainer.preferred.tech>





그림 3 스케치에 원하는 색을 가볍게 그리면 자동으로 채색해주는 인공지능

일반적인 프로그램은 정해진 환경에서 정해진 동작만 수행하는 반면 인공지능 프로그램은 변화하는 환경에 맞게 최적의 동작을 수행합니다. 그렇기에 인공지능이 변화를 정확하게 인지하는 것은 결정적 요소이며 이를 위해서는 잘 정제된 학습 데이터가 필요합니다. 요컨대, 정확한 데이터를 기반으로 학습된 인공지능 모델은 신뢰성과 응용 가치가 높다고 할 수 있습니다.

내용정리

- 인공지능 : 인간의 지적 능력을 모방한 컴퓨터 시스템
- 온디바이스 AI : 장치에 내장되어 서버 연결 없이 작동하는 인공지능
- 인공지능 사례 3 가지
 - 인식 : 데이터를 식별하기 위한 인공지능
 - 모방 : 데이터를 모방하기 위한 인공지능
 - 창작 : 데이터를 생성하기 위한 인공지능



그림 4 세계박람회의 Elektro

1939

Westinghouse 가 개발한 릴레이 기반 Elektro 로봇은 음성 명령에 반응하고 78rpm 으로 사전 기록된 농담을 했습니다. Elektro 는 1939 년 세계 박람회에 소개되었으며 머리와 팔을 움직일 수 있었고 담배를 피울 수도 있었습니다.

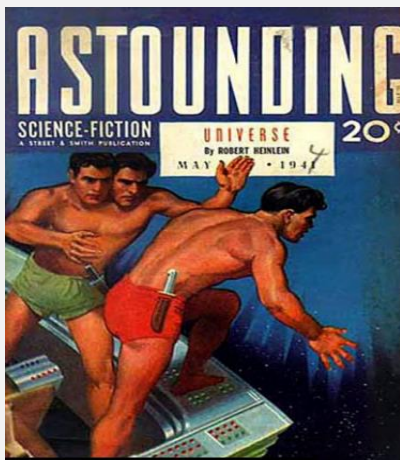


그림 5 Astounding 5 월호

1941

Isaac Asimov 는 'Astounding Science Fiction' 의

5 월호에서 공상 과학 단편 소설 'Liar!'를 발표했습니다. 이 소설에서 그는 로봇 공학의 세 가지 법칙을 최초로 소개했습니다. 1 법칙은 '로봇은 사람에게 상해를 입히거나 활동하지 않아서 사람이 해를 입을 수 없다' 2 법칙은 '로봇은 명령이 제 1 법칙과 상충하는 경우를 제외하고는 인간의 명령에 따라야 한다' 3 법칙은 '제 1 법칙 또는 제 2 법칙과 충돌하지 않는 한 로봇은 자신의 존재를 보호해야 한다' 입니다. 이것은 "로보틱스"라는 용어의 첫

2. 실습 환경

과거, 인공지능 학습은 인공지능에 특화된 고성능 컴퓨터에서나 가능했던 일입니다. 그러나 시간이 흘러 저비용 고성능 GPU가 개발되면서 쉽게 구현할 수 있게 되었습니다. 또, 소형 AI 모듈이 개발되며 소형 디바이스, 모빌리티 디바이스 등 다양한 응용 분야가 생겨났습니다.

2.1. AIoT AutoCar

AIoT AutoCar는 소형 AI 모듈인 Nvidia Jetson Nano를 탑재하고 있어 적절한 인공지능 성능을 기대할 수 있습니다. 한백전자에서 제공하는 Pop.AI 라이브러리를 이용해 쉽게 인공지능에 대해 배워보고 차량에 인공지능을 응용하는 다양한 예제들과 흥미로운 주제로 이루어진 실습들로 구성되어 있습니다. 더 나아가 카메라, 마이크, 6축 센서, 차량 구동체, 스피커 등을 이용해 다양한 인공지능 아이디어를 학습자 스스로 실현해볼 수 있습니다.

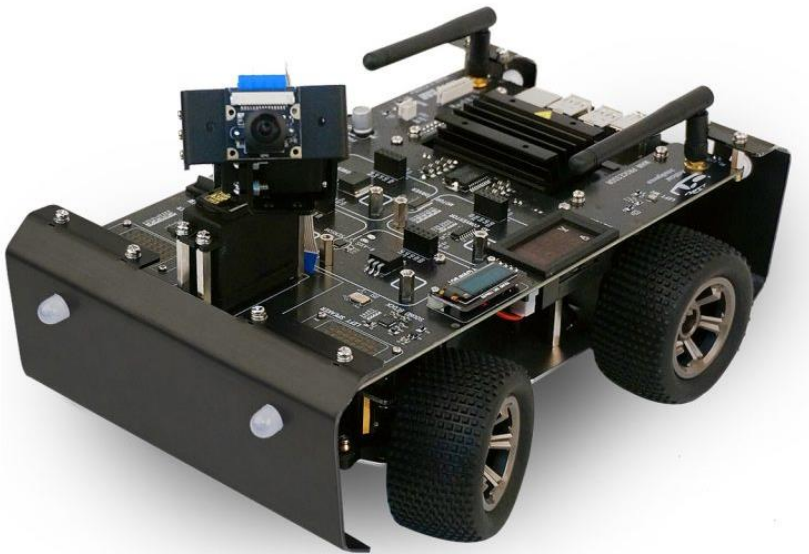


그림 6 AIoT AutoCar

AIoT AutoCar 에 전원을 연결한 후 스위치를 켜면 약 1 분 동안 내부 상태를 점검합니다. 그러나 부팅이 완료되어도 모니터가 없어서 화면을 볼 수 없기에 프로그래밍을 시작할 수 없습니다. 이 문제는 네트워크를 연결해 SSH, NoMachine 등 원격 제어 프로그램을 이용하여 해결할 수 있습니다. AIoT AutoCar 는 Micro USB 케이블로 간단히 네트워크를 연결할 수 있도록 구성되어있습니다.

2.1.1. AIoT AutoCAR Layout

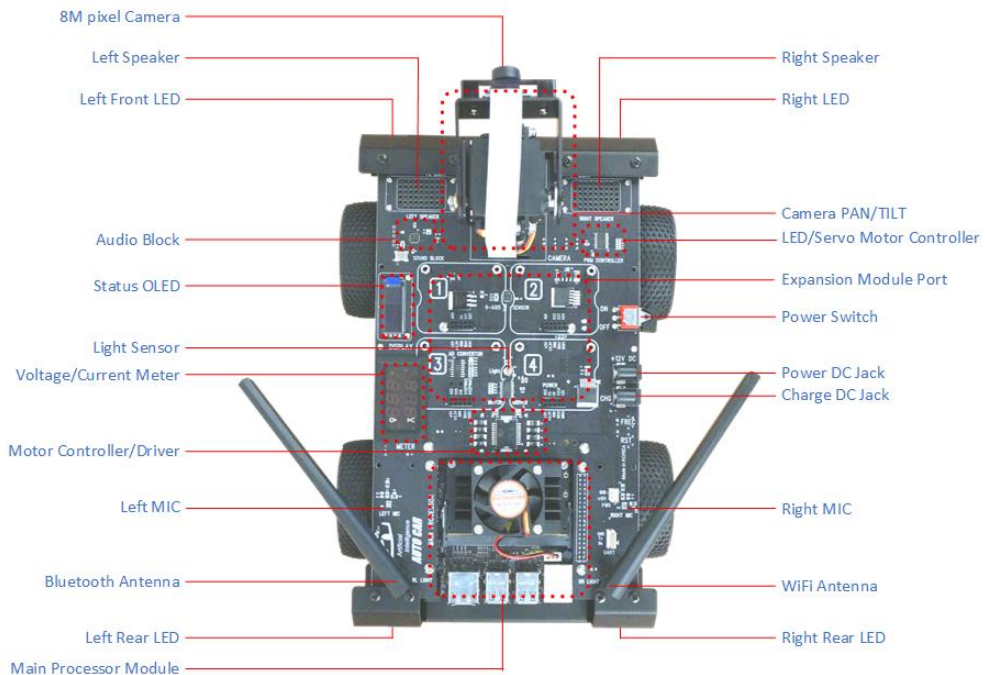


그림 7 AIoT AutoCAR Layout

AIoT AutoCAR 에 각 부분별 기능은 위 그림과 같습니다. AutoCAR 의 전원은 장착되어 있는 배터리를 활용하거나 차량 우측에 있는 DC Jack 을 통해 동작하도록 되어 있습니다. 전원 스위치를 켜면 배터리나 DC Jack 의 전원입력을 받아 동작하는 형태입니다.

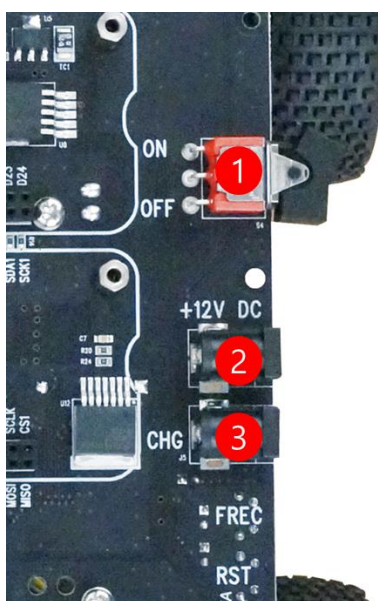


그림 8 AIoT AutoCar 전원부

DC Jack 을 연결하는 부분에는 입력 단자가 2 개가 장착되어 있습니다. 한 개는 DC 입력단자, 한 개는 배터리 충전 단자로 구성되어 있습니다. 위 그림을 참고하면 2 번이 DC 입력, 3 번이 충전입니다. DC Jack 을 이용하여 동작을 하게되면 배터리는 충전되지 않는 상태로 장비가 동작됩니다. 따라서 배터리 충전을 위해서는 꼭 충전 단자에 입력이 이루어져야 합니다. 다만 배터리 충전 단자에 입력이 이루어 지면 물리적으로 배터리와 장비가 분리되어 DC Jack 을 통한 전원 입력 없이는 장비가 동작되지 않습니다. 충전과 동시에 AutoCAR 를 활용하려면 두 단자에 모두 어댑터를 연결해야 합니다.



그림 9 AIoT AutoCAR Voltage Meter

AutoCAR 에는 현재 차량에 공급되는 배터리 또는 DC 어댑터를 통해 공급되는 전원과 사용하는 전류를 측정하기 위한 Voltage Meter 가 장착되어 있습니다. 장착되어 있는 배터리는 3Cell 배터리입니다. 이 배터리가 완전하게 방전되었을 때 전원을 켜면 +9V 가 출력되고 완전하게 충전되었을 때는 +12.6V 가 출력 됩니다. 하지만 장착되어 있는 LiPO 배터리의 특성상 완전하게 방전이되었을 때 재충전이 불가능할 수 있기 때문에 +10.5V 정도에서는 배터리를 충전해주는 것이 좋습니다.

2.2. PC 와 AIoT AutoCAR 의 RNDIS 네트워크 연결

AIoT AutoCar 는 RNDIS Remote Network Driver Interface Specification 네트워크를 지원합니다. RNDIS 는 USB, Bluetooth 등 동적 장치 환경에서 네트워크 인터페이스를 사용할 수 있도록하는 기능으로 테더링을 예로 들 수 있습니다. 즉, USB 케이블 연결로 네트워크 연결이 가능합니다.

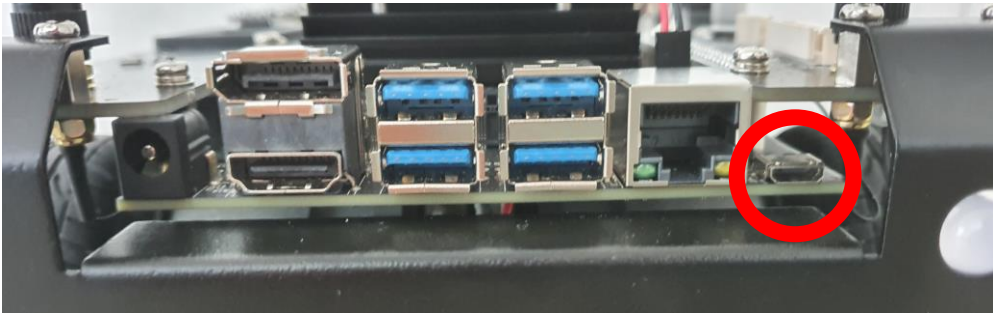


그림 10 PC 와 AIoT AutoCar 유선 네트워크 연결

USB 케이블로 AIoT AutoCar 와 PC 가 연결되면 AIoT AutoCAR 의 IP 는 192.168.55.1 으로 할당되고 PC 의 IP 는 192.168.55.100 으로 할당됩니다.

2.3. 개발 환경

AIoT AutoCar 는 Jupyter Lab 환경에서 실습이 이루어집니다.

2.3.1. Jupyter Lab

Jupyter Lab 은 웹 기반 통합 개발 환경입니다. 코드를 조각 단위로 실행하여 결과를 확인할 수 있어 코드가 진행되는 것을 차근차근 관찰할 수 있습니다. 또, 그래픽 요소를 표시할 수 있으므로 학습자의 실습 이해를 돕는 요소가 다수 포함되어 있습니다. 다음과 같이 PC 에서 웹 브라우저를 실행한 후 AIoT AutoCar 에 접속하면 프로그램을 작성할 수 있습니다.

| 1 | PC 에서 웹 브라우저(크롬 권장) 실행

| 2 | URL 은 AIoT AutoCar 주소과 Jupyter Lab 포트 번호를 지정해 192.168.55.1:8888 사용

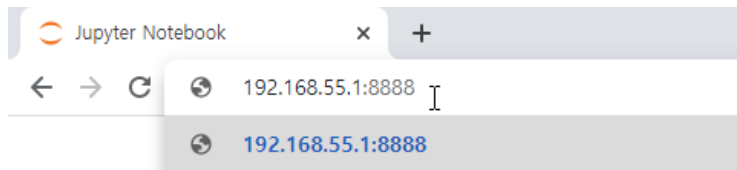


그림 11 Jupyter Lab 접속 1

| 3 | Password 에 soda 입력 후 Log in 버튼 클릭

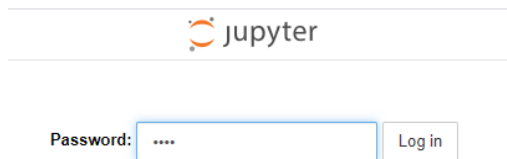


그림 12 Jupyter Lab 접속 2

| 4 | Python 3 클릭



그림 13 새파일 생성

| 5 | 코드 입력 후 RUN 버튼 클릭

```
01: p = 'Hello "World"'
02: print(p)
```

예제 1 Hello "World"

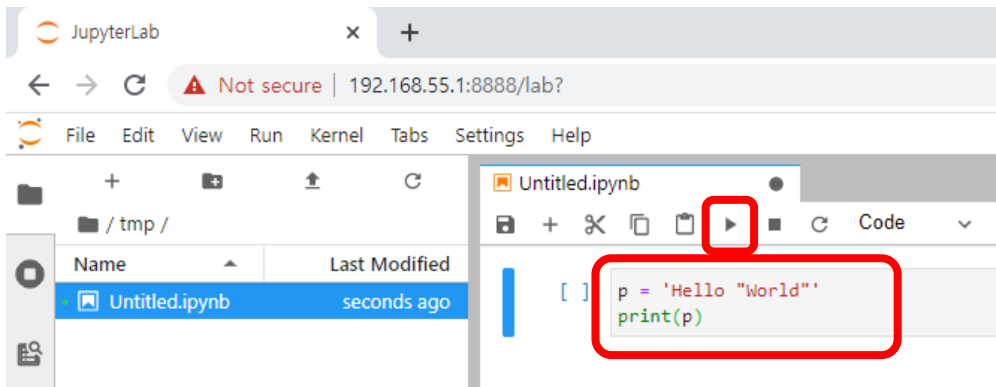


그림 14 코드작성 및 실행

| 6 | 결과 확인

```
[1]: p = 'Hello "World"'
      print(p)

      Hello "World"
```

그림 15 실행결과 확인

또, Jupyter 에서는 원격 터미널 기능도 제공하고 있습니다.

| 7 | 좌측 상단 [+] 버튼 클릭

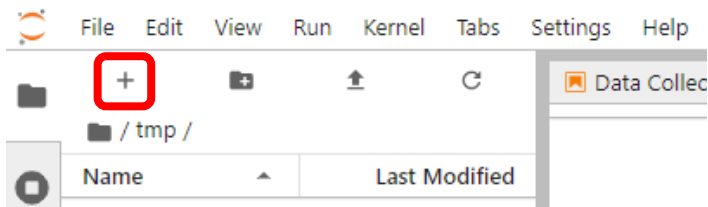


그림 16 작업 추가

| 8 | Other 그룹에서 Terminal 선택

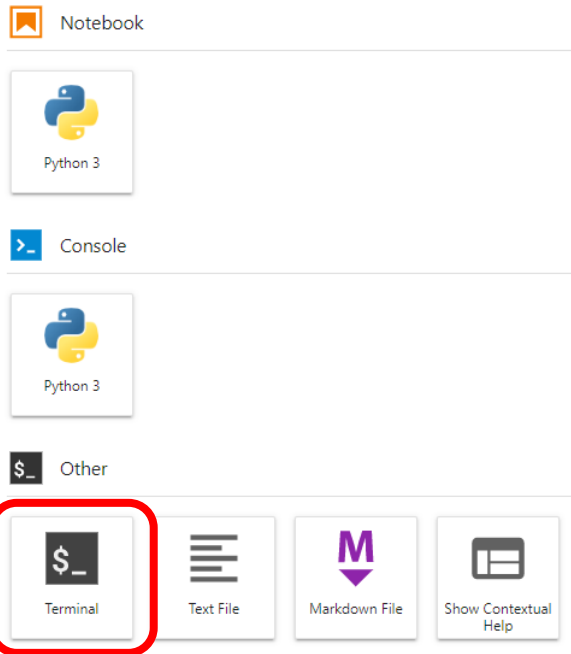


그림 17 Terminal 선택

| 9 | 원격 터미널 접속

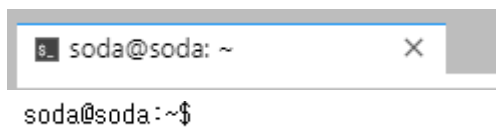


그림 18 원격 터미널 접속

내용 정리

- RNDIS 네트워크 : USB, Bluetooth 등 동적 장치를 이용해 연결한 네트워크
- Jupyter Lab 환경 : 웹 브라우저 기반 원격 개발 환경

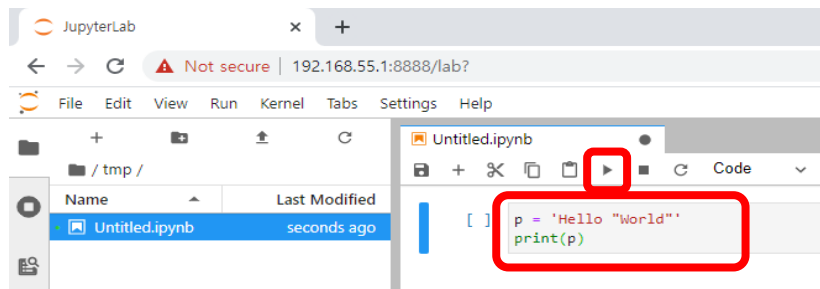




그림 19 Grace Hopper

1945

컴퓨터에서 ‘버그’는 어떤 형태의 오류 또는 실패를 의미합니다. 1945년 Grace Hopper는 컴퓨터 박람회에서 Harvard Mark II 컴퓨터의 릴레이 접점 사이에 나방이 붙어있는 상황에 착안하여 버그라는 단어를 처음 사용했습니다.



그림 20 ENIAC

1946

펜실베이니아 대학교의 John Mauchly와 J.Presper Eckert은 1943년부터 시작된 ENIAC 컴퓨터 시스템의 개발을 완료하였습니다. ENIAC은 프로그래밍을 위해 패널 간 배선 및 스위치를 사용했으며, 1,000 제곱 피트 이상을 차지했으며 약 18,000 개의 진공관을 사용했고 무게는 30 톤이었습니다. 당시 ENIAC은 10년 동안 모든 인류가 했던 계산보다 더 많은 계산을 수행했다고 알려졌습니다.

3. AIoT Autocar 제어

AIoT AutoCar 는 Pop.Pilot 라이브러리의 AutoCar 클래스를 이용해 쉽게 제어할 수 있습니다. 이 라이브러리에는 차량의 전/후진과 좌/우회전을 제어하는 메소드와 카메라를 위/아래로 움직이거나 좌/우로 회전시키는 메소드, 6 축 센서 값을 읽어오는 메소드가 있습니다.

3.1. 차량 제어

차량을 제어하는 주요 메소드는 다음과 같습니다.

- steering : 값을 -1 ~ 1 사이값으로 지정하면 차량의 앞바퀴를 좌/우로 조향
- setSpeed(value) : 값을 0~99 사이값으로 지정하여 차량의 속력을 제어
- stop() : 차량을 정지
- forward() : 차량을 전진
- backward() : 차량을 후진
- joystick() : 가상 조이스틱을 이용해 차량을 제어
- camPan(value) : 값을 0~180 사이값으로 지정하여 카메라를 좌/우로 제어
- camTilt(value) : 값을 -30~200 사이값으로 지정하여 카메라를 위/아래로 제어
- getGyro(axis) : 'x', 'y', 'z' 를 입력하면 해당 축의 자이로 값을 반환하고, 아무 입력없이 호출시 모든 축에 대한 값을 딕셔너리 반환
 - 기본값 : All
- getAccel(axis) : 'x', 'y', 'z' 를 입력하면 해당 축의 가속도 값을 반환하고, 아무 입력없이 호출시 모든 축에 대한 값을 딕셔너리로 반환
 - 기본값 : All

먼저, Pop 에서 Pilot 라이브러리를 불러오고 AutoCar 클래스를 생성합니다. 또, 코드 진행에 딜레이를 주고 관찰하기 위해 time 모듈도 불러옵니다.

```
01: import time
02: from pop import Pilot
03:
04: Car = Pilot.AutoCar()
```

AutoCar 객체가 생성되었다면 즉시 차량을 제어할 수 있습니다.

3.1.1. 조향

다음과 같이 AutoCar 의 steering 속성 값을 입력하면 차량을 좌우로 조향할 수 있습니다.

```
05: Car.steering = -1
06:
07: time.sleep(1)
08:
09: Car.steering = 1
```

조향 제어

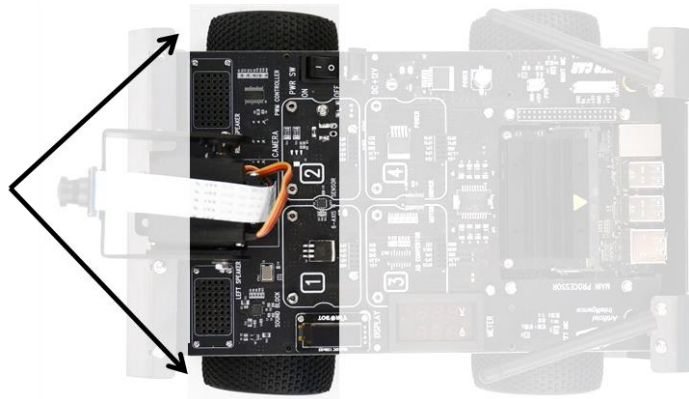


그림 21 조향 제어

3.1.2. 전진 또는 후진

forward() 메소드와 backward() 메소드를 사용하면 차량을 앞뒤로 움직일 수 있고, stop() 메소드를 사용하면 정지시킬 수 있습니다.

```
10: Car.forward()  
11:  
12: time.sleep(1)  
13:  
14: Car.backward()  
15:  
16: time.sleep(1)  
17:  
18: Car.stop()
```

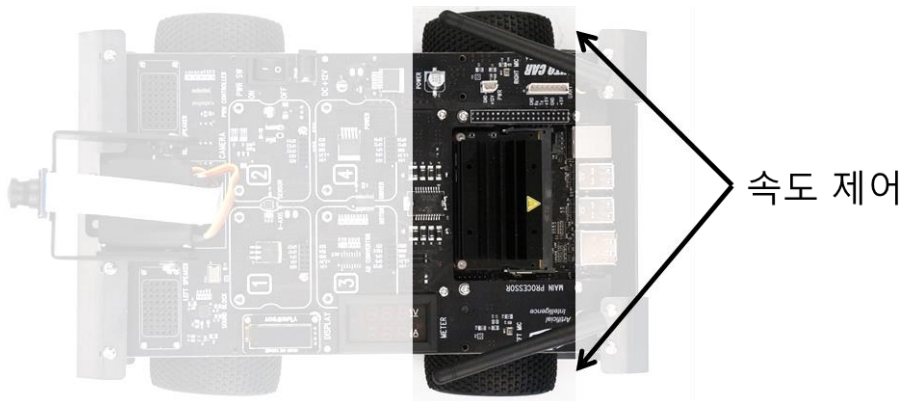


그림 22 속도 제어

3.1.3. 속도 제어

setSpeed(value) 메소드를 사용하거나 forward(), backward() 메소드의 파라미터로 속도를 입력하면 주행 속도를 조절할 수 있습니다. 속도값의

범위는 0~99 입니다.

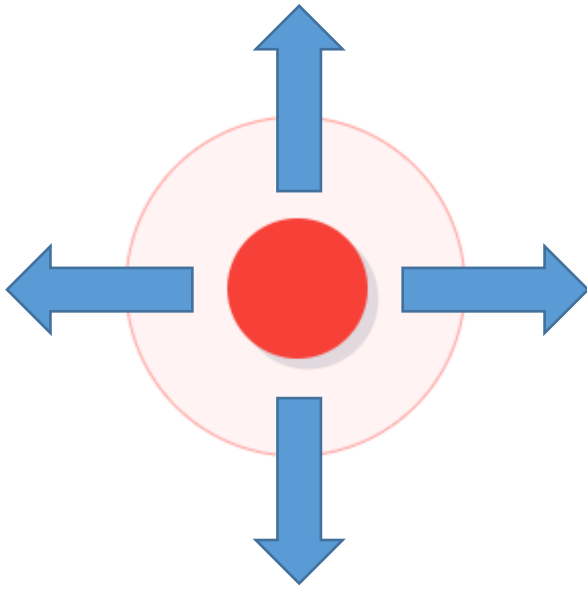
```
19: Car.setSpeed(50)
20: Car.forward()
21:
22: time.sleep(1)
23:
24: Car.backward(99)
25:
26: time.sleep(1)
27:
28: Car.stop()
```

3.1.4. 가상 조이스틱

편리한 주행 제어를 위해 가상의 조이스틱 GUI 를 사용할 수도 있습니다. joystick() 메소드를 호출하면 가상의 조이스틱이 표시되고 앞, 뒤, 좌, 우로 조작하면 차량이 움직입니다.

```
29: Car.joystick()
```

[출력]



3.2. 카메라 제어

카메라를 좌우 또는 위아래로 움직이려면 `camPan()` 메소드와 `camTilt()` 메소드를 사용하여 조작할 수 있습니다.

```
30: Car.camPan(0)
31:
32: time.sleep(1)
33:
34: Car.camPan(180)
35:
36: time.sleep(1)
37:
38: Car.camTilt(180)
39:
40: time.sleep(1)
41:
```

```
42: Car.camTilt(0)
```

camPan() : 좌 우 제어
camTilt() : 위 아래 제어

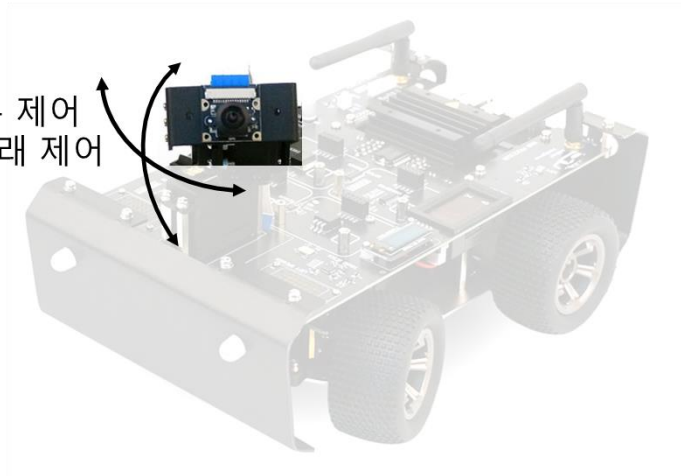


그림 23 카메라 제어

3.3.6 축 센서

getGyro() 메소드를 사용하면 6 축 센서의 자이로 값을 읽을 수 있습니다.

파라미터를 입력하지 않으면 모든 축에 대한 값을 반환하는데, 파라미터로 'x', 'y', 'z'를 입력하면 해당 축의 값만 반환하고 자원을 덜 소모하므로 특정 축만 사용할 때는 파라미터로 해당 축을 지정하는 것이 좋습니다.

```
43: value = Car.getGyro()  
44: print(value)
```

```
45:
46: time.sleep(1)
47:
48: value = Car.getGyro('z')
49: print(value)
```

[출력]

```
{'x': -534, 'y': -84, 'z': 43}
38
```

또, 이와 비슷하게 getAccel() 메소드를 사용하면 6 축 센서의 가속도 값을 읽을 수 있습니다.

```
50: value = Car.getAccel()
51: print(value)
52:
53: time.sleep(1)
54:
55: value = Car.getAccel('z')
56: print(value)
```

[출력]

```
{'x': 1176, 'y': 356, 'z': 15812}
15561
```

6 축 센서의 위치는 아래와 같습니다.

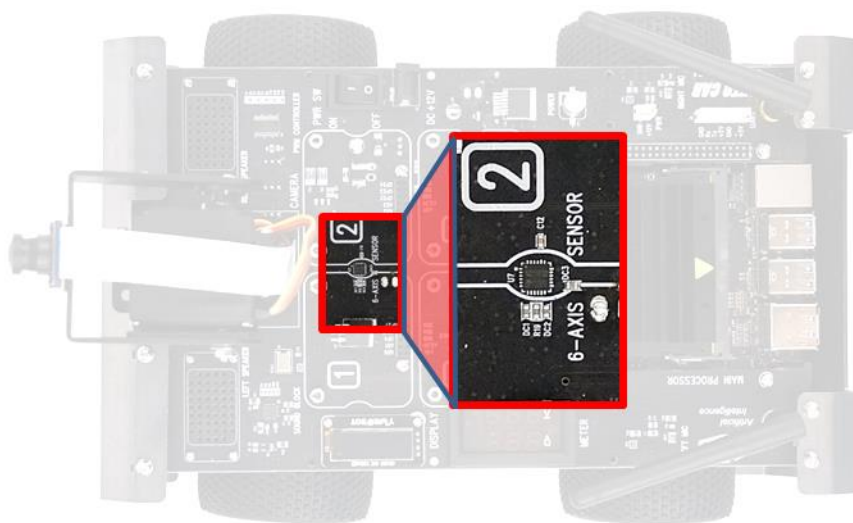


그림 24 6 축 센서 위치

내용정리

- 차량 제어 주요 메소드
 - steering : -1 ~ 1 값을 지정하면 차량의 앞바퀴를 좌/우로 조향
 - setSpeed(value) : 0~99 값으로 차량의 속력을 제어
 - stop() : 차량을 정지
 - forward() : 차량을 전진
 - backward() : 차량을 후진
 - joystick() : 가상 조이스틱을 이용해 차량을 제어
- 카메라 제어 주요 메소드
 - camPan(value) : 0~180 값으로 카메라를 좌/우로 제어
 - camTilt(value) : -30~200 값으로 카메라를 위/아래로 제어
- 6 축 센서 주요 메소드
 - getGyro(axis) : 'x', 'y', 'z' 를 입력하면 해당 축의 자이로 값을 반환하고,
아무 입력없이 호출시 모든 축에 대한 값을 딕셔너리 반환
 - getAccel(axis) : 'x', 'y', 'z' 를 입력하면 해당 축의 가속도 값을 반환하고,
아무 입력없이 호출시 모든 축에 대한 값을 딕셔너리로 반환

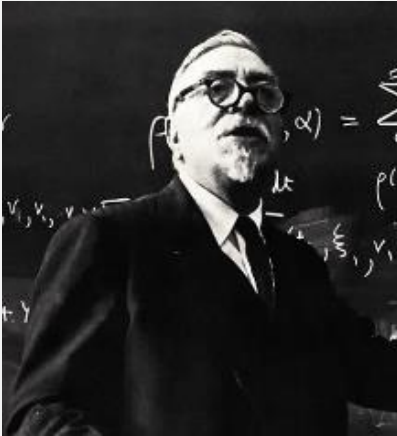


그림 25 Norbert Wiener

1948

Norbert Wiener 는 인공 지능 및 제어 시스템에 대한 연구에 큰 영향을 미치는 'Cybernetics'라는 책을 발간합니다.

Wiener 는 레이더 이미지를 해석하여 적의 비행기 코스를 예상하는 대공 시스템으로 2 차 세계 대전 실험을 시작했습니다.

Wiener 는 "steersman"이라는 그리스어 단어에서 "cybernetics"라는 용어를 만들었습니다.



그림 26 Frederix Williams and Tom Kilburn

1948

맨체스터 대학의 연구원인 Frederic Williams,

Tom Kilburn, Geoff Toothill은 맨체스터 "Baby"로 더 잘 알려진 소규모 실험기 (SSEM)를 개발합니다. Baby는 Williams and Kilburn이 개발한 새로운 메모리 기술(Williams Tube라고 알려진 최초의 컴퓨터용 고속 전자 랜덤 액세스 메모리)을 테스트하기 위해 만들어졌습니다. 17 개의 명령으로 구성되었으며 Kilburn이 작성한 첫 번째 프로그램은 1948년 6월 21일에 실행되었습니다. 이 프로그램은 역사상 최초의 디지털 전자 저장 프로그램 컴퓨터에서 실행된 프로그램입니다.

4. 카메라 활용

카메라는 렌즈와 이미지 센서 및 프로세서로 구성됩니다. 렌즈로 들어오는 빛을 이미지 센서가 전기 신호로 변환하면 프로세서가 신호를 분석해 하나의 디지털 이미지로 만듭니다.

이미지 센서는 잠자리의 눈처럼 수많은 센서 배열로 구성되어 있습니다. 센서 배열을 이루는 각 센서에서는 색상 필터에 빛을 투과시켜 특정 색상의 밝기를 메모리에 저장합니다. 이때 센서 배열은 초록을 더 많이 배치하는 베이어 패턴 Bayer Pattern 을 사용해 초록빛을 더 많이 수용하는데, 이는 사람의 눈이 초록빛에 더 민감하게 반응하기 때문입니다.

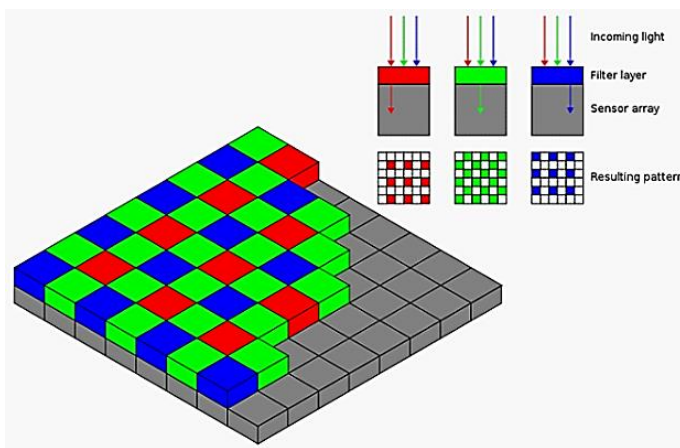


그림 27 베이어 패턴 이미지 센서

이미지 신호처리 프로세서는 색상필터들이 감지한 밝기를 분석해 픽셀을 만드는데, 100% 풀 컬러를 만들려면 400% 베이어 어레이 이미지 센서가 필요합니다.

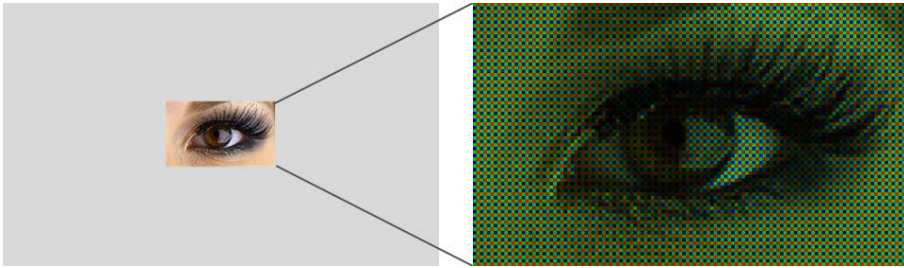


그림 28 이미지 센서와 픽셀

4.1. OpenCV

OpenCV는 인텔에서 오픈소스 기반으로 개발된 실시간 이미지 프로세싱 전용 라이브러리입니다. 윈도우에서 C++ 기반으로 개발을 시작했으나 현재는 Java, Python 등으로 확장되고 리눅스, 안드로이드, OS X 등 다양한 운영체제를 지원하고 있습니다.

이진화를 비롯해 노이즈 제어, 외곽선 검출, 이미지 변환, 패턴 인식, 기계 학습에 이르기까지 대부분의 알고리즘이 구현되어 있으므로 이미지 프로세싱 입문이 곧 OpenCV 입문이라고 봐도 무방합니다.

4.1.1. OpenCV 설치

OpenCV는 부록 518쪽을 참고해 직접 컴파일해서 설치할 수 있습니다. 그러나 Soda OS에는 이미 4.2.0 버전이 설치되어 있으므로 설치 과정을 수행하지 않아도 됩니다.

4.1.2. 이미지 활용

OpenCV 는 간단한 메소드로 이미지를 읽고 쓸 수 있습니다.

이미지를 표시해주는 메소드인 imshow() 메소드는 창을 띄워 이미지를 출력합니다. 그러나 Jupyter 환경에서는 창을 띄울 수 없기에 동작하지 않습니다. 이 문제는 한백전자에서 제공하는 유틸리티 패키지 Pop.Util 의 enable_imshow() 메소드를 사용하여 해결할 수 있습니다.

enable_imshow() 메소드는 창을 띄우는 것 대신 웹 그래픽으로 대체하여 표시하도록 OpenCV 의 imshow() 메소드를 변경합니다.

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
```

코드 1 imshow 를 구현한 pop 라이브러리 사용

4.1.2.1. 이미지 읽기

이미지를 읽을 때는 다음과 같이 imread() 메소드를 이용합니다. 이때 Blue, Green, Red 순으로 채널을 읽습니다.

- imread(fileName, flag): 파일에서 이미지를 읽어 numpy 의 ndarray 객체 반환
 - fileName: 이미지 파일 경로
 - flag: 읽기 옵션
 - cv2.IMREAD_COLOR: 기본값으로 컬러로 읽음
 - cv2.IMREAD_GRAYSCALE: 회색톤으로 읽음
 - cv2.IMREAD_UNCHANGED: 알파 채널을 포함해 읽음

- 반환: numpy.ndarray 객체

먼저, 예제에 사용할 이미지 파일이 필요합니다. Pop.Util 라이브러리에서 createIMG() 메소드를 실행하면 테스트용 이미지 파일을 'img.jpg'라는 이름으로 생성합니다.

```
01: from pop import Util
02: Util.createIMG()
```



그림 29 생성된 img.jpg 파일

다음 예제는 경로에서 이미지를 읽은 후 너비, 높이, 채널 수를 출력합니다.

```
01: import cv2
02:
03: image = cv2.imread("img.jpg", cv2.IMREAD_COLOR)
04: height, width, channel = image.shape
```

```
05:
06: print("width: %d, height: %d, channel: %d"%(width, height, channel))
```

[출력]

width: 640, height: 480, channel: 3

4.1.2.2. 이미지 보기

이미지 출력은 앞서 말한 `imshow()` 메소드를 사용합니다. `imread()` 메소드와 마찬가지로 BGR 타입의 이미지를 표시합니다.

- `imshow(title, image)`: 창에 이미지 표시
 - title: 창 제목
 - image: BGR 배열

opencv 메소드 사용을 위한 `cv2` 라이브러리와 Jupyter 에서 `imshow()` 메소드를 사용하기 위해 `pop` 의 `Util` 라이브러리를 불러옵니다. 그리고 `enable_imshow()` 메소드로 `imshow()` 메소드를 사용 가능하도록 만들어 줍니다.

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
```

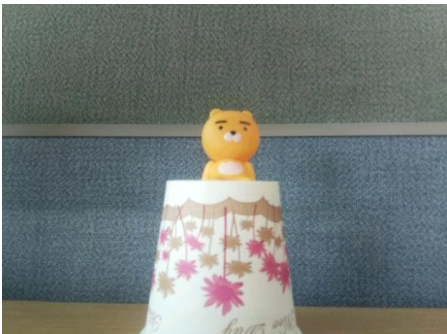
불러올 이미지 파일명을 `filename` 에 저장합니다. 이후 `imread` 메소드를 이용해 같은 사진을 컬러와 회색톤으로 읽고 각각 `imColor` 와 `imGray` 에 저장합니다.

```
05: filename = "img.jpg"
06:
07: imgColor = cv2.imread(filename, cv2.IMREAD_COLOR)
08: imgGray = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
```

불러온 이미지를 “Color”란 이름의 창과 “GrayScale”이라는 이름의 창에 맞춰 출력합니다.

```
09: cv2.imshow("Color", imgColor)
10: cv2.imshow("GrayScale", imgGray)
```

[출력]



전체 소스 코드

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
05:
06: filename = "img.jpg"
07:
08: imgColor = cv2.imread(filename, cv2.IMREAD_COLOR)
```

```
09: imgGray = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
10:
11: cv2.imshow("Color", imgColor)
12: cv2.imshow("GrayScale", imgGray)
```

Jupyter 환경이 아닌 경우 imshow() 메소드는 즉시 닫히므로 창을 유지하려면 대기가 필요합니다. 이 때 waitKey() 메소드를 사용할 수 있습니다.

- waitKey(n): 키 입력 대기
 - n: 밀리초 단위 대기 제한 시간
 - 키 입력이 없어도 대기 제한 시간을 초과하면 반환
 - 0은 대기 제한 시간 사용 안함
 - 반환: 입력 키 값
 - 키 입력 없이 대기 제한 시간으로 반환되면 -1

또, 한 번에 모든 창을 닫을 때는 destroyAllWindows() 메소드를 사용합니다.

- destroyAllWindows(): 모든 창 닫기

```
13: cv2.waitKey(0)
14: cv2.destroyAllWindows()
```

4.1.2.3. 이미지 채널 변경

BGR 순서의 이미지 데이터는 기본적으로 RGB의 순서를 따르는 다른 이미지 뷰어와 호환이 되지 않을 수도 있습니다. 그렇기 때문에 BGR 순서의 채널을 RGB 순서로 변경해봅니다.

기본적으로 필요한 라이브러리를 불러옵니다. 그리고 Jupyter 환경에서 imshow를 사용할 수 있도록 만들어주고 이미지를 가져옵니다.

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
05: imgOrigin = cv2.imread("img.jpg", cv2.IMREAD_COLOR)
```

split() 메소드는 다차원 ndarray 객체를 1차원 ndarray 객체로 바꾸는데, 이는 이미지 데이터를 채널별로 분리해 줍니다. 이렇게 분리된 채널을 Red, Green, Blue의 순서로 merge() 메소드를 통해 다시 묶습니다.

```
06: b, g, r = cv2.split(imgOrigin)
07: imgNew = cv2.merge([r, g, b])
```

이렇게 변경된 이미지를 imshow()로 출력하고 대기합니다. 키 입력이 들어올 경우 모든 창을 종료시킵니다.

```
08: cv2.imshow("Origin", imgOrigin)
09: cv2.imshow("New", imgNew)
10:
11: cv2.waitKey(0)
12: cv2.destroyAllWindows()
```

[출력]



원본 이미지와 비교했을 때 Red - Blue 채널이 바뀐 이미지가 표시됩니다.

앞의 조각 코드들의 전체 소스코드는 다음과 같습니다.

전체 소스 코드

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
05:
06: imgOrigin = cv2.imread("img.jpg", cv2.IMREAD_COLOR)
07:
```

```
08: b, g, r = cv2.split(imgOrigin)
09: imgNew = cv2.merge([r, g, b])
10:
11: cv2.imshow("Origin", imgOrigin)
12: cv2.imshow("New", imgNew)
13:
14: cv2.waitKey(0)
15: cv2.destroyAllWindows()
```

4.1.2.4. 이미지 저장

앞서 진행한 예제에서는 RGB 순서의 채널로 변경했으나 OpenCV 라이브러리는 BGR 순서로 처리하기 때문에 결과가 잘못된 것처럼 보일 수 있습니다. 변경된 사진을 저장하고 확인해봅니다.

imwrite() 메소드는 BGR 순서의 배열을 다른 이미지 뷰어와의 호환성을 유지하기 위해 RGB 순서로 바꿔 이미지 파일로 저장합니다. 저장할 때에는 다른 이름으로도 저장할 수 있습니다.

- imwrite(filename, image): BGR 순서의 ndarray 객체를 RGB 순서의 이미지 파일로 저장
 - filename: 이미지 파일 경로
 - image: ndarray 객체

이번 예제에서는 같은 이미지를 Color 와 Grayscale 로 변환 후 저장해봅니다. Color 는 기존 이미지를 덮어쓰고 Gray 는 새로운 이미지 파일을 생성합니다.

```
01: import cv2
02:
03: imgOrigin = cv2.imread("img.jpg", cv2.IMREAD_COLOR)
04: imgGray = cv2.imread("img.jpg", cv2.IMREAD_GRAYSCALE)
```

```
05:
06: cv2.imwrite("img.jpg", imgColor)
07: cv2.imwrite("imgGray.jpg", imgGray)
```

4.1.3. 카메라 활용

OpenCV 에서는 이미지뿐만 아니라 영상 처리도 가능합니다.

카메라나 파일에서 비디오 프레임을 읽을 때에는 GStreamer 프레임워크와 VideoCapture 클래스를 사용합니다.

4.1.3.1. GStreamer 프레임워크

OpenCV 에서 카메라에 접근할 때는 GStreamer 라는 프레임워크를 통해 접근합니다. GStreamer 는 파이프라인 기반의 멀티미디어 프레임워크로 단순 오디오 재생, 비디오 재생, 녹음, 스트리밍 및 편집 등 다양한 미디어를 프로그래머가 관리할 수 있는 기능을 제공합니다.

GStreamer 를 사용하려면 해상도, 프레임, 색상 채널 등의 설정이 필요합니다. 다음과 같이 필요한 설정값을 아래 문장에서 수정하고 VideoCapture 클래스 생성자로 입력하면 카메라에 접근할 수 있습니다.

```
01: import cv2
02:
03: cam = "nvarguscamerasrc ! video/x-raw(memory:NVMM), width=(int)640,
    height=(int)480, format=(string)NV12, framerate=(fraction)30/1 !
    nvvidconv flip-method=0 ! video/x-raw, width=(int)640,
    height=(int)480, format=(string)BGRx ! videoconvert ! video/x-raw,
    format=(string)BGR ! appsink"
04:
05: cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
```

그러나 문장이 너무 길어 매번 입력하기 힘드므로 Pop.Util 라이브러리에 미리 정의되어 있는 gstrmer() 메소드를 사용합니다.

```
01: import cv2
02: from pop import Util
03:
04: cam = Util.gstrmer(width=640, height=480, fps=30, flip=0)
05: cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
```

4.1.3.2. 카메라 캡처

카메라 캡처를 위한 VideoCapture 클래스의 주요 내용은 다음과 같습니다.

- VideoCapture([filename | device]): VideoCapture 객체 생성
 - filename: 파일 경로: .avi 과 같은 비디오 파일 또는 "img_%02d.jpg"와 같은 이미지 시퀀스
 - device: 연결된 카메라 인덱스로 하나의 카메라가 연결되어 있으면 0. 이후 1 씩 증가
- isOpened(): 카메라에 접근 가능한 지에 대한 여부
- release(): 비디오 파일 또는 캡처 장치를 닫음
- read(): 다음 비디오 프레임을 캡처해 BGR 로 디코딩한 ndarray 객체 반환
 - 반환: 튜플 타입 (retval, frame)
 - retval: 성공하면 True. 아니면 False
 - frame: BGR 순서의 ndarray 객체
- get(propId): VideoCapture 속성 반환
- set(propId, value): VideoCapture 속성 설정
 - 비디오 속성은 부록 521 쪽 또는 아래 링크를 참고

<https://github.com/hanback-docs/OpenCV-Examples#2>



다음 예제는 VideoCapture 클래스를 이용해 카메라 데이터를 640x480 해상도로 창에 표시하는 예제입니다.

필요한 라이브러리를 불러오고, Jupyter 환경에서 imshow() 메소드를 사용하기 위해 Util 의 enable_imshow() 메소드로 활성화해줍니다.

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
```

Util 의 gstrmer() 메소드로 카메라 해상도를 지정하고 VideoCapture 객체를 생성합니다.

```
05: cam = Util.gstrmer(width=640, height=480)
06: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
07: if not camera.isOpened():
08:     print("Not found camera")
```

폭과 높이에 대한 정보를 얻고 출력합니다.

```
09: width = camera.get(cv2.CAP_PROP_FRAME_WIDTH)
10: height = camera.get(cv2.CAP_PROP_FRAME_HEIGHT)
11: print("init width: %d, init height: %d" % (width,height))
```

read() 메소드는 한 프레임씩 반환하므로 for 문을 이용해 읽기를 반복하며 총 120 개의 프레임을 출력합니다. 만약 반환되는 데이터가 없으면 반복을 종료합니다.

```
12: for _ in range(120):
13:     ret, frame = camera.read()
14:     if not ret:
```

```
15:         break
16:
17:     cv2.imshow("soda", frame)
```

반복이 종료된 이후에는 카메라 장치를 닫고 출력 창 또한 닫습니다.

```
18: camera.release()
19: cv2.destroyAllWindows()
```

[출력]



전체 소스 코드

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
05:
06: cam = Util.gstrmer(width=640, height=480)
07:
08: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
09: if not camera.isOpened():
10:     print("Not found camera")
11:
12: width = camera.get(cv2.CAP_PROP_FRAME_WIDTH)
```

```

13: height = camera.get(cv2.CAP_PROP_FRAME_HEIGHT)
14: print("init width: %d, init height: %d" % (width,height))
15:
16: for _ in range(120):
17:     ret, frame = camera.read()
18:     if not ret:
19:         break
20:
21:     cv2.imshow("soda", frame)
22:
23: camera.release()
24: cv2.destroyAllWindows()

```

4.1.3.3. 비디오 저장

OpenCV 로 비디오를 저장할 때는 VideoWriter 클래스를 사용하며 주요 기능은 다음과 같습니다.

- VideoWriter(filename, fourcc, fps, frameSize): VideoWrite 객체 생성
 - filename: 확장자 .avi 인 파일 이름
 - fourcc: 프레임을 압축하는데 필요한 코덱 코드
 - VideoWriter_fourcc(c1, c2, c3, c4) 메소드로 생성
 - 'D', 'I', 'V', '4': DivX MPEG-4
 - 'P', 'I', 'M', '1': MPEG-1 코덱
 - 'M', 'J', 'P', 'G': 모션 JPEG 코덱
 - 'X', '2', '6', '4': H.264
 - 'X', 'V', 'I', 'D': XVID MPEG-4
 - fps: 프레임 속도
 - frameSize: (width, height): 프레임 크기 (튜플)
- write(image): 비디오 프레임 쓰기
 - image: ndarray 객체

- set(propId, value): VideoCapture 속성 설정

이번에는 Camera 로 읽은 프레임을 파일로 저장하는 예제입니다.

우선 필요한 모듈들을 불러오고 imshow() 메소드를 사용할 수 있도록 합니다.

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
```

Util 의 gstrmer() 메소드로 입력받을 카메라 해상도를 지정하고 VideoCapture 객체를 생성합니다.

```
05: cam = Util.gstrmer(width=640, height=480)
06: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
07: if not camera.isOpened():
08:     print("Not found camera")
```

H.264 코덱을 사용하는 VideoWriter 객체를 생성합니다. 이때 결과 영상은 640*480, 30FPS 이며 soda.avi 라는 이름을 가지게 됩니다.

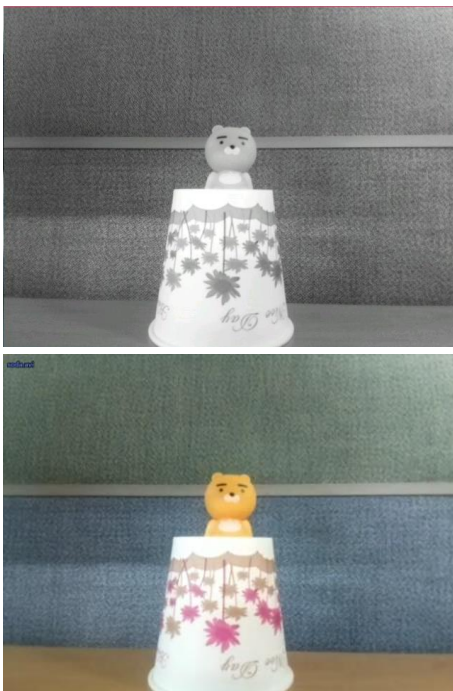
```
09: fourcc = cv2.VideoWriter_fourcc(*"X264")
10: out = cv2.VideoWriter("soda.avi", fourcc, 30, (640,480))
```

read() 메소드는 한 프레임씩 반환하므로 for 문을 이용하여 120 개의 프레임을 읽어옵니다. 프레임을 회색톤으로 변환시킨 뒤 창에 출력하고 원

본 프레임은 영상으로 저장합니다.

```
11: for _ in range(120):  
12:     ret, frame = camera.read()  
13:     framGray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)  
14:     out.write(frame)  
15:  
16:     cv2.imshow("soda", framGray)
```

[출력]



반복이 종료된 이후에는 카메라 장치를 닫고 출력 창 또한 닫습니다.

```
17: camera.release()  
18: cv2.destroyAllWindows()
```

전체 소스 코드

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
05:
06: cam = Util.gstrmer(width=640, height=480)
07: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
08: if not camera.isOpened():
09:     print("Not found camera")
10:
11: fourcc = cv2.VideoWriter_fourcc(*"X264")
12: out = cv2.VideoWriter("soda.avi", fourcc, 30, (640,480))
13:
14: for _ in range(120):
15:     ret, frame = camera.read()
16:     framGray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
17:     out.write(frame)
18:
19:     cv2.imshow("soda", framGray)
20:
21: camera.release()
22: cv2.destroyAllWindows()
```

프로그램을 실행하면 창에 회색톤 화면이 표시되지만 프로그램을 종료된 후 녹화된 파일을 확인해 보면 컬러로 저장되어 있습니다.

4.1.4. 외곽선 검출

영상에서 외곽선은 밝기가 낮은 값에서 높은값으로 변하거나, 또는 이와 반대로 변하는 지점을 가리킵니다. 다른 표현으로 외곽선은 영상안에 있는 객체들의 경계를 가리키는것으로 이를통해 모양, 방향성 등을 탐지할 수 있습니다.

OpenCV 에는 여러가지 외곽선 검출 알고리즘이 포함되어 있습니다. 그 중 캐니 엣지 검출 알고리즘은 잡음에 민감하지 않으며 명확한 외곽선을 검출하는데 목적을 두고 있는 알고리즘입니다. 캐니 엣지 검출은 Canny() 메소드를 통해 사용 가능합니다.

- Canny(image, threshold1, threshold2): 입력 이미지에서 외곽선을 검출하여 반환
 - image: 입력 이미지
 - threshold1, threshold2: 외곽선 검출을 위한 임계치. threshold1 은 최소값, threshold2 는 최대값

필요한 모듈들을 불러오고 imshow() 메소드를 사용할 수 있도록 합니다.

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
```

Pop.Util 의 gstrmer 메소드를 이용해 카메라 해상도를 지정하고 VideoCapture 객체를 생성합니다.

```
05: cam = Util.gstrmer(width=640, height=480)
06: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
07: if not camera.isOpened():
08:     print("Not found camera")
```

read() 메소드는 한 프레임씩 반환하므로 for 문을 이용하여 120 개의 프레임을 읽어옵니다. Canny() 메소드를 통해 프레임에서 외곽선을 검출합니다. Canny() 메소드에서는 8-bit 로 변환 후 임계치 범위를 벗어나는 값들을 제거합니다. 이렇게 변환된 이미지를 창에 출력합니다.

```
09: for _ in range(120):
10:     ret, frame = camera.read()
11:     img = cv2.Canny(frame,100, 200)
12:
13:     cv2.imshow("soda", img)
```

[출력]



프로그램을 실행하면 영상에서 각 객체의 외곽선 검출을 진행한 화면이 출력됩니다.

반복이 종료된 이후에는 카메라 장치를 닫고 출력 창 또한 닫습니다.

```
14: camera.release()
15: cv2.destroyAllWindows()
```

전체 소스 코드

```
01: import cv2
02: from pop import Util
03:
```

```
04: Util.enable_imshow()
05:
06: cam = Util.gstrmer(width=640, height=480)
07: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
08: if not camera.isOpened():
09:     print("Not found camera")
10:
11: for _ in range(120):
12:     ret, frame = camera.read()
13:     img = cv2.Canny(frame,100, 200)
14:
15:     cv2.imshow("soda", img)
16:
17: camera.release()
18: cv2.destroyAllWindows()
```

4.1.5. 얼굴 인식

OpenCV 라이브러리에는 얼굴인식 관련 데이터 모델과 얼굴 인식 알고리즘을 기본 제공합니다. 이를 활용하여 카메라를 통해 입력되는 영상에서 사람 얼굴을 인지하는 프로그램을 작성해 보도록 하겠습니다.

4.1.5.1. Haar Cascades

Haar Cascade 는 머신 러닝 기반의 객체 검출 알고리즘입니다. “Rapid Object Detection using a Boosted Cascade of Simple Features” 논문에서 제안한 특징을 기반으로 이미지나 비디오에서 객체를 검출할 때 사용됩니다. Haar Cascade 는 직사각형 영역으로 구성되는 특징을 사용하기 때문에 픽셀단위로 객체를 검출하는 방법보다 동작 속도 측면에서 빠른 검출 속도를 보입니다. 검출하기 위한 객체가 포함된 이미지와 포함되지

많은 이미지를 활용하여 특징 분류기를 통해 학습을 진행하고, 학습이 완료되면 분류기를 활용하여 객체를 검출합니다. 이 알고리즘은 크게 4 단계로 분류할 수 있습니다.

- Haar Feature Selection : 특징 선택
 - 사각형 형태의 커널을 가지고 특징 계산을 위해 이미지 전체를 스캔
 - 이미지를 스캔하며 이동하는 인접한 사각 영역내에 있는 픽셀의 합의 차이를 활용
- Integral Images : 적분 이미지
 - 사각 영역 내부의 픽셀들을 빠르게 더하고 연산하기 위해 적분 이미지를 사용
- Adaboost Training : 특징 학습
 - 선택한 특징을 활용하여 학습을 진행
 - 선택된 특징중 객체를 검출하기 위한 특징을 선별
 - 선별된 특징을 이용하여 학습에 사용되는 이미지에 특징을 적용
 - 잘못 분류될 가능성이 있기 때문에 어려움이 낮은 특징을 선택
- Cascade Classifier : 특징 분류
 - 학습이 완료되면 입력 이미지를 통해 객체를 검출
 - 입력 이미지에서 객체가 있는 영역인지 단계별로 체크하여 검출

OpenCV에서는 이러한 과정을 거쳐 생성된 얼굴 인식 분류기를 xml 형태의 파일로 제공합니다. 제공되는 분류기는 아래의 경로에 포함되어 있습니다. 분류기의 경로는 OpenCV를 설치할 때 지정하는 설치 경로에 따라서 약간씩 다르게 설치될 수 있습니다.

- /usr/local/share/opencv4/haarcascades/

설치된 분류기를 사용할 때는 CascadeClassifier를 활용합니다. 인자로 미리 학습된 분류기의 경로를 넣을 경우 로드 후 사용가능합니다. 덕분에 상당한 시간이 소요되는 학습과정 없이 빠르게 활용이 가능합니다.

- CascadeClassifier(cascPath): 분류기를 로드하여 반환
 - cascPath: 분류기 파일의 경로
- detectMultiScale(image, scaleFactor, minNeighbors, minSize):
 - image: 얼굴을 인식할 이미지 파일
 - scaleFactor:: 스캔되는 이미지의 축소비율
 - minNeighbors: 발견된 특징이 인정(사용)되기 위한 주변 특징의 개수
 - minSize: 객체로 인식되기 위한 최소 크기

필요한 모듈들을 불러오고 imshow() 메소드를 사용할 수 있도록 합니다.

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
```

OpenCV 에서 제공하는 Haar Cascade 분류기를 로드합니다.

```
05: haar_face=
    '/usr/local/share/opencv4/haarcascades/haarcascade_frontalface_default.xml'
06: face_cascade = cv2.CascadeClassifier(haar_face)
```

Pop.Util 의 gstrmer() 메소드를 이용해 카메라 해상도를 지정하고 VideoCapture 객체를 생성합니다.

```
07: cam = Util.gstrmer(width=640, height=480)
08: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
09: if not camera.isOpened():
```

```
10:     print("Not found camera")
```

입력받은 프레임을 회색톤으로 변환하고 detectMultiScale() 메소드를 이용해 얼굴을 검출합니다. scaleFactor의 값을 줄일 경우 정확도가 늘어날 수가 있지만 속도가 느려지며, minNeighbors의 값을 늘릴 경우에도 정확도가 늘어날 수 있지만 해상도가 떨어지는 이미지에서는 검출에 실패할 수도 있습니다.

```
11: for _ in range(300):
12:     ret, img = camera.read()
13:     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
14:     faces= face_cascade.detectMultiScale(gray, scaleFactor=1.3 ,minNeighbors=1,minSize=(100,100))
```

이후 원본 이미지에서 찾은 객체들의 위치에 사각형을 그려줍니다.

```
15:     for (x,y,w,h) in faces:
16:         cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
17:     cv2.imshow('img',img)
```

이렇게 완성된 이미지를 창에 출력합니다.

```
18: cam.release()
19: cv2.destroyAllWindows()
```

전체 소스 코드

```

01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
05:
06: haar_face=
    '/usr/local/share/opencv4/haarcascades/haarcascade_frontalface_defau
    lt.xml'
07: face_cascade = cv2.CascadeClassifier(haar_face)
08:
09: cam = Util.gstrmer(width=640, height=480)
10: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
11: if not camera.isOpened():
12:     print("Not found camera")
13:
14: for _ in range(300):
15:     ret, img = camera.read()
16:     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
17:     faces= face_cascade.detectMultiScale(gray, scaleFactor=1.3 ,minNe
18:                                         ighbors=1,minSize=(100,100))
19:
20:     for (x,y,w,h) in faces:
21:         cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
22:         cv2.imshow('img',img)
23:
24: cam.release()
25: cv2.destroyAllWindows()

```

코드 2 얼굴 인식

프로그램을 실행하면 영상에서 얼굴을 검출하여 검출된 위치에 파란색 사각형을 그려 화면에 출력합니다.

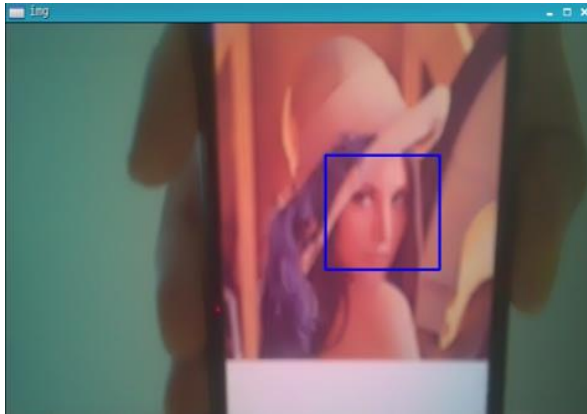


그림 30 얼굴 인식 1

사용되는 분류기를 `haarcascade_smile.xml` 로 변경하면 웃는 얼굴을 검출하는 형태로 활용도 가능합니다.

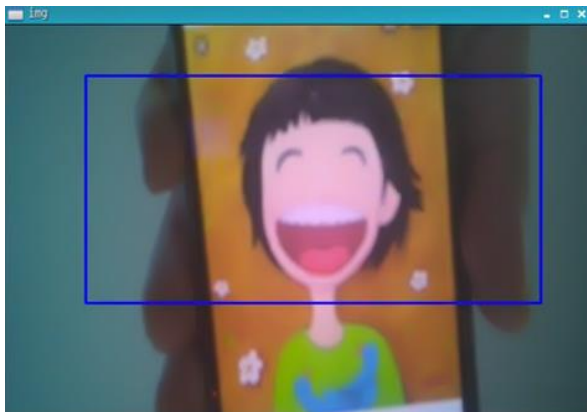


그림 31 얼굴 인식 2

내용정리

- 리눅스 커널은 카메라 하위 시스템인 V4L2로 카메라 장치를 추상화함
- OpenCV : 인텔에서 오픈소스 기반으로 개발된 실시간 이미지 프로세싱 전용 라이브러리
- 파이썬 확장 라이브러리인 numpy의 ndarray 객체를 이용해 다차원 이미지 데이터를 관리
- imread() 메소드 : 이미지를 읽기. 채널 순서를 RGB에서 BGR로 바꿔 읽음
- imshow() 메소드 : ndarray 객체를 이미지로 바꿔 창에 표시
- imwrite() 메소드 : BGR 순서의 ndarray 객체를 RGB로 바꿔 파일로 저장
- 주피터개발환경에서는 OpenCV의 imshow() 메소드가 정상적으로 동작하지 않으므로 파이썬 확장 라이브러리인 Matplotlib로 시각화
- GStreamer : 파이프라인 기반의 멀티미디어 프레임워크
 - 오디오 재생, 비디오 재생, 녹음, 스트리밍 및 편집 등 다양한 미디어를 프로그래머가 관리할 수 있는 기능 제공
- VideoCapture 클래스 : 카메라나 파일에서 비디오 프레임을 읽을 때 사용
- Haar Cascade : 머신 러닝 기반의 객체 검출 알고리즘



그림 32 AT & T 모뎀

1949

컴퓨터는 모뎀을 이용하여 일반 음성 전화선을 통해 "통화"를 시작했습니다. 공군 케임브리지 연구소(AFCRC)의 Jack Harrington 그룹이 레이더 신호를 전송하기 위해 1949년에 개발한 이 모뎀은 디지털 데이터를 사운드로 변조하고 수신된 사운드를 디지털 데이터로 복조합니다. 컴퓨터가 일반 음성 전화선을 사용하게 함으로써 전용 전신 또는 임대 데이터 회선보다 더 넓은 적용 범위와 저렴한 비용으로 사용하게 되었습니다.



그림 33 OXO 스크린

1952

캠브리지 대학 박사 학위를 받은 Alexander Douglas 는 최초의 컴퓨터 게임 중 하나인 Tic-Tac-Toe (영국에서는 'Naughts and Crosses'라고 알려진 OXO)를 개발했습니다. 최초의 컴퓨터들 중 하나인 EDSAC 컴퓨터에서 플레이한 OXO는 플레이어가 기계를 시작하거나 기계를 처음 움직일 수 있었습니다. 회선식 전화 다이얼을 사용하여 움직임을 입력하면 EDSAC는 35 x 15 도트 음극선 튜브에 게임 보드를 표시했습니다.

연습문제

문제 1

다음 코드는 OpenCV 라이브러리를 이용하여 이미지를 읽어 너비, 높이, 채널을 출력하는 코드입니다. 질문을 읽고 답해보세요.

```
01: import cv2
02:
03: image = cv2.  (  )
04: height, width, channel = image.shape
05:
06: print("width: %d, height: %d, channel: %d"%(width, height, channel))
```

- A. 이미지를 읽기 위해 사용되는 메소드 A가 무엇인지 답해보세요.
- B. 'Flash.jpg'라는 이미지 파일을 컬러로 읽으려 할 때 빈 칸 B에 들어갈 내용을 답해보세요.
- C. 이미지의 사이즈가 500x500 일 때 코드의 출력을 답해보세요.

문제 2

다음 코드는 OpenCV 라이브러리를 이용하여 카메라 영상을 읽은 후 출력하는 코드입니다. 질문을 읽고 답해보세요.

```
01: import cv2
02: from pop import Util
03:
04: Util.enable_imshow()
```

```
05:
06: cam = Util.gstrmer(width=640, height=480)
07:
08: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
09: if not camera.isOpened():
10:     print("Error!!")
11:
12: for _ in range(120):
13:     ret, frame = camera.read()
14:     if not ret:
15:         break
16:
17:     cv2.imshow("soda", frame)
18:
19: camera.release()
20: cv2.destroyAllWindows()
```

- A. 코드를 실행했을 때, “Error!!” 가 출력되었다면 그 원인이 무엇인지 답해보세요.
- B. 코드를 실행했을 때, 총 몇 개의 프레임이 출력되는지 답해보세요.
- C. 영상을 회색톤으로 변환하여 출력하는 코드를 작성해보세요.

문제 3

다음 코드는 카메라를 읽어 얼굴을 인식하는 코드입니다. 질문을 읽고 답해보세요.

```
01: import cv2
02: from pop import Util
```

```

03:
04: Util.enable_imshow()
05:
06: haar_face=
    '/usr/local/share/opencv4/haarcascades/haarcascade_frontalface_default.xml'
07: face_cascade = cv2.CascadeClassifier(haar_face)
08:
09: cam = Util.gstrmer(width=640, height=480)
10: camera = cv2.VideoCapture(cam, cv2.CAP_GSTREAMER)
11: if not camera.isOpened():
12:     print("Not found camera")
13:
14: for _ in range(300):
15:     ret, img = camera.read()
16:     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
17:     faces= face_cascade.detectMultiScale(gray, scaleFactor=1.3 ,minNeighbors
        =1,minSize=(100,100))
18:
19:     for (x,y,w,h) in faces:
20:         cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
21:         cv2.imshow('img',img)
22:
23: camera.release()
24: cv2.destroyAllWindows()

```

- A. 코드를 실행했을 때 얼굴이 인식되는지 확인해보고 그 좌표를 출력해 보세요.
- B. 다음 코드는 Pop.Pilot 라이브러리를 이용해 카메라를 움직이는 코드입니다. 이를 응용해 A의 좌표를 기반으로 카메라가 얼굴을 따라 움직이는 코드를 작성해보세요.

```

01: from pop import Pilot
02:
03: Car = Pilot.AutoCar()
04:
05: Car.camTilt(90)

```

5. 오디오 활용

음성 명령으로 센서를 제어하고 프로그램 상태를 음성으로 알려주는 프로그램을 만들려면 오디오 시스템을 활용할 수 있어야 합니다.

마이크로 소리를 녹음하고 스피커로 음악을 출력하는 장치를 사운드 카드라 하는데, 오디오 컨트롤러와 코덱으로 이뤄져 있으며 종류가 매우 많습니다. 따라서 관련 응용프로그램은 다양한 사운드 카드를 고려해 구현되어야 하지만 이는 매우 힘든 일입니다. 리눅스 커널은 이러한 문제점을 해결하기 오디오 하위 시스템을 제공합니다.

5.1. 오디오 하위 시스템

오디오 하위 시스템은 다양한 사운드 카드를 추상화해 사용자가 일관된 방법으로 접근할 수 있도록 표준화된 사운드 인터페이스를 제공합니다.

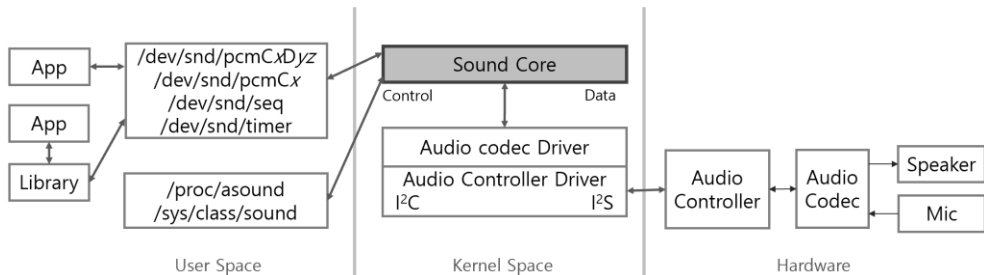


그림 34 리눅스 오디오 하위 시스템

오디오 컨트롤러는 마이크의 아날로그 입력을 디지털로 변환해 입력 스트림에 전달하고 디지털 출력 스트림을 아날로그로 변환해 스피커에 전달합니다. 코덱은 입출력 스트림에 대해 압축/해제와 같은 인코딩/디코딩을 수

행합니다.

오디오 하위 시스템은 오디오 코덱 드라이버와 오디오 컨트롤러 드라이버로 사운드 카드를 제어하는데, 초기에는 유닉스 및 유닉스 계열 운영체제에서 사운드 생성 및 캡처 인터페이스로 사용된 OSS Open Sound System를 사용했으나 현재는 새롭게 설계한 ALSA Advanced Linux Sound Architecture를 사용합니다.

ALSA는 완전히 모듈화 된 사운드 드라이버로 하드웨어 기반 미디 합성 및 여러 개의 채널에 대한 하드웨어 믹싱을 지원하며 멀티프로세서 및 스레드에 안전하게 설계되었습니다. 또한 OSS 바이너리 호환 인터페이스를 통해 대부분의 기존 OSS 프로그램도 수정 없이 사용할 수 있습니다.

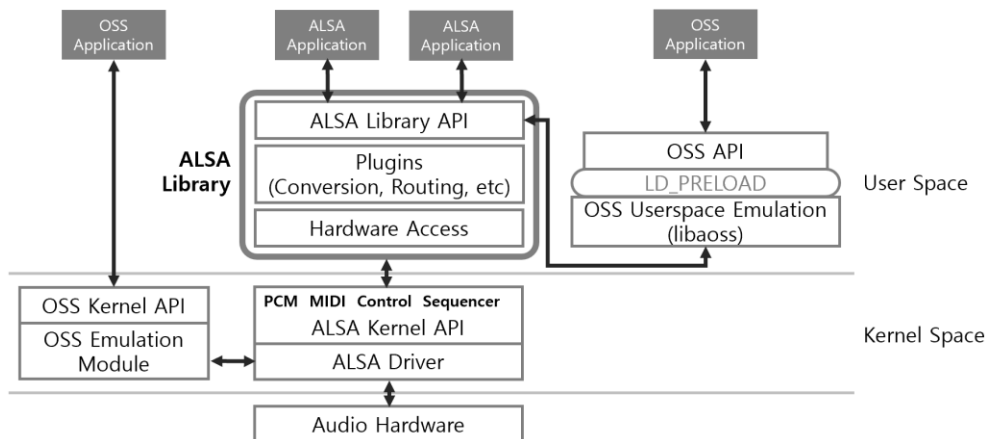


그림 35 ALSA 구조

다음은 ALSA의 주요 기능입니다.

- Control: 사운드 카드 제어를 위한 레지스터 관리
- PCM: 디지털 오디오 캡처와 출력을 위한 인터페이스.
- Raw MIDI: 전자 음악 장비의 표준인 MIDI Musical Instrument Digital Interface 지원
- Time: 사운드 이벤트의 동기화에 사용되는 사운드 카드의 하드웨어 타이밍 접근

- Sequencer: MIDI 를 위한 고수준 인터페이스. 보다 많은 미디 프로토콜과 타이밍 관리
- Mixer: 최 상단에서 사운드 볼륨 제어. 입력 시그널의 선택과 사운드 카드 제어

5.1.1. 오디오 라이브러리

alsa-lib 와 같은 오디오 라이브러리는 커널의 ALSA 오디오 하위 시스템을 응용프로그램에서 접근할 수 있도록 추상화한 것으로 한 응용프로그램의 오디오 출력을 다른 응용 프로그램에 전달하거나 실시간 오디오 및 음악 프로그램을 작성하고 기기 간 공유할 수 있도록 도와줍니다.

하지만 alsa-lib 에서 제공하는 기능은 너무 많고 복잡해 간단한 응용프로그램을 작성할 때도 큰 노력이 필요합니다. 따라서 이를 좀 더 쉽게 사용할 수 있도록 JACK, PulseAudio, PortAudio 과 같은 고수준 라이브러리들이 등장했는데, 이들은 리눅스뿐만 아니라 윈도우나 맥과 같은 다른 플랫폼도 함께 지원합니다.

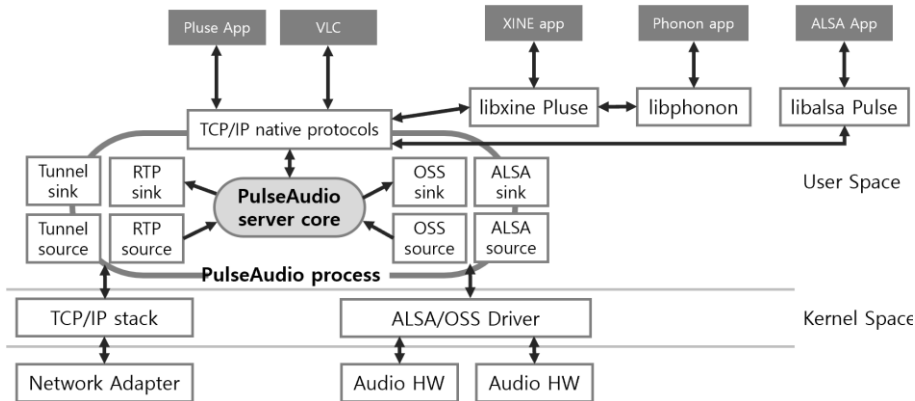


그림 36 PulseAudio 구조

5.1.2. 셸 환경에서 오디오 재생 및 녹음

PCM (Pulse Code Modulation) 은 아날로그인 음성/소리 신호를 디지털 신호로 변경하는 대표적인 방법입니다. 아날로그를 디지털로 변경해 주는 것을 ADC 이고 디지털 신호를 아날로그 신호로 다시 복원해 주는 것을 DAC (Digital to Analog Converter) 라 합니다.

아날로그 신호를 디지털로 신호 바꿀 때는 일정한 주기로 신호 값을 읽어 서 일련의 디지털 값들로 나열하게 됩니다. 이때, 만들어지는 하나의 디지털 값을 샘플이라 하고, 이렇게 샘플들을 만드는 과정을 샘플링, 샘플링을 위한 주기를 샘플링 레이트 또는 샘플링 주파수라고 부릅니다. 샘플링 레이트는 원음에서 사용하는 주파수 대역보다 최소한 2 배 이상 많아 야 원음을 복원할 때 왜곡이 발생하지 않습니다.

AIoT Home 는 셸 환경에서 오디오를 녹음하고 재생하는 명령을 다음과 같이 제공합니다. 특히 ‘sox -t alsa plughw:1 -d’ 명령은 마이크와 스피커를 동시에 테스트하므로 오디오를 사용하기 전에 실행해 봐야 합니다. 만약 문제가 있다면 앞서 소개한 “오디오 설정”을 확인합니다.

- speaker-test: 오디오 출력 테스트
 - speaker-test -c2 -D hw:1,0 -t wav
 - 사운드 카드 1번, 장치 0번에서 2채널 테스트용 wav 재생
 - speaker-test -c2 -t wav:
 - 기본 장치에서 2채널 테스트용 wav 재생
- arecord: raw 또는 wav 포맷으로 오디오 녹음
 - arecord -c 2 -D hw:1,0 --format=S16_LE --duration=5 --rate=16000 --file-type=raw out.raw
 - out.raw 파일에 5초간 16bit 16000 샘플링 레이트, stereo, raw 포맷 녹음
- aplay: raw 또는 wav 포맷 재생
 - aplay -D hw:1,0 --format=S16_LE --rate=16000 out.raw
 - 16bit 16000 샘플링 레이트, raw 포맷으로 녹음된 out.raw 재생
- sox: 다양한 포맷으로 오디오 녹음, 재생, 효과 지원

- `sox -t alsa plughw:1 -d`
 - 사운드 카드 1번의 마이크 입력을 기본 오디오 출력(스피커)으로 전달
- `rec`: 녹음용 심볼릭 링크
 - `rec hello.wav`: 오디오 입력을 wav 포맷으로 hello.wav 에 녹음
- `play`: 재생용 심볼릭마이크
 - `play hello.wav`: wav 포맷의 hello.wav 재생
 - `play example.mp3`

`speaker-test` 와 `aplay`, `arecord` 는 기본 포함되어 있으나 `sox` 은 다음 명령으로 추가 설치해야 하는데, 온디바이스에서 사용하는 Soda OS 에는 기본 포함되어 있습니다.

- `sudo apt-get install sox`

만약 WAV 파일에서 추가로 압축 포맷인 MP3 나 OGG 파일을 만들 때는 `lame` 또는 `oggenc` 명령을 사용하며, `sox`(또는 `play`)로 재생합니다. 이들은 다음 명령으로 설치하는데, Soda OS 에는 기본 포함되어 있습니다.

- `sudo apt-get install lame vorbis-tools`

WAV 파일에서 MP3 또는 OGG 파일을 만드는 명령은 다음과 같습니다. `lame` 는 반대로 MP3 를 WAV 로도 변환할 수 있습니다.

- `lame hello.wav`: WAV 에서 MP3 생성
- `lame --decode my.mp3 my.wav`: MP3 를 WAV 생성
- `oggenc hello.wav`: WAV 에서 OGG 생성

sox 은 aplay, arecord 기능 외에도 다양한 형식의 오디오 파일을 다른 형식으로 변환할 수 있으며, 리눅스를 비롯해 윈도우, Mac OS X 등 다양한 플랫폼을 지원합니다. lame 또는 oggenc 이 설치되어 있다면 MP3, OGG 녹음도 가능합니다.

sox 의 주요 기능은 다음과 같습니다.

- 현재 마이크 입력을 .mp3 로 변환하면서 녹음
 - `rec -t wav - | lame - hello2.mp3`
- 현재 마이크 입력을 .ogg 로 변환하면서 녹음
 - `rec -t wav - | oggenc - -o hello2.ogg`
- 스테레오(2 개 채널)로 일정 시간 녹음
 - `rec -c 2 demo.wav trim 0 00:10`
- 여러 효과를 적용한 후 16bit 깊이로 저장
 - `sox demo.wav -b 16 demo2.wav channels 1 rate 16k fade 3 norm`
 - 하나의 채널로 다운 믹스, 샘플 속도 변경, 페이드 인, 노말라이즈
- 속도 변환
 - `sox demo2.wav demo3.wav speed 1.527`
- 두 개의 오디오 파일 결합
 - `sox demo2.wav demo3.wav demo4.wav`
- 두 개의 오디오 파일을 함께 믹싱
 - `sox -m demo2.wav demo3.wav demo5.flac`
- 저음 부스트 효과를 적용하면서 출력
 - `sox demo5.flac -d bass +20`
- 파이프 오르간 사운드로 합성된 '마이너 7th' 코드 연주
 - `play -n -c1 synth sin %-12 sin %-9 sin %-5 sin %-2 fade h 0.1 1 0.1`

5.2. PyAudio

PyAudio 는 고수준 오디오 라이브러리 중 하나인 PortAudio 에 대한 파이썬 버전으로 리눅스를 비롯해 윈도우, Mac 등을 지원하며 파이썬으로 WAVE 파일을 재생하거나 녹음하는 프로그램을 작성할 수 있습니다.

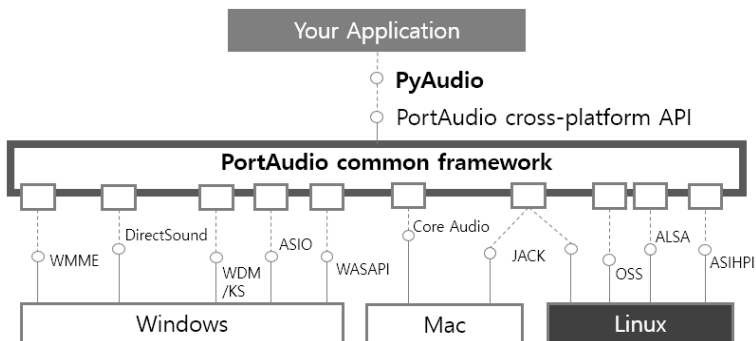


그림 37 PyAudio 구조

PortAudio 와 PyAudio 는 다음 명령으로 설치하는데, Soda OS 에는 미리 설치되어 있습니다.

- `sudo apt install portaudio19-dev`
- `pip3 install PyAudio`

오디오 입출력 작업은 커널의 오디오 하위 시스템에서 수행하므로 사용자는 필요할 때마다 커널에 작업을 요청해야 합니다. 이때 커널이 작업을 완료하고 결과를 반환할 때까지 응용프로그램이 대기하는 것을 블로킹 Blocking 모드라 합니다. 그에 반해 논블로킹 Non-Blocking 모드는 응용프로그램이 중단없이 자신의 작업을 이어갑니다.

PyAudio 는 블로킹과 논블로킹 모드를 모두 지원하는데, 블로킹 모드에서는 `read()` 또는 `write()`로 녹음과 재생을 수행하고 논블로킹 모드는

start_stream()으로 녹음과 재생을 수행하는 내부 스레드를 시작합니다. 둘 다 일시 정지는 stop_stream() 메소드를 사용하고 스트림 종료는 close(), PyAudio 작업 종료는 terminate()입니다.

5.2.1. 주요 메소드와 클래스

Pyudio 는 MP3 와 같은 무손실 압축 포맷을 지원하지 않고 무손실 무압축 포맷인 WAV Waveform Audio Format 파일만 지원합니다. 오디오 작업을 수행할 때는 pyaudio 모듈에 포함된 메소드와 PyAudio, Stream 클래스를 비롯해 WAVE 파일을 읽고 쓰는 wave 모듈의 open() 메소드와 Wave_read 및 Wave_write 객체의 메소드를 함께 사용합니다.

pyaudio 모듈의 get_format_from_width() 메소드는 Stream 객체의 open() 메소드 인자에 전달할 PortAudio 형식의 샘플 크기 얻을 때 사용하고 get_sample_size() 메소드는 PortAudio 형식의 샘플 크기를 int 형식으로 바꿀 때 사용합니다.

- get_format_from_width(width, unsigned=True): 폭에 해당하는 PyAudio 형식 샘플 크기 반환
 - width: 바이트 단위 샘플 폭. 1, 2, 3 또는 4
 - unsigned: True 이면 unsigned. False 는 signed
 - 반환 값: paFloat32, paInt32, paInt24, paInt16, paInt8, paUInt8
- get_sample_size(type): PyAudio 형식 샘플 크기를 정수 타입으로 반환
 - type: paFloat32, paInt32, paInt24, paInt16, paInt8, paUInt8
 - 반환 값: 4(paFloat32, paInt32), 3(paInt24), 2(paInt16), 1(paInt8, paUInt8)

pyaudio 모듈의 PyAudio 클래스로 객체를 만든 후 open()으로 Stream 객체를 만들면 close() 나 terminate()로 종료할 때까지 Stream 객체를 이용해 WAV 파일을 재생하거나 녹음할 수 있습니다.

- `PyAudio()`: PortAudio 리소스에 연결한 PyAudio 객체 생성
 - 반환 값: PyAudio 객체
- `close(stream)`: 스트림을 닫음
 - 일반적으로 이 메소드 대신 Stream 객체의 `close()` 메소드 사용
- `get_format_from_width(width, unsigned=True)`: 폭에 해당하는 PyAudio 형식 샘플 크기 반환
 - `get_format_from_width()` 메소드와 같음
- `get_sample_size(type)`: PyAudio 형식 샘플 크기를 정수 타입으로 반환
 - `get_sample_size()` 메소드와 같음
- `open(*args, **kwargs)`: Stream 객체 생성
 - `args, kwargs`: Stream 객체의 생성자(`Stream()`)에 인자로 전달.
 - 반환 값: Stream 객체
- `terminate()`: PortAudio 리소스 해제

`open()` 메소드로 만들어지는 Stream 객체는 실제 WAV 파일을 읽고 쓰면서 오디오를 재생하거나 녹음합니다. Stream 객체의 생성자인 `Stream()`의 인자는 PyAudio 객체의 `open()` 메소드를 통해 전달됩니다.

- `Stream(PA_manager, rate, channels, format, input=False, output=False, input_device_index=None, output_device_index=None, frames_per_buffer=1024, start=True, ... , stream_callback=None)`: 입력 또는 출력 스트림 초기화
 - `PA_manager`: PyAudio 객체 참조
 - `rate`: 샘플링 속도
 - `channels`: 채널 수
 - `format`: PortAudio 샘플 형식의 샘플링 크기
 - `input`: True 이면 입력 스트림. 기본값은 False
 - `output`: True 이면 출력 스트림. 기본값은 False
 - `input_device_index`: 사용할 입력 장치 인덱스
 - 기본값은 None 으로 기본 장치 사용

- output_device_index: 사용할 출력 장치 인덱스
 - 기본값은 None 으로 기본 장치 사용
- frames_per_buffer: 버퍼 당 프레임 수 지정. 기본값은 1024
- start: True 이면 즉시 스트림을 시작. 기본값은 True
- stream_callback: 논블로킹 작업을 위한 콜백 메소드 지정
 - 기본값은 None 으로 콜백 메소드 없음
- callback(in_data, frame_count, time_info, status_flags): 논블로킹 모드일 때 호출되는 콜백 메소드
 - in_data: 입력 스트림이면 bytes 타입의 읽은 데이터
 - frame_count: 처리 가능한 프레임 수
 - time_info: dict 타입의 시간 정보
 - input_buffer_adc_time, current_time, output_buffer_dac_time
 - status_flags: PortAudio 콜백 플래그
 - 반환 값: 튜플 (out_data, flag)
 - 반환 값을 통해 추가 작업 여부를 전달함
 - out_data: 길이가 (frame_count * channels * bytes-per-channel)인 바이트 배열
 - 출력 스트림: 출력할 데이터
 - 입력 스트림: None 으로 채워진 데이터
 - flag: paContinue, paComplete, paAbort 중 하나
 - paContinue: 작업할 데이터가 더 남아있음
 - paComplete: 작업할 데이터가 없음 (작업 종료)
 - paAbort: 작업 중 오류 발생 (비정상 종료)
- close(): 스트림을 닫음
- get_cpu_load(): 논블로킹 모드에서 CPU 사용률 반환
 - 반환 값: float 타입 CPU 사용률
 - 블로킹 모드에서는 항상 0.0
- get_input_latency(): 입력 대기 시간 반환
 - 반환 값: float 타입 시간
- get_output_latency(): 출력 대기 시간 반환

- 반환 값: float 타입 시간
- `get_read_available()`: 기다리지 않고 읽을 수 있는 프레임 수 반환
 - 반환 값: int 타입 프레임 수
- `get_time()`: 스트림 시간 반환
 - 반환 값: float 타입 시간
- `get_write_available()`: 기다리지 않고 쓸 수 있는 프레임 수 반환
 - 반환 값: int 타입 프레임 수
- `is_active()`: 스트림이 활성 상태인지 반환
 - 반환 값: True 이면 활성, 아니면 False
- `is_stopped()`: 스트림이 멈췄는지 반환
 - 반환 값: True 이면 멈춤, 아니면 False
- `read(num_frames, exception_on_overflow=True)`: 블로킹 모드 스트림에서 샘플 읽기
 - `num_frames`: 읽을 프레임 수
 - `exception_on_overflow`: True 이면 스트림 오버플로를 `IOError` 예외로 알림. 기본값은 True
 - 반환 값: string 객체
- `start_stream()`: 논블로킹 모드일 때 스트림 읽기 시작
- `stop_stram()`: 스트림 중단
 - `start_stream()`으로 다시 시작할 수 있음
- `write(frames, num_frames=None, exception_on_underflow=False)`: 블로킹 모드 스트림에 샘플 쓰기
 - `frames`: 데이터 프레임
 - `num_frames`: 쓸 프레임 수. None 은 자동 계산
 - `exception_on_underflow`: True 이면 스트림 언더플로를 `IOError` 예외로 알림. 기본값은 False

스트림 객체에서 읽고 쓸 WAV 파일은 표준 라이브러리인 `wav` 모듈의 `open()` 메소드로 지정한 후 `Wave_read` 및 `Wave_write` 객체의 메소드로 접근합니다.

- `open(file, mode=None)`: WAV 파일을 열고 `Wave_read` 또는 `Wave_write` 객체 반환
 - `file`: 파일 이름
 - `mode`
 - `'rb'`: 읽기 전용
 - `'wb'`: 쓰기 전용
 - 반환 값: `mode`에 따라 `Wave_read` 또는 `Wave_write` 객체

WAV 파일을 읽을 때 사용하는 `Wave_read` 객체의 주요 메소드는 다음과 같습니다.

- `close()`: wave 객체 제거
- `getnchannels()`: 오디오 채널 수 반환
 - 반환 값: 1: 모노, 2: 스테레오
- `getsampwidth()`: 샘플링 폭 반환
 - 반환 값: int 타입 샘플링 폭. 기본값은 2
- `getframerate()`: 샘플링 빈도 반환
 - 반환 값: int 타입 샘플링 빈도. 기본값은 48000
- `getnframes()`: 전체 프레임 수 반환
 - 반환 값: int 타입 프레임 수
- `readframes(n)`: 오디오 프레임 읽기
 - `n`: 프레임 수
 - 반환 값: bytes 객체

WAV 파일을 쓸 때 사용하는 `Wave_write` 객체의 주요 메소드는 다음과 같습니다.

- `close()`: wave 객체 제거
- `setnchannels(n)`: 채널 수 설정

- n: 채널 수
- setsampwidth(n): 샘플 폭 설정
 - n: 바이트 단위 샘플 폭
- setframerate(n): 샘플링 빈도 설정
 - n: 샘플링 빈도
- setnframes(n): 프레임 수 설정
 - n: 프레임 수
- writeframes(data): 파일에 오디오 프레임 쓰기
 - data: 오디오 프레임

5.2.2. WAV 파일 재생

wave 모듈의 open() 메소드로 WAV 파일을 읽기 전용으로 열면 Wave_read 객체가 반환됩니다. Wave_read 객체의 readframes() 메소드는 프레임 수만큼 데이터를 읽으면서 파일 포인터를 이동하는데, getnframes()로 전체 개수를 파악한 후 한 번에 전체를 읽을 수도 있습니다.

예제들을 실행하려면 wav 파일이 필요합니다. 만약 준비된 파일이 없다면 ‘/usr/share/sounds/alsa/’ 경로의 테스트용 WAV 파일을 사용할 수 있습니다.

5.2.2.1. 블로킹 모드 재생

블로킹 모드에서는 WAV 파일에서 읽은 데이터를 Output Stream 객체의 write() 메소드에 전달하면 스피커에 출력됩니다. Output Stream 객체는 PyAudio 객체의 open() 메소드 인자 중 output에 True를 전달해 만듭니다. open() 메소드에는 그 밖에 샘플링 형식이나 채널 수, 샘플링 비율

등도 함께 전달합니다.

```
01: import pyaudio
02: import wave
03:
04: w = wave.open("/usr/share/sounds/alsa/Side_Left.wav", "rb")
05: data = w.readframes(w.getnframes())
06: w.close()
07:
08: p = pyaudio.PyAudio()
09: stream = p.open(format=p.get_format_from_width(2),
10:                 channels=1,
11:                 rate=48000,
12:                 output=True)
13:
14: stream.write(data)
15:
16: stream.stop_stream()
17: stream.close()
18: p.terminate()
```

프로그램을 실행하면 “Side Left”가 스피커로 출력되는데, 커널 버전에 따라 표준 출력에 “alsa_... failed...” 오류가 표시되더라도 소리 출력에 문제가 없다면 무시합니다.

```
Expression 'alsa_snd_pcm_hw_params_set_period_size_near( pcm, hwParams,
&alsaPeriodFrames, &dir )' failed in 'src/hostapi/alsa/pa_linux_alsa.c',
line: 924
...
```

PyAudio 객체의 `open()` 메소드로 출력 스트림 객체를 만들 때 포맷 폭과, 채널, 샘플링 비율은 WAV 파일을 만들 때 설정된 값과 같아야 합니다.

다. 따라서 상수를 사용하는 것보다 WAVE_Read 객체의 해당 메소드 사용을 권장합니다.

```
19: stream = p.open(format=p.get_format_from_width(w.getsampwidth()),
20:                  channels=w.getnchannels(),
21:                  rate=w.getframerate(),
22:                  output=True)
```

WAV 파일 크기가 크면 readframes() 메소드로 한 번에 모든 프레임을 읽어 출력하는 것보다 블록(또는 청크) 크기로 읽어 출력하는데, readframes() 메소드의 반환 값이 0 일 때까지 반복하면 됩니다.

```
23: data = w.readframes(1024)
24: while len(data) > 0:
25:     stream.write(data)
26:     data = w.readframes(1024)
```

변경된 내용을 적용한 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: import pyaudio
02: import wave
03:
04: w = wave.open("/usr/share/sounds/alsa/Side_Left.wav", "rb")
05: data = w.readframes(w.getnframes())
06: w.close()
07:
08: p = pyaudio.PyAudio()
09: stream = p.open(format=p.get_format_from_width(2),
10:                 channels=1,
```

```

11:             rate=48000,
12:             output=True)
13:
14: stream.write(data)
15:
16: stream.stop_stream()
17: stream.close()
18: p.terminate()
19: p.terminate()
20:
21: stream = p.open(format=p.get_format_from_width(w.getsampwidth()),
22:                 channels=w.getnchannels(),
23:                 rate=w.getframerate(),
24:                 output=True)
25:
26: data = w.readframes(1024)
27: while len(data) > 0:
28:     stream.write(data)
29:     data = w.readframes(1024)

```

5.2.2.2. 논블로킹 모드 재생

블로킹 모드에서는 소리 출력이 완료될 때까지 응용프로그램은 대기 상태에 놓이므로 다른 작업은 불가능합니다. 하지만 논블로킹 모드는 대기 상태에 빠지지 않아 배경 음악처럼 소리를 출력할 수 있습니다.

논블로킹 모드에서는 커널이 재생 가능한 프레임 수를 사용자 메소드에 콜백으로 전달하므로 사용자 콜백 메소드는 호출될 때마다 WAV 파일에서 해당 개수만큼 프레임을 읽어 반환 값으로 커널에 넘겨주면 됩니다.

```

01: def callback(in_data, frame_count, time_info, status):
02:     data = w.readframes(frame_count),

```

```
03:     return (data, pyaudio.paContinue)
```

위의 콜백 메소드를 open 메소드의 stream_callback 인자에 넣어주면 start_stream() 메소드를 통해 비동기 재생을 시작합니다.

```
04: stream = p.open(format=p.get_format_from_width(w.getsampwidth()),
05:                 channels=w.getnchannels(),
06:                 rate=w.getframerate(),
07:                 output=True
08:                 stream_callback=callback)
09:
10: stream.start_stream()
```

그리고 반복문과 is_active()로 재생이 끝났을 경우 프로그램을 종료시킵니다.

```
11: while stream.is_active(): # WAV 재생이 끝나면 False
12:     print("main work...")
13:     time.sleep(0.1)
14:
15: stream.stop_stream()
16: stream.close()
17: p.terminate()
```

블로킹 모드에서 논블로킹 모드로 바뀐 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```

01: import pyaudio
02: import wave
03: import time
04:
05: def callback(in_data, frame_count, time_info, status):
06:     data = w.readframes(frame_count)
07:     return (data, pyaudio.paContinue)
08:
09: w = wave.open("/usr/share/sounds/alsa/Side_Left.wav", "rb")
10: p = pyaudio.PyAudio()
11:
12: stream = p.open(format=p.get_format_from_width(w.getsampwidth()),
13:                 channels=w.getnchannels(),
14:                 rate=w.getframerate(),
15:                 output=True,
16:                 stream_callback=callback)
17:
18: stream.start_stream()
19:
20: while stream.is_active(): # WAV 재생이 끝나면 False
21:     print("main work...")
22:     time.sleep(0.1)
23:
24: stream.stop_stream()
25: stream.close()
26: p.terminate()

```

사용자 콜백 메소드가 커널이 요구한 프레임 수보다 더 적은 데이터를 반환하면 커널은 해당 데이터만 출력하고 더는 사용자 콜백을 호출하지 않습니다. 따라서 배경 음악처럼 재생이 끝날 때마다 다시 처음으로 되감아 재생할 때는 Wave_read 객체의 `rewind()` 메소드로 WAV 파일 포인터를 처음으로 옮긴 후 커널이 요구한 프레임 수에서 부족한 만큼 추가로 더 읽어 커널에 반환해야 합니다.

```

27: def callback(in_data, frame_count, time_info, status):
28:     data = w.readframes(frame_count)
29:     mod = frame_count - len(data) // w.getsampwidth()
30:     if mod != 0:
31:         w.rewind()
32:         data += w.readframes(mod)
33:
34:     return (data, pyaudio.paContinue)

```

5.2.3. 웨이브폼 출력

오디오 데이터는 sin 곡선의 주기 데이터이므로 파이썬 확장 라이브러리 중 하나인 numpy 로 출력할 샘플링 주파수와 옥타브 및 음계 주파수를 이용해 sin 곡선의 웨이브 폼을 만들면 WAV 파일이 없어도 소리 출력이 가능합니다.

numpy 로 샘플링 주파수 및 지속 시간과 음계 주파수로 웨이브 폼을 만드는 식은 다음과 같습니다.

- `numpy.sin(2 * numpy.pi * numpy.arange(fs * duration) * f / fs).astype(numpy.float32)`
 - `numpy.pi`: 파이 값
 - `fs`: 샘플링 주파수
 - `duration`: 지속 시간
 - `f`: 음계 주파수

5.2.3.1. 특정 음 출력

numpy 웨이브 폼 식을 이용해 주파수가 440.0Hz 인 4 옥타브 ‘라’를 PyAudio 로 출력하는 코드는 다음과 같습니다.

```

01: import pyaudio
02: import numpy as np
03:
04: volume = 0.5
05: fs = 48000 # 샘플링 주파수 (연속적인 신호에서 얻어진 단위시간(초) 당
    샘플링 횟수
06: duration = 1.0
07: f = 440.0 # 4 옥타브 '라'
08:
09: data = (np.sin(2 * np.pi * np.arange(fs * duration) * f/fs)).astype(np.
    float32)
10:
11: p = pyaudio.PyAudio()
12: stream = p.open(format=pyaudio.paFloat32, channels=1, rate=fs,
    output=True)
13: stream.write(volume * data)
14:
15: stream.stop_stream()
16: stream.close()
17: p.terminate()

```

5.2.3.2. Tone 클래스 구현

numpy 웨이브 폼 식을 이용하면 사용자가 원하는 음을 출력할 수 있으므로 재사용성을 높이기 위해 이를 Tone 클래스를 구현해 봅니다. Tone 객체를 생성하는 `__init__()` 메소드는 `volume` 과 `rate`, `channels` 를 인자로 받아 PyAudio 및 Output Stream 객체를 만듭니다.

```

01: import pyaudio
02: import numpy as np
03:
04: class Tone:
05:     def __init__(self, volume=.5, rate=48000, channels=1):
06:         self.volume = volume

```

```
07:         self.rate = rate
08:         self.channels = channels
09:         self.p = pyaudio.PyAudio()
10:         self.stream = self.p.open(format=pyaudio.paFloat32,
                                channels=self.channels, rate=self.rate, output=True)
```

play() 메소드는 octave 와 note, duration 을 인자로 받아 numpy 웨이브 폼 식으로 만든 데이터를 재생합니다. duration 은 4 가 약 1 초에 해당합니다.

```
11:     def play(self, octave, note, duration):
12:         f = 2**(octave) * 55 * 2**(((note) - 10) / 12)
13:         sample = (np.sin(2 * np.pi * np.arange(self.rate * duration)
                        * f / self.rate)).astype(np.float32)
14:         self.stream.write(self.volume * sample)
```

stop() 메소드는 출력을 멈추고 Stream 과 PyAudio 객체를 닫습니다.

```
15:     def stop(self):
16:         self.stream.stop_stream()
17:         self.stream.close()
18:         self.p.terminate()
```

with 구문에서도 사용할 수 있도록 __enter__() 메소드와 __exit__() 메소드를 구현하는데, __enter__() 는 Tone 개체를 반환하고 __exit__() 는 stop() 메소드를 호출합니다.

```
19:     def __enter__(self):
20:         return self
```

```
21:
22:     def __exit__(self, type, value, traceback):
23:         self.stop()
```

Tone 객체를 with 구문으로 만들어 3 옥타브 ‘도’에서 ‘시’까지 약 1 초 간격으로 출력해 봅니다.

```
24: with Tone() as tone:
25:     for n in [1, 3, 5, 7, 8, 10, 12]:
26:         tone.play(3, n, 4)
```

지금까지 설명한 Tone 클래스와 이를 사용하는 전체 코드는 다음과 같습니다.

전체 소스 코드

```
01: import pyaudio
02: import numpy as np
03:
04: class Tone:
05:     def __init__(self, volume=.5, rate=48000, channels=1):
06:         self.volume = volume
07:         self.rate = rate
08:         self.channels = channels
09:         self.p = pyaudio.PyAudio()
10:         self.stream = self.p.open(format=pyaudio.paFloat32,
11:                                   channels=self.channels, rate=self.rate, output=True)
12:
13:     def __enter__(self):
14:         return self
15:
16:     def __exit__(self, type, value, traceback):
```

```

16:         self.stop()
17:
18:     def stop(self):
19:         self.stream.stop_stream()
20:         self.stream.close()
21:         self.p.terminate()
22:
23:     def play(self, octave, note, duration):
24:         f = 2**(octave) * 55 * 2**(((note) - 10) / 12)
25:         sample = (np.sin(2 * np.pi * np.arange(self.rate * duration)
        * f / self.rate)).astype(np.float32)
26:         self.stream.write(self.volume * sample)
27:
28: with Tone() as tone:
29:     for n in [1, 3, 5, 7, 8, 10, 12]:
30:         tone.play(3, n, 4)

```

5.2.3.3. Pop 라이브러리의 Tone 클래스

Pop 라이브러리에는 다음과 같은 Tone 클래스가 구현되어 있으므로 단순한 멜로디를 출력할 때는 이를 이용하면 됩니다.

- Tone(tempo=100, volume=.5, rate=48000, channels=1): Tone 객체 생성
 - tempo: 빠르기. 기본값은 100
 - volume: 볼륨. 기본값은 0.5
 - rate: 샘플링 비율. 기본값은 48000
 - channels: 채널 수. 기본값은 1
- close(): 오디오 자원 해제
- setTempo(tempo): 빠르기 설정
 - tempo: 빠르기
- rest(duration): 쉼표
 - duration: 길이. 1, 1/2, 1/4, 1/8, 1/16, ...

- play(octave, pitch, duration): 음 출력
 - octave: 옥타브. 1 ~ 8
 - pitch: 음. "DO", "DO#", "RE", "RE#", "MI", "FA", "SOL", "SOL#", "RA", "RA#", "SI"
 - duration: 길이. 1, 1/2, 1/4, 1/8, 1/16, ...

Pop 라이브러리의 Tone 객체로 “학교종”을 출력해 봅니다. Tone 객체를 만든 후 setTempo()로 빠르기를 200 으로 설정한 후 play()에 옥타브와 음표, 길이를 입력하면 됩니다.

```

01: from pop import Tone
02:
03: shoolBall1 = ((4, "SOL", 1/4), (4, "SOL", 1/4), (4, "RA", 1/4), (4,
    "RA", 1/4), (4, "SOL", 1/4), (4, "SOL", 1/4), (4, "MI", 1/2),
    (4, "SOL", 1/4), (4, "SOL", 1/4), (4, "MI", 1/4), (4, "MI",
    1/4), (4, "RE", 1/2 + 1/4))
04:
05: shoolBall2 = (*shoolBall1[:7], (4, "SOL", 1/4), (4, "MI", 1/2), (4,
    "SOL", 1/4), (4, "MI", 1/4), (4, "RE", 1/4), (4, "MI", 1/4),
    (4, "DO", 1/2 + 1/4))
06:
07: with Tone() as tone:
08:     tone.setTempo(200)
09:
10:     for n in shoolBall1:
11:         tone.play(*n)
12:         tone.rest(1/4)
13:
14:     for n in shoolBall2:
15:         tone.play(*n)
16:         tone.rest(1/4)

```

5.2.4. WAV 파일 녹음

wave 모듈의 open() 메소드로 WAV 파일을 쓰기 전용으로 열면 해당 경로에 WAV 파일이 만들어지고 Wave_write 객체가 반환됩니다.

Wave_read 객체의 setnchannels(), setsampwidth(), setframerate()로 샘플링 타입, 채널 수, 샘플링 비율을 설정한 후 PyAudio 객체의 open() 메소드로 만든 입력 Stream에서 읽은 데이터를 쓰는데 모드에 따라 읽고 쓰는 방식이 다릅니다.

```
01: import wave
02:
03: w = wave.open("./out.wav", "wb")
04: w.setsampwidth(p.get_sample_size(pyaudio.paInt16))
05: w.setnchannels(1)
06: w.setframerate(RATE)
```

5.2.4.1. 블로킹 모드 녹음

블로킹 모드로 녹음할 때는 Input Stream 객체의 read() 메소드로 마이크 입력을 읽은 후 Wave_read 객체의 writeframes()에 이를 전달해 스피커에 출력합니다.

녹음에 필요한 모듈을 import 합니다. 이후 필요한 변수를 선언합니다.

```
01: import pyaudio
02: import wave
03:
04: CHUNK = 1024
05: RATE = 48000
```

Pyaudio 객체와 스트림을 생성한 뒤, 녹음이 저장될 WAV 파일을 열고 샘플링 타입, 채널 수, 샘플링 비율을 설정합니다.

```
06: p = pyaudio.PyAudio()
07: stream = p.open(format=pyaudio.paInt16,
08:                 channels=1,
09:                 rate=RATE,
10:                 input=True,
11:                 frames_per_buffer=CHUNK)
12:
13: w = wave.open("./out.wav", "wb")
14: w.setsampwidth(p.get_sample_size(paudio.paInt16))
15: w.setnchannels(1)
16: w.setframerate(RATE)
```

그리고 while 을 이용해 계속 녹음을 진행합니다. Ctrl + C 를 누를 경우 녹음이 종료됩니다. (Jupyter 에서는 Interrupt the Kernel 버튼)

```
17: try:
18:     while True:
19:         w.writeframes(stream.read(CHUNK))
20: except KeyboardInterrupt:
21:     pass
22:
23: w.close()
24: stream.stop_stream()
25: stream.close()
26: p.terminate()
```

프로그램을 실행하기 전에 다음 명령을 실행해 마이크 입력이 정상적으로 스피커에 출력되는지 확인합니다. 명령 중 옵션 plughw:1 은 캡처 장치 (마이크)로 사운드 카드 1 을 지정하고, -d 는 기본 사운드 카드로 설정된

플레이 장치(스피커)에 캡처한 오디오 데이터를 전달하란 뜻으로 Soda 는 USB 사운드 카드가 사운드 카드 1 이자 기본 사운드 카드입니다.

- `sox -t alsa plughw:1 -d`

마이크가 정상적으로 동작하면 프로그램을 실행해 녹음을 시작합니다. 마이크 입력은 현재 경로의 `out.wav` 에 저장되므로 결과를 확인하려면 `<Ctrl>c` 를 눌러 녹음을 멈춘 후 `play` 명령을 실행합니다.

- `play out.wav`

프로그램에서 녹음 시간을 제한할 때는 `for` 루프 구문을 사용하는데, 입력 Stream 에서 읽은 데이터를 리스트에 추가한 후 루프를 탈출하면 리스트 내용을 모두 파일에 저장하는 방식을 많이 사용합니다. 이때, 녹음 시간을 지나치게 길게 설정하면 메모리가 부족할 수 있으므로 주의합니다.

```
27: TIME = 5    # 5 초
28: data = []
29:
30: for _ in range(0, int(RATE / CHUNK * TIME)):
31:     d = stream.read(CHUNK)
32:     data.append(d)
33:
34: w.writeframes(b''.join(data))
```

아래는 녹음 시간 설정 부분이 적용된 전체 소스 코드입니다.

전체 소스 코드

```
01: import pyaudio
02: import wave
03:
04: CHUNK = 1024
05: RATE = 48000
06:
07: p = pyaudio.PyAudio()
08: stream = p.open(format=pyaudio.paInt16,
09:                 channels=1,
10:                 rate=RATE,
11:                 input=True,
12:                 frames_per_buffer=CHUNK)
13:
14: w = wave.open("./out.wav", "wb")
15: w.setnchannels(1)
16: w.setsampwidth(p.get_sample_size(pyaudio.paInt16))
17: w.setframerate(RATE)
18:
19: TIME = 5    # 5 초
20: data = []
21:
22: for _ in range(0, int(RATE / CHUNK * TIME)):
23:     d = stream.read(CHUNK)
24:     data.append(d)
25:
26: w.writeframes(b''.join(data))
27:
28: w.close()
29: stream.stop_stream()
30: stream.close()
31: p.terminate()
```

5.2.4.2. 논블로킹 모드 녹음

논블로킹 모드로 녹음할 때도 재생과 같이 사용자 콜백 메소드를 사용합니다.

블로킹 모드와 마찬가지로 필요한 모듈을 import 합니다. 이후 필요한 변수들을 선언합니다.

```
01: import pyaudio
02: import wave
03: import time
04:
05: CHUNK = 1024
06: RATE = 48000
07: isStop = False
```

Pyaudio 객체를 생성한 뒤, 녹음이 저장될 WAV 파일을 열고 샘플링 타입, 채널 수, 샘플링 비율을 설정합니다.

```
08: p = pyaudio.PyAudio()
09:
10: w = wave.open("out.wav", 'wb')
11: w.setsampwidth(p.get_sample_size(pyaudio.paInt16))
12: w.setnchannels(1)
13: w.setframerate(RATE)
```

논블로킹 모드이므로 프레임 수 대신 Wave_write 객체의 writeframes() 메소드로 오디오 데이터를 저장합니다. 그리고 저장한 오디오 데이터 크기만큼 빈 데이터를 만들고 paContinue 또는 paComplete 와 함께 튜플로 반환합니다.

```
14: def callback(in_data, frame_count, time_info, status):
15:     w.writeframes(in_data)
16:     data = chr(0) * len(in_data)
17:     return (data, pyaudio.paContinue if not isStop else pyaudio.
        paComplete)
```

앞서 만든 사용자 콜백 메소드를 이용해 논블로킹 모드로 스트림의 데이터를 저장할 수 있습니다.

```
18: stream = p.open(format=pyaudio.paInt16,
19:                 channels=1, rate=RATE, input=True,
20:                 frames_per_buffer=CHUNK,
21:                 stream_callback=callback)
```

start_stream() 메소드를 호출한 뒤 녹음을 시작하고 녹음이 진행 중이라는 것을 알 수 있도록 'recording...'을 출력시킵니다. Ctrl + C 를 누를 경우 녹음이 종료됩니다. (Jupyter 에서는 Interrupt the Kernel 버튼)

```
22: stream.start_stream()
23: try:
24:     print("Recording..", end='')
25:     while True:
26:         print(".", end="")
27:         time.sleep(0.5)
28: except KeyboardInterrupt:
29:     isStop = True
30:     time.sleep(1) # 녹음을 완료할 때까지 대기
31:
32: w.close()
33: stream.stop_stream()
34: stream.close()
35: p.terminate()
```

논블로킹 모드로 녹음을 진행하는 전체 소스 코드입니다.

전체 소스 코드

```

01: import pyaudio
02: import wave
03: import time
04:
05: CHUNK = 1024
06: RATE = 48000
07: isStop = False
08:
09: p = pyaudio.PyAudio()
10:
11: w = wave.open("out.wav", 'wb')
12: w.setsampwidth(p.get_sample_size(pyaudio.paInt16))
13: w.setnchannels(1)
14: w.setframerate(RATE)
15:
16: def callback(in_data, frame_count, time_info, status):
17:     w.writeframes(in_data)
18:     data = chr(0) * len(in_data)    # 현재 읽은 데이터 크기 기준으로
    다음에 읽을 빈 버퍼 생성
19:     return (data, pyaudio.paContinue if not isStop else
    pyaudio.paComplete)
20: stream = p.open(format=pyaudio.paInt16,
21:                 channels=1, rate=RATE, input=True,
22:                 frames_per_buffer=CHUNK,
23:                 stream_callback=callback)
24: stream.start_stream()
25: try:
26:     print("Recording..", end='')
27:     while True:
28:         print(".", end="")
29:         time.sleep(0.5)
30: except KeyboardInterrupt:
31:     isStop = True
32:     time.sleep(1)    # 녹음을 완료할 때까지 대기
33:
34: w.close()
35: stream.stop_stream()

```

```
36: stream.close()  
37: p.terminate()
```

5.2.4.3. 마이크를 주변 소음 크기 측정 센서로 활용

주변 소음은 Input Stream 에서 읽은 오디오 데이터의 RMS 를 계산하면 구할 수 있습니다. audioop 모듈은 원시 오디오 데이터 가공을 위한 여러 메소드를 제공하는데, 그 중 RMS 계산용 rms() 메소드도 포함되어 있습니다. 아래는 RMS 계산 및 소음 크기에 비례한 가로 막대 그래프를 그리는 사용자 콜백 메소드입니다.

```
01: def callback(in_data, frame_count, time_info, status):
02:     rms = audioop.rms(in_data, 2)
03:     print(' ' * (rms//50), '*(', rms, ')')
04:     data = chr(0) * len(in_data)
05:     return (data, pyaudio.paContinue if not isStop else pyaudio.paAbort)
```

변경된 사용자 콜백 메소드를 적용한 전체 소스 코드입니다.

전체 소스 코드

```
01: import pyaudio
02: import audioop
03: import time
04:
05: CHUNK = 1024
06: RATE = 48000
07: isStop = False
08:
09: p = pyaudio.PyAudio()
10:
11: def callback(in_data, frame_count, time_info, status):
12:     rms = audioop.rms(in_data, 2)
13:     print('=' * (rms//50), '*(', rms, ')')
14:     data = chr(0) * len(in_data)
```

```
15:     return (data, pyaudio.paContinue if not isStop else
    pyaudio.paAbort)
16:
17: stream = p.open(format=pyaudio.paInt16,
18:                 channels=1,
19:                 rate=RATE,
20:                 input=True,
21:                 frames_per_buffer=CHUNK,
22:                 stream_callback=callback)
23: stream.start_stream()
24:
25: try:
26:     while True:
27:         time.sleep(0.1)
28: except KeyboardInterrupt:
29:     isStop = True
30:
31: stream.stop_stream()
32: stream.close()
33: p.terminate()
```

프로그램을 실행하면 마이크에서 감지한 소음과 관련된 그래프가 출력됩니다.

```
soda@soda:~ $ python3 code101.py
```

```

*( 284 )
*( 297 )
====*( 302 )
====*( 961 )
====*( 2629 )
====*( 3077 )
====*( 3307 )
====*( 3791 )
====*( 3790 )
====*( 3224 )
====*( 2741 )
====*( 2287 )
====*( 1503 )
====*( 1030 )
...

```

5.2.4.4. Pop 라이브러리의 SoundMeter 클래스

Pop 라이브러리에는 마이크를 고성능 주변소음 측정기로 활용할 수 있는 SoundMeter 클래스가 포함되어 있으며 다음과 같은 기능이 제공됩니다.

- SoundMeter(sType=pyaudio.paInt16, chNum=1, chunk=1024, sRate=48000): SoundMeter 객체 생성
 - sType: 샘플링 타입. 기본값은 paInt16
 - chNum: 채널 수. 기본값은 1
 - chunk: 버퍼 크기. 기본값은 1024
 - sRate: 샘플링 비율. 기본값은 48000
- setCallback(func, *args): 사용자 콜백 메소드 등록 및 측정 시작
 - func: 사용자 콜백 메소드
 - args: 사용자 콜백 메소드가 호출될 때 함께 전달할 인자. 생략 가능
- callback(rms, inData, agrs, ...): 사용자 콜백 메소드
 - rms: 입력 데이터의 RMS
 - inData: 입력 데이터
 - agrs, ...: setCallback()에서 전달한 인자들. 생략 가능
- stop (): 측정을 중단하고 오디오 자원 해제

SoundMeter 를 이용해 주변 소음 크기가 일정값 이상일 경우에 출력하는 예제입니다.

```
01: import time
02: from pop import SoundMeter
03:
04: sm = SoundMeter()
05:
06: def onSoundMeter(rms, inData):
07:     if(rms>600):
08:         print(rms)
09:
10: sm.setCallback(onSoundMeter)
11:
12: input("input something")
13:
14: sm.stop()
```

5.2.5. Pop.AudioPlay, Pop.AudioPlayList, Pop.AudioRecord 클래스

Pop 라이브러리에는 앞서 소개한 Tone 및 SoundMeter 클래스 외에 PyAudio 기반으로 응용프로그램에서 더 쉽게 오디오를 녹음하고 재생할 수 있도록 AudioPlay 와 AudioPlayList, AudioRecord 클래스를 제공합니다.

다음은 AudioPlay 클래스의 주요 기능으로 WAV 파일을 재생할 때 블로킹과 논블로킹 모드를 모두 지원합니다.

- AudioPlay(file, blocking=True, cont=False): AudioPlay 객체 생성
 - file: 재생할 WAV 파일 이름

- blocking: True 이면 블로킹 모드, False 는 논블로킹 모드. 기본값은 True
- cont: 논블로킹 모드일 때 True 이면 반복재생, False 이면 단일 재생. 기본값은 False
- run(): 재생 시작
- isPlay(): 논블로킹 모드일 때 재생 상태 반환
 - True 이면 재생 중, False 이면 종료
- stop(): 논블로킹 모드일 때 멈춤
- close(): 명시적으로 오디오 자원 해제
 - 프로그램이 종료 등으로 AudioPlay 객체가 제거될 때 자동 호출

AudioPlayList 클래스는 파일 리스트를 인자로 전달하고, run() 메소드에 재생할 리스트의 인덱스를 지정할 수 있는 것을 제외하면 AudioPlay 와 같습니다.

- AudioPlayList(files, blocking=True, cont=False): AudioPlayList 객체 생성
 - files: 재생할 WAV 파일 리스트
 - blocking: True 이면 블로킹 모드, False 는 논블로킹 모드. 기본값은 True
 - cont: 논블로킹 모드일 때 True 이면 연속 재생, False 이면 단일 재생. 기본값은 False
- run(pos=0): 재생 시작
 - pos: 리스트 인덱스. 기본값은 0

AudioRecord 클래스의 주요 기능은 다음과 같습니다. 사용자가 원할 때 즉시 녹음을 중단할 수 있어야 하므로 논블로킹 모드만 지원합니다.

- AudioRecord(file, sFormat=8, sChannel=1, sRate=48000, sFramePerBuffer=1024): AudioRecord 객체 생성
 - file: 저장할 WAV 파일 이름
 - sFormat: 샘플링 타입. 8 (paInt16), 2 (paInt32), 1 (paFloat32). 기본값은 8
 - sChannel: 채널 수. 기본값은 1

- sRate: 샘플링 속도. 기본값은 48000
- sFramePerBuffer: 버퍼 당 프레임 수 지정. 기본값은 1024
- run(): 재생 시작
- isPlay(): 논블로킹 모드일 때 재생 상태 반환
 - True 이면 재생 중. False 이면 종료
- stop(): 논블로킹 모드일 때 멈춤
- close(): 명시적으로 오디오 자원 해제
 - 프로그램이 종료 등으로 AudioRecord 객체가 제거될 때 자동 호출

audio_play() 메소드는 단순히 해당 파일을 재생합니다.

- audio_play(file): 파일 재생
 - file: 재생할 WAV 파일 이름

이렇게 Pop 라이브러리의 오디오 관련 클래스들을 이용하면 앞서 사용했던 PyAudio 보다 비교적 간단한 코드로 같은 기능을 이용할 수 있습니다. AudioRecord 와 AudioPlay 클래스를 이용해 녹음 및 재생하는 예제를 만듭니다.

AudioRecord 와 with 키워드를 이용해 약 5 초간 녹음한 내용을 “my_record.wav”라는 이름으로 저장합니다.

```
01: with AudioRecord("my_record.wav") as record:
02:     record.run()
03:     print("Start Recording...")
04:
05:     for _ in range(5):
```

```
06:     time.sleep(1)
07:
08:     record.stop()
09:     print("Stop Recording...")
```

그리고 AudioPlay 와 with 키워드를 이용해 논블로킹으로 12 초간 “my_record.wav”를 반복재생합니다.

```
10: with AudioPlay("my_record.wav", False, True) as play:
11:     play.run()
12:     print("Start Play...")
13:     for _ in range(12):
14:         time.sleep(1)
15:
16:     play.stop()
17:     print("Stop play...")
```

다음은 녹음과 재생을 합친 전체 소스 코드입니다.

전체 소스 코드

```
01: from pop import AudioRecord, AudioPlay
02: import time
03:
04: with AudioRecord("my_record.wav") as record:
05:     record.run()
06:     print("Start Recording...")
07:
08:     for _ in range(5):
09:         time.sleep(1)
```

```
10:
11:     record.stop()
12:     print("Stop Recording...")
13:
14: # 논블로킹 모드, 반복재생
15: with AudioPlay("my_record.wav", False, True) as play:
16:     play.run()
17:     print("Start Play...")
18:     for _ in range(12):
19:         time.sleep(1)
20:
21:     play.stop()
22:     print("Stop play...")
```

5.3. 텍스트 음성 변환

텍스트를 음성으로 변환할 때 사용하는 TTS Text to Speech 는 모델로 선정된 사람의 말소리를 녹음해 일정한 음성 단위로 나눈 후 부호를 붙여 합성기에 저장했다가 필요에 따라 해당 음성 단위만을 다시 합쳐 인위적인 말소리를 만드는 기술입니다. 일반적으로 음성의 분절음을 합성하는 것이 어려워 분절음의 경계를 기준으로 음성의 앞과 뒤를 함께 기록해 음성합성을 합니다.

텍스트 음성 변환 시스템은 프론트엔드와 백엔드로 나뉘지는데, 프론트엔드는 사용자가 입력한 텍스트를 정규화해 숫자나 생략된 표현을 어떻게 처리할지 결정하고 각 단어를 발음 기호로 변환하면서 단어나 숙어, 문장 단위로 분할한 결과를 백엔드로 보냅니다. 백엔드는 이를 토대로 보다 자연스러운 음성으로 만들기 위해 운율 등을 조정한 다음 실제 음성 데이터로 합성해 출력합니다.

5.3.1. 구글 텍스트 음성 변환기

구글 텍스트 음성 변환기 안드로이드 스마트폰 화면의 텍스트를 소리 내어 읽도록 개발한 다국어 지원 화면 판독기 응용프로그램으로 시작해, 현재는 AI 전문 자회사인 DeepMind 의 WaveNet 에 이식되어 클라우드 버전으로 진화하고 있습니다.

DeepMind 의 AI 음성 합성 기술은 특히 고급스럽고 현실적입니다. 애플의 Siri 를 포함한 대부분의 음성 합성기는 연결합성 방식을 사용하지만 WaveNet 은 개별 음절을 저장한 후 이를 조합해 단어와 문장을 구성하며 기계 학습으로 음성을 생성합니다. 초당 24,000 개의 샘플 속도로 재생할 수 있도록 사람의 음성 데이터베이스를 통해 파형을 생성하며 최종 결과는 악센트와 같은 미묘한 처리까지 포함됩니다.

5.3.2. gTTS

gTTS는 클라우드 기반 구글 번역기에 포함된 텍스트 음성 변환 API에 대한 파이썬 라이브러리로 명령행 기반 툴도 함께 제공합니다. 구글 클라우드 서비스로 텍스트를 전달하면 음성합성 결과를 mp3 파일로 보내 줍니다.

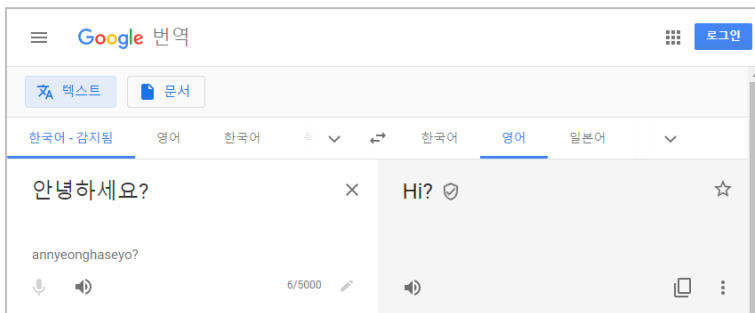


그림 38 클라우드 기반 구글 번역기 (<https://translate.google.com>)

gTTS는 발음을 교정할 수 있는 유연한 전처리와 적절한 억양, 약어, 숫자 등을 유지하면서 길이 제한 없이 텍스트를 읽을 수 있는 맞춤형 음성 분석기 및 언어 자동 감지 기능을 제공합니다.

다음 명령으로 파이썬 라이브러리와 명령행 툴을 설치하는데, Soda OS에는 미리 설치되어 있습니다.

- `sudo pip3 install gTTS`

5.3.2.1. 명령행 툴

gtts-cli 명령은 셸에서 사용자가 입력한 텍스트를 구글 클라우드로 보낸 후 mp3 결과 파일을 받아 현재 경로에 저장하는데, 재생은 play 명령을

사용합니다.

다음은 gtts-cli 의 주요 옵션입니다.

- -f, --file <file>: 파일로부터 텍스트 읽음
- -s, --slow: 천천히 발음
- -l, --lang=<lang_word>: IETF 언어 태그로 언어 설정
 - 기본값은 en(영어). ko(한국어)
- --all: 적용 가능한 IETF 언어 태그 출력
- -o, --output <.mp3>: 결과를 파일로 저장
 - 기본값은 표준 출력으로 결과(mp3 내용) 전달

AIoT AutoCar 는 인터넷에 연결한 후 한국어 텍스트 ‘안녕하세요’에 대한 합성 결과를 스피커로 출력하는 명령은 다음과 같은데, 영어 발음이 적용되어 어색하게 들립니다.

- gtts-cli “안녕하세요” | play -t mp3 -

하지만 다음과 같이 --lang=ko 옵션을 추가하면 훨씬 현실적으로 들립니다.

- gtts-cli “안녕하세요” --lang=ko | play -t mp3 -

결과를 파일로 저장할 때는 --output 옵션을 사용하며 play 로 재생합니다.

- gtts-cli “안녕하세요” --lang=ko --output hello.mp3
- play hello.mp3

5.3.2.2. gTTS 클래스

사용자 코드에서 gTTS 를 사용할 때는 gTTS 클래스를 사용하며 다음과 같은 기능을 제공합니다.

- gTTS(text, lang='en', slow=False, lang_check=True, ...): gTTS 객체 생성
 - text: 읽을 텍스트
 - lang: 텍스트를 읽을 언어. IETF 언어 태그
 - 기본값은 영어. 한글은 'ko'
 - slow: True 이면 더 느리게 읽음
 - lang_check: True 이면 언어 오류 감지
 - 언어 선택이 잘못되면 ValueError 예외 발생
- save(savefile): TTS API 요청 수행 및 결과 저장
 - savefile: 저장할 파일 이름
- write_to_fp(fp): save() 메소드와 같으나 결과를 파일 객체에 저장
 - fp: 쓰기 모드의 바이너리 파일 객체

gTTS 문서 사이트 (<https://gtts.readthedocs.io/en/latest/module.html>) 를 방문하면 좀 더 자세한 설명을 찾을 수 있습니다.

gTTS 객체를 만든 후 save() 메소드를 통해 영문 텍스트를 영어로 합성한 후 MP3 로 저장해 봅니다. 결과는 play 명령으로 확인합니다.

```
01: from gtts import gTTS
02: import subprocess
03:
04: text = "And the saddest thing. under the sun above"
05: filename = "en_tts.mp3"
06:
07: tts = gTTS(text)
08: tts.save(filename)
```


gTTS() 생성자의 lang 인자에 'ko'를 전달하면 한글 텍스트를 합성할 수 있습니다. 코드에서 MP3 파일도 함께 재생하려면 subprocess 모듈의 Popen() 메소드에 쉘 명령인 play 와 파일 이름을 인자로 전달하면 됩니다.

```
09: with subprocess.Popen(['play', filename]) as p:  
10:     p.wait()
```

subprocess 모듈을 응용해 시 「너의 하늘을 보아」를 gTTS() 메소드를 통해 낭송시킬 수 있습니다.

```
01: from gtts import gTTS  
02: import subprocess  
03:  
04: FILE_NAME = '너의 하늘을 보아.mp3'  
05:  
06: TEXT = ""네가 자꾸 쓰러지는 것은  
07: 네가 꼭 이를 것이 있기 때문이야  
08: 네가 지금 길을 잃어버린 것은  
09: 네가 가야만 할 길이 있기 때문이야  
10: 네가 다시 울며 가는 것은  
11: 네가 꽃피워 낼 것이 있기 때문이야  
12: 힘들고 앞이 안 보일 때는  
13: 너의 하늘을 보아""  
14:  
15: tts = gTTS(TEXT, lang='ko')  
16: tts.save(FILE_NAME)  
17:  
18: with subprocess.Popen(['play', FILE_NAME]) as p:  
19:     p.wait()
```

클라우드 기반 음성 합성 기능을 실시간으로 프로그램 인터페이스에 적용하면 네트워크 지연 및 음성 합성 시간이 문제가 됩니다. 예를 들어 print()로 출력할 메시지를 음성으로 들려줄 때 클라우드 기반 음성 합성 기능을 실시간으로 적용하면 시간 지연 때문에 오히려 불편할 수 있습니다.

따라서 이런 경우는 save() 메소드를 이용해 프로그램에서 출력할 모든 메시지를 정의한 후 이를 사전에 MP3 파일로 만들어 두고 필요할 때마다 이를 재생하는 것이 더 낫습니다.

다음은 파일 이름과, 텍스트, 언어를 입력하면 Kernel 이 shutdown 될 때까지 지속해서 음성 합성 결과를 생성하는 간단한 소스로 매번 결과를 확인할 수 있도록 재생 기능까지 추가되어 있습니다.

```
01: from gtts import gTTS
02: import subprocess
03:
04: try:
05:     while True:
06:         f = input("Enter of file name: ")
07:         t = input("Enter of Text: ")
08:         l = input("Select language (ko | en): ")
09:
10:         tts = gTTS(t, lang=l)
11:         tts.save(f + ".mp3")
12:
13:         with subprocess.Popen(["play", f + ".mp3"]) as p:
14:             p.wait()
15: except KeyboardInterrupt:
16:     pass
```

5.4. 구글 어시스턴트

구글 어시스턴트는 구글 클라우드에서 제공하는 AI 기반 음성 인식 및 행동 실행 서비스입니다. 구글 클라우드 서버와 연동하는 어시스턴트 SDK는 안드로이드 폰과 엣지 디바이스로 나뉘는데, 라즈베리 파이는 엣지 디바이스용 SDK를 사용해야 하며, 학습이 아닌 상용 배포는 구글과의 계약이 필요합니다.

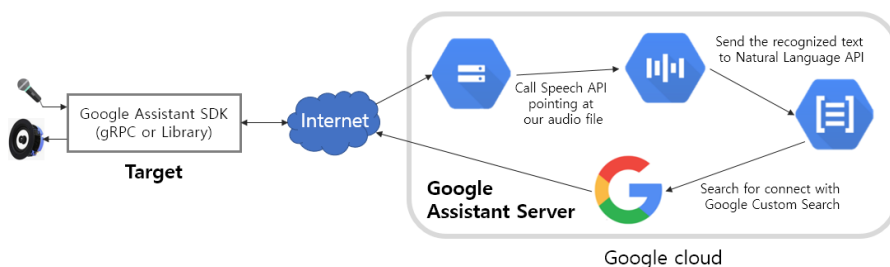


그림 39 구글 어시스턴트 서비스

5.4.1. 구글 어시스턴트 사용 인증

라즈베리 파이와 같은 엣지 디바이스에 구글 어시스턴트를 사용하려면 Google Assistant SDK 설치를 비롯해 구글 어시스턴트 사용 인증이 필요합니다. 구글 어시스턴트 사용 인증은 구글 계정을 기반으로 진행되며 Actions on Google에 새 프로젝트를 만들고 모델을 등록해 자격 증명서를 다운받고 Google Cloud Platform에서 Google Assistant API 사용을 허용한 후 다운받은 자격 증명서로 엣지 디바이스를 인증하는 것으로 마무리됩니다.

Soda OS에는 이 절차가 미리 적용되어 있으므로 바로 서비스를 사용할 수 있지만, 계정마다 엣지 디바이스의 무료 할당량이 1일 500회, 1분 60회로 제한되므로 여러 사용자가 함께 사용하는 Soda OS의 구글 테스트 계정은 1일 할당량을 초과할 가능성이 높습니다. 따라서 Google

Assistant SDK 다시 설치할 필요는 없지만 자신의 계정으로 구글 어시스턴트 사용 인증을 새로 진행할 필요는 있습니다. 다음 주소는 옛지 디바이스에서 구글 어시스턴트 사용에 필요한 세부 절차를 안내합니다.

- <https://developers.google.com/assistant/sdk/guides/service/python/>

5.4.1.1. 활동 제어 설정

첫 번째 단계는 자신의 구글 계정에 대한 활동 제어를 설정하는 것으로 PC에서 크롬 브라우저를 실행한 후 구글 계정에 로그인합니다. 구글 클라우드 서비스를 사용하는 것이므로 계정이 없으면 새로 만들어야 하는데, 기존 계정을 사용하는 것보다는 프로젝트용 계정을 새로 만드는 것을 추천합니다. 프로젝트 계정을 만들기 위해 게스트 세션으로 크롬 브라우저를 실행합니다.

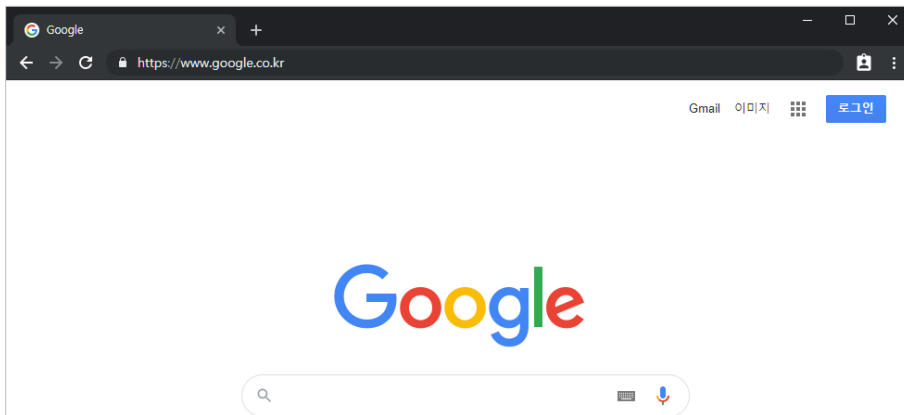


그림 40 게스트 세션으로 크롬 브라우저 실행

오른쪽 상단의 ‘로그인’ 누르고 “계정 만들기”에서 “본인 계정”을 선택합니다.

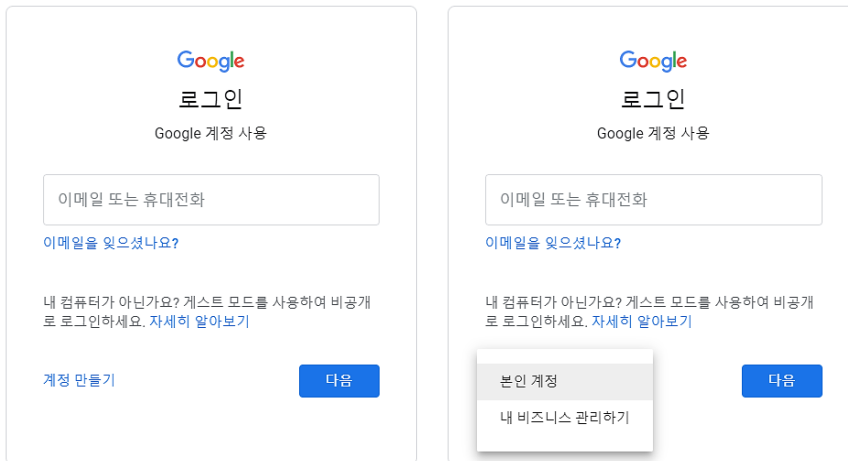


그림 41 프로젝트용 본인 계정 만들기

계정 정보를 입력한 후 ‘다음’을 선택합니다.

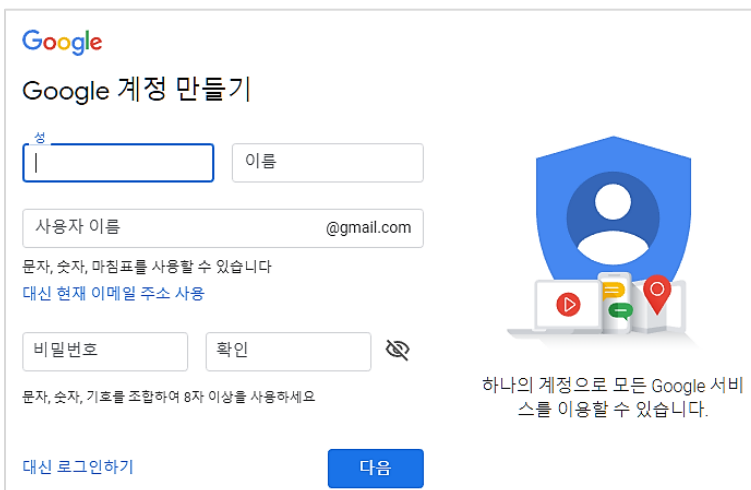


그림 42 계정 정보 입력

자신의 휴대폰 번호를 입력한 후 ‘다음’을 선택합니다.

그림 43 휴대폰 번호 입력

전화번호를 인증하기 위한 인증 코드가 문자로 오면 이를 입력한 후 합니다.

그림 44 전화번호 인증

개인 정보를 입력한 후 ‘다음’을 선택하는데, 생일과 성별을 제외한 옵션 항목은 생략해도 됩니다.



Google에 오신 것을 환영합니다





전화번호는 계정 보안용으로 사용됩니다. 다른 사용자에게는 전화번호가 표시되지 않습니다.

계정을 안전하게 보호하기 위해 사용합니다

연도

월

일

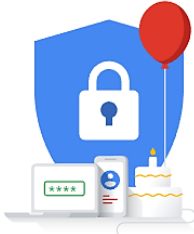
생일

성별

[이 정보가 필요한 이유](#)

뒤로

다음



개인정보를 비공개로 안전하게 유지합니다.

그림 45 개인 정보 입력

스크롤바를 아래로 내려 개인정보 보호 및 약관에 동의한 후 ‘계정 만들기’를 선택합니다.

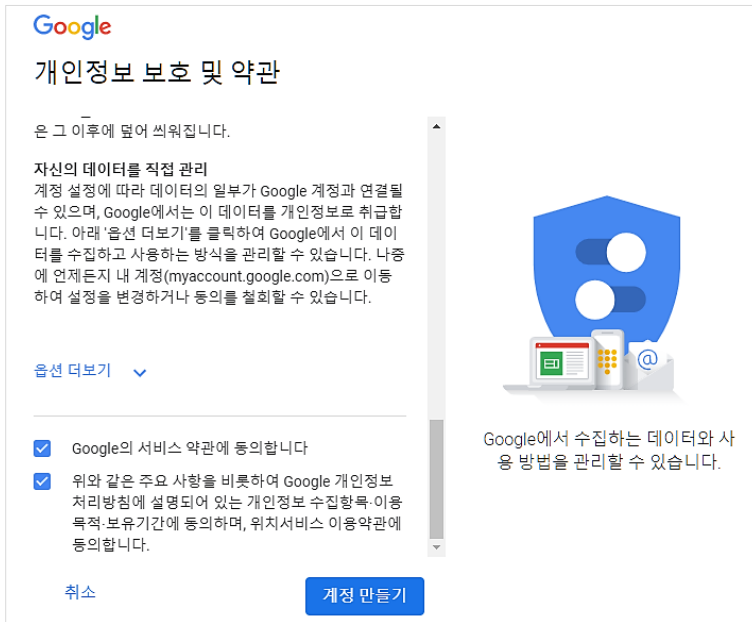


그림 46 개인정보 보호 및 약관 동의

계정 만들기가 완료되면 크롬 브라우저는 새로 만든 계정으로 로그인한 상태입니다. 이제 이 계정에 대한 활동 제어를 설정하기 위해 다음 주소로 이동합니다.

- <https://myaccount.google.com/activitycontrols>

여기서 활동 제어란 현재 계정의 활동 정보를 구글과 공유하는 것으로 이를 통해 개인별 맞춤형 클라우드 서비스가 이뤄지는데, 옛지 디바이스에서 구글 어시스턴트를 사용하려면 “웹 및 앱 활동”과 “기기 정보”, “음성 및 오디오 활동”이 활성화되어야 합니다. “웹 및 앱 활동”은 활성화된 상태이므로 나머지를 활성화합니다.

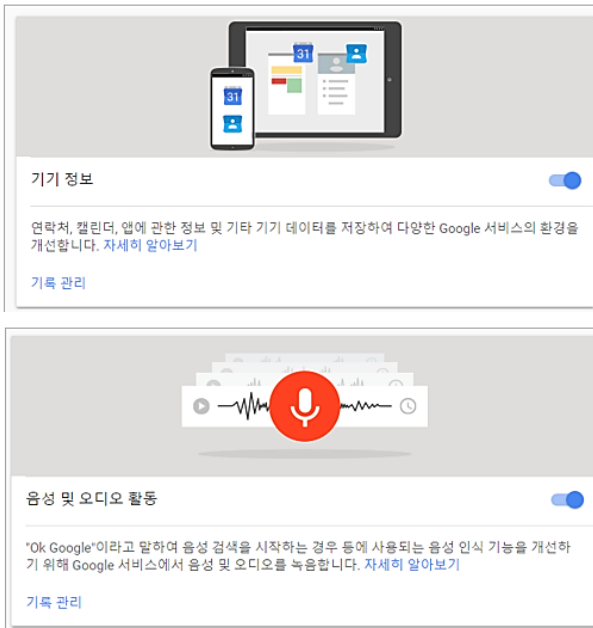


그림 47 프로젝트 계정의 활동 제어 설정

5.4.1.2. Actions on Google 프로젝트 생성

엡지 디바이스를 인증하는 자격 증명서를 다운받으려면 Google Cloud Platform 서비스 중 하나인 Actions on Google 콘솔에서 새 프로젝트를 만든 후 장치 모델을 등록해야 합니다.

Actions on Google 은 구글 어시스턴트의 핵심 서비스 중 하나로 장치마다 탑재된 구글 어시스턴트 앱이나 라이브러리 및 서비스가 요청한 음성 명령을 분석해 지정된 액션 실행하는데, 대부분은 구글이 AI로 학습시킨 음성 답변이나 웹 브라우저 실행이지만 엡지 디바이스 등에서 새로운 액션을 실행하도록 위임하거나 타사 웹 서비스를 실행하도록 새로운 액션을 정의하는 것도 가능합니다.

Actions on Google 콘솔에서 새 프로젝트를 만들기 위해 다음 주소로 이

동합니다.

- <https://console.actions.google.com/?pli=1>

기존 프로젝트가 있으면 목록이 표시되지만 처음에는 비어 있습니다.
“New Project”를 선택해 새 프로젝트 만들기를 시작합니다.

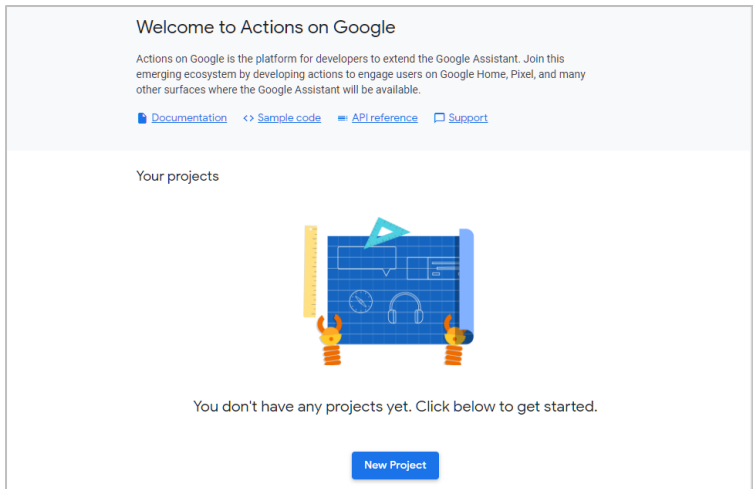


그림 48 Action on Google 프로젝트 생성

서비스 약관에 동의한 후 “Agree and continue”를 선택합니다. 업데이트 및 팁 받거나 설문조사, 파일럿 프로그램 참여는 옵션입니다.

Actions on Google

Terms of Service

I agree to adhere to the [Actions on Google policies](#) and [Terms of Service](#). I understand that creating projects in Actions on Google Console will also allow me to use Firebase services, and I agree that my use of Firebase services will adhere to the applicable [Firebase Terms of Service](#).

☒ Yes ☐ No

Country of residence

South Korea

Send me updates and tips to help me get the most out of Actions on Google.

☐ Yes ☐ No

Invite me to participate in surveys and pilots to improve Actions on Google.

☐ Yes ☐ No

Agree and continue

그림 49 서비스 약관 동의

프로젝트 이름을 입력한 후 언어는 “Korean”, 나라는 “South Korea”를 선택합니다. “Create project”를 선택하면 프로젝트가 만들어지는데 시간이 약간 걸릴 수 있습니다.

New Project

Project Name

soda

Choose a language for your Actions project (you can add more Korean languages later)

Choose your country or region South Korea

Cancel Create project

그림 50 새 프로젝트 생성

5.4.1.3. Actions on Google 장치 모델 등록

Action on Google 콘솔에서 새 프로젝트 만들기가 완료되면 장치 모델 등록 화면이 표시되는데, 옛지 디바이스를 위한 간단한 정보를 입력하고 이를 토대로 자격 증명서를 발급받으며 선택적으로 간단한 액션을 지정하는 절차입니다. 이때 발급받은 자격 증명서로 옛지 디바이스를 인증해야 Google Assistant API 를 사용할 수 있으므로 이 절차는 매우 중요합니다. 장치 모델을 등록하기 위해 스크롤바를 가장 아래로 내린 후 More options 에서 “Device registration”을 선택합니다.

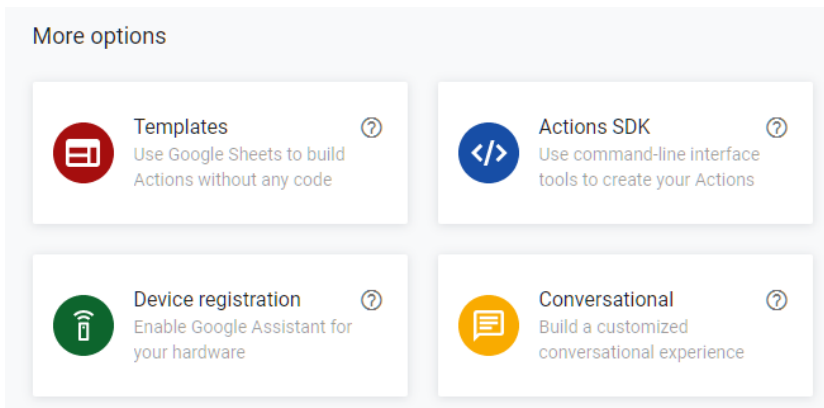


그림 51 장치 등록

옛지 디바이스 모델 등록 화면이 표시되면 “REGISTER MODEL”을 선택해 등록을 시작합니다.

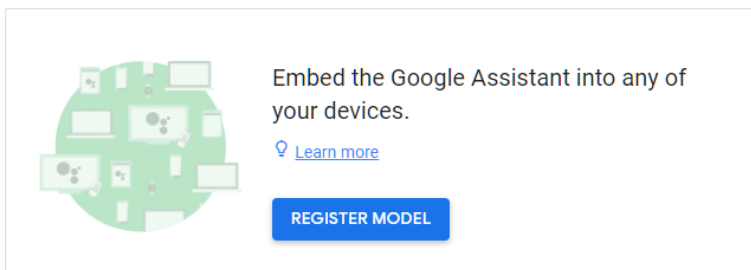


그림 52 모델 등록 시작

장치 등록은 3 단계로 나뉘지는데 첫 번째 단계에서 제품명과 제조사를 입력한 후 목록에서 장치 종류를 선택합니다.

Register model ✕

1 2 3
Create model Download credentials Specify traits

Product name ⓘ
pop

Manufacturer name ⓘ
soda

Device type ⓘ
Camera ▼

Model id ⓘ
soda-fde1c-pop-mo7mwj ✎

Cancel REGISTER MODEL

그림 53 모델 생성

가장 중요한 모델 ID는 제품명과 제조사를 조합해 자동으로 만들어지는데, 구글 어시스턴트 응용프로그램을 처음 실행할 때 프로젝트 ID (또는 디바이스 ID)와 함께 인자로 전달해야 하므로 따로 적어 둡니다. 프로젝트 ID (또는 디바이스 ID)는 제품명 앞의 2 개 필드입니다.

“REGISTER MODEL”을 선택해 두 번째 단계로 이동한 후 “Download OAuth 2.0 credentials”을 선택해 자격 증명서를 다운받습니다. 이 증명서는 나중에 구글 어시스턴트를 사용할 옛지 디바이스를 인증하는데 필요 하므로 잘 보관해 둡니다.

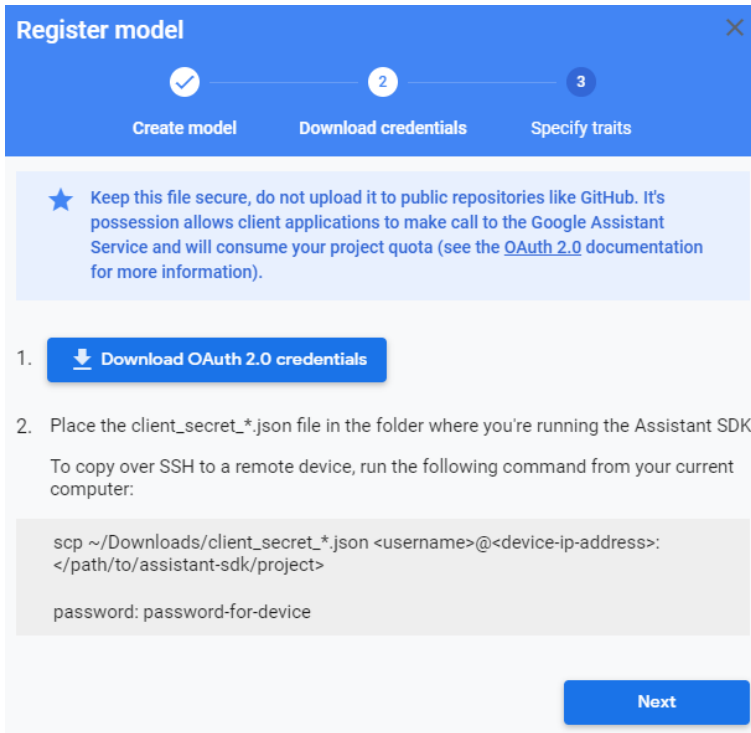


그림 54 자격 증명서 다운로드

“Next”를 눌러 마지막 단계로 이동하면 엣지 디바이스를 위한 몇 가지 서비스 특성을 선택할 수 있습니다. 선택한 서비스 특성에 따라 Actions on Google에는 특정 음성 명령에 대한 액션이 추가되는데, 사용자가 직접 정의할 수 있으므로 지금은 선택하지 않고 “SKIP”를 선택해 모델 등록을 완료합니다.

Register model

1 Create model 2 Download credentials 3 Specify traits

Search All traits ▾

☐ All 7 traits

☐ Brightness
This trait covers how to control the brightness of a device. Absolute brightness setting is in a normalized range from 0 to 100 (individual lights may not support every point in the range based on their LED configuration).
[View details](#)

☐ ColorSpectrum
This trait belongs to any device that is able to set a color spectrum. This applies to full...

SKIP **SAVE TRAITS**

그림 55 특성 지정

장치 모델 등록이 완료되면 목록이 표시되는데, 이곳에서 모델을 제거하거나 해당 모델의 자격 증명서를 다시 다운받거나 특성과 같은 설정을 변경할 수 있습니다.

Device registration

REGISTER MODEL

Product name	Manufacturer name	Model ID	Device Type	Last updated time
pop	soda	soda-fde1c-pop-m...	action.devices.typ...	Aug 08, 2019, 10:22...

그림 56 장치 모델 목록

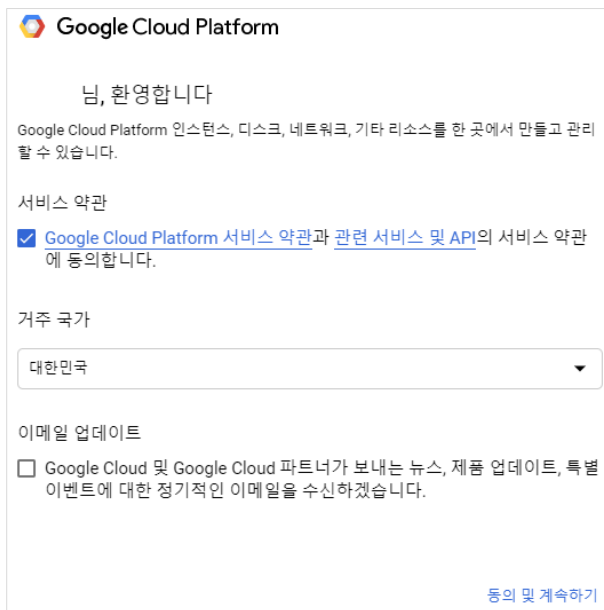
5.4.1.4. Google Assistant API 사용 설정

Actions on Google 콘솔에서 만든 새 프로젝트가 Google Assistant API를 사용하도록 설정하기 위해 다음 주소의 Google Cloud Platform 콘솔

로 이동합니다.

- <https://console.developers.google.com/apis/library/embeddedassistant.googleapis.com>

이때 크롬 브라우저는 프로젝트 계정 로그인을 유지하고 있어야 합니다. 처음 접속하면 Google Cloud Platform 에 대한 서비스 약과 동의 화면이 표시될 수 있습니다. “동의 및 계속하기”를 선택합니다.



The image shows a Google Cloud Platform (GCP) Terms of Service Agreement screen. At the top, it says "Google Cloud Platform" with the logo. Below that, it says "님, 환영합니다" (Hello, [Name]). The main text describes the GCP services and states that the user is agreeing to the terms. There is a checkbox labeled "Google Cloud Platform 서비스 약관과 관련 서비스 및 API의 서비스 약관에 동의합니다." (I agree to the Google Cloud Platform service terms and the service terms of related services and APIs). The checkbox is checked. Below this, there is a dropdown menu for "거주 국가" (Country of residence) with "대한민국" (South Korea) selected. At the bottom, there is a checkbox for "이메일 업데이트" (Update email) with the text "Google Cloud 및 Google Cloud 파트너가 보내는 뉴스, 제품 업데이트, 특별 이벤트에 대한 정기적인 이메일을 수신하겠습니다." (I will receive regular emails from Google Cloud and Google Cloud partners about news, product updates, and special events). The checkbox is unchecked. At the bottom right, there is a button labeled "동의 및 계속하기" (Agree and continue).

그림 57 Google Cloud Platform 약관 동의

상단 메뉴바에서 “프로젝트 선택”을 눌러 Actions on Google 에서 만든 프로젝트를 선택한 후 “열기”를 선택합니다. 만약 목록에 자신이 만든 프로젝트가 표시되지 않으면 업데이트 시간이 필요하므로 기다렸다가 다시 시도합니다.

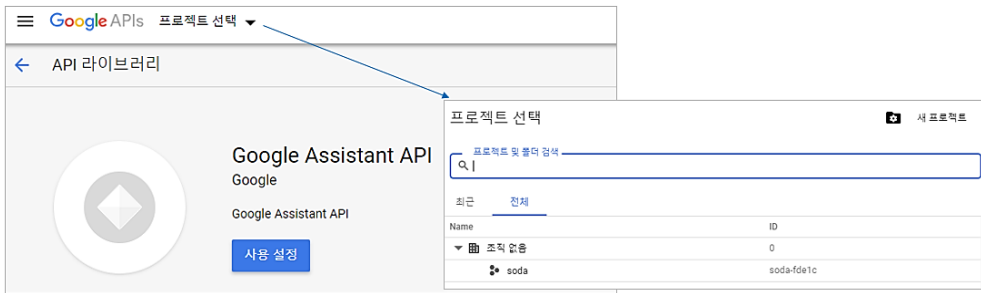


그림 58 Actions on Google 에서 만든 프로젝트 선택

Google Assistant API “사용 설정”을 눌러 Actions on Google 에서 만든 프로젝트가 Google Assistant API 를 사용할 수 있도록 허용합니다.

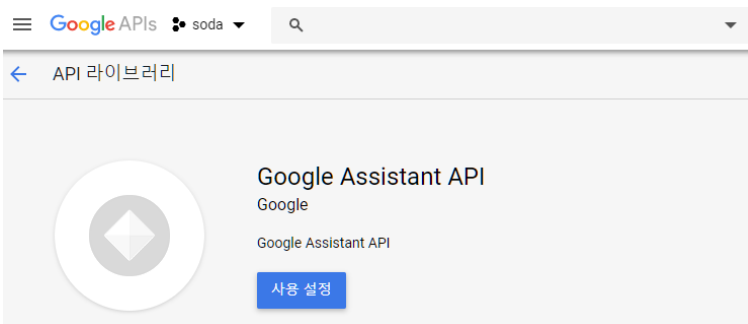


그림 59 Google Assistant API 사용 허용

OAuth 는 사용자 정보를 제 3 자에게 제공하지 않고 인증 또는 인가를 허용하는 개방형 표준 프로토콜로, OAuth Open Authorization, Open Authentication 동의 화면을 구성하기 위해 왼쪽 메뉴바에서 “사용자 인증 정보”를 선택합니다. 처음에는 저장한 동의 화면이 없으므로 “동의 화면 구성”을 선택합니다.

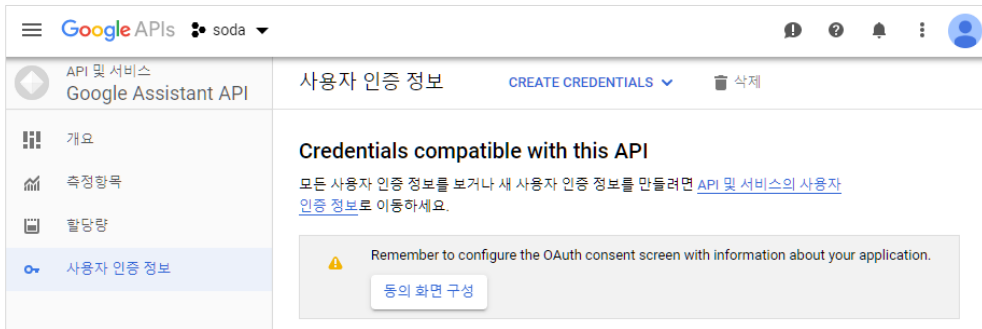


그림 60 사용자 인증 정보 설정

가운데 “OAuth 동의 화면”이 표시되면 이를 선택한 후 지원 이메일에서 자신의 이메일을 선택한 다음 아래로 이동해 “저장”을 선택합니다.

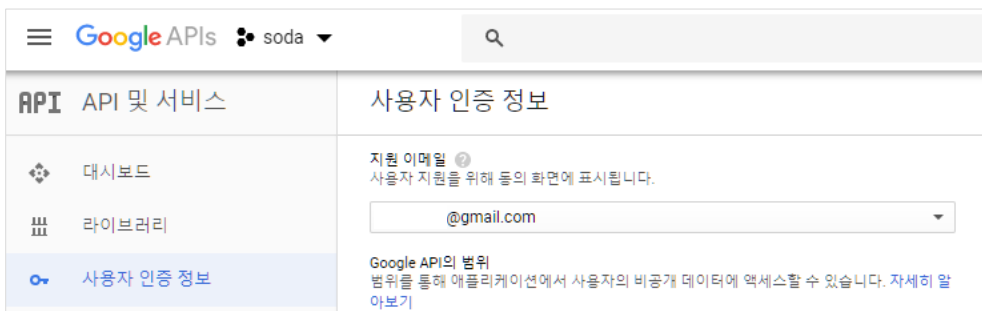


그림 61 지원 이메일 선택

OAuth 인증은 API를 제공하는 서버에서 진행하고, 사용자는 발급받은 액세스 토큰으로 응용프로그램을 실행함으로써 세션이나 쿠키로 클라이언트의 정보를 유지할 필요 없이 서비스 제공자의 API를 안전하게 사용합니다.

대시보드에 표시되는 트래픽 관련 정보는 Google Cloud Platform 전체 정보입니다.

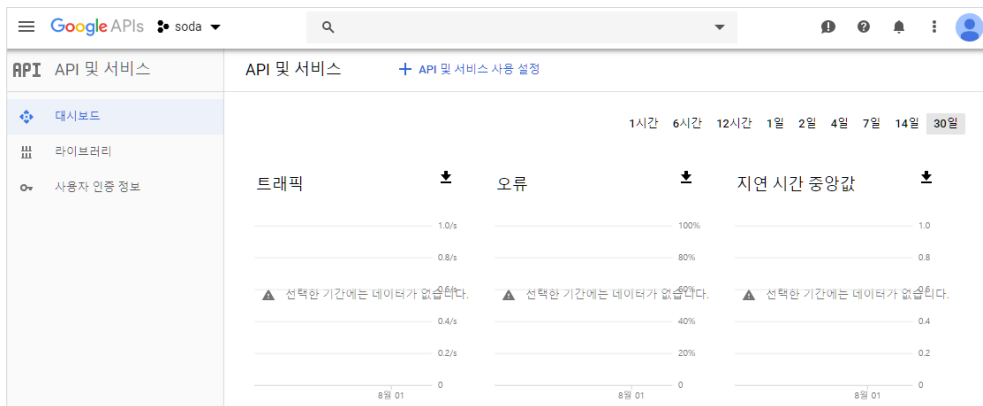


그림 62 Google Cloud Platform 트래픽 정보

Google Assistant API 만 보려면 아래 필터 목록에서 Google Assistant API 를 선택합니다.

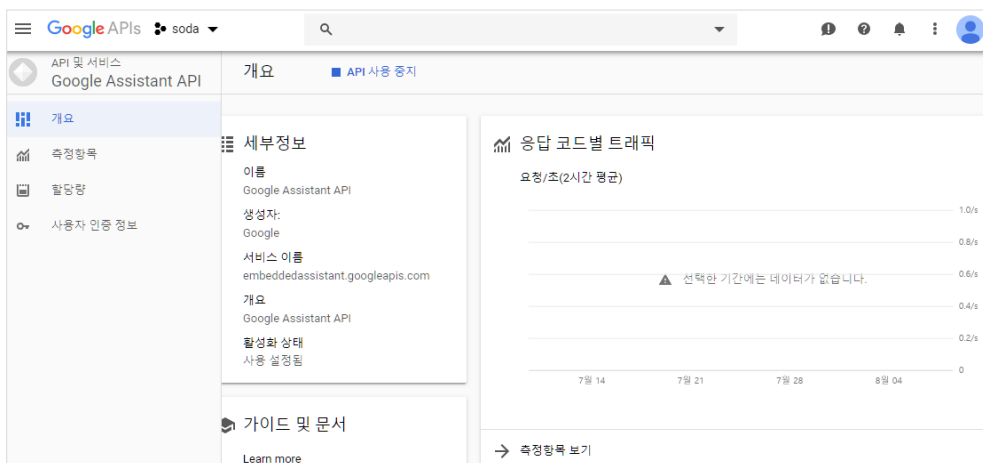


그림 63 Google Assistant API 트래픽 정보

5.4.1.5. 엡지 디바이스 자격 증명

AIoT AutoCar 를 네트워크 드라이버로 연결한 후 Actions on Google 콘솔에서 장치 모델 등록을 통해 다운받은 자격 증명서를 AIoT AutoCar 에 복사합니다.

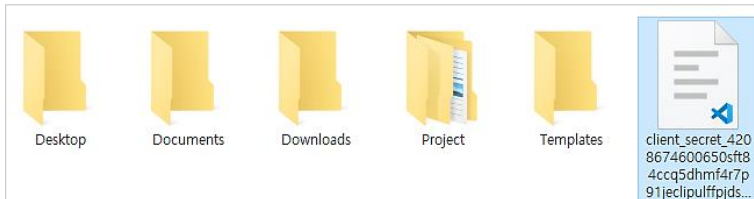


그림 64 엡지 디바이스에 자격 증명서 복사

셸에서 다음 명령으로 Soda OS 에 포함된 기존 자격 증명을 제거합니다.

- `rm -rf ~/.config/google*`

복사한 자격 증명서 경로에서 새 자격 증명을 등록하기 위해 다음 명령을 실행합니다. <json_file> 은 “client_secret_”으로 시작하는 자격 증명서 파일 이름입니다.

- `google-oauthlib-tool --scope https://www.googleapis.com/auth/assistant-sdk-prototype --save --headless --client-secrets <json_file>`

명령을 실행하면 OAuth 기반으로 응용프로그램을 인증하는데 필요한 코드를 받을 수 있는 URL 이 출력되고 마지막 줄에 인증 코드를 입력하라는 메시지가 출력됩니다. 이 URL 은 명령을 실행할 때마다 바뀌며 반드시 해당 URL 을 통해 생성한 인증 코드 전체를 입력해야 InvalidGrantError 가 발생하지 않습니다.

```
google-oauthlib-tool --scope https://www.googleapis.com/auth/assistant-sdk-prototype
--save --headless --client-secrets client_secret_4208674600650sft84ccq5dhmf4r7p91j
pulff.json
Please visit this URL to authorize this applications: https://accounts.google.com/o/oauth
uth84ccq5dhmf4r4p92jecIpulffpjdS.apps.googleusercontent.com\&redirect_uri=urn%3Aietf%3Aw
oauth$.com%2Fault%2Fassistant-sdk-prototype\&state\=L3uRbuAjMpFiMsZgmZdupPVNiuF30U\&prom
consent\&acceLcedPPiP5EcPHdJikk0dfd_GHo\&code-hallenge_method\=S256
Enter the authorization code:
```

그림 65 엣지 디바이스 OAuth 인증

셸 작업을 잠시 멈추고, 해당 URL 을 크롬 브라우저로 복사해 이동하면 OAuth 인증을 위한 계정 선택 화면이 표시됩니다. Actions on Google 콘솔에서 새 프로젝트를 만들 때 사용한 계정을 선택합니다.

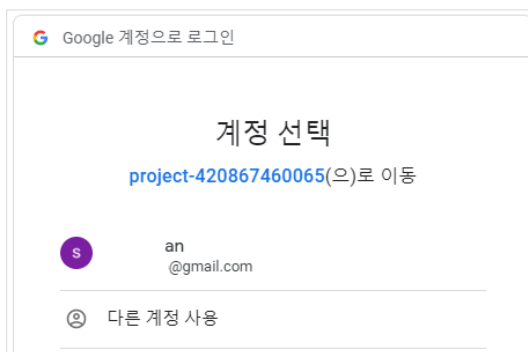


그림 66 OAuth 인증 계정 선택

구글 어시스턴트 사용에 관한 권한 부여 선택 창이 표시되면 모두 “허용”을 선택합니다.

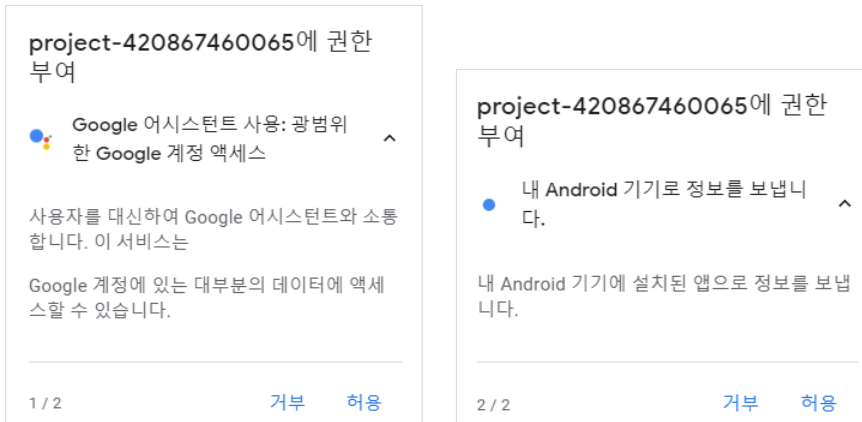


그림 67 구글 어시스턴트 사용 권한 허용

최종적으로 부여한 권한 확인 화면이 표시되면 “허용”을 선택합니다. 이후 “4/”로 시작하는 인증 코드가 표시되는데 이를 복사합니다.

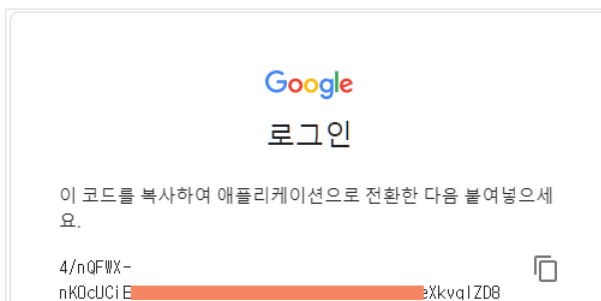


그림 68 인증 코드 확인

멈췄던 셸 작업으로 돌아가 복사한 인증코드를 마지막 줄에 붙여넣고 <Enter>를 눌러 인증을 완료합니다. 인증에 성공하면 ~/.config/google-oauthlib-tool/ 경로에 Google Assistant API 를 호출하는데 필요한 액세스 토큰이 저장됩니다.

```
google-oauthlib-tool --scope https://www.googleapis.com/auth/assistant-sdk-prototype \
--save --headless --client-secrets client_secret_4208674600650sft84ccq5dhmf4r7p91jecL
pulff.json
Please visit this URL to authorize this applications: https://accounts.google.com/o/oauth2/
uth84ccq5dhmf4r4p92jeclpulffpjd.s.apps.googleusercontent.com\&redirect_uri=urn%3Aietf%3Awg%3A
oauth$.com%2Fault%2Fassistant-sdk-prototype\&state\=L3uRbuAjMpFiMsZgmZdupPVNiuF30U\&prompt\=
consent\&accelcdPPiP5EcPHdJikk0dfd_GHo\&code-hallenge_method\=S256
Enter the authorization code: 4/nQG-i4mLzkHKVAhFsvZkFPi4NGK8I11PAIh2JjwoBvzZlnFotW90tEI
credentials saved: /home/soda/.config/google-oauthlib-tool/credentials.json
```

그림 69 엡지 디바이스 OAuth 인증 완료

5.4.1.6. 인스턴스 ID 생성

Google Assistant API 를 사용하는 응용프로그램은 디바이스 모델과 연관된 인스턴스를 통해 실행됩니다. 인스턴스는 모델 ID 와 프로젝트 ID(또는 디바이스 ID)로 응용프로그램과 디바이스 모델을 연관시키며 재사용을 위해 인스턴스 ID 를 로컬에 저장합니다. 따라서 같은 디바이스 모델을 사용할 때는 인자를 생략할 수 있습니다. 하지만 인스턴스 ID 가 저장되지 않은 상태에서 인자 없이 응용프로그램을 실행하면 오류 메시지를 출력하고 프로그램은 종료합니다.

```
googlesamples-assistant-hotword
Traceback (most recent call last):
  File "/usr/local/bin/googlesamples-assistant-hotword", line 10, in <module>
    sys.exit(main())
  File "/usr/local/lib/python3.7/site-packages/googlesamples/assistant/library/hotword.py", line 119, in main
    raise Exception('Missing --device-model-id option')
Exception: Missing --device-model-id option_
```

그림 70 모델 ID 오류

Soda OS 는 인스턴스 ID 를 제공하므로 인자없이 응용프로그램을 실행할 수 있지만 사용자가 자신의 계정으로 구글 어시스턴스 사용 인증을 새로 했다면 최초 한 번은 프로젝트 ID 와 모델 ID 를 인자로 전달해야 이후 인자를 생략할 수 있습니다.

gRPC 서비스 기반의 googlesamples-assistant-pushtotal 는 다음과 실행해 같이 인스턴스 ID 를 생성합니다. <Enter> 키를 눌러 음성 명령을 시작하며 기본 언어는 영어입니다.

- `googlesamples-assistant-pushtotal --project-id <project_id> --device-model-id <model_id>`

5.4.2. 구글 어시스턴트 SDK 설치

구글 어시스턴트 SDK 는 라이브러리와 SDK 예제, gRPC Google Remote Procedure Call, OAuth 인증 토큰으로 이뤄져 있으면 Soda OS 에는 미리 설치되어 있습니다.

구글 어시스턴트 라이브러리(libassistant_embedder.so)는 몇 가지 오픈소스 소프트웨어를 사용하는데, PortAudio 는 오디오 입출력에 사용되고 libffi 는 파이썬과 라이브러리 사이 호출 인터페이스는 맞춰줍니다. libssl 은 Google Cloud Platform 과 엣지 디바이스 사이 암호화된 통신 채널을 만들어주고 libmpg123 은 MP3 포맷의 데이터 압축과 해제를 수행합니다.

어시스턴트 라이브러리에서 사용하는 오픈소스 소프트웨어를 설치하는 명령은 다음과 같습니다.

- `sudo apt install portaudio19-dev libffi-dev libssl-dev libmpg123-dev`

구글 어시스턴트 라이브러리를 설치하는 명령은 다음과 같습니다.

- `sudo pip3 install --upgrade google-assistant-library`

구글 어시스턴트 서비스는 높은 성능의 오픈소스 범용 RPC 프레임워크인 gRPC 를 기반으로 동작합니다. 구글은 오래전부터 마이크로 서비스 아키텍처를 연구해왔고 내부 서비스들을 연결하기 위해서 Stubby 를 만들었습니다. 이후 Stubby 를 오픈소스로 변경하면서 구글 내부, 외부 서비스뿐만 아니라 모바일, IoT 등 다양한 엔드포인트 endpoint 에서 활용하기 위해서 gRPC 를 만들었습니다.

구글 어시스턴트 예제와 OAuth 인증 톨을 설치할 때 일반 사용자 계정에서 sudo 명령을 함께 사용하면 실패하므로 “sudo su” 명령으로 루트 계정용 셸을 만들어 진행합니다.

구글 어시스턴트 예제와 gRPC, OAuth 인증 톨을 설치하는 명령은 다음과 같습니다.

- sudo su
- pip3 install --upgrade google-assistant-sdk[samples]
- pip3 install --upgrade google-assistant-grpc
- pip3 install --upgrade google-auth-oauthlib[tool]
- exit

구글 어시스턴트 예제는 파이썬 확장 라이브러리 설치 경로 아래 두개의 폴더에 위치하며, 자신의 응용프로그램에 Google Assistant API 를 적용할 때는 이들을 분석하면 됩니다.

- grpc 폴더: gRPC 서비스 기반 Google Assistant 클라이언트
 - pushtotalk.py: 키 입력이 있을 때마다 Google Assistant 실행
 - 한국어를 포함한 다국어 지원
 - /usr/local/bin/googlesamples-assistant-pushtotalk 에서 호출
 - textinput.py: 텍스트 기반으로 Google Assistant 실행
 - /usr/local/bin/googlesamples-assistant-textinput 에서 호출

- library 폴더: 라이브러리 기반 Google Assistant 클라이언트
 - hotword.py: 'Ok google' 핫워드로 구글 어시스턴트 실행
 - 한국어를 포함한 다국어 지원
 - /usr/local/bin/googlesamples-assistant-hotword 에서 호출

5.4.3. 구글 어시스턴트 SDK 예제

구글 어시스턴트 SDK 예제는 Google Assistant API 의 사용법을 보여주는 응용프로그램들로 라이브러리와 gRPC 서비스 버전이 있습니다.

5.4.3.1. 서비스 기반 응용프로그램

인자없이 google-assistant-pushtotalk 를 실행하면 로컬에 저장된 모델 ID 와 프로젝트 ID 를 찾아 인스턴스를 만든 후 디폴트 마이크와 스피커를 초기화하고 키 입력이 있을 때까지 대기합니다. 키가 입력되면 나머지 절차는 google-assistant-pushtotalk 와 같으며 스피커로 결과가 출력한 뒤에는 다시 키 입력 대기 상태가 됩니다.

```
googlesamples-assistant-pushtotalk
INFO:root:Connectiong to embeddedassistant.googleapis.com
INFO:root:Using device model soda-fde1c-pop-mo7mwj and device id 69fdfaa2-b9ab-11e9-9ae9-
Press Enter to send a new request...
INFO:root:Recording audio request.
INFO:root:Transcript to user request: "오늘".
INFO:root:Transcript to user request: "오늘 나".
INFO:root:Transcript to user request: "오늘 날".
INFO:root:Transcript to user request: "오늘 날씨".
INFO:root:Transcript to user request: "오늘 날씨 알".
INFO:root:Transcript to user request: "오늘 날씨 알려".
INFO:root:Transcript to user request: "오늘 날씨 알려 줘".
INFO:root:Transcript to user request: "오늘 날씨 알려 주세요".
INFO:root:Transcript to user request: "오늘 날씨 알려 주세요".
INFO:root:Transcript to user request: "오늘 날씨 알려 주세요".
INFO:root:End of audio request detected.
INFO:root:Stopping recording.
INFO:root:Transcript to user request: "오늘 날씨 알려 주세요".
INFO:root:Transcript to user request: "오늘 날씨 알려 주세요".
INFO:root:Playing assistant response.
INFO:root:Finished playing assistant response.
Press Enter to send a new request...
```

그림 71 googlesamples-assistant-pushtotalk

5.4.4. Google Assistant API 활용

응용프로그램에서 Google Assistant API 를 활용하려면 구글이 제공하는 응용프로그램 소스를 분석해야 하지만 난이도가 높은 편이어서 상당한 기반 지식이 있어야 합니다.

5.4.4.1. Pop 라이브러리

Pop 라이브러리 중 popAssist 모듈은 인터넷에 연결된 옛지 디바이스에서 사용자가 좀 더 쉽게 Google Assistant API 를 사용할 수 있도록 고수준으로 구현한 create_conversation_stream() 메소드와 create_device_handler() 메소드 및 GAssistant 클래스를 제공합니다.

create_conversation_stream()은 타킷의 마이크와 스피커를 제어하는 ConversationStream 객체를 생성하는데 내부에서는 PortAudio 라이브러리의 파이썬 바인더인 sounddevice 를 사용합니다. ConversationStream 객체는 오디오 소스로 마이크, 오디오 싱크로 스피커 사용하며 응답으로

받은 오디오 데이터는 volume_percentage 프로퍼티에 의해 스케일이 조정된 상태로 출력됩니다.

ConversationStream 객체를 반환하는 create_conversation_stream() 메소드의 인자 의미는 다음과 같으며, 주로 인자 없이(기본값 사용) 호출합니다.

- create_conversation_stream(audio_sample_rate=16000, audio_sample_width=2, audio_block_size=6400, audio_flush_size=25600, audio_iter_size=3200): ConversationStream 객체를 만들어 반환
 - audio_sample_rate: 샘플 비율. 기본값은 16000
 - audio_sample_width: 바이트 단위의 각 샘플 크기. 기본값인 2 이상 허용
 - audio_block_size: 바이트 단위의 각 읽기 및 쓰기 작업의 크기. 기본값은 6400
 - audio_flush_size: 플러시 동작 때 만들어진 바이트 단위 묶음 데이터 크기. 기본값은 25600
 - audio_iter_size: 각 반복마다 읽을 데이터 크기 (바이트 단위). 기본값은 3200

create_device_handler() 메소드는 DeviceRequestHandler 객체를 반환하는데, 이를 이용해 Actions on Google 에서 정의한 사용자 액션을 처리합니다.

다음은 GAssistant 클래스에서 제공하는 기능으로 주로 생성자와 Assistant 를 실행하는 assist() 메소드 사용합니다. 이때 이 둘에 공통적으로 적용되는 conversation_stream 인자는 서로 배타적이므로 한 쪽에만 ConversationStream 객체를 전달합니다.

- GAssistant(conversation_stream=None, local_device_handler=None, google_device_handler=None, lang="ko-KR", display=False): GAssistant 객체 생성
 - conversation_stream: 음성을 인식하고 결과를 출력할 ConversationStream 객체.
 - None(기본값)일 때는 assist() 메소드의 인자에 전달해야 함
 - local_device_handler: 로컬 사용자 액션 핸들러. 기본값은 None
 - google_device_handler: Actions on Google 서비스 연계 액션 핸들러 객체. 기본값은 None

- lang: 사용 언어. "ko-KR" (한국어), "en-US" (미국식 영어) 등. 기본값은 "ko-KR"
 - <https://developers.google.com/actions/localization/languages-locales> 참조
- display: 결과를 화면에 출력할 때 사용하는 웹 브라우저 객체. 기본값은 False
- conversation_stream_close(): ConversationStream 객체의 오디오 접근을 닫음
 - 주로 ConversationStream 객체의 Close() 메소드 직접 호출
- get_device_id(): DeviceRequestHandler 객체를 만들 때 필요한 device_id 반환
- get_device_handler(): 현재 GAssistant 객체에 설정된 DeviceRequestHandler 객체 반환
- assist(rec_request_handler=None, resp_handler=None, conversation_stream=None): Assistant 실행
 - rec_request_handler: 음성 인식을 시작할 때 호출되는 사용자 메소드. 기본값은 None
 - resp_handler: 응답을 출력한 후 호출되는 사용자 메소드. 기본값은 None
 - conversation_stream: ConversationStream 객체
 - None(기본값)일 때는 생성자의 인자로 전달해야 함

ConversationStream 객체는 GAssistant 객체 내부에서 오디오 접근을 위해 사용하지만 몇 가지 기능은 사용자가 호출할 수 있습니다.

- volume_percentage: 볼륨 조절 프로퍼티
 - volume_percentage = <n>: 현재 볼륨 설정. 1 ~ 100
 - volume = volume_percentage: 현재 볼륨 반환
- close(): 오디오 자원 해제

5.4.4.2. 구글 어시스턴트 실행

PopAssist의 GAssistant 클래스를 이용해 구글 어시스턴트를 일화성으로 실행해 봅니다. GAssistant 객체를 만들 때 필요한

ConversationStream 객체는 create_conversation_stream() 메소드로 만듭니다. 구글 클라우드 플랫폼을 사용하므로 옛지 디바이스는 반드시 인

터넷에 연결되어 있어야 합니다.

```
01: from popAssist import *
02:
03: stream = create_conversation_stream()
04: ga = GAssistant(stream)
05:
06: print("Taking about...")
07: ga.assist()
08:
09: print("Bye...")
10: stream.close()
```

프로그램을 실행한 후 “Taking about...” 메시지가 출력되면 “오늘 날씨가 어때?”와 같은 음성 명령을 입력하면 Google Cloud Platform 에서 가져온 결과를 출력한 후 종료합니다. 10 초 동안 소리 입력이 없어도 종료합니다.

```
soda@soda:~ $ python3 code111.py
```

```
Taking about...
```

```
Bye...
```

음성 명령 처리

응답에 데이터의 스케일을 늘리거나 줄이면 스피커 볼륨 조 효과를 볼 수 있습니다. 스케일 조정은 ConversationStream 객체의 volume_percentage 프로퍼티를 이용하며 값의 범위는 1 ~ 100 입니다.

```
01: from popAssist import *
02:
03: stream = create_conversation_stream()
04: ga = GAssistant(stream)
05:
06: print("Current volume: %d"%(stream.volume_percentage))
07: stream.volume_percentage = 100 #1 ~ 100 사잇값 사용
08:
```

```
09: print("Taking about...")
10: ga.assist()
11:
12: print("Bye...")
13: stream.close()
```

응답 스케일은 ConversationStream 객체가 만들어질 때 50 으로 설정됩니다.

```
soda@soda:~ $ python3 code112.py
Current volume: 50
Taking about...
Bye...
```

assist() 메소드의 rec_request_handler 와 resp_handler 인자에는 사용자 메소드를 전달하면 음성 인식이 시작되거나 응답이 완료될 때 해당 메소드를 호출해 줍니다.

```
01: from popAssist import *
02:
03: stream = create_conversation_stream()
04: ga = GAssistant(stream)
05:
06: def onStart(): # 음성 인식을 시작할 때 호출
07:     print(">>> start recording....")
08:
09: def onStop(): # 응답의 출력이 완료되면 호출
10:     print(">>> stop response...")
11:
12: ga.assist(onStart, onStop)
13: stream.close()
```

프로그램을 실행한 후 음성 인식을 시작할 수 있을 때 onStart() 메소드가 호출되고 응답이 완료되면 onStop() 메소드가 호출됩니다.

```
soda@soda:~ $ python3 code113.py
>>> start recording...
>>> stop response...
```

응답이 완료될 때 호출되는 사용자 메소드에서 다시 assist() 메소드를 호출하면 계속해서 구글 어시스턴트를 실행할 수 있습니다. 응답은 내부 스레드에서 처리하므로 assist() 메소드가 실행될 때 프로그램이 종료하지 않도록 주의합니다.

```
01: from popAssist import *
02: import time
03:
04: stream = create_conversation_stream()
05: ga = GAssistant(stream)
06:
07: try:
08:     def onStart():
09:         print(">>> start recording....")
10:
11:     def onStop():
12:         ga.assist(onStart, onStop)    #응답의 출력이 완료되면 다시
13:         assist() 호출
14:
15:     ga.assist(onStart, onStop)
16:     while True:
17:         print("main work...")
18:         time.sleep(2)
19: except KeyboardInterrupt:
20:     stream.close()
```

프로그램을 실행한 후 시작 메시지가 출력되면 음성 인식이 가능한 상태입니다. 내부 스레드를 통해 응답을 처리하므로 묵음이 10 초 이상 지속되어 음성 인식을 다시 시작하거나 응답을 출력하는 중에도 while 루프는 계속 실행됩니다.

```
soda@soda:~ $ python3 code114.py
>>> start recording...
main work...
>>> start recording...
main work...
main work...
main work...
```

5.4.4.3. 사용자 장치 액션

사용자 장치 액션은 음성 인식 결과를 옛지 디바이스의 특정 명령에 반영하는 행위로 사용자 계정과 연계된 Actions on Google 서비스를 사용하여 하므로 반드시 “구글 어시스턴트 사용 인증”을 수행한 후 진행해야 합니다.

Actions on Google 에서 관리하는 액션은 특성과 사용자 장치 액션으로 나뉘는데, 특성은 밝기를 제어하는 Brightness 나 장치를 켜고 끄는 OnOff 와 같이 구글에서 미리 지정한 액션이고, Actions SDK 와 연계되는 사용자 장치 액션은 사용자가 추가한 액션입니다.

옛지 디바이스가 액션 패키지를 정의한 후 Actions on Google 에 등록하면 Google Assistant 서버는 음성 인식 결과를 액션 패키지와 비교해 일치하는 명령을 옛지 디바이스에 반환하고 옛지 디바이스는 그에 대응하는 사용자 코드를 실행합니다.

JSON 포맷으로 정의한 액션 패키지에 포함되는 내용은 다음과 같습니다.

- 사용자 질의와 일치시키려는 패턴

- 일치하는 질의와 연결할 사용자 장치 액션
- 장치가 액션을 지원하면 사용자에게 전달할 응답
- 매개 변수와 함께 장치로 전송되는 명령 이름

다음은 확장자가 .json 인 JSON 포맷의 액션 패키지 구조입니다.

```
01: {  
02:   "manifest": {  
03:     object(Manifest)  
04:   },  
05:   "accountLinking": {  
06:     object(AccountLinking)  
07:   },  
08:   "actions": [  
09:     {  
10:       object(Action)  
11:     }  
12:   ],  
13:   "types": [  
14:     {  
15:       object(Type)  
16:     }  
17:   ],  
18:   "conversations": {  
19:     string: {  
20:       object(ConversationFulfillment)  
21:     },  
22:     ...  
23:   },  
24: }
```

액션 패키지의 항목 의미는 다음과 같습니다.

manifest	object(Manifest) 앱의 세부 정보
accountLinking	object(AccountLinking) 앱의 계정 연결에 대한 세부 정보
actions[]	object(Action) 앱이 처리할 수 있는 작업 목록.
types[]	object(Type) 개발자가 정의한 타입 목록
conversations	map (key: string, value: object(ConversationFulfillment)) 액션 간에 공유할 수 있는 대화 맵 "key": value 쌍 목록으로 구성 예: { "name": "wrench", "mass": "1.3kg", "count": "3" }.

AIoT AutoCar에는 카메라가 포함되어 있으므로 음성 명령으로 사진을 찍어보겠습니다. 먼저 사진을 찍는 액션 패키지를 다음과 같이 picture.json 파일로 정의합니다.

```

01: {
02:   "manifest": {
03:     "displayName": "Take Picture",
04:     "invocationName": "Take Picture",
05:     "category": "PRODUCTIVITY"
06:   },
07:   "locale": "ko",
08:   "actions": [
09:     {
10:       "name": "TaskPicture",
11:       "availability": {
12:         "deviceClasses": [ { "assistantSdkDevice": {} } ]
13:       },
14:       "intent": {
15:         "name": "com.example.intents.TakePicture",
16:         "trigger": { "queryPatterns": [ "사진 찍어" ] }

```

```

17:     },
18:     "fulfillment": {
19:       "staticFulfillment": {
20:         "templatedResponse": {
21:           "items": [
22:             {
23:               "simpleResponse": { "textToSpeech": "사진을
찍습니다" }
24:             },
25:             {
26:               "deviceExecution": { "command": "TakePicture" }
27:             }
28:           ]
29:         }
30:       }
31:     }
32:   }
33: ]
34: }

```

한국어 음성 인식을 사용하려면 “locale” 값을 “ko”로 지정합니다. “intent”에 포함된 “trigger”의 “queryPatterns” 값은 사진을 찍는 구문 패턴으로 여러 개일 수 있습니다. “deviceExecution”의 “command” 값은 옛지 디바이스로 반환되는 명령 내용입니다.

정의한 액션 패키지는 다음과 같이 gactions 명령으로 Actions on Google 에 등록하는데, gactions 명령으로 액션 패키지 등록을 요청하면 이를 허용하도록 프로젝트 ID 를 함께 전달합니다.

- gactions update --action_package <action.json> --project <project-id>

```
gactions update --action_package picture.json --project soda-fde1c
Gactions needs access to your Google account. Please copy & paste the URL below into a web browser. Then copy and paste the authorization code from the browser back here.
Visit this URL:
https://accounts.google.com/o/oauth2/auth?access_type=offline&client_id=237807841406-06v96e2d0c.apps.googleusercontent.com&redirect_uri=urn%3Aietf%3Aauth%3A2.0%3Aauth%3Aresponse%3A%2F%2Fwww.googleapis.com%2Factions.builder&state=state
Enter authorization code:
```

그림 72 액션 패키지 등록 시작

처음 액션 패키지를 등록하면 “옛지 디바이스 자격 증명” 때와 같이 인증 코드를 얻을 수 있는 URL 이 표시되는데, 작업을 잠시 멈추고 크롬 브라우저에 붙여넣어 이동합니다. 프로젝트 계정을 선택하면 Assistant CLI (명령 라인 인터페이스) 사용을 허용할지 묻는데 “허용”을 선택합니다.

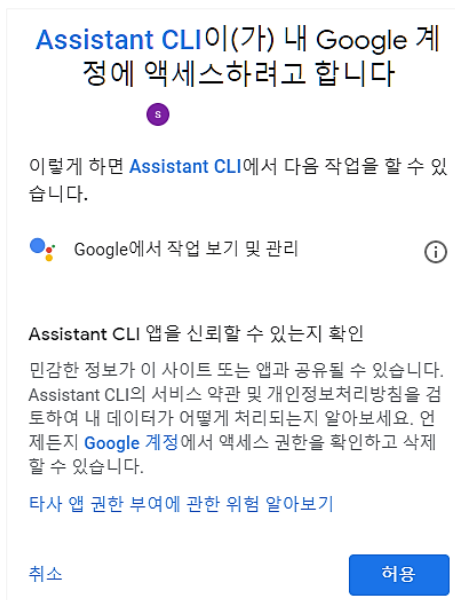


그림 73 Assistant CLI 접근 허용

인증 코드가 표시되면 이를 복사한 후 멈췄던 작업으로 돌아가 “Enter authorization code:” 줄 아래 붙여넣고 <Enter>를 눌러 액션 패키지 등록을 완료합니다. 인증이 완료되면 현재 경로에 creds.data 파일이 만들

여기 다음에 수정된 내용을 반영할 때는 인증 절차가 생략됩니다.

```
gactions update --action_package picture.json --project soda-fde1c
Gactions needs access to your Google account. Please copy & paste the URL below into a web browser.
Visit this URL:
https://accounts.google.com/o/oauth2/auth?access_type=offline&client_id=237807841406-96e2d0c.apps.googleusercontent.com&redirect_uri=urn%3Aietf%3Awww.googleapis.com%3A%2F%2Fwww.googleapis.com%2Factions.builder&state=state
Enter authorization code:
4/nQGlaZ7UE_TiC2eu4jpPYWJJ8XRN84_nov08ydHTeZc2fLnLcXWRrE
Your app for the Assistant for project soda-fde1c was successfully updated with your
on Google console to finish registering your app and submit it for review at https://console.actions.google.com/project/soda-fde1c/overview
```

그림 74 액션 패키지 등록 완료

Actions on Google 에 등록된 액션 목록은 다음 주소를 통해 확인할 수 있습니다.

- <https://console.actions.google.com/project/<project-id>/overview>
 - “Build your Action” > “Add Actions” 선택

Korean			Modify languages
Actions Below is a list of Actions you have built			Test
Name	Fulfillment type	Fulfillment tool	
com.example.intents.TakePicture	Conversational	Actions SDK	>

그림 75 Actions on Google 사용자 장치 액션 목록

목록에서 해당 액션을 선택하면 세부 내용을 확인할 수 있으며 액션 호출 구문이나 언어 변경과 같은 수정도 가능합니다.

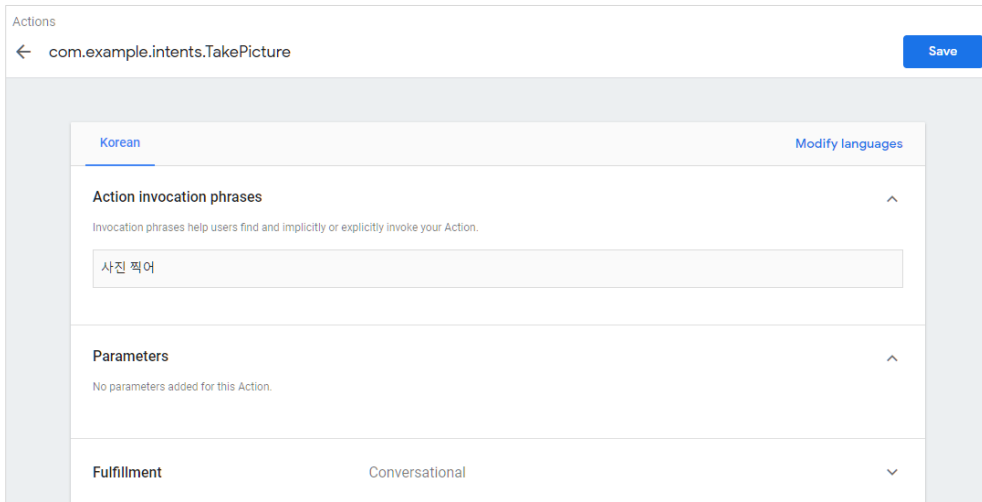


그림 76 사용자 액션 내용

Actions on Google 에 사진을 찍는 액션 패키지를 등록했으면 실제 사진을 찍는 사용자 코드를 구현하는데, 앞서 진행한 카메라 활용에서 사용했던 cv2 모듈의 imwrite() 메소드와 pop 라이브러리의 Camera 객체들을 사용합니다.

Actions on Google 에서 수신한 액션 항목 중 “deviceExecution”의 “command”가 일치할 때 실행할 사용자 메소드를 등록하기 위해 create_device_handler() 메소드로 DeviceRequestHandler 객체를 만든 후 이를 데커레이터로 사용자 메소드를 정의하면 자동으로 DeviceRequestHandler 객체에 추가됩니다. 동일한 방법으로 계속 다른 액션에 대한 사용자 메소드를 정의하면서 추가할 수 있습니다.

```
01: device_handler = create_device_handler()
02: cam = Camera(width=300, height=300)
03:
04: @device_handler.command("TakePicture")
05: def takePicture():
06:     print("Take picture...")
```

```
07: cv2.imwrite("picture.png", cam.value)
```

마지막으로 GAssistant 객체를 만들 때 google_device_handler 인자에 사용자 메소드가 추가된 DeviceRequestHandler 객체를 전달하면 assist() 메소드가 호출될 때마다 내부에서 음성 인식에 대한 액션 응답을 분석해 실행할 사용자 메소드가 있으면 호출해 줍니다.

```
01: from popAssist import *
02: import subprocess
03: import time
04:
05: device_handler = create_device_handler()
06: stream = create_conversation_stream()
07: cam = Camera(width=300, height=300)
08:
09: @device_handler.command("TakePicture")      # deviceExecution 의
      command 가 "TakePicture"일 때
10: def takePicture():
11:     print("Take picture...")
12:     cv2.imwrite("picture.png", cam.value)
13:
14: ga = GAssistant(stream, google_device_handler=device_handler)
15:
16: try:
17:     def onStart():
18:         print(">>> Start recording....")
19:
20:     def onStop():
21:         ga.assist(onStart, onStop)
22:
23:     ga.assist(onStart, onStop)
24:
```

```
25:     while True:
26:         time.sleep(1)
27: except KeyboardInterrupt:
28:     stream.close()
```

프로그램을 실행한 후 “Start recording...” 메시지가 출력되면 음성 인식을 시작할 수 있습니다. “사진 찍어”란 음성이 인식되면 “사진을 찍습니다”란 응답과 함께 AIoT Home에 장착된 카메라로 사진을 찍어 현재 경로에 저장합니다.

```
soda@soda:~ $ python3 code117.py
>>> Start recording...
Take picture...
>>> Start recording...
```

5.4.4.4. GAssistant 기반 사용자 장치 액션

사용자 장치 액션은 옛지 디바이스의 특정 명령이나 동작 수행에 필요한 액션 패턴 분석과 이를 실행하는 사용자 코드를 분리해 주지만 액션 패턴이 지나치게 단순하고 JSON 형식의 액션 패키지를 추가로 만들어 등록하는 부담감을 덜어주기 위해 GAssistant 클래스에 자체적으로 사용자 장치 액션을 처리하는 기능이 추가되었습니다.

기본적으로 Google Assistant 서버는 등록된 액션 패키지에서 처리할 내용이 없으면 음성을 인식한 텍스트와 응답 파일 2가지를 반환합니다. GAssistant 클래스는 반환된 음성 인식 텍스트를 이용해 원하는 명령이나 동작을 수행하는 것으로 GAssistant 객체를 만들 때 local_device_handler 인자에 사용자 메소드를 전달하면 음성을 인식할 때마다 수신한 텍스트를 인자로 사용자 메소드를 호출합니다.

따라서 사용자 메소드는 이 문자열을 분석해 원하는 작업을 수행하면 됩니다. 예를 들어 “모터를 전진해 주세요” 또는 “모터 전진”이란 음성 명령이 있을 때 사용자는 반환된 텍스트에 “모터”와 “전진”이 있는지 비교한 후 모터 제어를 수행하는 식입니다. 이때 사용자 메소드가 True 를 반환하면 수신한 응답 파일을 재생하지 않습니다.

```
01: from popAssist import *
02: import subprocess
03:
04: def userAction(text): # 로컬 사용자 액션 핸들러
05:     print(text)
06:
07:     return True
08:
09: stream = create_conversation_stream()
10: ga = GAssistant(stream, local_device_handler=userAction)
11:
12: try:
13:     def onStart():
14:         print(">>> Start recording....")
15:
16:     while True:
17:         ga.assist(onStart)
18: except KeyboardInterrupt:
19:     stream.close()
```

프로그램을 실행한 한 후 ‘>>> Start recording...’ 메시지가 출력되면 음성 인식을 시작하는데, 인식된 내용은 텍스트로 출력됩니다.

```
soda@soda:~ $ python3 code118.py
```

```
>>> Start recording...
오늘 며칠이야
>>> Start recording...
오늘 무슨 요일이지
>>> Start recording...
```

앞에서 소개한 음성 명령으로 사진 찍기를 GAssistant 기반 사용자 장치 액션으로 바꿔봅니다. 앞서 소개한 사용자 장치 액션은 ‘사진 찍어’에만 반응했지만 로컬 사용자 장치 액션은 ‘사진’과 ‘찍어’가 순서대로만 나오면 ‘사진을 찍어 주세요.’, ‘빨리 사진 찍어야 합니다’ 등과 같이 나머지 단어와 관계없이 사진을 찍습니다.

```
01: from popAssist import *
02: from pop import Camera
03: import cv2
04: import subprocess
05:
06: cam = Camera(width=300, height=300)
07:
08: def userAction(text):
09:     action = False
10:
11:     r = text.find("사진")
12:     if r != -1 and text.find("찍어", r) != -1:
13:         print("Take picture...")
14:         cv2.imwrite("picture.png", cam.value)
15:
16:         action = True
17:
18:     return action
19:
20: stream = create_conversation_stream()
21: ga = GAssistant(stream, local_device_handler=userAction)
```

```

22:
23: try:
24:     def onStart():
25:         print(">>> Start recording....")
26:
27:     while True:
28:         ga.assist(onStart)
29: except KeyboardInterrupt:
30:     stream.close()

```

5.4.4.5. 음성으로 AIoT AutoCar 제어

하위드가 인식되면 AIoT AutoCar의 모터를 제어하는 기능을 구현해 봅니다. 음성으로 ‘전진’, ‘후진’, ‘정지’의 명령어에 따라 모터가 제어되도록 합니다.

GAssistant 객체를 만들 때 local_device_handler 인자에 userAction() 메소드를 인자로 전달해 로컬 사용자 장치 액션을 실행하도록 설정하고 GAssistant 객체의 assist() 메소드를 호출하도록 설정합니다.

userAction() 메소드가 호출될 때 인자로 받은 문자열과 비교해 모터를 구현합니다.

음성으로 AutoCar를 제어하기 위해 Pilot 모듈과 popAssistant 모듈을 import 합니다. 이후 AutoCar 객체를 생성합니다.

```

01: from pop import Pilot
02: from popAssist import *
03:
04: Car = Pilot.AutoCar()

```

음성 명령을 받고 동작할 메소드를 정의합니다. 메소드는 “전진”, “후진”, “정지”라는 키워드를 찾고 키워드에 맞는 AutoCar 객체의 메소드를 호출

합니다.

```
05: def userAction(text):
06:     action = False
07:
08:     print(text)
09:
10:     if text.find("전진") != -1:
11:         Car.forward()
12:         action = True
13:     elif text.find("후진") != -1:
14:         Car.backward()
15:         action = True
16:     elif text.find("정지") != -1:
17:         Car.stop()
18:         action = True
19:
20:     return action
```

ConversationStream 객체와 GAssistant 객체를 생성합니다. 이후 Assistant 를 실행시킵니다.

```
21: stream = create_conversation_stream()
22: ga = GAssistant(stream, local_device_handler=userAction)
23:
24: try:
25:
26:     def onStart():
27:         print(">>> Start recording....")
28:
29:     while True:
30:         ga.assist(onStart)
```

Ctrl + C 를 누를 경우 스트림을 종료합니다. (Jupyter 에서는 Interrupt the Kernel 버튼)

```
31: except KeyboardInterrupt:
32:     stream.close()
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import Pilot
02: from popAssist import *
03:
04: Car = Pilot.AutoCar()
05:
06: def userAction(text):
07:     action = False
08:
09:     print(text)
10:
11:     if text.find("전진") != -1:
12:         Car.forward()
13:         action = True
14:     elif text.find("후진") != -1:
15:         Car.backward()
16:         action = True
17:     elif text.find("정지") != -1:
18:         Car.stop()
19:         action = True
20:
21:     return action
22:
23: stream = create_conversation_stream()
24: ga = GAssistant(stream, local_device_handler=userAction)
```

```
25:
26: try:
27:
28:     def onStart():
29:         print(">>> Start recording....")
30:
31:     while True:
32:         ga.assist(onStart)
33:
34: except KeyboardInterrupt:
35:     stream.close()
```

프로그램을 실행한 후 ‘Start recording’ 메시지가 출력되는데 이후부터 음성 인식을 시작합니다. ‘전진’, ‘후진’, ‘정지’로 음성 명령을 내리면 해당 모터 제어를 합니다.

내용 정리

- 리눅스 커널은 이러한 문제점을 해결하기 오디오 하위 시스템을 제공
- 오디오 하위 시스템 : 다양한 사운드 카드를 추상화해 표준화된 사운드 인터페이스를 제공. ALSA 사용
- PyAudio : PortAudio에 대한 파이썬 버전
 - 파이썬으로 WAVE 파일을 재생하거나 녹음하는 프로그램을 작성 가능
 - 블로킹과 논블로킹 모드 지원
 - WAV 파일만 지원
- TTS : 텍스트를 음성으로 변환할 때 사용
- gTTS : 클라우드 기반 구글 번역기에 포함된 텍스트 음성 변환 API에 대한 파이썬 라이브러리
 - 명령행 기반 툴도 함께 제공
 - 구글 클라우드 서비스로 텍스트를 전달하면 음성합성 결과를 mp3 파일로 변환
- 구글 어시스턴트 : 구글 클라우드에서 제공하는 AI 기반 음성 인식 및 행동 실행 서비스
- 사용자 장치 액션 : 음성 인식 결과를 엡지 디바이스의 특정 명령에 반영

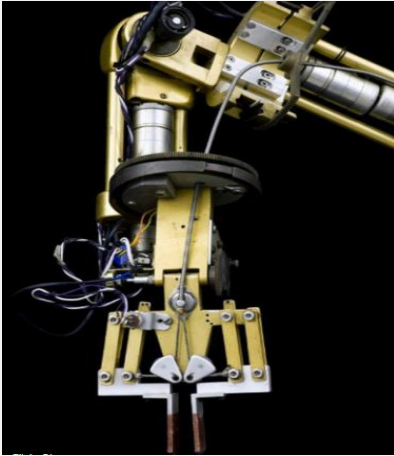


그림 77 The Rancho Arm

1963

The Rancho Arm 로봇은 장애인용 장치로 설계되었습니다. Rancho Arm 의 6 개 관절은 인간의 팔과 같은 유연성을 제공했습니다. 1963 년 Stanford University 에 인수된 이 장치는 컴퓨터로 제어할 수 있는 최초의 인공 로봇 팔 중 하나입니다.



그림 78 Ed Feigenbaum

1965

Ed Feigenbaum 교수, Joshua Lederberg 및 Carl Djerassi 가 이끄는 Stanford 팀은 최초의 "전문가 시스템"인 DENDRAL 을 만듭니다. DENDRAL 은 전문가의 축적된 전문 지식을 문제 해결에 적용하도록 설계된 인공 지능 프로그램입니다. 전문 분야는 화학과 물리학이었으며 유기 화합물의 분자 구조를 식별하기 위해 "if-then"규칙의 배터리를 적용했는데, 경우에 따라 전문가보다 더 정확한 결과를 얻을 수 있었습니다

연습문제

문제 4

다음 코드는 wave 라이브러리를 이용하여 오디오 파일을 읽어 변수에 저장하는 코드입니다. 질문을 읽고 답해보세요.

```
01: import wave
02:
03: w = wave.open()
04: data = w.readframes(w.getnframes())
05: w.close()
```

- A. 'sound.wav' 라는 오디오 파일을 읽으려 할 때 빈 칸에 들어갈 코드를 작성해보세요.
- B. getnframes() 메소드의 역할이 무엇인지 답해보세요.
- C. PyAudio 라이브러리를 이용해 읽어온 오디오 데이터를 재생하는 코드를 작성해보세요.

문제 5

gTTS 라이브러리를 사용하여 'Audio output' 이라는 문장을 음성 파일로 저장하는 코드를 작성해보세요.

문제 6

다음 코드는 popAssist 라이브러리를 이용해 음성으로 AutoCar 를 제어하는 코드입니다. gTTS 를 이용하여 ‘전진’이 인식되면 ‘전진합니다’ 와 같이 음성으로 응답을 하는 코드를 작성해보세요.

```
06: from pop import Pilot
07: from popAssist import *
08:
09: Car = Pilot.AutoCar()
10:
11: Car.stop()
12:
13: def userAction(text):
14:     action = False
15:
16:     print(text)
17:
18:     if text.find("전진") != -1:
19:         Car.forward()
20:         action = True
21:     elif text.find("후진") != -1:
22:         Car.backward()
23:         action = True
24:     elif text.find("정지") != -1:
25:         Car.stop()
26:         action = True
27:
28:     return action
29:
30: stream = create_conversation_stream()
31: ga = GAssistant(stream, local_device_handler=userAction)
32:
33: try:
34:
35:     def onStart():
36:         print(">>> Start recording....")
37:
```

```
38:     while True:
39:         ga.assist(onStart)
40:
41: except KeyboardInterrupt:
42:     stream.close()
```

6. 데이터 사이언스

데이터는 빛, 속도, 온도 등과 그것들을 응용한 데이터까지 다양하고 방대하게 수집할 수 있습니다. 데이터가 세분되고 양이 많아질수록 정밀한 설계가 가능하지만 그만큼 불필요한 데이터도 많아집니다. 수많은 데이터 속에서 필요한 데이터를 정제해내는 방법과 기술을 데이터 사이언스라고 합니다.

파이썬에는 데이터를 쉽게 다양한 형태로 응용할 수 있도록 도움을 주는 패키지들이 있습니다. 데이터를 다루는 패키지인 Numpy 와 Pandas, 데이터를 보기 쉽게 표시해주는 패키지인 Matplotlib 가 있습니다.

6.1. Numpy

Numpy(넘파이)는 빠른 고성능 연산을 위해 C 언어로 구현되었으며 벡터 및 텐서 연산에 편리한 기능들을 사용할 수 있기에 선형 대수 계산에 유용한 패키지입니다. 파이썬의 연산 속도는 느린 편이기에 C 언어의 연산 속도를 얻고자 라이브러리로 만들어졌습니다. 파이썬 기반 프로그램에서 인공지능 연산을 수월하게 하여 파이썬의 편의성과 C 언어의 연산 능력을 동시에 이용할 수 있게 합니다.

Numpy 는 다음 명령으로 설치하는데, Soda OS 에는 미리 설치되어 있습니다.

- `pip3 install numpy`

6.1.1. 배열

Numpy 에서 ndarray 라는 타입을 사용하며 빠른 연산을 위한 Numpy 의

배열 타입입니다. 배열을 생성, 초기화, 변형하는 주요 메소드는 다음과 같습니다.

- `array(list)`: 파이썬의 리스트, 튜플, 배열 타입을 Numpy의 `ndarray` 타입으로 생성
- `zeros(shape)`: 값을 0으로 채운 `shape` 형태로 `ndarray`를 생성
- `ones(shape)`: 값을 1으로 채운 `shape` 형태로 `ndarray`를 생성
- `full(shape, n)`: 값을 `n`으로 채운 `shape` 형태로 `ndarray`를 생성
- `eye(size)`: `size x size` 크기의 `ndarray`를 대각선은 1, 나머지는 0으로 채워 생성
- `random.random(shape)`: 값을 랜덤으로 채운 `shape` 형태로 `ndarray`를 생성
- `arange(start, stop, step)`: `start` 부터 `stop` 까지 `step` 만큼 증감시킨 값을 갖는 `ndarray`를 생성
 - `start` 파라미터만 입력해 생성 가능. 이 경우 0부터 `start` 까지 1씩 증감하는 값으로 생성
- `reshape(shape)`: `ndarray`를 `shape` 형태로 재구성

6.1.1.1. 생성

`ndarray`는 `array()` 메소드로부터 생성되고, 파이썬의 리스트, 튜플, 배열 타입을 입력받을 수 있습니다.

```
01: import numpy as np
02:
03: arr = np.array([1,2,3])
04: print(type(arr))
05: print(arr)
```

[출력]

```
<class 'numpy.ndarray'>
[1,2,3]
```

또, `ndarray`는 파이썬의 배열 타입처럼 다차원으로 생성할 수 있고, `ndim`

과 shape 속성을 이용해 차원과 배열 형태를 확인할 수 있습니다

```
01: import numpy as np
02:
03: arr = np.array([[1,2,3],[4,5,6]])
04: print(arr.ndim)
05: print(arr.shape)
```

[출력]

2

(2, 3)

arange() 메소드는 리스트 중에서 [1, 2, 3, 4, 5]와 같이 패턴이 반복되는 형태 배열을 생성할 때 사용할 수 있는 메소드입니다. 1 개나 3 개의 파라미터를 받을 수 있습니다. 파라미터를 1 개만 받은 경우에는 0 부터 n 까지 1 씩 증감하는 형태로 생성되고, 3 개를 받아 값 범위와 증가 폭을 지정할 수 있습니다.

```
01: import numpy as np
02:
03: arr = np.arange(5)
04: print(arr)
05: arr = np.arange(5,15,2)
06: print(arr)
```

[출력]

[0, 1, 2, 3, 4]

[5, 7, 9, 11, 13]

6.1.1.2. 초기화

복잡한 배열 또는 반복되는 배열을 만들거나 배열의 값을 사전 정의할 수 없을 때 값을 지정하여 ndarray 를 생성할 수는 없습니다. 이럴 때 특정 값을 채워 초기화시킨 ndarray 를 생성할 수 있습니다. 먼저, 값을 0 으로 채워 생성하는 경우입니다.

numpy.zeros 는 배열의 형태를 입력받고 값을 0 으로 채운 ndarray 배열을 생성하는 메소드입니다.

```
01: import numpy as np
02:
03: arr = np.zeros(5)
04: print(arr)
```

[출력]

[0. 0. 0. 0. 0.]

또, Shape 를 여러 가지 형태로 생성할 수 있습니다.

```
01: import numpy as np
02:
03: arr = np.zeros((2,5))
04: print(arr)
05: arr = np.zeros((2,2,2))
06: print(arr)
```

[출력]

[[0. 0. 0. 0. 0.],
 [0. 0. 0. 0. 0.]]

```
[[[0. 0.], [0. 0.]],  
 [[0. 0.], [0. 0.]]]
```

값이 0 이 아닌 1 로 채울 경우에는 `numpy.ones` 메소드를 사용해 값을 1 로 채운 `ndarray` 배열을 생성합니다.

```
01: import numpy as np  
02:  
03: arr = np.ones(5)  
04: print(arr)  
05: arr = np.ones((2,2))  
06: print(arr)
```

[출력]

```
[1.  1.  1.  1.  1.]
```

```
[[1.  1.], [1.  1.]]
```

값을 직접 지정하여 배열을 생성할 때는 `numpy.full` 메소드를 이용해 입력한 파라미터 값으로 채운 `ndarray` 배열을 생성합니다.

```
01: import numpy as np  
02:  
03: arr = np.full(5,-1)  
04: print(arr)  
05: arr = np.full((2,2),-1)  
06: print(arr)
```

[출력]

```
[-1. -1. -1. -1. -1.]
```

```
[[ -1. -1.], [ -1. -1.]]
```

특별한 경우로, 대각선만 1로 채운 형태의 배열이 필요한 경우가 있습니다. 이런 경우에는 `numpy.eye` 메소드를 사용해 사이즈 값을 파라미터로 받아 대각선으로 1을 채우고 나머지는 0을 채운 2차원 `ndarray` 배열을 생성합니다.

```
01: import numpy as np
02:
03: arr = np.eye(5)
04: print(arr)
```

[출력]

```
[[1. 0. 0. 0. 0.],
 [0. 1. 0. 0. 0.],
 [0. 0. 1. 0. 0.],
 [0. 0. 0. 1. 0.],
 [0. 0. 0. 0. 1.]]
```

Numpy는 랜덤 값을 채운 배열 생성이 가능하고, 머신러닝 알고리즘에서 랜덤 노이즈나 가중치의 초기값 등으로 이러한 랜덤 값 배열을 활용합니다.

`numpy.random` 클래스의 `random` 메소드에 `shape`를 파라미터로 입력하면 `ndarray`를 반환합니다.

```
01: import numpy as np
02:
03: arr = np.random.random(5)
```

```
04: print(arr)
05: arr = np.random.random((2,2))
06: print(arr)
```

[출력]

[0.79952745, 0.90090867, 0.79834555, 0.44890384, 0.43928767]

[[0.08159691, 0.42507803],
[0.66439239, 0.8743432]]

6.1.1.3. 변형

다차원 배열 간 연산할 때 형태를 재조정해 맞춰주거나 필요한 부분만 추출해 연산하는 등 다양한 연산 형태가 있습니다. Numpy에서는 형태 변형이나 배열 추출에 관한 메소드를 제공합니다.

arange 메소드를 이용해 패턴이 있는 배열을 생성하면 1차원 배열로 생성됩니다. 이 배열을 다차원 배열로 변환할 때 사용하는 메소드로 ndarray.reshape가 있습니다. 파라미터로 shape를 입력하면 호출한 ndarray를 shape 형태로 재구성합니다.

```
01: import numpy as np
02:
03: arr = np.arange(10)
04: print(arr)
05: reshaped = arr.reshape(2,5)
06: print(reshaped)
```

[출력]

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```
[[0, 1, 2, 3, 4],  
 [5, 6, 7, 8, 9]]
```

Numpy 에서 특정 배열을 잘라내는 방법은 파이썬의 리스트를 잘라내는 방법과 같습니다. 배열의 원소에 접근할 때 하나만 지정하거나 범위를 지정해 여러 개를 추출할 수 있습니다.

```
01: import numpy as np  
02:  
03: arr = np.arange(10)  
04: print(arr)  
05: s1 = arr[5]  
06: print(s1)  
07: s2 = arr[:5]  
08: print(s2)  
09: s3 = arr[2:5]  
10: print(s3)
```

[출력]

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
5
```

```
[0, 1, 2, 3, 4]
```

```
[2, 3, 4]
```

또, 다차원 배열의 원소에 접근할 때 특정 원소, 행 또는 열 하나만 지정하거나 범위를 지정해 여러 개를 추출할 수 있습니다.

```
01: import numpy as np
```

```
02:
03: arr = np.arange(10).reshape(2,5)
04: print(arr)
05: s1 = arr[1,4]
06: print(s1)
07: s2 = arr[1:]
08: print(s2)
09: s3 = arr[0, :]
10: print(s3)
11: s4 = arr[:, 0]
12: print(s4)
```

[출력]

```
[[0, 1, 2, 3, 4],
 [5, 6, 7, 8, 9]]
```

9

```
[[5, 6, 7, 8, 9]]
```

```
[0, 1, 2, 3, 4]
```

```
[0, 5]
```


내용정리

- Numpy : C 언어 기반으로 만들어져 고속 연산이 가능하도록 하며 벡터 및 텐서 연산에 편리한 기능들을 제공하는 패키지
- ndarray : 빠른 연산을 위한 Numpy 의 배열 타입
- array(list): 파이썬의 리스트, 튜플, 배열 타입을 ndarray 타입으로 생성
- zeros(shape): (shape) 형태에 값을 0 으로 채워 ndarray 를 생성
- ones(shape): (shape) 형태에 값을 1 로 채워 ndarray 를 생성
- full(shape, n): (shape) 형태에 값을 n 으로 채워 ndarray 를 생성
- eye(size): (size, size) 형태의 ndarray 를 대각선은 1, 나머지는 0 으로 채워 생성
- random.random(shape): (shape) 형태에 값을 랜덤으로 채워 ndarray 를 생성
- arange(start, stop, step):
 - start 부터 stop 까지 step 만큼 증감시킨 수열을 갖는 ndarray 를 생성
 - start 파라미터만 입력하면 0 부터 start 까지 1 씩 증가하는 수열로 생성
- reshape(shape): ndarray 를 (shape) 형태로 재구성



그림 79 HP 2116A

1966

HP의 첫 번째 컴퓨터인 2116A는 HP의 프로그래머블 테스트 및 측정 제품들을 위한 다목적 계측기 컨트롤러로 개발되었습니다. 2116A는 다양한 표준 실험실 장비와 인터페이스하여 고객이 장비 시스템을 전산화 할 수 있었습니다. 2116A는 상용 제품에 HP가 처음으로 집적 회로를 사용한 컴퓨터였습니다.



그림 80 NLS setup

1968

Douglas Engelbart와 그의 SRI 팀은 샌프란시스코의 컴퓨팅 컨퍼런스에서 'The Mother of All Demos'로 알려진 실험 'oN-Line System'을 공개했습니다. 90분 동안 약 천 명의 목격자들이 공동 편집, 화상 회의, 워드 프로세싱 및 "마우스"라고 하는 이상한 포인팅 장치를 목격했습니다. NLS 시스템의 모든 단락 또는 기타 요소는 하이퍼 텍스트 링크를 사용하여 다른 요소에 연결될 수 있었습니다.

연습문제

문제 7

다음 코드를 읽고 출력을 작성해보세요.

```
01: import numpy as np
02:
03: arr = np.array([[1,2,3],[4,5,6]])
04:
05: print(arr.ndim)
06: print(arr.shape)
```

문제 8

다음 코드를 읽고 출력을 작성해보세요.

```
01: import numpy as np
02:
03: arr = np.zeros((3,3))
04:
05: print(arr)
```

문제 9

다음과 같은 출력을 하는 코드를 작성해보세요.

[출력]

[-2, 0, 2, 4, 6, 8, 10]

문제 10

다음 코드는 Cds 센서를 이용하여 주변 밝기 값을 출력하는 코드입니다.

```
01: import numpy as np
02: from pop import Cds, delay
03:
04: cds = Cds(7)
05:
06: for i in range(10):
07:     value = cds.readAverage()
08:     print(value)
09:     delay(500)
```

- A. 코드를 실행시켜 결과가 어떻게 나오는지 답해보세요.
- B. 10 개의 data 를 저장할 수 있는 ndarray 배열을 0 으로 초기화하여 생성하는 코드를 작성해보세요.
- C. Cds 센서의 값 10 개를 B 에서 생성한 배열에 저장하는 코드를 작성해보세요.
- D. C 에서 생성된 배열을 (2,5)의 shape 로 변형하는 코드를 작성해보세요.
- E. D 에서 변형된 배열 중 첫 행만 추출하여 출력해보세요.

6.1.2. 연산

Numpy 연산에는 파이썬에서 사용하는 연산자(+, -, *, /)를 사용할 수 있고, 연산 메소드로도 사용할 수 있습니다. 또, 파이썬의 일반적 연산과 달리 Numpy 는 행렬곱을 지원합니다.

연산에 사용되는 주요 연산자와 연산 메소드는 다음과 같습니다.

- + 또는 add(x, y): x 와 y 로 입력된 ndarray 배열을 덧셈하여 반환
- - 또는 subtract(x, y): 입력된 ndarray 배열 x 에 y 를 뺄셈하여 반환
- * 또는 multiply(x, y): x 와 y 로 입력된 ndarray 배열을 곱셈하여 반환
- / 또는 divide(x, y): 입력된 ndarray 배열 x 에 y 를 나눗셈하여 반환
- matmul(x, y): 입력된 ndarray 배열 x 에 y 를 행렬곱하여 반환
- abs(x): 입력된 ndarray 배열 x 를 절대값으로 반환
- square(x): 입력된 ndarray 배열 x 를 제곱하여 반환
- sqrt(x): 입력된 ndarray 배열 x 를 제곱근으로 반환
- log(x): 입력된 ndarray 배열 x 를 로그값으로 반환

6.1.2.1. 사칙 연산

Numpy 에서 기본적인 사칙연산을 할 때의 이점은 연산 속도가 빠르다는 점입니다. 작은 사이즈의 배열로 연산할 때는 파이썬 기본 연산과의 차이가 크지 않지만, 배열이 커질수록 그 차이는 점점 벌어집니다. 그러므로 Numpy 에서 지원하는 연산을 사용하는 것이 좋습니다.

```
01: import numpy as np
02:
03: x = np.arange(4).reshape(2,2)
04: print(x)
05: y = np.arange(3,-1,-1).reshape(2,2)
06: print(y)
```

```
07:
08: s1 = x + y
09: print(s1)
10: s2 = x - y
11: print(s2)
12: s3 = x * y
13: print(s3)
14: s4 = x / y
15: print(s4)
```

[출력]

[[0, 1], [2, 3]]

[[3, 2], [1, 0]]

[[3, 3], [3, 3]]

[[-3, -1], [1, 3]]

[[0, 2], [2, 0]]

[[0., 0.5], [2., inf]]

또, 다음과 같이 Numpy 의 메소드를 사용해 사칙연산을 처리할 수도 있습니다.

```
01: import numpy as np
02:
03: x = np.arange(4).reshape(2,2)
04: print(x)
05: y = np.arange(3,-1,-1).reshape(2,2)
06: print(y)
07:
08: s1 = np.add(x,y)
09: print(s1)
```

```
10: s2 = np.subtract(x,y)
11: print(s2)
12: s3 = np.multiply(x,y)
13: print(s3)
14: s4 = np.divide(x,y)
15: print(s4)
```

[출력]

[[0, 1], [2, 3]]

[[3, 2], [1, 0]]

[[3, 3], [3, 3]]

[[-3, -1], [1, 3]]

[[0, 2], [2, 0]]

[[0., 0.5], [2., inf]]

6.1.2.2. 행렬곱

머신러닝을 프로그래밍할 때 일반적으로 사용되는 것이 행렬곱입니다. Numpy 는 연산 속도도 빠르면서 행렬곱까지 지원하기 때문에 머신러닝 프로그래밍에서 매우 중요한 패키지로 사용됩니다. 행렬곱 메소드는 `matmul` 이며 파라미터로 `ndarray` 2 개를 받습니다.

```
01: import numpy as np
02:
03: x = np.arange(4).reshape(2,2)
04: print(x)
05: y = np.arange(3,-1,-1).reshape(2,2)
06: print(y)
```

```
07:
08: s1 = np.matmul(x,y)
09: print(s1)
```

[출력]

[[0, 1], [2, 3]]

[[3, 2], [1, 0]]

[[1, 0], [9, 4]]

6.1.2.3. 기타 연산

배열의 절대값, 제곱, 제곱근, 로그를 구하는 등 유용한 연산도 Numpy 에서 지원합니다.

```
01: import numpy as np
02:
03: x = np.arange(4).reshape(2,2)
04: print(x)
05:
06: s1 = np.abs(x)
07: print(s1)
08: s2 = np.square(x)
09: print(s2)
10: s3 = np.sqrt(x)
11: print(s3)
12: s4 = np.log(x)
13: print(s4)
```

[출력]

[[0, 1], [2, 3]]

[[0, 1], [2, 3]]

`[[0, 1], [4, 9]]`

`[[0., 1.], [1.414, 1.732]]`

`[[ $-\infty$ , 0.], [0.693, 1.098]]`

내용정리

- 파이썬에서 사용하는 연산자(+, -, *, /) 사용 가능
- `add(x, y)`: ndarray 배열을 덧셈하여 반환
- `subtract(x, y)`: 입력된 ndarray 배열 `x`에 `y`를 뺄셈하여 반환
- `multiply(x, y)`: ndarray 배열을 곱셈하여 반환
- `divide(x, y)`: 입력된 ndarray 배열 `x`에 `y`를 나눗셈하여 반환
- `matmul(x, y)`: 입력된 ndarray 배열 `x`에 `y`를 행렬곱하여 반환
- `abs(x)`: 입력된 ndarray 배열을 절대값으로 반환
- `square(x)`: 입력된 ndarray 배열을 제곱하여 반환
- `sqrt(x)`: 입력된 ndarray 배열을 제곱근으로 반환
- `log(x)`: 입력된 ndarray 배열을 로그값으로 반환

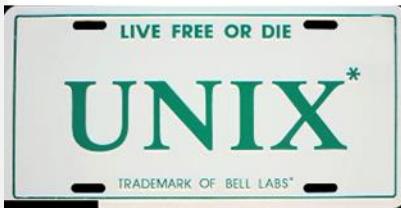


그림 81 UNIX

1969

AT & T Bell Labs 프로그래머 Kenneth Thompson 과 Dennis Ritchie 는 여분의 DEC 미니 컴퓨터에서 UNIX 운영 체제를 개발합니다. UNIX 는 Multics 에서 제공하는 많은 시간 공유 및 파일 관리 기능을 결합하여 이름을 변경했습니다.

UNIX 운영 체제는 특히 엔지니어와 과학자들 사이에서 폭 넓은 추종자들을 빠르게 확보했으며 오늘날의 세계의 많은 컴퓨팅 인프라의 기초가 되었습니다.

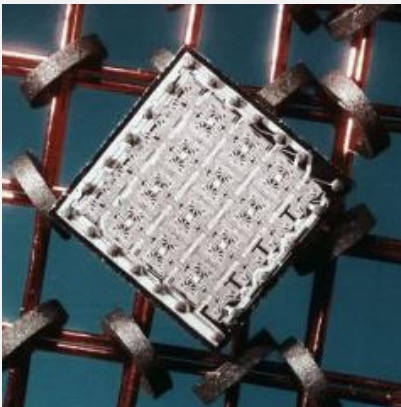


그림 82 코어 상단의 IC 메모리

1970

IBM 은 자기 코어 메모리 대신 주 메모리에 반도체 메모리를 사용하는 최초의 컴퓨터인 System 370 Model 145 메인 프레임 컴퓨터를 출시했습니다. Model 145 는 코어 메모리를 사용하는 컴퓨터와 비교하여 절반의 공간에 동등한 양의 데이터를 저장할 수 있습니다.

연습문제

문제 11

다음 코드를 읽고 출력을 작성해보세요.

```
01: import numpy as np
02:
03: arr = np.ones(5)
04:
05: a1 = arr + arr
06: a2 = np.add(arr, arr)
07:
08: print(a1)
09: print(a2)
```

문제 12

다음 코드를 읽고 출력을 작성해보세요.

```
01: import numpy as np
02:
03: arr = np.array([[1,2], [3,4]])
04:
05: a1 = np.matmul(arr, arr)
06:
07: print(a1)
```

문제 13

다음 코드를 읽고 arr 을 제공하여 출력하는 코드를 작성해보세요.

```
01: import numpy as np
02:
03: arr = np.array([0.1, 1, 10, 11])
```

문제 14

다음은 OLED 의 (1,1)좌표에 ‘!’을 출력하는 예제입니다.

```
01: from pop import Oled
02:
03: display = Oled()
04:
05: display.setCursor(1,1)
06: display.print('!')
```

- A. 코드를 실행시켜 결과를 확인해 보세요.
- B. 다음 코드에서 sqrt 와 matmul 메소드만 사용하고, arr1 과 arr2 배열만 사용하여 OLED 에 ‘!’가 (2,3) 좌표에 출력되도록 빈 칸을 채워보세요.

```
01: from pop import Oled
02: import numpy as np
03:
04: display = Oled()
05:
06: arr1 = np.array([[1,2], [4,1]])
07: arr2 = np.array([2, 1])
08:

...

```

```
11:
12: arr = np.abs(arr)
13:
14: x = int(arr[0])
15: y = int(arr[1])
16:
17: display.setCursor(x,y)
18: display.print('!')
```

6.1.3. 데이터 처리

Numpy 에서 데이터를 다양하게 활용할 수 있도록 데이터 처리 관련 메소드를 지원합니다. 데이터 필터링, 통계, 정렬 등 유용하게 사용할 수 있는 메소드를 소개합니다.

- `where(cond, x, y)`: 조건 연산자 또는 `True, False` 배열을 이용해 참이면 `x` 의 원소, 거짓이면 `y` 의 원소를 선택해 새로운 배열을 반환
- `sum(x)`: 입력된 `ndarray` 배열의 모든 원소 값을 덧셈하여 반환
- `mean(x)`: 입력된 `ndarray` 배열의 모든 원소 값의 평균을 반환
- `max(x)`: 입력된 `ndarray` 배열의 모든 원소 값 중 최댓값을 반환
- `min(x)`: 입력된 `ndarray` 배열의 모든 원소 값 중 최솟값을 반환
- `std(x)`: 입력된 `ndarray` 배열의 표준편차를 반환
- `var(x)`: 입력된 `ndarray` 배열의 분산을 반환
- `cumsum(x)`: 입력된 `ndarray` 배열의 원소 순 누적 합을 반환
- `cumprod(x)`: 입력된 `ndarray` 배열의 원소 순 누적 곱을 반환
- `sort(x)`: 입력된 `ndarray` 배열을 정렬하여 반환
- `unique(x)`: 입력된 `ndarray` 배열의 중복을 제거하여 반환
- `union1d(x, y)`: 입력된 `ndarray x` 와 `y` 배열의 합집합을 반환
- `setdiff1d(x, y)`: 입력된 `ndarray x` 에 대한 `y` 배열의 차집합을 반환

6.1.3.1. 조건부 선택

두 배열에서 특정 조건에 따라 어느 배열의 원소를 가져올지 순서대로 선택해 새로운 배열을 만들어낼 수 있습니다. 예를 들어, 두 배열을 비교해 더 큰 값을 가져와 배열을 만들고자 할 때 다음과 같이 프로그래밍할 수 있습니다.

```
01: import numpy as np
```

```
02:
03: x = np.arange(4).reshape(2,2)
04: print(x)
05: y = np.arange(3,-1,-1).reshape(2,2)
06: print(y)
07:
08: s1 = np.where(x > y, x, y)
09: print(s1)
```

[출력]

```
[[0, 1], [2, 3]]
```

```
[[3, 2], [1, 0]]
```

```
[[3, 2], [2, 3]]
```

6.1.3.2. 배열 통계

배열에서 평균, 최댓값, 최솟값을 찾거나 원소들의 합을 구하는 등 유용한 연산도 Numpy 에서 지원합니다. 그중 가장 많이 쓰이는 평균, 합, 최대, 최소 연산부터 설명합니다.

```
01: import numpy as np
02:
03: x = np.arange(4).reshape(2,2)
04: print(x)
05:
06: s1 = np.sum(x)
07: print(s1)
08: s2 = np.mean(x)
09: print(s2)
10: s3 = np.max(x)
11: print(s3)
12: s4 = np.min(x)
13: print(s4)
```

[출력]

[[0, 1], [2, 3]]

6

1.5

3

0

이어서 표준편차, 분산, 원소 순 누적 합과 곱 등 연산도 가능합니다.

```
01: import numpy as np
02:
03: x = np.arange(4).reshape(2,2)
04: print(x)
05:
06: s1 = np.std(x)
07: print(s1)
08: s2 = np.var(x)
09: print(s2)
10: s3 = np.cumsum(x)
11: print(s3)
12: s4 = np.cumprod(x)
13: print(s4)
```

[출력]

[[0, 1], [2, 3]]

1.118033988749895

1.25

[0, 1, 3, 6]

[0, 0, 0, 0]

6.1.3.3. 집합 연산

배열을 비교하거나 오름차순으로 접근해야 하는 등 정렬이 필요한 경우, 배열의 중복을 제거하거나 두 배열의 합집합을 구하는 등 집합 연산에도 유용한 메소드를 사용할 수 있습니다.

```
01: import numpy as np
02:
03: x = np.ones((2,2))
04: print(x)
05: y = np.arange(3,-1,-1).reshape(2,2)
06: print(y)
07:
08: s1 = np.sort(y)
09: print(s1)
10: s2 = np.unique(x)
11: print(s2)
12: s3 = np.union1d(x,y)
13: print(s3)
14: s4 = np.setdiff1d(y,x)
15: print(s4)
```

[출력]

[[0, 1], [2, 3]]

[[3, 2], [1, 0]]

[[2, 3], [0, 1]]

[1.]

[0., 1., 2., 3.]

[0, 2, 3]

6.1.3.4. 파일 저장 및 로드

프로그램을 종료한 뒤 다음 실행에도 데이터를 유지해야 하는 경우 시스템에 파일 형태로 저장하는 방법을 사용할 수 있습니다. Numpy에서는

다음과 같이 save 와 load 메소드로 구현할 수 있습니다. 파일 저장 메소드는 save(name, x)형태로 사용하며 name 으로 지정한 파일명 뒤에 자동으로 '.numpy' 확장자가 붙어 바이너리 형식으로 저장됩니다. 파일 로드 메소드는 load(name) 형태로 사용하며 파일명은 확장자까지 모두 포함되어야 합니다.

```
01: import numpy as np
02:
03: x = np.random.random(20)
04: print(x)
05:
06: np.save("randoms", x)
07:
08: y=np.load("randoms.npy")
09: print(y)
```

[출력]

```
[0.37688615, 0.58605648, 0.68608808, 0.73656025, 0.66878285, 0.10704113,
 0.43355633, 0.22134522, 0.6735362, 0.28959784, 0.34256772, 0.80990703,
 0.96233255, 0.69791836, 0.47497396, 0.09502128, 0.63275289, 0.9446321,
 0.47604674, 0.25210486]
```

```
[0.37688615, 0.58605648, 0.68608808, 0.73656025, 0.66878285, 0.10704113,
 0.43355633, 0.22134522, 0.6735362, 0.28959784, 0.34256772, 0.80990703,
 0.96233255, 0.69791836, 0.47497396, 0.09502128, 0.63275289, 0.9446321,
 0.47604674, 0.25210486]
```

여러 개의 배열을 한 파일로 저장하고 싶다면 savez 메소드를 사용할 수 있습니다. savez(name, arr_name=x, ...) 형태로 사용하며 name 은 파일명, arr_name 은 배열 x 의 이름을 지정할 수 있습니다. 파일 확장자는 '.numpy'가 아닌 '.npz'로 저장되며 바이너리 형식입니다.

이 파일을 로드하면 덱서너리와 같은 방법으로 배열을 가져올 수 있습니다. npz 파일은 사용이 끝났다면 close 메소드를 이용해 파일을 닫아줘야 합니다.

```
01: import numpy as np
02:
03: x = np.random.random(20)
04: print(x)
05: y = np.random.random(20)
06: print(y)
07:
08: np.savez("randoms", arr1=x, arr2=y)
09:
10: z=np.load("randoms.npz")
11: print(z["arr1"])
12: print(z["arr2"])
13: z.close()
```

[출력]

```
[0.74794297, 0.04710173, 0.45330354, 0.97561827, 0.82609087, 0.14017323,
 0.43741512, 0.66098091, 0.39292845, 0.20467854, 0.53466003, 0.78627536,
 0.08068616, 0.28534034, 0.13186805, 0.6501074, 0.40426906, 0.00993878,
 0.50939675, 0.14722917]
[0.17388481, 0.31467584]
```

```
[0.74794297, 0.04710173, 0.45330354, 0.97561827, 0.82609087, 0.14017323,
 0.43741512, 0.66098091, 0.39292845, 0.20467854, 0.53466003, 0.78627536,
 0.08068616, 0.28534034, 0.13186805, 0.6501074, 0.40426906, 0.00993878,
 0.50939675, 0.14722917]
[0.17388481, 0.31467584]
```

파일을 직접 열었을 때도 읽기 쉽도록 텍스트 파일 형식으로도 저장할 수 있습니다. Numpy 패키지의 savetxt 와 loadtxt 메소드를 사용합니다. 파

일 저장은 `savetxt(name, x)` 형태로 사용하고 파일 로드는 `loadtxt(name)` 형태로 사용합니다.

```
01: import numpy as np
02:
03: x = np.arange(20)
04: print(x)
05:
06: np.savetxt("texts.txt", x)
07:
08: y=np.loadtxt("texts.txt")
09: print(y)
```

[출력]

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
```

```
[ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17.
 18. 19.]
```

texts.txt 파일을 메모장 등의 에디터로 열어봤을 때 다음과 같이 저장되어 있습니다.

1	0.0000000000000000e+00
2	1.0000000000000000e+00
3	2.0000000000000000e+00
4	3.0000000000000000e+00
5	4.0000000000000000e+00
6	5.0000000000000000e+00
7	6.0000000000000000e+00
8	7.0000000000000000e+00
9	8.0000000000000000e+00
10	9.0000000000000000e+00
11	1.0000000000000000e+01
12	1.1000000000000000e+01
13	1.2000000000000000e+01
14	1.3000000000000000e+01
15	1.4000000000000000e+01
16	1.5000000000000000e+01
17	1.6000000000000000e+01
18	1.7000000000000000e+01
19	1.8000000000000000e+01
20	1.9000000000000000e+01

그림 83 texts.txt

내용정리

- `where(cond, x, y)` : 조건 `cond` 가 참이면 `x`, 거짓이면 `y`를 선택해 새로운 배열 반환
- `sum(x)`: 입력된 `ndarray` 배열의 모든 원소 값을 덧셈하여 반환
- `mean(x)`: 입력된 `ndarray` 배열의 모든 원소 값의 평균을 반환
- `max(x)`: 입력된 `ndarray` 배열의 모든 원소 값 중 최대값을 반환
- `min(x)`: 입력된 `ndarray` 배열의 모든 원소 값 중 최소값을 반환
- `std(x)`: 입력된 `ndarray` 배열의 표준편차를 반환
- `var(x)`: 입력된 `ndarray` 배열의 분산을 반환
- `cumsum(x)`: 입력된 `ndarray` 배열의 원소 순 누적 합을 반환
- `cumprod(x)`: 입력된 `ndarray` 배열의 원소 순 누적 곱을 반환
- `sort(x)`: 입력된 `ndarray` 배열을 정렬하여 반환
- `unique(x)`: 입력된 `ndarray` 배열의 중복을 제거하여 반환
- `union1d(x, y)`: 입력된 `ndarray` `x` 와 `y` 배열의 합집합을 반환
- `setdiff1d(x, y)`: 입력된 `ndarray` `x` 에 대한 `y` 배열의 차집합을 반환
- `save(name, x)` : `x` 를 `name` 파일명에 '.npy' 확장자인 바이너리 형식으로 저장
- `load(name)` : 파일 로드 메소드. 파일명은 확장자까지 모두 포함해야 함
- `savez(name, arr_name=x, ...)` : 여러 개의 배열을 한 파일로 저장하는 메소드
 - `name` 은 파일명, `arr_name` 은 배열 `x` 의 이름을 지정
 - 파일 확장자는 '.npz'로 저장되며 바이너리 형식으로 저장
 - 파일 사용 종료시 `close` 메소드를 이용해 파일을 닫아줘야 함
- `savetxt(name, x) / loadtxt(name)` : `x` 를 `name` 이름의 텍스트 파일 저장 / 로드



그림 84 Shakey

1970

SRI International 의 Shakey 는 인공 지능으로 제어되는 최초의 모바일 로봇입니다. 이 로봇은 감지 장치가 장착되어 있고 STRIPS 라는 문제 해결 프로그램으로 구동되며 환경에 대한 정보를 경로에 적용하여 SRI 홀 주변을 탐색했습니다. Shakey 는 TV 카메라, 레이저 거리 측정기 및 범프 센서를 사용하여 데이터를 수집한 다음 메인컴퓨터인 DEC PDP-10 및 PDP-15로 전송했습니다. 컴퓨터는 무선 링크를 통해 Shakey 에게 명령을 보냈으며 Shakey 는 시가당 2 미터의 속도로 움직



그림 85 Ray Tomlinson

1971

1970년대 초 전자 메일은 시분할 시스템에서 새롭게 떠오르는 컴퓨터 네트워크로 이동합니다. 네트워크가 있는 곳 어디에서나 메시지가 자유롭게 이동하고 이메일의 사용이 폭발적으로 늘어납니다. Newman 의 Ray Tomlinson 은 ARPAnet 에서 네트워크 이메일 프로토콜로 iconic “@” 표시를 선택했습니다.

연습문제

문제 15

다음 코드를 읽고 출력을 작성해보세요.

```
01: import numpy as np
02:
03: arr = np.array([1,2,3,4])
04:
05: a1 = np.sum(arr)
06: a2 = np.min(arr)
07: a3 = np.mean(arr)
08:
09: print(a1)
10: print(a2)
11: print(a3)
```

문제 16

다음 코드를 읽고 출력을 작성해보세요.

```
01: import numpy as np
02:
03: arr1 = np.array([1,2,3,4])
04: arr2 = np.array([4,3,2,1])
05:
06: a1 = np.where(arr1>arr2, arr1, arr2)
07:
08: print(a1)
```

문제 17

다음 코드를 읽고 arr 을 정렬하여 출력하는 코드를 작성해보세요.

```
01: import numpy as np
02:
03: arr = np.array([5, 1, -10, 11, 3])
```

문제 18

다음 코드는 Cds 센서의 밝기 값을 10 개씩 arr1 과 arr2 에 저장하고 출력하는 코드입니다.

```
01: from pop import Cds,delay
02: import numpy as np
03:
04: cds = Cds(7)
05:
06: arr1 = np.zeros(10)
07: arr2 = np.zeros(10)
08:
09: for i in range(len(arr1)):
10:     arr1[i] = cds.readAverage()
11:     delay(300)
12:
13: for i in range(len(arr2)):
14:     arr2[i] = cds.readAverage()
15:     delay(300)
16:
17: print(arr1)
18: print(arr2)
```

A. 코드를 실행시키고 CDS 센서에 변화를 주었을 때 출력을 확인해 보

세요.

- B. arr1, arr2 두개의 배열을 비교하여 작은 값이 저장되는 새로운 배열 arr3 을 생성해보세요.
- C. B 에서 생성된 배열을 오름차순으로 정렬해보세요.
- D. C 에서 정렬된 배열을 ARRAY.txt 로 저장하고 로드해보세요.

6.2. Pandas

Pandas(판다스)는 엑셀, 스프레드시트, SQL 과 비슷하게 행과 열이 있는 데이터 테이블을 사용합니다. 색인별 데이터 조회, 조건별 데이터 조회 등 데이터 통계와 분석에 유용한 패키지입니다. 인공지능을 학습시키기 전, 데이터 필터링을 위해 사용되며 대량의 데이터를 확인하기 쉽도록 합니다.

Pandas 는 다음 명령으로 설치하는데, Soda OS 에는 미리 설치되어 있습니다.

- `pip3 install pandas`

6.2.1. 데이터 프레임

Pandas 에서 데이터 테이블과 비슷한 개념으로 데이터를 관리합니다. 인덱스와 값만 있는 1 차원 테이블을 시리즈라 하고 열과 인덱스, 값이 있는 2 차원 테이블을 데이터 프레임이라고 합니다. 시리즈와 데이터 프레임을 관리하는 주요 메소드는 다음과 같습니다.

- `Series(list, index)`: 각각 값과 인덱스를 담고있는 리스트 2 개를 입력받아 Pandas 의 Series 타입으로 생성
- `DataFrame(list, index, columns)`: 값, 인덱스, 열을 담고있는 리스트 3 개를 입력받아 Pandas 의 DataFrame 타입으로 생성
- `DataFrame.head(x)`: 데이터 프레임의 앞 부분 x 개의 튜플 조회
- `DataFrame.tail(x)`: 데이터 프레임의 뒷 부분 x 개의 튜플 조회
- `DataFrame[name]`: 데이터 프레임에서 특정 열을 조회

6.2.1.1. 시리즈 생성

Series 는 인덱스와 값만 갖는 1 차원 구조이며, 파이썬의 딕셔너리와 비슷하게 인덱스를 참조해 값에 접근합니다. Pandas.Series 메소드로부터 생성되고 인덱스와 값에 해당하는 리스트 2 개를 입력받아야 합니다.

```
01: import pandas as pd
02:
03: value = [32, 68, 220, 72]
04: index = ["온도", "습도", "강수량", "불쾌지수"]
05: sr = pd.Series(value, index=index)
06:
07: print(type(sr))
08: print(sr)
09: print(sr["온도"])
```

[출력]

```
<class 'pandas.core.series.Series'>
```

```
온도      32
습도      68
강수량    220
불쾌지수   72
```

```
32
```

Series 에는 값과 인덱스를 각각 접근하기 위해 values 와 index 라는 속성이 있습니다. values 는 시리즈가 갖고 있는 값들을 반환하고, index 는 인덱스를 반환합니다.

```
10: print(sr.index)
11: print(sr.values)
```

[출력]

```
Index(['온도', '습도', '강수량', '불쾌지수'], dtype='object')
[32  68 220  72]
```

6.2.1.2. 데이터 프레임 생성

DataFrame 은 Series 에 이어서 인덱스와 값뿐만 아니라 열도 갖는 2 차원 구조이며, 특정 값에 접근하려면 인덱스와 열을 동시에 참조해야 합니다. 열과 인덱스를 지정하여 특정 값을 추출하려면

DataFrame[index][column] 형식으로 가져올 수 있습니다.

Pandas.DataFrame 메소드로부터 생성되고 파라미터로 값만 입력해 생성한 경우 인덱스와 열에 자동으로 순번을 지정합니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: df = pd.DataFrame(value)
07:
08: print(df)
09: print(df[0][1])
```

[출력]

	0	1	2	3
0	32	68	220	72
1	28	30	0	12
2	38	81	0	91

28

인덱스, 값, 열을 직접 지정하여 데이터 프레임을 생성할 수 있습니다.

이 경우 해당하는 리스트 3 개를 파라미터로 입력받아야 합니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: print(df)
11: print(df["온도"]["늦봄"])
```

[출력]

	온도	습도	강수량	불쾌지수
초여름	32	68	220	72
늦봄	28	30	0	12
한여름	38	81	0	91

28

DataFrame 에는 인덱스, 행, 값에 각각 접근하기 위해 index, columns, values 라는 속성이 있습니다. index 는 각 행의 색인을 반환하고, columns 는 각 열의 색인을 반환합니다. values 는 시리즈와 달리 2 차원 배열로 값을 반환합니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
```

```
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: print(df.index)
11: print(df.columns)
12: print(df.values)
```

[출력]

```
Index(['초여름', '늦봄', '한여름'], dtype='object')
```

```
Index(['온도', '습도', '강수량', '불쾌지수'], dtype='object')
```

```
[[ 32  68 220  72]
 [ 28  30   0  12]
 [ 38  81   0  91]]
```


내용정리

- Pandas : 데이터 통계와 분석에 유용한 패키지
 - 행과 열이 있는 데이터 테이블을 사용
 - 표의 개념으로 데이터를 관리
- Series(list, index) : Pandas 의 Series 타입으로 생성
 - 각각 값과 인덱스를 담고있는 리스트 2 개를 입력
 - values 속성 : 시리즈가 갖고 있는 값 반환
 - index 속성 : 인덱스를 반환합니다.
- DataFrame(list, index, columns) : Pandas 의 DataFrame 타입으로 생성
 - 값, 인덱스, 열을 담고있는 리스트 3 개를 입력
 - 인덱스, 열 없이 값만 입력해 생성한 경우 인덱스와 열에 자동 순번 지정
 - index 속성 : 각 행의 색인 반환
 - columns 속성 : 각 열의 색인 반환
 - values : 배열로 값 반환
- DataFrame.head(x) : 데이터 프레임의 앞 부분 x 개의 튜플 조회
- DataFrame.tail(x) : 데이터 프레임의 뒷 부분 x 개의 튜플 조회
- DataFrame[name] : 데이터 프레임에서 특정 열을 조회



그림 86 Ken Thompson and Dennis Ritchie

1972

C 프로그래밍 언어가 개발되었습니다. Dennis Ritchie 와 그의 팀은 초기 언어 BCPL (Basic Combined Programming Language)을 기반으로 C 를 만들었고 곧 C 로 Unix 의 소스 코드를 다시 작성했습니다. 따라서 Unix 는 다른 컴퓨터로 쉽고 이식되어 신속하게 퍼졌습니다. C 는 오늘날에도 여전히 널리 사용됩니다.

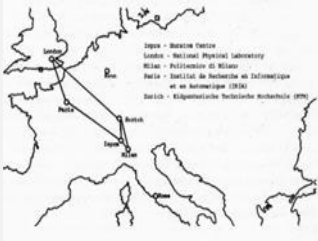


그림 87 1973 년 EIN

1973

초기 네트워크는 컴퓨터를 성공적으로 연결했지만 다른 종류의 네트워크를 서로 연결할 수는 없어 온라인 커뮤니티의 규모가 제한되었습니다. 프랑스의 CYCLADES 와 영국의 NPL 네트워크는 1973 년 EIN (European Informatics Network) 에서 다른종류의 네트워크를 연결하는 인터넷워킹 실험을 하고 있었고 Xerox PARC 는 같은 해에 PUP (PARC Universal Packet) 프로토콜을 사용하여 이더넷을 다른 네트워크와 연결하기 시작합니다. 이 두 가지 노력 모두 ARPA 의 TCP / IP 인터넷 프로토콜개발에

연습문제

문제 19

다음 코드를 읽고 출력을 작성해보세요.

```
01: import pandas as pd
02:
03: value = [[1, 2, 3, 5],
04:          [7, 11, 13, 17],
05:          [19, 23, 29, 31]]
06:
07: df = pd.DataFrame(value)
08:
09: print(df[2][2])
```

문제 20

다음 코드를 읽고 출력을 작성해보세요.

```
01: import pandas as pd
02:
03: value = [1, 5, 9, 13]
04: index = ["A", "B", "C", "D"]
05: sr = pd.Series(value, index=index)
06:
07: print(sr["C"])
```

문제 21

데이터 프레임을 사용해 다음과 같은 출력을 하는 코드를 작성해보세요.

[출력]

	A	B	C	D
a	0	2	4	6
b	1	3	5	7
c	2	4	6	8

문제 22

다음 코드는 Cds 센서의 밝기 값을 출력하는 코드입니다.

```
01: from pop import Cds, delay
02:
03: cds = Cds(7)
04:
05: arr1, arr2, arr3 = [], [], []
06:
07: for i in range(3):
08:     arr1.append(cds.readAverage())
09:     delay(300)
10:     arr2.append(cds.readAverage())
11:     delay(300)
12:     arr3.append(cds.readAverage())
13:     delay(300)
14:
15: arr = [arr1, arr2, arr3]
16:
17: print(arr)
```

- A. 코드를 실행시켜 결과를 확인해 보세요.
- B. arr 를 value 로 갖고, ['A', 'B', 'C']를 index 로 갖는 시리즈를 생성해 보세요.
- C. arr 를 value 로 갖고, ['A', 'B', 'C']를 columns, ['D', 'E', 'F']를 index 로 갖는 데이터 프레임을 생성해 보세요.

6.2.2. 데이터 조회 및 처리

Pandas 는 데이터 테이블을 특정 조건에 맞는 값을 검색하거나 행렬 반전, 인덱스 제어 등 다양하게 활용할 수 있도록 유용한 기능을 지원합니다. 데이터 조회 및 제어의 주요 메소드와 속성은 다음과 같습니다.

- `DataFrame.loc[x, y]`: x 행 y 열에 있는 값을 반환
- `DataFrame[Condition]`: 조건 연산 Condition 에 True 인 결과를 반환
- `DataFrame.T`: 데이터 프레임의 행과 열을 반전하여 반환
- `DataFrame.append(DataFrame)`: 새로운 데이터 프레임을 추가
- `DataFrame.insert(i, name, values)`: name 이라는 이름으로 values 를 i 번째 열에 삽입
- `DataFrame.drop(name, axis=0|1)`: 데이터 프레임에서 name 행 또는 열을 삭제
 - axis 가 0 이면 행, 1 이면 열을 삭제
- `DataFrame.sort_index()`: 인덱스를 기준으로 데이터 프레임을 정렬하여 반환
- `DataFrame.sort_values(column)`: column 열을 기준으로 데이터 프레임을 정렬하여 반환

6.2.2.1. 데이터 조회

데이터 프레임에서 특정 행과 열에 해당하는 값이나 특정 범위에 있는 값을 조회하고자 할 때 `loc()` 메소드를 사용할 수 있습니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: print(df.loc["초여름", "온도"])
```

```
11: print(df.loc[:, "온도"]) #데이터 프레임에서 모든 행에 대한 "온도"
    열을 조회
12: print(df.loc[:, "습도": "강수량"]) #데이터 프레임에서 모든 행에 대한
    "습도" 부터 "강수량" 열을 조회
```

[출력]

32

초여름 32

늦봄 28

한여름 38

	습도	강수량
초여름	68	220
늦봄	30	0
한여름	81	0

SQL 의 Select 문처럼 특정 조건에 해당하는 값들을 조회해야 할 때 DataFrame[Condition] 형식으로 조건을 줄 수 있습니다. 예를 들어, 온도가 30 이상인 튜플들을 찾는다고 가정하면 다음과 같이 할 수 있습니다. DataFrame[Condition]은 DataFrame 을 반환합니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: cond = df["온도"]>=30
11: print(df[cond])
```

[출력]

	온도	습도	강수량	불쾌지수
초여름	32	68	220	72
한여름	38	81	0	91

조금 더 복잡하게 온도가 30 이상인 튜플 중에서 습도만 찾는다고 가정하면 다음과 같이 할 수 있습니다.

```
12: cond = df["온도"]>=30
13: print(df[cond]["습도"])
```

[출력]

초여름	68
한여름	81

6.2.2.2. 테이블 반전

데이터 프레임에서 행과 열을 반전시키고자 할 때 T 속성을 사용할 수 있습니다. DataFrame.T 형식으로 사용하면 행과 열이 반전된 DataFrame을 반환합니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: print(df.T)
```

[출력]

	초여름	늦봄	한여름
온도	32	28	38
습도	68	30	81
강수량	220	0	0
불쾌지수	72	12	91

6.2.2.3. 데이터 추가

데이터 프레임에 새로운 데이터 프레임을 추가할 때 `append` 메소드를 사용할 수 있습니다. 추가하려는 데이터 프레임에 중복되는 행이 있어도 따로 추가되고, 중복되는 열이 있으면 병합합니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: value2 = [[37, 90, 120, 94]]
11: index2 = ["한여름"]
12: df2 = pd.DataFrame(value2, index=index2, columns=columns)
13:
14: print(df.append(df2))
```

[출력]

	온도	습도	강수량	불쾌지수
초여름	32	68	220	72
늦봄	28	30	0	12
한여름	38	81	0	91
한여름	37	90	120	94

데이터 프레임에 새로운 열을 추가할 때 `insert` 메소드를 사용할 수 있습

니다. 리스트로 각 행에 열에 대한 새로운 값을 지정해야 하며, 리스트가 아닌 변수 또는 상수를 입력하면 모든 행이 그 값으로 초기화됩니다.

Insert(number, column_name, values) 형태로 사용합니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: df.insert(0, "자외선", [6, 3, 7])
11: print(df)
```

[출력]

	자외선	온도	습도	강수량	불쾌지수
초여름	6	32	68	220	72
늦봄	3	28	30	0	12
한여름	7	38	81	0	91

값을 상수 또는 변수로 지정하여 모든 행의 값을 초기화할 수 있습니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
```



```
10: df.insert(0, "자외선", 0)
11: print(df)
```

[출력]

	자외선	온도	습도	강수량	불쾌지수
초여름	0	32	68	220	72
늦봄	0	28	30	0	12
한여름	0	38	81	0	91

6.2.2.4. 데이터 삭제

데이터 프레임에서 특정 데이터를 삭제할 때 drop 메소드를 사용할 수 있습니다. drop(name, axis=0|1) 형태로 사용하며 axis 가 0 이면 행, 1 이면 열에 있는 name 이 삭제됩니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: print(df.drop("불쾌지수", 1))
```

[출력]

	온도	습도	강수량
초여름	32	68	220
늦봄	28	30	0
한여름	38	81	0

```
11: print(df.drop("초여름", 0))
```

[출력]

	온도	습도	강수량	불쾌지수
늦봄	28	30	0	12
한여름	38	81	0	91

6.2.2.5. 데이터 정렬

데이터 프레임에서 인덱스의 오름차순으로 정렬할 때 `sort_index` 메소드를 사용할 수 있습니다.

```
01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:          [28, 30, 0, 12],
05:          [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: print(df.sort_index())
```

[출력]

	온도	습도	강수량
늦봄	28	30	0
초여름	32	68	220
한여름	38	81	0

데이터 프레임에서 특정 열의 오름차순으로 정렬할 때 `sort_values` 메소드를 사용할 수 있습니다. `sort_values(name)` 형태로 사용하며 `name` 으로 지정된 열을 기준으로 오름차순 정렬합니다.

```
01: import pandas as pd
02:
```

```

03: value = [[32, 68, 220, 72],
04:    [28, 30, 0, 12],
05:    [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09:
10: print(df.sort_values("습도"))

```

[출력]

	온도	습도	강수량
늦봄	28	30	0
초여름	32	68	220
한여름	38	81	0

6.2.2.6. 파일 저장 및 로드

Pandas에서는 csv 파일을 저장하거나 읽어올 수 있도록 메소드를 지원하고 있습니다. 파일 저장에는 DataFrame의 to_csv(name) 메소드를 사용하고, 파일 로드에는 Pandas의 read_csv(name) 메소드를 사용합니다.

```

01: import pandas as pd
02:
03: value = [[32, 68, 220, 72],
04:    [28, 30, 0, 12],
05:    [38, 81, 0, 91]]
06: columns = ["온도", "습도", "강수량", "불쾌지수"]
07: index = ["초여름", "늦봄", "한여름"]
08: df = pd.DataFrame(value, index=index, columns=columns)
09: print(df)
10:
11: df.to_csv("weather.csv")
12:
13: csv=pd.read_csv("weather.csv", index_col=0)
14: print(csv)

```

[출력]

	온도	습도	강수량	불쾌지수
늦봄	28	30	0	72
초여름	32	68	220	12
한여름	38	81	0	91

	온도	습도	강수량	불쾌지수
늦봄	28	30	0	72
초여름	32	68	220	12
한여름	38	81	0	91

저장된 csv 파일을 메모장 등의 일반 에디터로 열었을 경우 다음과 같이 저장된 것을 확인할 수 있습니다.

```

1 ,온도,습도,강수량,불쾌지수
2 초여름,32,68,220,72
3 늦봄,28,30,0,12
4 한여름,38,81,0,91

```

그림 88 weather.csv

스프레드시트, 엑셀 등 csv 뷰어 프로그램으로 열었을 경우 다음과 같이 저장된 것을 확인할 수 있습니다.

	column 1	column 2	column 3	column 4	column 5
1		온도	습도	강수량	불쾌지수
2	초여름	32	68	220	72
3	늦봄	28	30	0	12
4	한여름	38	81	0	91

그림 89 weather.csv

내용정리

- `DataFrame.loc[x, y]`: x 행 y 열에 있는 값을 반환
- `DataFrame[Condition]`: 조건 Condition 에 참인 값들을 반환
- `DataFrame.T`: 데이터 프레임의 행과 열을 반전하여 반환
- `DataFrame.append(DataFrame)`: 새로운 데이터 프레임을 추가
- `DataFrame.insert(i, name, values)`: name 이름으로 values 를 i 번째 열에 삽입
- `DataFrame.drop(name, axis=0|1)`: 데이터 프레임에서 name 행 또는 열을 삭제
- axis 가 0 이면 행, 1 이면 열을 삭제
- `DataFrame.sort_index()`: index 기준 데이터 프레임 오름차순 정렬
- `DataFrame.sort_values(column)`: column 기준 데이터 프레임 오름차순 정렬
- `to_csv(name)` : 데이터 프레임을 CSV 파일로 저장
- `read_csv(name)` : CSV 파일을 데이터 프레임으로 로드



그림 90 Apple 1

1976

Steve Wozniak, Steve Jobs 가 제작한 Apple-1 은 애호가들을 위한 단일 보드 컴퓨터입니다. 이후 Apple Computer Inc 라는 이름으로 회사를 설립하였습니다. Apple 이 다음 모델을 발표하기 전까지 약 200 개의 보드가 판매되었습니다.



그림 91 IBM PC

1981

IBM 의 개인용 컴퓨터 (PC) 발표에 대한 대규모 마케팅으로 개인용 컴퓨터 시장의 빠른 성장을 촉진합니다. 공식적으로 IBM Model 5150 으로 알려진 최초의 IBM PC 는 4.77MHz Intel 8088 마이크로 프로세서를 기반으로 하며 Microsoft 의 MS-DOS 운영 체제를 사용했습니다. IBM PC 는 업계에서 가장 널리 채택 된 최초의 PC 가 됨으로써 비즈니스 컴퓨터에 대한 혁명을 일으켰습니다.

연습문제

문제 23

다음 코드를 읽고 출력을 작성해보세요.

```
01: import pandas as pd
02:
03: value = [[1, 2, 3, 5],
04:          [7, 11, 13, 17],
05:          [19, 23, 29, 31]]
06:
07: df = pd.DataFrame(value)
08:
09: print(df.loc[1][3])
```

문제 24

다음 코드를 읽고 출력을 작성해보세요.

```
01: import pandas as pd
02:
03: value = [[0, 2, 4, 8],
04:          [1, 3, 5, 9],
05:          [2, 4, 6, 10]]
06:
07: columns = ["A", "B", "C", "D"]
08: index = ["a", "b", "c"]
09:
10: df = pd.DataFrame(value, index=index, columns=columns)
11:
12: print(df.drop("C", 1))
```


문제 25

다음 코드를 읽고 데이터 프레임 df를 "C"열을 기준으로 정렬하여 출력하는 코드를 작성해보세요.

```
13: import pandas as pd
14:
15: value = [[5, 0, 7, 1],
16:          [3, 1, 9, 5],
17:          [0, 10, 2, 4]]
18:
19: columns = ["A", "B", "C", "D"]
20: index = ["a", "b", "c"]
21:
22: df = pd.DataFrame(value, index=index, columns=columns)
```

문제 26

아래의 코드는 Cds 센서의 밝기 값을 출력하는 코드입니다. 질문을 읽고 답해보세요.

```
01: import pandas as pd
02: from pop import Cds, delay
03:
04: cds = Cds(7)
05:
06: arr1, arr2, arr3 = [], [], []
07:
08: for i in range(3):
09:     arr1.append(cds.readAverage())
10:     delay(300)
11:     arr2.append(cds.readAverage())
12:     delay(300)
```

```
13:     arr3.append(cds.readAverage())
14:     delay(300)
15:
16: arr = [arr1, arr2, arr3]
17:
18: C = ["A", "B", "C"]
19: I = ["D", "E", "F"]
20:
21: df = pd.DataFrame(arr,index=I,columns=C)
22: print(df)
```

- A. 코드를 실행시켜 결과를 확인해 보세요.
- B. 빈 배열을 생성하고, Cds 센서값을 저장하여 데이터 프레임 df의 “G” 행에 추가해 보세요.
- C. “B” 열의 value가 1200 이하인 데이터 프레임 중 “C” 열을 출력해 보세요.
- D. B의 결과에서 첫 열과 끝 열에 0으로 채워진 열을 삽입해보세요.
- E. D의 데이터 프레임을 “data.csv” 이라는 이름의 CSV 파일로 저장하고 로드하여 출력해 보세요.

6.2.3. 통계와 연산

데이터 프레임에서 요약 통계를 내거나 기본적인 연산을 할 수 있도록 지원합니다. Pandas 에서 제공하는 통계 및 연산 메소드는 다음과 같습니다.

- `DataFrame.add(x)`: 데이터 프레임에 다른 데이터 프레임 `x`를 덧셈하여 반환
- `DataFrame.sub(x)`: 데이터 프레임에 다른 데이터 프레임 `x`를 뺄셈하여 반환
- `DataFrame.mul(x)`: 데이터 프레임에 다른 데이터 프레임 `x`를 곱셈하여 반환
- `DataFrame.div(x)`: 데이터 프레임에 다른 데이터 프레임 `x`를 나눗셈하여 반환
- `DataFrame.sum(axis)`: 데이터 프레임의 각 행 또는 열의 합을 반환
- `DataFrame.mean(axis)`: 데이터 프레임의 각 행 또는 열의 평균을 반환
- `DataFrame.std(axis)`: 데이터 프레임의 각 행 또는 열의 표준편차를 반환
- `DataFrame.var(axis)`: 데이터 프레임의 각 행 또는 열의 분산을 반환

6.2.3.1. 사칙 연산

데이터 프레임의 각 행과 열의 값을 사칙 연산으로 계산할 수 있는 메소드가 있습니다. 덧셈, 뺄셈, 곱셈, 나눗셈 순서대로 각각 `add(x)`, `sub(x)`, `mul(x)`, `div(x)`이며 연산에 사용되는 데이터 프레임의 형태가 맞지 않아 연산할 수 없는 부분은 `NaN`을 반환합니다.

```
01: import pandas as pd
02:
03: value = [[7, 5],
04:          [5, 1],
05:          [3, -1]]
06: df = pd.DataFrame(value)
07:
08: value2 = [[1, -3]]
09: df2 = pd.DataFrame(value2)
10:
11: print(df.add(df2))
```

```
12: print(df.sub(df2))
13: print(df.mul(df2))
14: print(df.div(df2))
```

[출력]

	0	1
0	8.0	2.0
1	NaN	NaN
2	NaN	NaN

	0	1
0	6.0	8.0
1	NaN	NaN
2	NaN	NaN

	0	1
0	7.0	-15.0
1	NaN	NaN
2	NaN	NaN

	0	1
0	7.0	-1.667
1	NaN	NaN
2	NaN	NaN

NaN은 다른 연산에서도 매우 큰 방해가 될 수 있습니다. 두 번째 파라미터로 `fill_value`를 입력하면 NaN이 나오는 것을 방지하고, 겹치지 않는 행 또는 열에 값을 지정해 연산할 수 있습니다.

```
15: print(df.add(df2, fill_value=0))
16: print(df.sub(df2, fill_value=0))
```

```
17: print(df.mul(df2, fill_value=1))
18: print(df.div(df2, fill_value=1))
```

[출력]

	0	1
0	8.0	2.0
1	5.0	1.0
2	3.0	-1.0

	0	1
0	6.0	8.0
1	5.0	1.0
2	3.0	-1.0

	0	1
0	7.0	-15.0
1	5.0	1.0
2	3.0	-1.0

	0	1
0	7.0	-1.667
1	5.0	1.0
2	3.0	-1.0

6.2.3.2. 통계

데이터 프레임의 각 행과 열의 값으로 기본적인 통계를 낼 수 있는 메소드가 있습니다. 많은 통계 메소드가 있으나 대표적으로 자주 사용되는 메소드 `sum(axis)`, `mean(axis)`, `std(axis)`, `var(axis)`를 소개합니다. `axis`가 0 이면 각 열에 대한 통계 연산 결과를 반환합니다.

```
01: import pandas as pd
02:
03: value = [[7, 5],
04:          [5, 1],
05:          [3, -1]]
06: df = pd.DataFrame(value)
07:
08: print(df.sum(0))
09: print(df.mean(0))
10: print(df.std(0))
11: print(df.var(0))
```

[출력]

```
0      15
1       5
```

```
0      5.000
1      1.667
```

```
0      2.000
1      3.055
```

```
0      4.000
1      9.333
```

또, 각 행에 대한 통계치를 구하고 싶다면 axis 를 1 로 입력합니다.

```
print(df.sum(1))
print(df.mean(1))
print(df.std(1))
print(df.var(1))
```

[출력]

0	12
1	6
2	2

0	6.0
1	3.0
2	1.0

0	1.414
1	2.828
2	2.828

0	2.0
1	8.0
2	8.0

내용정리

- `DataFrame.add(x)`: 데이터 프레임에 다른 데이터 프레임 `x`를 덧셈하여 반환
- `DataFrame.sub(x)`: 데이터 프레임에 다른 데이터 프레임 `x`를 뺄셈하여 반환
- `DataFrame.mul(x)`: 데이터 프레임에 다른 데이터 프레임 `x`를 곱셈하여 반환
- `DataFrame.div(x)`: 데이터 프레임에 다른 데이터 프레임 `x`를 나눗셈하여 반환
- `DataFrame.sum(axis)`: 데이터 프레임의 각 행 또는 열의 합을 반환
- `DataFrame.mean(axis)`: 데이터 프레임의 각 행 또는 열의 평균을 반환
- `DataFrame.std(axis)`: 데이터 프레임의 각 행 또는 열의 표준편차를 반환
- `DataFrame.var(axis)`: 데이터 프레임의 각 행 또는 열의 분산을 반환



그림 92 Minitel

1981

1981 년 France Telecom 은 모든 전화 가입자에게 무료 Minitel 단말기를 제공하여 최초의 "웹"을 시작했습니다. Minitel 은 1990 년까지 수천만 명의 사용자와 신문, 기차표, 세금 신고, 이메일 및 채팅과 같은 온라인 서비스를 제공 했습니다. 고객은 전화 요금으로 청구되는 Minitel 서비스 에 대한 액세스 요금을 분 단위로 지불했습니다.

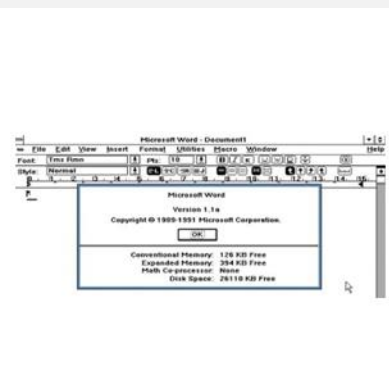


그림 93 Microsoft Word

1983

Microsoft 가 Word(Multi-Tool Word)를 발표했습니다. Microsoft 는 마케팅적으로 PC World 잡지 11 월호에 Word 프로그램의 데모 버전이 포함 된 45 만 개의 디스크를 배포 하여 독자들에게 무료로 프로그램을 시험해볼 수있는 기회를 제공했습니다. 워드 프로세서 프로그램으로서 시장 점유율을 위해 WordPerfect 와 경쟁했으며, 1989 년에 Windows 용 Microsoft Word 가 도입되어 글로벌 표준이되었습니다.

연습문제

문제 27

다음 코드를 읽고 출력을 작성해보세요.

```
01: import pandas as pd
02:
03: value = [[5, -5],
04:          [-5, 5],
05:          [5, -5]]
06: df = pd.DataFrame(value)
07:
08: value2 = [[-5, 5]]
09: df2 = pd.DataFrame(value2)
10:
11: print(df.add(df2))
```

문제 28

다음 코드를 읽고 출력을 작성해보세요.

```
01: import pandas as pd
02:
03: value = [[5, -5],
04:          [-5, 5],
05:          [5, -5]]
06: df = pd.DataFrame(value)
07:
08: print(df.mean(0))
```

문제 29

다음 코드와 출력을 읽고 틀린 부분을 고쳐보세요.

```
01: import pandas as pd
02:
03: value = [[5, -5],
04:          [-5, 5],
05:          [5, -5]]
06: df = pd.DataFrame(value)
07:
08: value2 = [[-5, 5]]
09: df2 = pd.DataFrame(value2)
10:
11: print(df.mul(df2))
```

[출력]

	0	1
0	-25.0	-25.0
1	-10.0	10.0
2	10.0	-10.0

문제 30

다음 코드는 Cds 센서 밝기 값을 출력하는 코드입니다. 질문을 읽고 답해보세요.

```
01: from pop import Cds, delay
02: import pandas as pd
03:
04: cds = Cds(7)
05:
06: arr1, arr2 = [], []
07:
08: for i in range(3):
09:     arr1.append(cds.readAverage())
```

```
10:     delay(300)
11:     arr2.append(cds.readAverage())
12:     delay(300)
13:
14: arr = [arr1, arr2]
15:
16: df = pd.DataFrame(arr)
17: print(df)
```

- A. 코드를 실행시켜 결과를 확인해 보세요.
- B. 새로운 데이터 프레임을 생성하고, arr1 을 데이터 프레임으로 저장해 보세요.
- C. B 의 데이터 프레임에서 df 를 덧셈, 뺄셈, 곱하기, 나누기하여 NaN 없이 출력하세요.
- D. 데이터프레임 df 에 대하여 B 의 데이터 프레임의 합계, 평균, 표준편차, 분산을 출력하세요.

6.3. Matplotlib

Matplotlib(맷플롯 라이브러리)는 파이썬 기본 데이터 형식과 Numpy, Pandas 데이터를 알아보기 쉽게 그래프로 시각화할 수 있는 패키지입니다. 인공지능 학습을 위해 수집한 데이터를 시각적으로 확인해보고 데이터 분석, 전처리를 쉽게 할 수 있도록 합니다.

Matplotlib 는 다음 명령으로 설치하는데, Soda OS 에는 미리 설치되어 있습니다.

- `pip3 install matplotlib`

Matplotlib 는 그래프를 표현하기 위해 GUI 환경을 사용합니다. GUI 환경에서 그래프를 표현하기 위한 공간에 대한 객체가 Subplot 이고, 하나 이상의 Subplot 을 Window 에 표현하는 객체가 Figure 입니다.

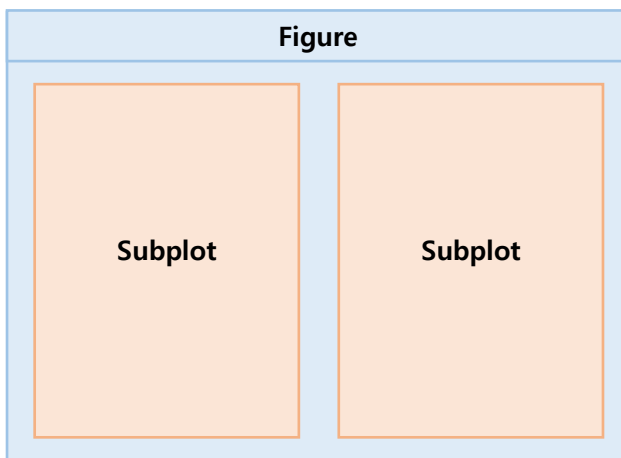


그림 94 Figure 와 Subplot

Subplot 에서는 대표적으로 선형 그래프, 막대그래프, 히스토그램, 산점도를 표현할 수 있습니다.

6.3.1. 그래프

Matplotlib의 pyplot은 선형 그래프, 막대그래프, 히스토그램, 산점도 등을 표현할 수 있습니다. 그래프에 사용되는 주요 메소드와 클래스는 다음과 같습니다.

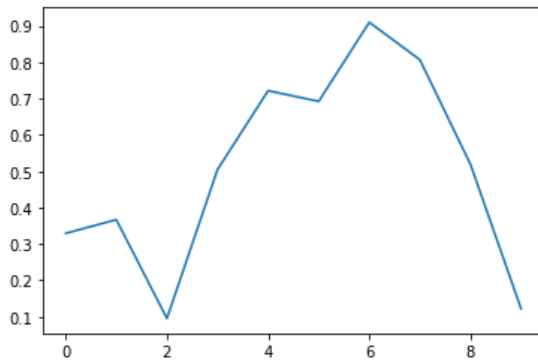
- plot(list): 지점들을 선으로 이은 그래프를 생성
- bar(index, list): 각 index 별 값을 세로 막대로 표현한 그래프를 생성
- barh(index, list): 각 index 별 값을 가로 막대로 표현한 그래프를 생성
- pie(list): 각 값을 원판으로 표현한 그래프를 생성
- hist(list): 값들의 분포를 표현한 그래프를 생성
- scatter(list_x, list_y): x, y 좌표에 점을 찍어 표현한 그래프 생성

6.3.1.1. 선형 그래프

선형 그래프는 가장 기본적인 형태로 1 차원 배열 값을 순서대로 나열해 표현합니다. Pyplot의 plot 메소드를 사용하며 plot(list) 형식으로 사용합니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(10)
04: plt.plot(data)
05: plt.show()
```

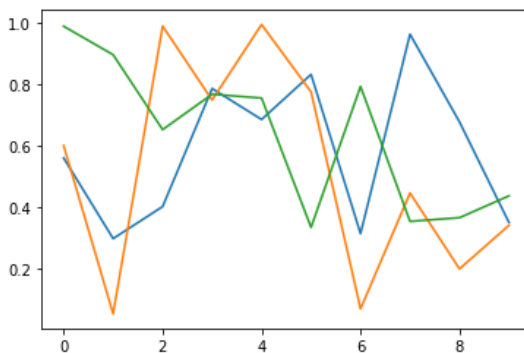
[출력]



plot 메소드에 2 차원 배열을 파라미터로 입력할 경우 하나의 Subplot 에 여러 개의 선형 그래프를 겹쳐 표현할 수도 있습니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(30).reshape(10,3)
04: plt.plot(data)
05: plt.show()
```

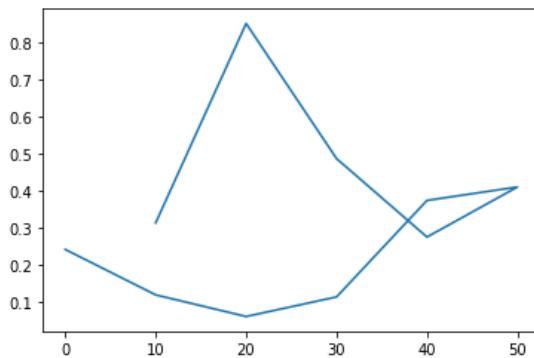
[출력]



plot 메소드에 2 번째 파라미터를 입력할 경우 X 축과 Y 축 모두 직접 표현할 수 있습니다. 단, 축을 순서 없이 지정할 수 있기에 선이 되돌아가는 형태로 만들 수도 있습니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(10)
04: plt.plot([0,10,20,30,40,50,40,30,20,10], data)
05: plt.show()
```

[출력]



이러한 특성을 응용하면 다음과 같은 그래프도 만들어 볼 수 있습니다.

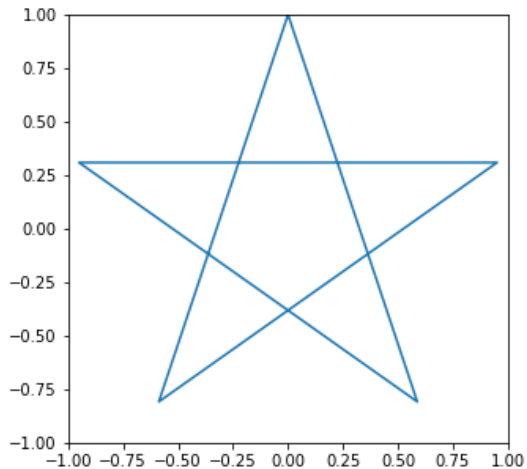
```
06: points=[np.deg2rad(90),
07:          np.deg2rad(18),
08:          np.deg2rad(306),
09:          np.deg2rad(234),
10:          np.deg2rad(162)]
11:
12: x=[np.cos(points[0]),
13:    np.cos(points[3]),
14:    np.cos(points[1]),
15:    np.cos(points[4]),
16:    np.cos(points[2]),
17:    np.cos(points[0])]
18:
19: y=[np.sin(points[0]),
```

```

20: np.sin(points[3]),
21: np.sin(points[1]),
22: np.sin(points[4]),
23: np.sin(points[2]),
24: np.sin(points[0])]
25:
26: plt.ylim(-1,1)
27: plt.xlim(-1,1)
28: plt.rcParams["figure.figsize"] = (5,5)
29: plt.plot(x,y)
30: plt.show()

```

[출력]

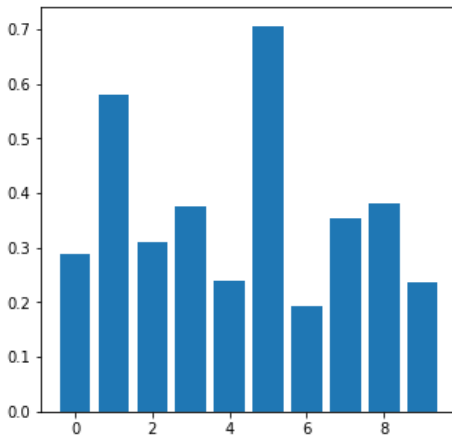


6.3.1.2. 세로형 막대그래프

막대그래프는 데이터를 비교하기 쉬운 형태로 1 차원 배열 값을 나열해 표현합니다. Pyplot 의 bar 메소드를 사용하며 bar(x, y) 형식으로 사용합니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(10)
04: plt.bar(np.arange(10), data)
05: plt.show()
```

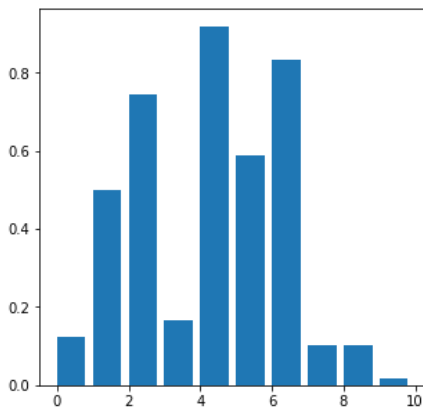
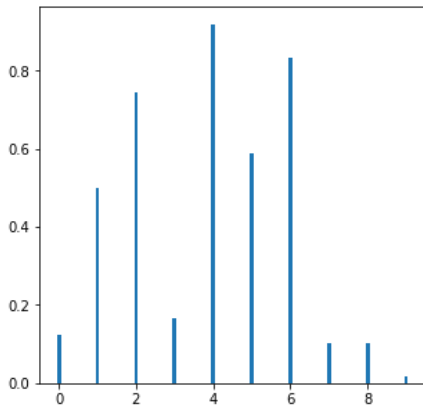
[출력]



bar 메소드에 추가 파라미터로 width 또는 align 입력할 경우 막대의 두께를 조절하거나 값의 모서리, 값의 중앙에 정렬해 표현할 수도 있습니다. align 파라미터에는 'center' 또는 'edge'만 사용할 수 있습니다.

```
06: plt.bar(np.arange(10), data, width=0.1)
07: plt.show()
08: plt.bar(np.arange(10), data, align='edge')
09: plt.show()
```

[출력]



여러 개의 데이터를 하나의 Subplot 에 표현하고자 할 때, color 파라미터를 사용해 색상으로 구분하고서 표현할 수 있습니다. 색상 명칭에 대한 표는 다음과 링크를 참고할 수 있습니다.

- https://github.com/hanback-docs/Matplot_color

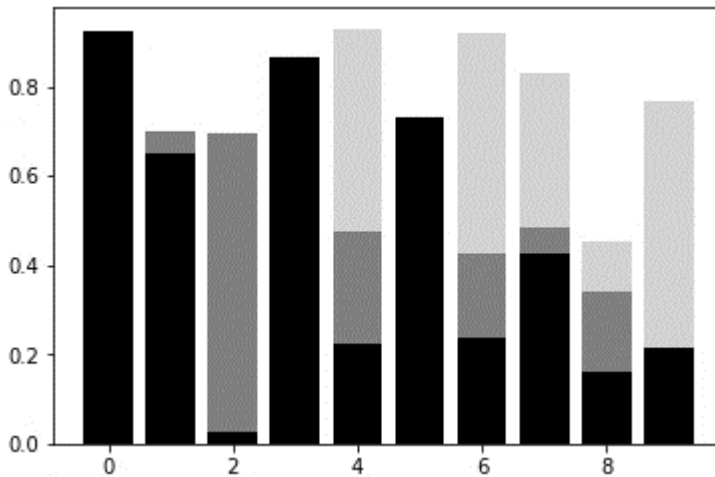


원하는 색을 color 파라미터로 입력해 막대그래프를 생성하면 다음과 같이 생성됩니다.

```
01: import matplotlib.pyplot as plt
```

```
02:
03: data=np.random.random(30).reshape(3,10)
04: plt.bar(np.arange(10), data[0], color='lightgray')
05: plt.bar(np.arange(10), data[1], color='gray')
06: plt.bar(np.arange(10), data[2], color='black')
07: plt.show()
```

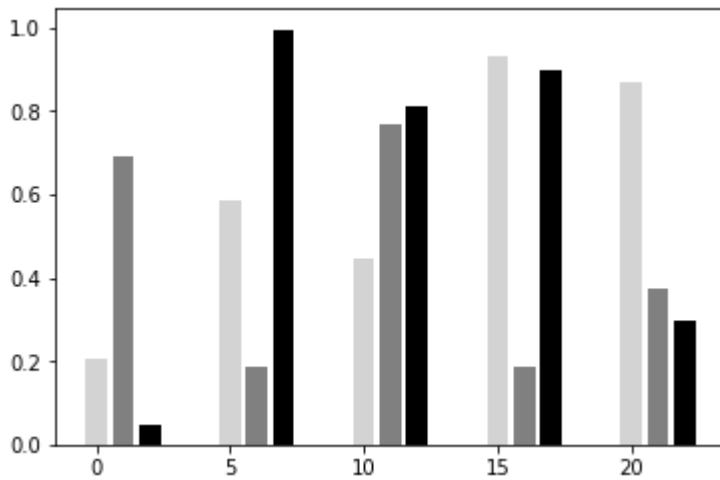
[출력]



만약 막대그래프가 겹치지 않게 나열하고 싶다면 x 값을 조절해 표현할 수 있습니다.

```
08: data=np.random.random(15).reshape(3,5)
09: plt.bar(np.arange(0,25,5), data[0], color='lightgray')
10: plt.bar(np.arange(0,25,5)+1, data[1], color='gray')
11: plt.bar(np.arange(0,25,5)+2, data[2], color='black')
12: plt.show()
```

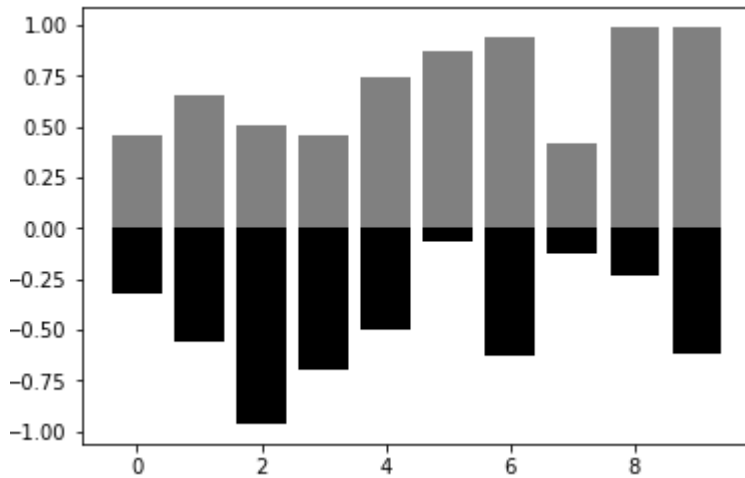
[출력]



막대그래프에서 color 파라미터를 응용해 Back to Back 그래프를 생성할 수 있습니다. 다음과 같이 x 값을 반전시키고 color 파라미터를 입력합니다.

```
01: import matplotlib.pyplot as plt
02: import numpy as np
03:
04: data=np.random.random(20).reshape(2,10)
05: plt.bar(np.arange(10), data[0], color='gray')
06: plt.bar(np.arange(10), -data[1], color='black')
07: plt.show()
```

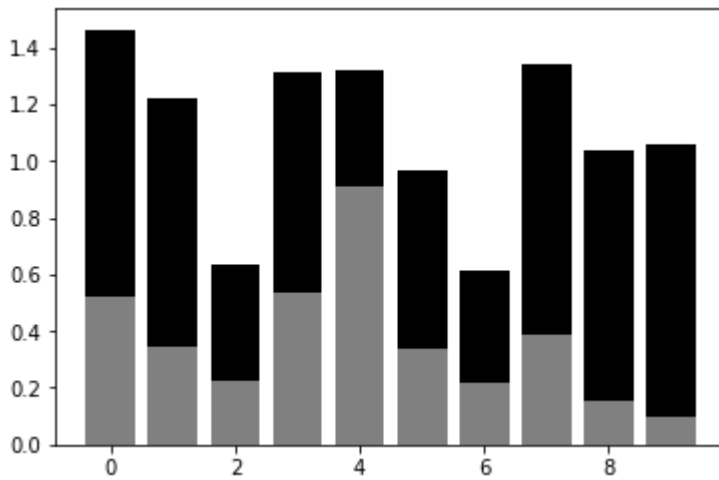
[출력]



막대 위에 막대를 쌓는 스택 바 그래프는 `bottom` 파라미터를 이용해 구현할 수 있습니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(20).reshape(2,10)
04: plt.bar(np.arange(10), data[0], color='gray')
05: plt.bar(np.arange(10), data[1], color='black', bottom=data[0])
06: plt.show()
```

[출력]

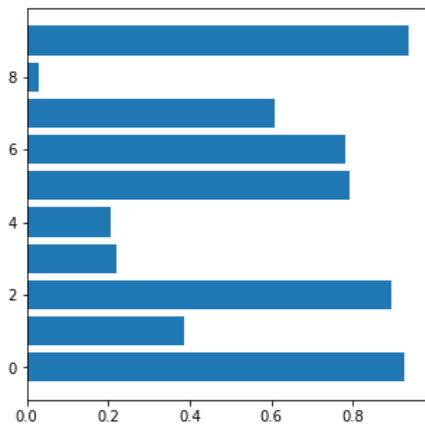


6.3.1.3. 가로형 막대그래프

Matplotlib 는 세로형 막대그래프뿐만 아니라 가로형 막대그래프 역시 지원합니다. Pyplot 의 `barh` 메소드를 사용하며 `barh(x, y)` 형식으로 사용합니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(10)
04: plt.barh(np.arange(10), data)
05: plt.show()
```

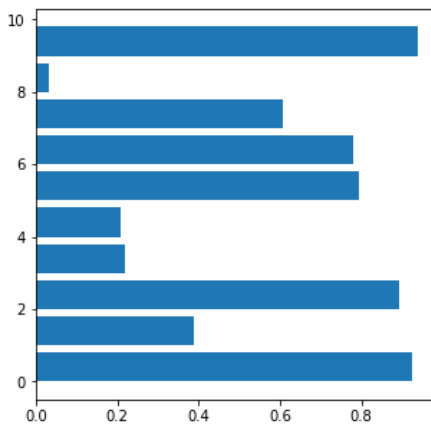
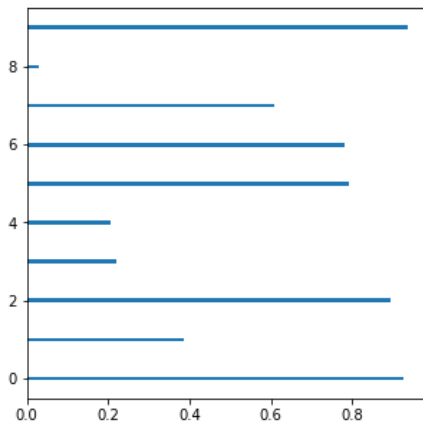
[출력]



가로형 막대그래프에 추가 파라미터로 height 또는 align 입력할 경우 막대의 두께를 조절하거나 값의 모서리, 값의 중앙에 정렬해 표현할 수도 있습니다. align 파라미터에는 'center' 또는 'edge'만 사용할 수 있습니다.

```
01: plt.barh(np.arange(10), data, height=0.1)
02: plt.show()
03: plt.barh(np.arange(10), data, align='edge')
04: plt.show()
```

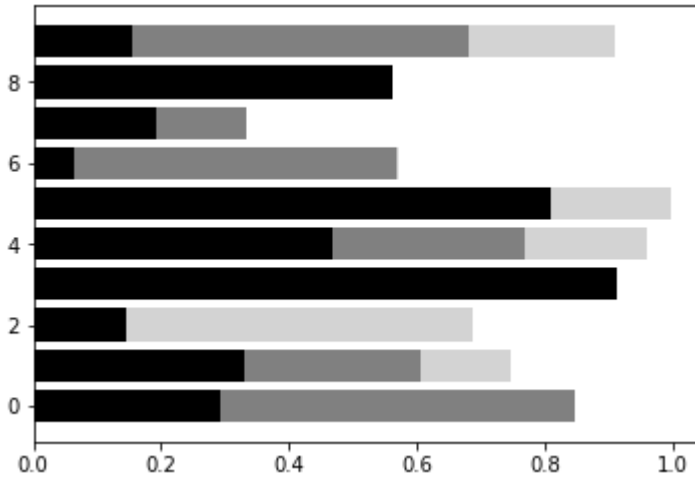
[출력]



가로형 막대그래프에서 color 파라미터로 입력해 생성하면 다음과 같이 생성됩니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(30).reshape(3,10)
04: plt.barh(np.arange(10), data[0], color='lightgray')
05: plt.barh(np.arange(10), data[1], color='gray')
06: plt.barh(np.arange(10), data[2], color='black')
07: plt.show()
```

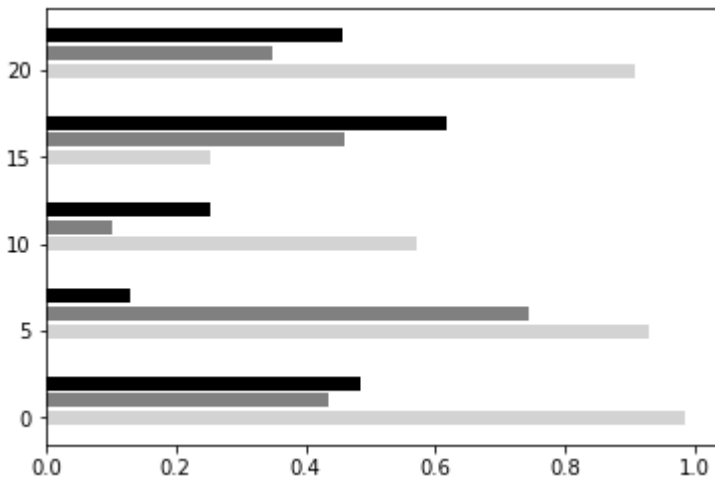
[출력]



만약 막대그래프가 겹치지 않게 나열하고 싶다면 x 값을 조절해 표현할 수 있습니다.

```
01: data=np.random.random(15).reshape(3,5)
02: plt.barh(np.arange(0,25,5), data[0], color='lightgray')
03: plt.barh(np.arange(0,25,5)+1, data[1], color='gray')
04: plt.barh(np.arange(0,25,5)+2, data[2], color='black')
05: plt.show()
```

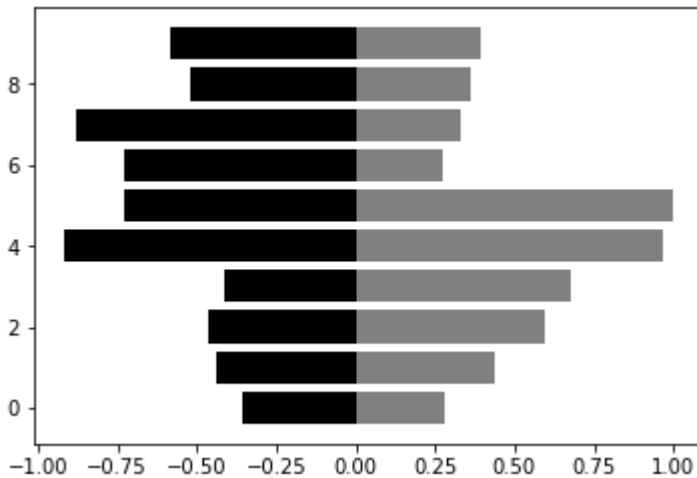
[출력]



가로 막대 그래프에서 color 파라미터를 응용해 Back to Back 그래프를 생성할 수 있습니다. 다음과 같이 x 값을 반전시키고 color 파라미터를 입력합니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(20).reshape(2,10)
04: plt.barh(np.arange(10), data[0], color='gray')
05: plt.barh(np.arange(10), -data[1], color='black')
06: plt.show()
```

[출력]

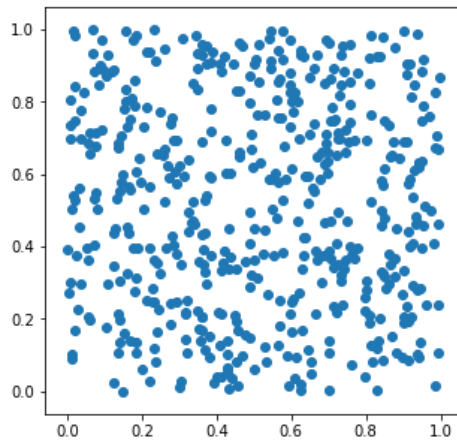


6.3.1.4. 산점도 그래프

산점도 그래프는 값을 점으로 표현해 데이터 분포를 파악하기 쉬운 그래프입니다. Pyplot의 scatter 메소드를 사용하며 scatter(x, y) 형식으로 사용합니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(1000).reshape(2,500)
04: plt.scatter(data[0], data[1])
05: plt.show()
```

[출력]

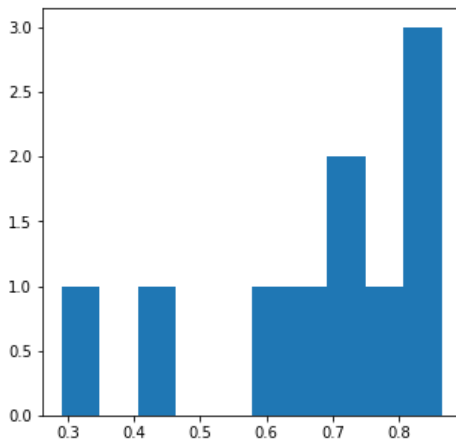


6.3.1.5. 히스토그램

히스토그램은 값의 분포를 막대 그래프로 표현해 구간별 확률분포나 밀도를 비교하기 좋은 그래프입니다. Matplotlib 에서 지원하는 `hist(list)` 메소드를 이용해 구현할 수 있습니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(10)
04: plt.show()
```

[출력]

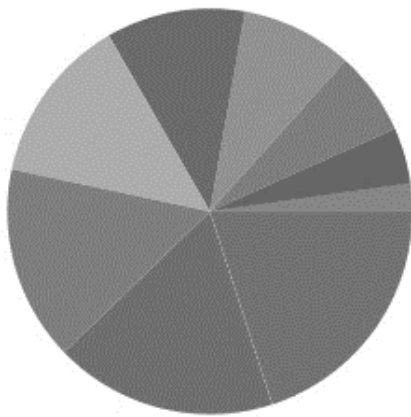


6.3.1.6. 원판 그래프

각 값의 비율을 한눈에 비교하기 좋은 원판 또는 파이 그래프는 `pie(list)` 메소드를 이용해 구현할 수 있습니다. 입력되는 `list`의 값들은 100 개까지 표현할 수 있습니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(10)
04: plt.pie(np.arange(10))
05: plt.show()
```

[출력]



내용정리

- Matplotlib : 데이터를 그래프로 시각화할 수 있는 패키지
- `plot(list)`: list 의 값들을 선으로 이은 그래프를 생성
- `bar(index, list)`: 각 index 별 값을 세로 막대로 표현한 그래프를 생성
- `barh(index, list)`: 각 index 별 값을 가로 막대로 표현한 그래프를 생성
- `pie(list)`: 각 값을 원판으로 표현한 그래프를 생성
- `hist(list)`: 값들의 분포를 표현한 그래프를 생성
- `scatter(list_x, list_y)`: x, y 좌표에 점을 찍어 표현한 그래프 생성



그림 95 NES 게임콘솔

1985

1983 년 이래로 미국의 전자 게임 시장은 성능이 떨어지는 게임과 콘솔로 인해 침체기를 맞았습니다. 같은 해 Nintendo 는 일본에서 Famicom 게임 시스템을 출시했습니다. 1985 년 북미에서 Nintendo Entertainment System 으로 출시된 NES 는 미국 게임 산업의 운명을 뒤집기 시작했습니다. 이 시스템은 18 개의 타이틀로 시작했으며 배관공 마리오를 비디오 게임의 역사적인 캐릭터로 만드는 역할을 했습니다.



그림 96 Robert T. Morris

1988

23 세의 Robert T. Morris 는 인터넷을 통해 비파괴 웜을 전송하여 네트워크에 연결된 60,000 개의 호스트 중 약 6,000 명에게 며칠 동안 중대한 문제를 일으켜 광범위한 정전이 발생합니다. 이것은 실제 컴퓨터 시스템에 가장 큰 영향을 미치고 네트워크 보안의 중요성을 알리는 최초의 웜입니다. Morris 는 “컴퓨터 사기 및 학대 행위”에 따라 유죄 판결을 받은 첫 번째 사람입니다.

연습문제

문제 31

다음 데이터를 선형 그래프로 출력하는 코드를 작성해보세요.

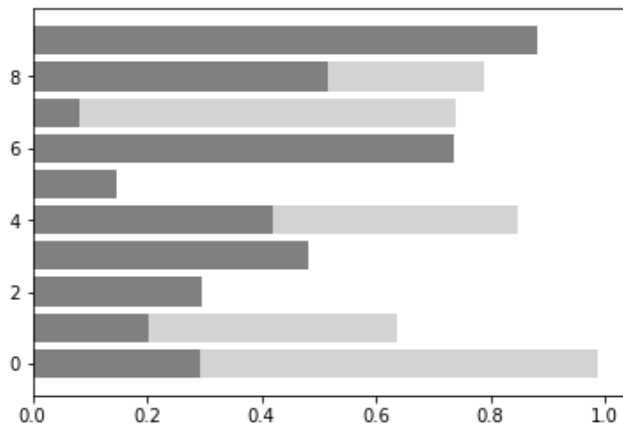
번호	1	2	3	4	5	6	7
값	10	15	13	17	18	15	21

문제 32

다음 코드와 출력을 읽고 틀린 부분을 고쳐보세요.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(20).reshape(2,10)
04:
05: plt.bar(np.arange(10), data[0], color='lightgray')
06: plt.bar(np.arange(10), data[1], color='gray')
07: plt.show()
```

[출력]



문제 33

다음 코드는 Cds 센서의 밝기 값을 그래프로 출력하는 코드입니다. 질문을 읽고 답해보세요.

```

01: from pop import Cds,delay
02: import numpy as np
03: import matplotlib.pyplot as plt
04:
05: cds = Cds(7)
06:
07: arr = [ ]
08:
09: for i in range(100):
10:     arr.append(cds.readAverage())
11:     delay(50)
12:
13: arr = np.array(arr)
14:
15: plt.plot(arr)
16: plt.show()

```

- A. 코드를 실행시켜 결과를 확인해보세요.
- B. 세로형 막대그래프로 출력하는 코드를 작성해 보세요.
- C. 산점도그래프로 출력하는 코드를 작성해 보세요.
- D. 히스토그램으로 출력하는 코드를 작성해 보세요.

6.3.2. 추가적인 표현

Matplotlib 의 메소드로 X 또는 Y 축의 라벨이나 그래프의 범례를 표현할 수 있습니다. 이를 위해 사용되는 주요 메소드는 다음과 같습니다.

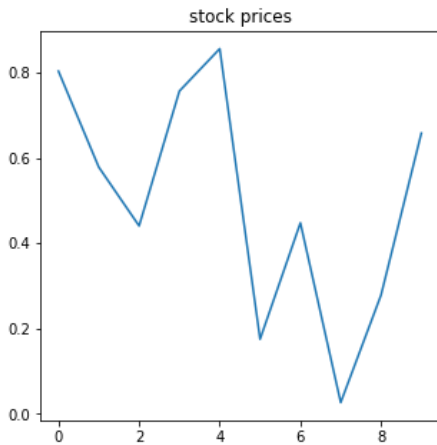
- title(name): 그래프 title 생성
- xlabel(name): 그래프 x label 생성
- ylabel(name): 그래프 y label 생성
- legend(list): 그래프 범례 생성

6.3.2.1. 그래프 제목

그래프의 주제를 가장 잘 파악할 수 있는 제목을 title 메소드를 이용해 표현할 수 있습니다. 다음과 같이 선형 그래프를 생성하고 title 을 표시합니다. 범례

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(10)
04: plt.title("stock prices")
05: plt.plot(data)
06: plt.show()
```

[출력]

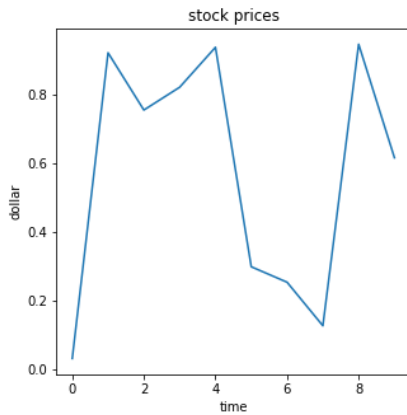


6.3.2.2. 축 라벨

그래프의 값이 의미하는 바를 잘 파악할 수 있는 축 라벨을 xlabel 또는 ylabel 메소드를 이용해 표현할 수 있습니다. 다음과 같이 선형 그래프를 생성하고 x 축과 y 축 라벨을 표시합니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(10)
04: plt.title("stock prices")
05: plt.xlabel("time")
06: plt.ylabel("dollar")
07: plt.plot(data)
08: plt.show()
```

[출력]

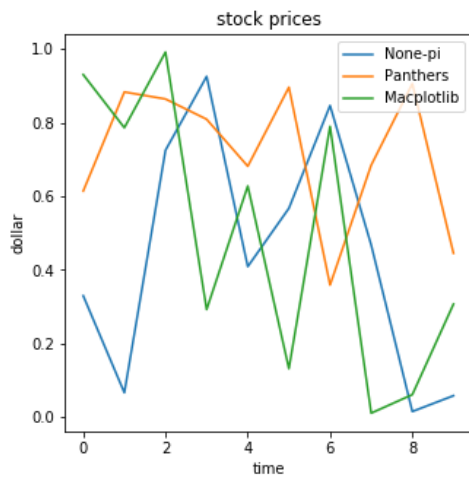


6.3.2.3. 범례

여러 그래프 사이에서 각 그래프를 구분할 수 있는 범례를 legend 메소드를 이용해 표현할 수 있습니다. 다음과 같이 여러 개의 선형 그래프를 생성하고 범례를 표시합니다.

```
01: import matplotlib.pyplot as plt
02:
03: data=np.random.random(30).reshape(10,3)
04: plt.title("stock prices")
05: plt.xlabel("time")
06: plt.ylabel("dollar")
07: plt.plot(data)
08: plt.legend(["None-pi", "Panthers", "Macplotlib"])
09: plt.show()
```

[출력]



내용정리

- Matplotlib 의 메소드로 X 또는 Y 축의 라벨이나 그래프의 범례를 표현 가능
- title(name): 그래프 title 생성
- xlabel(name): 그래프 x 축 라벨 생성
- ylabel(name): 그래프 y 축 라벨 생성
- legend(list): 그래프 범례 생성



그림 97 Windows 3.0

1990

Microsoft 는 DOS 프로그램과 호환되는 최초의 성공적인 Windows 3.0 버전을 출시하여 마침내 PC 사용자를 만족시키는 데 충분한 성능을 제공했습니다. 이 버전의 경우 Microsoft 는 인터페이스를 업데이트하여 PC 에서 처음으로 그래픽 응용 프로그램을 지원할 수 있는 디자인을 만들었습니다. 또한 여러 프로그램을 Intel 80386 마이크로 프로세서에서 동시에 실행 할 수 있었습니다. 결과적으로 PC 의 대중화가 촉진되었습니다.



그림 98 Linus Torvalds

1988

핀란드 대학생 Linus Torvalds 가 설계 한 Linux 커널이 출시됩니다. 사용자들은 주변 장치에 대한 지원을 추가하고 안정성을 향상시키는 등 개발 및 개선을 시작했습니다. 리눅스는 또한 GNU 운영 체제의 일부 요소를 통합했으며 오늘날 스마트 폰에서 슈퍼 컴퓨터에 이르는 장치에서 사용됩니다.

연습문제

문제 34

아래의 코드는 Cds 센서의 밝기 값을 그래프로 출력하는 코드입니다.

```
01: from pop import Cds,delay
02: import numpy as np
03: import matplotlib.pyplot as plt
04:
05: cds = Cds(7)
06:
07: arr = []
08:
09: for i in range(100):
10:     arr.append(cds.readAverage())
11:     delay(50)
12:
13: arr = np.array(arr).reshape(50,2)
14:
15: plt.plot(arr)
16: plt.show()
```

A. 코드를 실행시켜 결과를 확인해 보세요.

B. 아래와 같은 표현을 추가해 보세요.

그래프 제목	CdS Data
x 축 라벨	Count
y 축 라벨	Lux
첫 번째 범례	First
두 번째 범례	Second

7. 인공지능

인공지능은 지적 능력 수준에 따라 약인공지능 Weak AI, 강인공지능 Strong AI, 초인공지능 Super AI 으로 분류할 수 있습니다.

약인공지능 Weak AI 은 특정 분야에서만 인간보다 우수한 능력을 보이는 인공지능으로 지각, 추론, 행동과 같은 논리적 사고만 가능한 수준입니다. 대표적인 예로 바둑에 특화된 구글의 알파고 AlphaGo, 자연어* 상호작용에 특화된 IBM 의 왓슨 Watson 등이 있습니다.

- 자연어 : 인간이 사용하는 의사소통 언어. 영어, 한국어, 중국어 등.

강인공지능 Strong AI 은 모든 분야에서 인간 수준의 지적 능력을 갖추고 있으며 비대면 대화를 했을 때 인간과 구분이 힘든 수준의 인공지능입니다. 범용 인공지능이라고도 하며 대표적으로 영화 ‘터미네이터’의 스카이넷, 영화 ‘아이언맨’의 자비스가 있습니다.

초인공지능 Super AI 은 모든 분야에서 인간을 초월한 수준의 인공지능입니다. 강인공지능이 스스로 이 단계로 진행하리라 예측합니다. 인간이 인공지능을 이해할 수 없는 상황이 발생하고 이에 따라 인간이 인공지능으로 대체되거나 멸종할 가능성도 있다고 추측되고 있습니다.

현재는 약인공지능 수준에 머물러있으며 서포트 벡터 머신 SVM, 군집화 Clustering, 의사결정 트리 Decision Tree, 인공신경망 ANN 등으로 구현할 수 있습니다. 이러한 구현 방법들을 머신러닝 Machine Learning 이라고 합니다.

앞으로 설명할 머신러닝 예제에서는 Pop.AI 라이브러리를 사용합니다. Keras, Tensorflow 등 새로운 환경에 대해 익힐 시간을 절약하고 쉬운 이해와 빠른 이론 적용을 위해 한백전자에서 제작된 라이브러리입니다. 복잡한 개념과 어려운 로직을 구현할 필요 없이 6 줄 내외로 여러 이론을 직접 구현해볼 수 있도록 구성하였습니다. 또, 이 파트의 마지막 예제 실습으로 직접 Tensorflow 와 Keras 를 이용해 선형 회귀, 심층신경망 등

을 구현해볼 수 있습니다.

7.1. 머신러닝

머신러닝은 컴퓨터 시스템이 주어진 데이터를 학습하는 과정을 의미합니다. 학습 유도 방식에 따라 지도 학습, 비지도 학습, 강화 학습으로 분류할 수 있습니다.

지도 학습 Supervised Learning 은 훈련 데이터로부터 학습 모델을 유도할 때 학습 결과가 어떻게 출력되어야 하는지 알려주고, 학습 모델이 출력한 결과와 비교하며 모델을 최적화하는 학습 방법입니다.

비지도 학습 Unsupervised Learning 은 훈련 데이터로부터 학습 모델을 유도할 때 주어진 목푼값 없이 학습 모델 스스로 각 데이터의 특징을 추론하고, 특징값의 편차에 따라 데이터를 군집화하는 학습 방법입니다.

강화 학습 Reinforcement Learning 은 보상이라는 개념을 이용합니다. 학습 모델이 이전 출력과 비교해 더 나은 결과를 출력할 때 보상을 주며 오차를 줄여나가는 학습 방법입니다.

7.1.1. 지도 학습

지도 학습은 짧은 학습 시간을 투자해 낮은 오차를 얻을 수 있다는 장점이 있습니다. 그러나 데이터양 대비 감소하는 오차가 적은 편이기에 학습의 한계가 빨리 나타나는 편이며 복잡한 학습 모델일수록 필요한 데이터양은 급격히 증가합니다. 또, 훈련에 필요한 데이터를 가공하는 과정이 필요하므로 학습 데이터가 단순하고 목푼값의 종류가 적은 문제에 적합한 방법입니다. 기본적인 지도 학습 기법으로는 분류와 회귀가 있습니다.

7.1.1.1. 분류(Classification)

입력과 처리 결과로 이루어진 훈련 데이터에서 분석할 수 있는 기준을 학습하고, 학습된 기준에 따라 새로운 데이터를 어떤 종류 Class 로 구분할지 선택하는 기법입니다. 자세하게 설명하자면, 훈련 데이터는 값과 클래스 또는 라벨로 이루어져 있으며 분류 모델은 값과 클래스를 학습해 새로운 값이 입력되면 어떤 클래스를 부여할지 선택합니다.

분류 기법에는 클래스의 개수에 따라 크게 이진 분류와 다중 분류로 구분됩니다.

이진 분류 Binary Classification 는 클래스가 True 와 False 로 이루어져 있는 모델로, 데이터를 두 가지로 분류합니다. 로지스틱 회귀를 이용해 구현하기도 합니다. 대표적으로 양수와 음수를 구분하는 분류 모델, 목소리를 남성과 여성을 구분하는 분류 모델 등이 있습니다.

다중 분류 Multi-class Classification 는 클래스가 여러 개로 이루어져 있는 모델로, 데이터를 여러 클래스로 분류합니다. 소프트맥스 회귀를 이용해 구현하기도 합니다. 대표적으로 체온의 정상 범위를 구분하는 분류 모델, 고양이 이미지를 품종별로 구분하는 분류 모델이 있습니다.

7.1.1.2. 회귀(Regression)

입력과 처리 결과로 이루어진 훈련 데이터로부터 두 값의 관계를 학습하고, 어떤 데이터가 새롭게 입력될지 예측하는 기법입니다. 예컨대 입력 x 와 처리 결과 y 가 훈련 데이터로 주어진다고 가정했을 때 두 값의 관계 함수 $y=f(x)$ 를 찾아내는 과정이 회귀의 학습 과정이고 추측한 관계 함수 $y=f(x)$ 를 이용해 새로운 입력과 결과를 예측하는 것이 회귀의 응용 과정입니다.

회귀 기법에는 결괏값의 범위와 클래스의 개수에 따라 선형 회귀와 로지스틱 회귀, 소프트맥스 회귀로 구분할 수 있습니다.

선형 회귀 Linear Regression 는 입력과 처리 결과를 선형적 관계로 모델링하며 관계식이 1 차원 방정식입니다. 그러므로 결과값의 범위는 $-\infty \sim \infty$ 입니다. 단순히 입력이 어떤 결과를 가지는지에 대한 관계식을 갖습니다. 대표적으로 함수 회귀 모델, 기후 예측 모델이 있습니다.

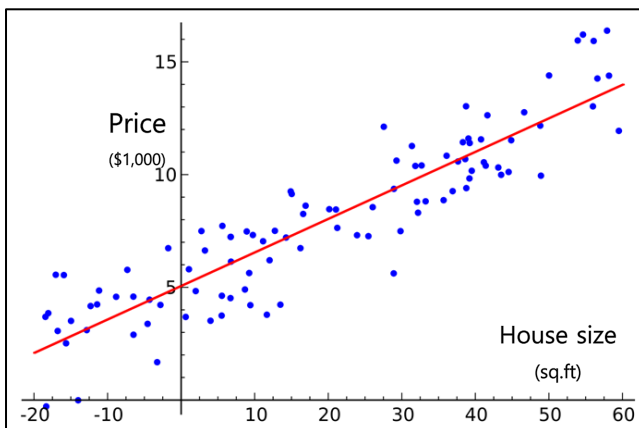


그림 99 집 면적 대비 가격에 대한 선형 회귀

로지스틱 회귀 Logistic Regression 는 입력과 처리 결과를 이항 확률 관계로 모델링하며 관계식의 결과는 2 개이며 값은 0 과 1 사이입니다. 입력이 True(1)와 False(0) 중 어느 것일 확률이 높은지에 대한 관계식을 갖습니다. 대표적으로 논리 회귀 모델, 시험 합격/불합격 예측 모델이 있습니다.

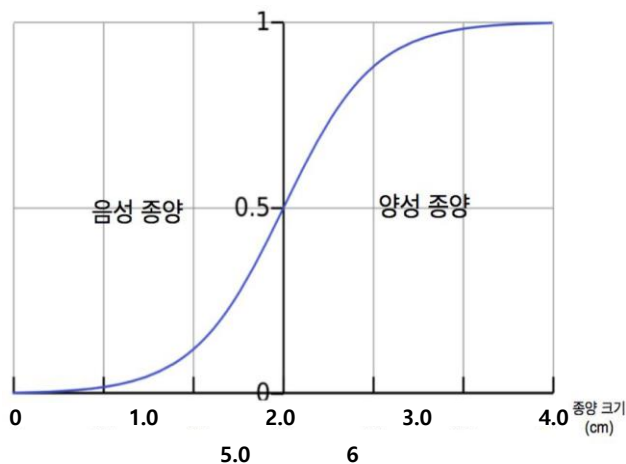


그림 100 종양의 크기에 따라 음성과 양성을 구분하는 로지스틱 회귀

소프트맥스 회귀 Softmax Regression 는 입력과 처리 결과를 다항 확률 관계로 모델링하며 관계식의 결과는 3 개 이상이며 값은 0 과 1 사이입니다. 입력이 3 개 이상의 클래스 중 각 클래스일 확률에 대한 관계식을 갖습니다. 대표적으로 품종 예측 모델이 있습니다.

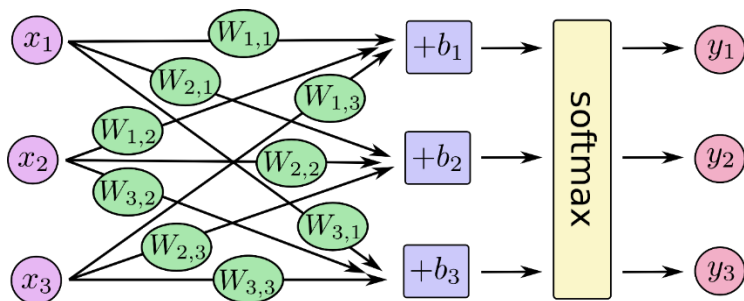


그림 101 소프트맥스 회귀

7.1.2. 비지도 학습

비지도 학습은 학습 초기 오차가 비교적 높지만, 시간이 지날수록 오차가 급격히 줄어들고 학습 데이터를 가공할 필요 없이 많은 학습을 진행할 수 있는 장점이 있습니다. 또, 데이터 특징을 스스로 찾아가기 때문에 설계자가 예측하지 못한 특징을 찾아낼 수도 있습니다. 그러나 학습 시간이 긴 편이고 목표가 분명하지 않기 때문에 원하지 않은 학습 모델이 나올 수 있습니다. 그러므로 주어진 학습 데이터가 비교적 복잡하거나 데이터 특징을 분석할 때 적합한 방법입니다. 기본적인 비지도 학습 기법으로는 군집화가 있습니다.

7.1.2.1. 군집화(Clustering)

입력만으로 이루어진 훈련데이터로부터 각 데이터가 갖는 값이 비슷한 것끼리 군집시키고 새로운 데이터에 대한 군집 기준을 만들어내는 기법입니다. 입력 x 가 주어졌을 때 군집 기준값과의 오차가 가장 낮은 군집에 소속시키고 군집 기준값을 조정합니다. 군집 기준값을 조정해나가며 최적의 값을 찾는 과정이 학습 과정이며 군집 기준값을 이용해 새로운 데이터의 군집을 결정하는 과정이 응용 과정입니다.

군집화 기법으로는 K 평균 군집화가 있습니다.

K-평균 군집화 K-means Clustering 는 군집의 개수 K 개의 군집을 데이터 밀도가 높은 곳에 군집화합니다. 입력과 관계없이 초깃값을 랜덤으로 정하기 때문에 원치 않는 결과가 나올 수 있으며 학습할 때마다 결과가 변합니다. 또, 군집마다 밀도 차이가 크거나 그 경계가 복잡하고 모호할수록 원하는 결과를 얻기 힘듭니다.

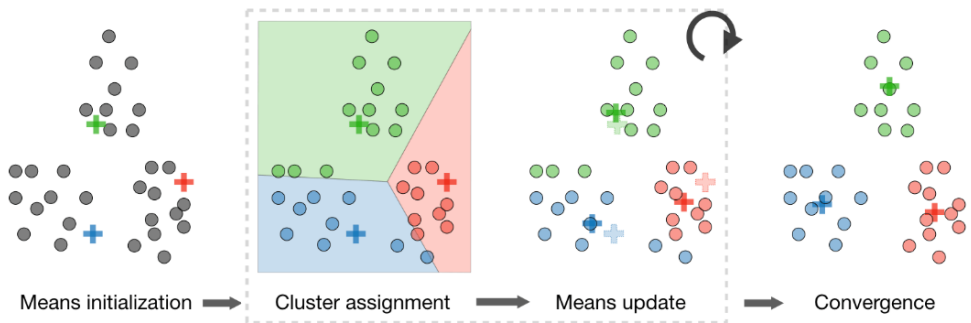


그림 102 K-평균 군집화

7.1.3. 머신러닝 기법

대표적인 머신러닝 기법은 앞서 언급된 선형 회귀, 로지스틱 회귀, 소프트맥스 회귀, K-평균 군집화 등이 있습니다. 이 기법들은 딥러닝의 기반이 되고 전처리 알고리즘으로 사용되기도 합니다.

7.1.3.1. 선형 회귀(Linear Regression)

선형 회귀는 입력에 대한 결과가 선형 관계를 갖습니다. 입력이 결과에 대해 직접적인 관계가 있을 때 선형 관계라고 합니다.

다음과 같이 키와 발 크기에 관한 데이터가 주어졌을 때 키에 따른 발 크기를 예측하기 위한 회귀 모델을 구하고자 합니다.

키(cm)	발 크기(mm)
173	270
171	275
162	245
187	280
157	230
169	265
177	270
159	250
182	275

표 1 데이터셋

주어진 데이터를 다음과 같이 그래프로 표현할 수 있습니다.

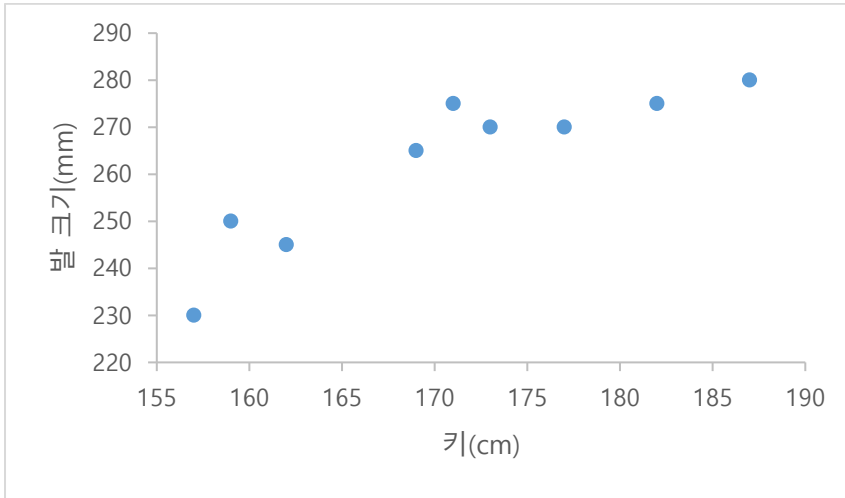


그림 103 데이터 그래프

이때 모든 데이터의 관계를 하나의 직선을 그어 표현할 때 다음과 같은 직선 $G(x)$ 를 그릴 수 있습니다. 이 직선 $G(x)$ 를 구하기 위해 선형 모델이 임의로 설정한 직선을 가설 Hypothesis 이라 하고 $H(x)$ 로 표현합니다. 선형 회귀 모델은 데이터와의 관계를 가장 잘 표현할 수 있는 가설 $H(x)$ 를 찾는 데, 이 과정을 최적화 Optimize 라고 합니다.

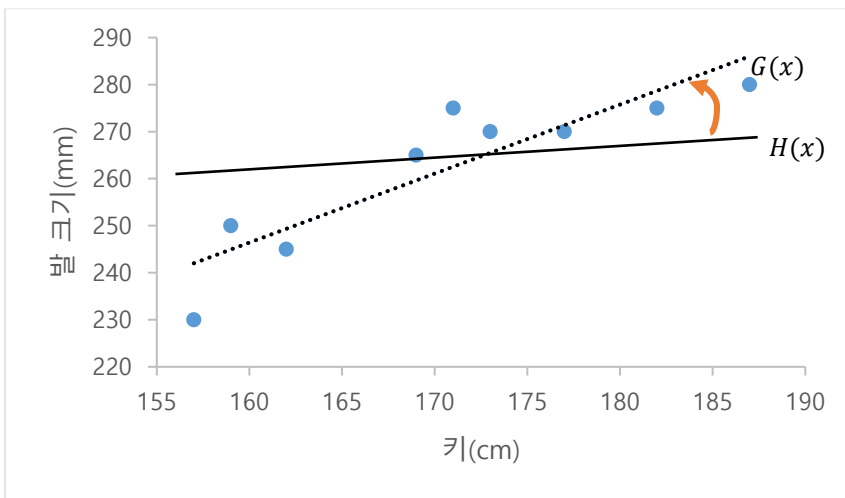


그림 104 데이터 그래프

성공적으로 가설 최적화를 끝낸 선형 회귀 모델은 새로운 키 데이터가 주어졌을 때 발 크기가 몇일지 선형적으로 예측할 수 있습니다.

키(cm)	발 크기(mm)
190	291
180	276
170	261
160	246
150	230

표 2 데이터셋

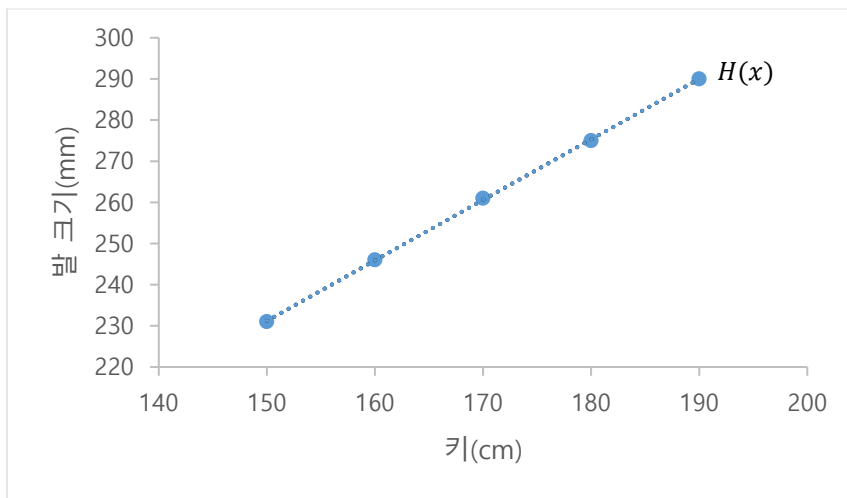


그림 105 예측 그래프

선형 회귀를 실습하는 과정은 Jupyter 환경에서 진행됩니다. Jupyter 환경에 대해서는 19 페이지를 참고할 수 있습니다. 키와 발 크기를 입력받아 선형 회귀하고 새로운 키 데이터를 입력받아 발 크기를 예측하는 모델을 실습합니다.

실습에 들어가기 앞서, Keras 와 Pop.AI 라이브러리를 비교해봅니다.

다음 소스코드는 Keras 를 이용해 구현한 선형 회귀입니다.

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(1,)))
06: model.add(keras.layers.Dense(1))
07:
08: model.compile(loss='MSE', optimizer='SGD')
09:
10: model.fit(X, Y, epochs=100)
```

또, 다음 소스코드는 Pop.AI 를 이용해 구현한 선형 회귀입니다.

```
01: from pop import AI
02:
03: LR = AI.Linear_Regression()
04:
05: LR.train()
06: LR.run()
```

Keras 는 범용성을 위해 만들어진 라이브러리로 선형 회귀를 구현하기 위해서는 학습 모델 설계부터 손실 함수, 최적화 함수 등 고려해야 할 것이 많습니다. 그러나 Pop.AI 라이브러리는 특정 모델을 쉽게 사용할 수 있도록 사전 설계되어 입문자가 빠르게 실습해볼 수 있도록 합니다. 단, Pop.AI 라이브러리는 Keras 를 기반으로 설계되었으며 Keras 를 이용한 학습 모델 설계, 손실 함수 등은 기본적인 개념을 익힌 뒤 자세히 다뤄보도록 합니다.

소스를 작성해 Pop.AI 라이브러리를 가져오고, Linear_Regression 객체를 LR 이라는 변수에 생성합니다.

Linear_Regression 객체의 파라미터에는 restore 와 ckpt_name 이 있습니다.

- restore: 최근 모델에 이어서 학습할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: False
- ckpt_name: 저장 및 불러올 모델 파일의 이름입니다.
 - 기본값: linear_regression

```
01: from pop import AI
02:
03: LR = AI.Linear_Regression()
```

Linear_Regression 객체의 속성으로 X_data 와 Y_data 가 있습니다. X_data 에는 입력 데이터들이 들어있어야 하고, Y_data 에는 입력에 대한 결과값 데이터들이 들어있어야 합니다. 입력할 데이터들은 앞서 설명에 이용한 데이터들을 활용합니다. 입력 리스트와 결과 리스트는 1 대 1 대응되어야 합니다.

```
04: LR.X_data = [[173],[171],[162],[187],[157],[169],[177],[159],[182]]
05: LR.Y_data = [[270],[275],[245],[280],[230],[265],[270],[250],[275]]
```

Linear_Regression 객체의 train() 메소드를 사용하면 회귀 학습을 시작할 수 있습니다. 파라미터에는 times 와 print_every 가 있고, 각각 학습할 횟수와 학습 상황을 몇 번째마다 출력할지를 의미합니다. 기본값은 100 과 10 으로 설정되어있습니다.

train 메소드를 실행하면 10 회마다 회귀 모델의 오차를 출력합니다.

06: LR.train()

[출력]

```
10 step loss: 33460.68
20 step loss: 27578.742
30 step loss: 22424.902
40 step loss: 17997.885
50 step loss: 14260.836
60 step loss: 11155.359
70 step loss: 8612.853
80 step loss: 6561.687
90 step loss: 4931.5103
100 step loss: 3655.764
Training is done.
Time spent: 0.7 s
Training speed: 138.3 step/s
```

Linear_Regression 객체의 run() 메소드를 사용하면 학습된 모델을 사용할 수 있습니다. 파라미터에는 inputs 가 있고, 모델에 사용할 데이터를 의미합니다. inputs 에는 [[172], [162]]와 같이 2 차원 리스트로 입력해야 합니다. 기본값은 X_data 를 사용하도록 설정되어있습니다.

run 메소드를 실행하면 입력에 대한 회귀 모델의 예측 발 크기를 출력합니다.

07: LR.run()

[출력]

```
run with : [[173], [171], [162], [187], [157], [169], [177], [159], [182]]
array([[204.85384],
       [202.49596],
```



```
[191.88553],  
[221.35895],  
[185.99084],  
[200.13808],  
[209.56958],  
[188.34871],  
[215.46426]], dtype=float32)
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import AI  
02:  
03: LR = AI.Linear_Regression()  
04:  
05: LR.X_data = [[173],[171],[162],[187],[157],[169],[177],[159],[182]]  
06: LR.Y_data = [[270],[275],[245],[280],[230],[265],[270],[250],[275]]  
07:  
08: LR.train()  
09: LR.run()
```

100 회 학습한 선형 회귀를 실행한 결과가 매우 부정확한 것을 확인할 수 있습니다. 추가 학습을 위해 train 메소드의 times 파라미터를 1000 으로 설정하고 학습합니다. 또, print_every 파라미터를 이용해 출력량을 조절 해도 좋습니다.

```
10: LR.train(times=1000, print_every=100)
```

[출력]

200 step loss: 121.05106

```
300 step loss: 49.99603
400 step loss: 49.772236
500 step loss: 49.772034
600 step loss: 49.772095
700 step loss: 49.77205
800 step loss: 49.771927
900 step loss: 49.77195
1000 step loss: 49.77183
1100 step loss: 49.771797
Training is done.
Time spent: 2.6 s
Training speed: 378.0 step/s
```

이전 학습 모델에 이어서 1,000 회 학습해 총 1,100 회를 학습했고, 100 회마다 학습 오차를 출력합니다. run 메소드를 이용해 학습 모델의 예측값을 출력해보겠습니다.

```
11: LR.run()
```

[출력]

```
run with : [[173], [171], [162], [187], [157], [169], [177], [159], [182]]
array([[265.58435],
       [262.5285 ],
       [248.7772 ],
       [286.97528],
       [241.13759],
       [259.47266],
       [271.69604],
       [244.19344],
       [279.33566]], dtype=float32)
```

예측값이 설정된 결괏값과 비슷한 수준으로 출력되는 것을 확인할 수 있습니다. run 메소드의 파라미터로 새로운 데이터 [[150], [160], [170], [180], [190]]을 입력하여 어떻게 출력하는지 확인해봅니다.

```
12: LR.run([[150], [160], [170], [180], [190]])
```

[출력]

```
array([[230.44212],  
       [245.72136],  
       [261.00058],  
       [276.27982],  
       [291.55905]], dtype=float32)
```

부가적으로 프로그램이 종료된 이후 학습 모델을 다시 불러와 사용하는 과정을 설명합니다. Jupyter Notebook 에서 프로그램 종료 상황을 가정하기 위해 다음과 같이 상단 툴바에서 커널을 재시작합니다.

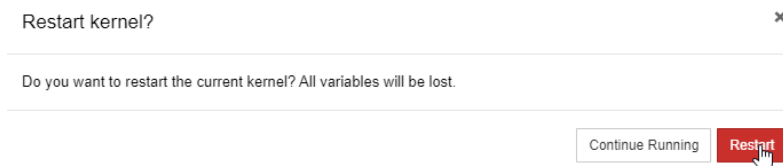
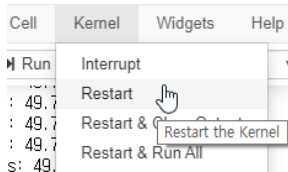


그림 106 커널 재시작

이어서, AI 모듈을 불러오고 Linear_Regression 객체를 생성할 때

restore 파라미터를 True 로 설정합니다.

```
01: from pop import AI
02:
03: LR = AI.Linear_Regression(restore=True)
```

Linear_Regression 객체에 X_data 와 Y_data 를 입력하고 run 메소드를 호출하면 이전에 학습된 모델의 출력 결과를 확인할 수 있습니다.

```
04: LR.X_data = [[173],[171],[162],[187],[157],[169],[177],[159],[182]]
05: LR.Y_data = [[270],[275],[245],[280],[230],[265],[270],[250],[275]]
06:
07: LR.run()
```

[출력]

```
run with : [[173], [171], [162], [187], [157], [169], [177], [159], [182]]
array([[265.58435],
       [262.5285 ],
       [248.7772 ],
       [286.97528],
       [241.13759],
       [259.47266],
       [271.69604],
       [244.19344],
       [279.33566]], dtype=float32)
```

또, train 메소드를 호출하면 이전 학습 모델에 이어서 학습할 수 있습니다.

```
08: LR.train()
```

[출력]

```
1110 step loss: 49.771706
1120 step loss: 49.771736
1130 step loss: 49.77168
1140 step loss: 49.77174
1150 step loss: 49.771748
1160 step loss: 49.771694
1170 step loss: 49.771664
1180 step loss: 49.77174
1190 step loss: 49.77171
1200 step loss: 49.771652
Training is done.
Time spent: 0.3 s
Training speed: 338.3 step/s
```

만약, Linear_Regression 객체를 생성할 때 Rester 파라미터를 설정하지 않거나 False 로 설정한 경우 이전 학습 모델에 덮어 씌워지므로 주의해야 합니다. 이 상황을 방지하고자 한다면 ckpt_name 파라미터에 학습 모델을 구분할 수 있는 이름을 지정하여 개별 저장하도록 할 수 있습니다.

```
09: LR = AI.Linear_Regression(restore=True, ckpt_name="model_1")
```

7.1.3.2. 로지스틱 회귀(Logistic Regression)

로지스틱 회귀는 입력에 대해 이항 확률 관계를 갖습니다. 입력이 주어졌을 때 결과를 0~1 사잇값으로 표현하며, 입력이 True(1)일 확률을 의미합니다.

다음과 같이 키와 성별에 관한 데이터가 주어졌을 때 키에 따른 성별을 예측하기 위한 회귀 모델을 구하고자 합니다. 성별 값은 True(1)가 여성, False(0)가 남성일 경우로 설정합니다.

키(cm)	성별
173	0
171	1
162	1
187	0
157	1
169	0
177	0
159	1
182	0

표 3 데이터셋

주어진 데이터를 다음과 같이 그래프로 표현할 수 있습니다.

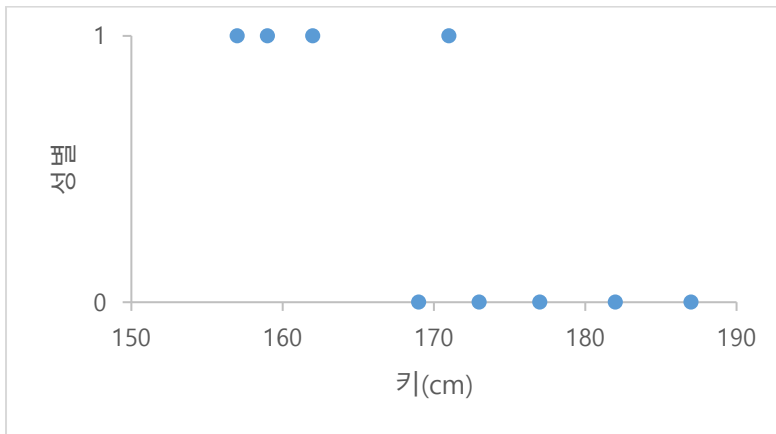


그림 107 데이터 그래프

이때 모든 데이터의 관계를 하나의 선을 그어 표현할 때 다음과 같은 선

$G(x)$ 를 그릴 수 있습니다. 이 선 $G(x)$ 를 구하기 위해 로지스틱 회귀 모델이 임의의 곡선을 가설 $H(x)$ 로 설정합니다. 로지스틱 회귀 모델은 이 가설 $H(x)$ 를 데이터의 관계를 가장 잘 표현하도록 최적화 Optimize 합니다.

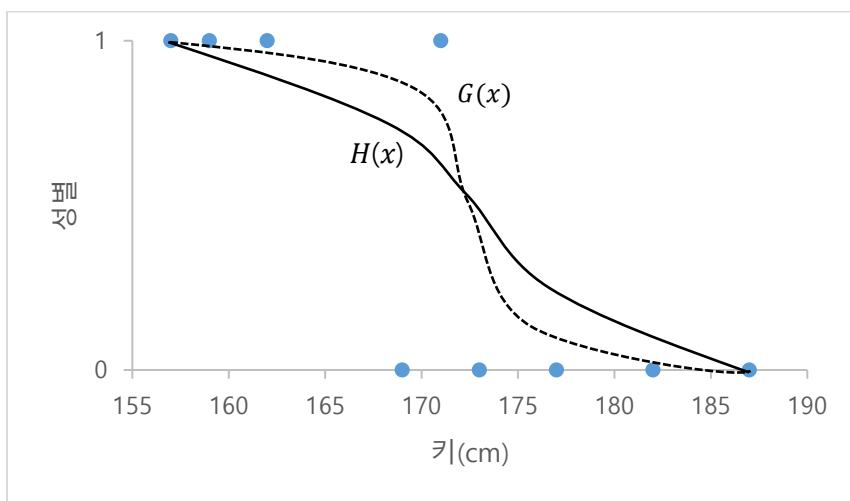


그림 108 데이터 그래프

최적화에 성공한 로지스틱 회귀 모델은 새로운 키 데이터가 주어졌을 때 여성일 확률을 예측할 수 있습니다. 단, 여성보다 키가 작은 남성이 존재하므로 확률로써 예측합니다.

키(cm)	성별
190	0.005
180	0.073
170	0.493
160	0.916
150	0.972

표 4 데이터셋

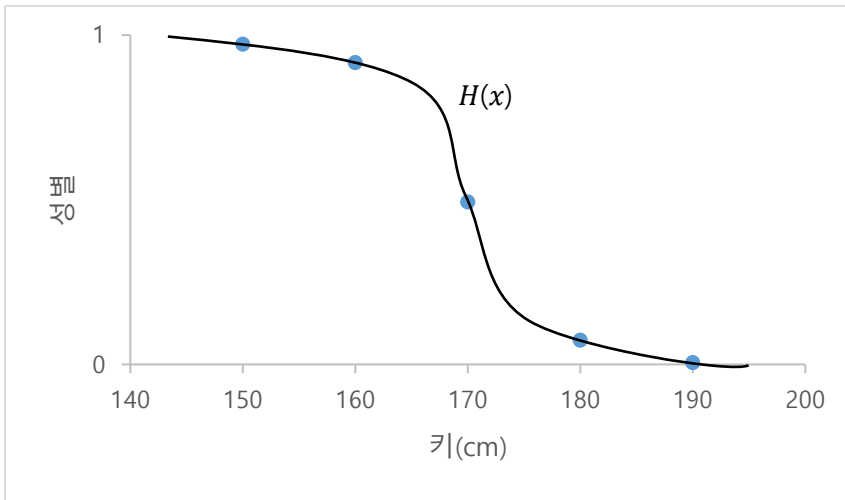


그림 109 데이터 그래프

로지스틱 회귀를 실습하는 과정 역시 Jupyter 환경에서 진행되며 Pop.AI 라이브러리를 사용합니다. Jupyter 환경에 대해서는 19 페이지를 참고할 수 있습니다. 키와 성별 데이터를 입력받아 로지스틱 회귀하고 새로운 키 데이터를 입력하면 성별을 예측하는 모델을 실습합니다.

다음 소스를 작성해 Pop.AI 라이브러리를 가져오고, Logistic_Regression 객체를 LR 이라는 변수에 생성합니다.

Logistic_Regression 객체의 파라미터에는 input_size, restore, ckpt_name 이 있습니다.

- input_size: 입력 데이터의 크기를 의미합니다. 예를 들어, 키와 발 크기를 이용해 성별을 구하고자 한다면 입력 데이터는 [[172, 270], [156, 245]]와 같은 형태이며 최하위 차원의 크기인 2 를 input_size 에 입력합니다.
 - 기본값: 1
- restore: 최근 모델에 이어서 학습할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: False

- ckpt_name: 저장 및 불러올 모델 파일의 이름입니다.
 - 기본값: logistic_regression

```
01: from pop import AI
02:
03: LR = AI.Logistic_Regression()
```

Logistic_Regression 객체의 속성에도 X_data 와 Y_data 가 있습니다. 입력할 데이터들은 앞서 설명에 이용한 데이터들을 활용합니다.

```
04: LR.X_data = [[173],[171],[162],[187],[157],[169],[177],[159],[182]]
05: LR.Y_data = [[0],[1],[1],[0],[1],[0],[0],[1],[0]]
```

Logistic_Regression 객체의 train() 메소드를 사용하면 회귀 학습을 시작할 수 있습니다. 파라미터에는 times 와 print_every 가 있습니다. 또, 기본값은 100 과 10 으로 설정되어있습니다.

train 메소드를 실행하면 10 회마다 회귀 모델의 오차를 출력합니다.

```
06: LR.train()
```

[출력]

```
10 step loss: 0.8034792
20 step loss: 0.7270934
30 step loss: 0.70982087
40 step loss: 0.7122097
50 step loss: 0.70981884
60 step loss: 0.70816445
70 step loss: 0.707251
```

```
80 step loss: 0.70657855
90 step loss: 0.70598745
100 step loss: 0.7054279
Training is done.
Time spent: 0.7 s
Training speed: 141.3 step/s
```

Logistic_Regression 객체의 run() 메소드를 사용하면 학습된 모델을 사용할 수 있습니다. 파라미터에는 inputs 가 있습니다. 또, 기본값은 X_data 를 사용하도록 설정되어있습니다.

run 메소드를 실행하면 입력에 대한 회귀 모델의 성별 확률을 출력합니다.

```
07: LR.run()
```

[출력]

```
run with : [[173], [171], [162], [187], [157], [169], [177], [159], [182]]
array([[0.42474964],
       [0.4225297 ],
       [0.41258067],
       [0.4403682 ],
       [0.40708318],
       [0.42031294],
       [0.42919803],
       [0.40927944],
       [0.43477482]], dtype=float32)
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import AI
02:
03: LR = AI.Logistic_Regression()
04:
05: LR.X_data = [[173],[171],[162],[187],[157],[169],[177],[159],[182]]
06: LR.Y_data = [[0],[1],[1],[0],[1],[0],[0],[1],[0]]
07:
08: LR.train()
09: LR.run()
```

100 회 학습한 로지스틱 회귀를 실행한 결과가 매우 부정확한 것을 확인할 수 있습니다. 추가 학습을 위해 train 메소드의 times 파라미터를 10000 으로 설정하고 학습합니다.

```
10: LR.train(times=10000, print_every=1000)
```

[출력]

```
1100 step loss: 0.5822015
2100 step loss: 0.47118405
3100 step loss: 0.39941132
4100 step loss: 0.3519534
5100 step loss: 0.31969684
6100 step loss: 0.2973455
7100 step loss: 0.28168958
8100 step loss: 0.27069926
9100 step loss: 0.26287797
10100 step loss: 0.257355
Training is done.
Time spent: 28.8 s
Training speed: 347.4 step/s
```

이전 학습 모델에 이어서 10,000 회 학습해 총 10,100 회를 학습했고, 1000 회마다 학습 오차를 출력합니다. run 메소드를 이용해 학습 모델의 예측값을 출력해보겠습니다.

```
11: Logistic_Regression.run()
```

[출력]

```
run with : [[173], [171], [162], [187], [157], [169], [177], [159], [182]]
```

```
array([[0.22458985],  
       [0.34181786],  
       [0.877867  ],  
       [0.00483832],  
       [0.9686939  ],  
       [0.48218343],  
       [0.08264586],  
       [0.94522667],  
       [0.02049813]], dtype=float32)
```

예측값이 설정된 결괏값과 비슷한 수준으로 출력되는 것을 확인할 수 있습니다. run 메소드의 파라미터로 새로운 데이터 [[150], [160], [170], [180], [190]]을 입력하여 어떻게 출력하는지 확인해봅니다.

```
12: LR.run([[150], [160], [170], [180], [190]])
```

[출력]

```
array([[0.9967699 ],  
       [0.941998  ],  
       [0.46083763],  
       [0.04304785],  
       [0.00236174]], dtype=float32)
```

Logistic_Regression 객체 역시 restore 파라미터를 True 로 설정하면

최근 사용한 학습 모델을 불러와 다시 사용할 수 있습니다.

```
13: LR = AI.Logistic_Regression(restore=True, ckpt_name="model_1")
```

7.1.3.3. 소프트맥스 회귀(Softmax Regression)

소프트맥스 회귀를 구현하기 위해서는 인공신경망 ANN 개념이 필요합니다. 그러나 인공신경망은 딥러닝에서 다루므로 인공신경망을 이야기하며 소프트맥스 회귀에 대해 더 자세히 다루도록 하고 현재 챕터에서는 다차원 방정식으로 대체하여 설명합니다.

소프트맥스 회귀는 입력에 대해 다항 확률 관계를 갖습니다. 입력이 주어졌을 때 각 클래스별로 0~1 사이값으로 표현되며 모든 클래스 값의 합은 1이 됩니다.

다음과 같이 키와 연령층에 관한 데이터가 주어졌을 때 키에 따른 연령층을 예측하기 위한 회귀 모델을 구하고자 합니다. 각 연령층에 관한 클래스는 유아, 어린이, 성인으로 설정하고 클래스의 개수 K 를 3으로 합니다.

키(cm)	유아	어린이	성인
73	1	0	0
62	1	0	0
83	1	0	0
110	0	1	0
139	0	1	0
123	0	1	0
177	0	0	1
159	0	0	1
182	0	0	1

표 5 데이터셋

주어진 데이터를 다음과 같이 3 개의 그래프로 표현할 수 있습니다.

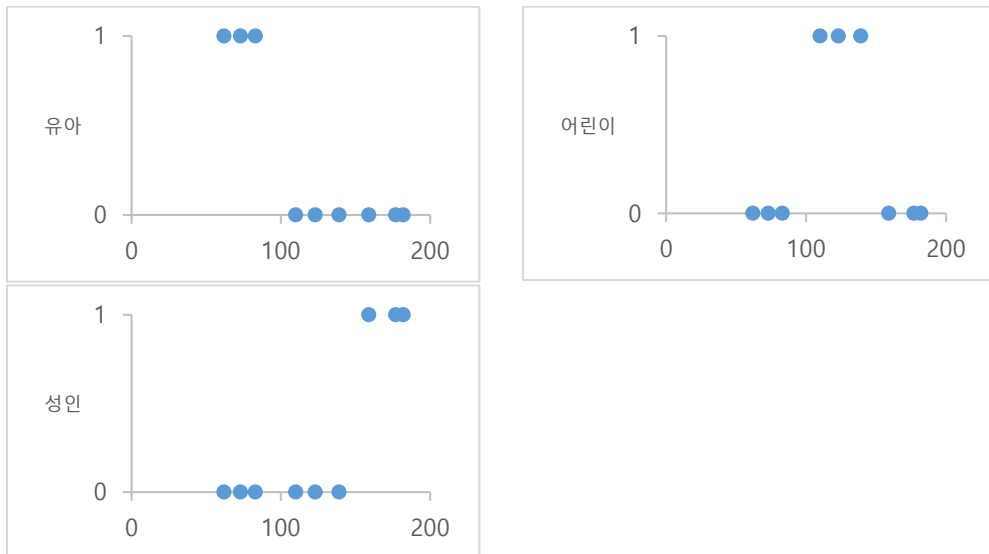


그림 110 데이터 그래프

이때 모든 데이터와 클래스 간의 관계를 각각 그래프에서 하나의 선을 그려 표현할 때 다음과 같은 선들 $G_i(x)$ 을 그릴 수 있습니다. 이 선들 $G_i(x)$ 을 구하기 위해 소프트맥스 회귀 모델이 임의의 곡선들을 가설 $H_i(x)$ 로 설정합니다. 소프트맥스 회귀 모델은 각 클래스에 대한 가설들 $H_i(x)$ 을 데이터의 관계를 가장 잘 표현하도록 최적화 Optimize 합니다.

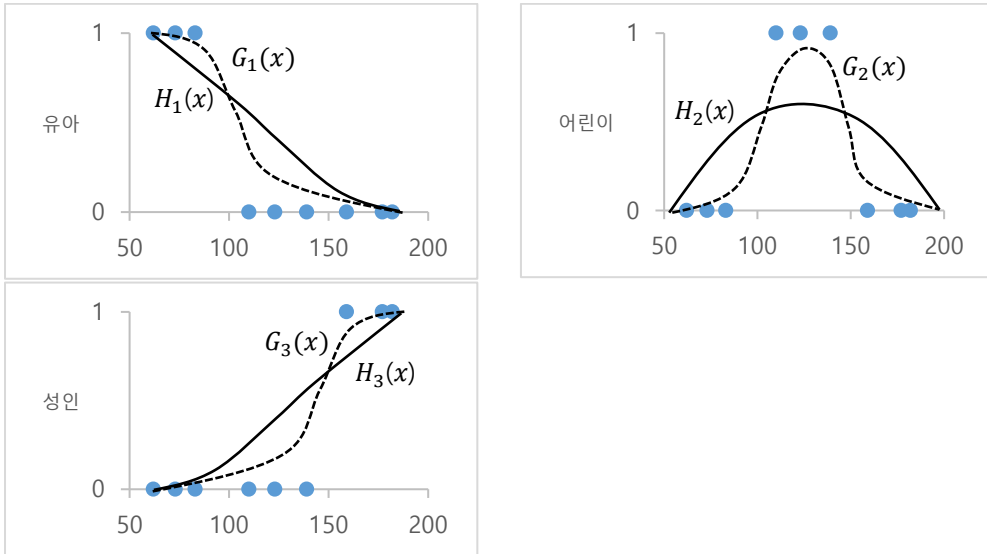


그림 111 데이터 그래프

최적화에 성공한 소프트맥스 회귀 모델은 새로운 키 데이터가 주어졌을 때 유아, 어린이, 성인일 확률을 예측할 수 있습니다. 단, 모든 클래스 값의 합이 1 이어야 하므로 소프트맥스 함수 Softmax Function 를 이용해 각 클래스 값을 조정합니다.

키(cm)	유아	어린이	성인
60	0.998	0.002	0
80	0.961	0.068	0.025
100	0.647	0.391	0.083
120	0.217	0.884	0.152
140	0.113	0.807	0.384
160	0.051	0.16	0.891
180	0.008	0.052	0.985

표 6 데이터셋

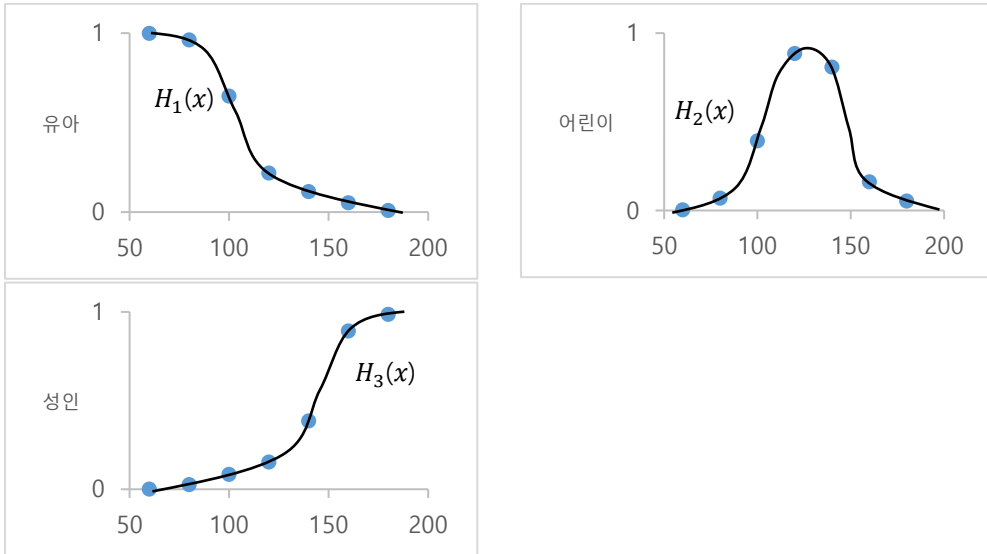


그림 112 데이터 그래프

만약, 과도하게 최적화 과정을 진행하여 다음과 같이 이상적인 최적화 모델 $G_i(x)$ 를 넘어 극단적인 형태의 모델 $H_i(x)$ 이 생성된다면 잘 최적화된 모델이라고 할 수 없습니다. 이러한 현상을 과적합 또는 오버피팅 **Overfitting** 이라고 하며 과도한 최적화로 인해 원하는 결과를 얻을 수 없는 상태입니다. 이와 반대되는 개념으로는 과소적합 또는 언더피팅 **Underfitting** 이 있으며 부족한 최적화로 인해 원하는 결과를 얻을 수 없는 상태입니다.

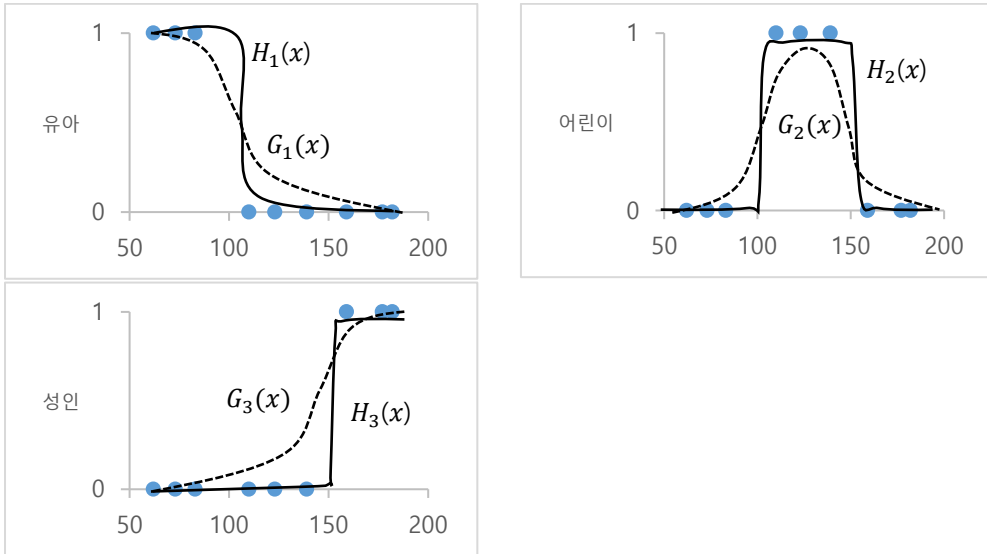


그림 113 과적합 데이터 그래프

소프트맥스 회귀에 대한 실습은 인공지능망 실습 과정에서 자세히 다루도록 합니다.

7.1.3.4. K-평균 군집화(K-means Clustering)

K-평균 군집화는 예측 Expectation 과 최대화 Maximization 을 반복하며 최적해로 수렴하는 EM 알고리즘을 기반하고 있습니다. 군집 기준점으로부터 가까운 데이터들을 묶고, 묶인 데이터들의 중심점을 군집 기준점으로 재설정하는 과정을 반복합니다.

다음과 같이 키와 나이에 대한 데이터가 주어졌을 때 이 데이터들의 특징에 따라 K 개의 군집을 구하고자 합니다.

키(cm)	나이
73	1
62	0.3

83	1.8
110	5.5
139	10.3
123	7.7
177	17.8
159	16.3
182	19.1

표 7 데이터셋

주어진 데이터를 다음과 같이 그래프로 표현할 수 있습니다.

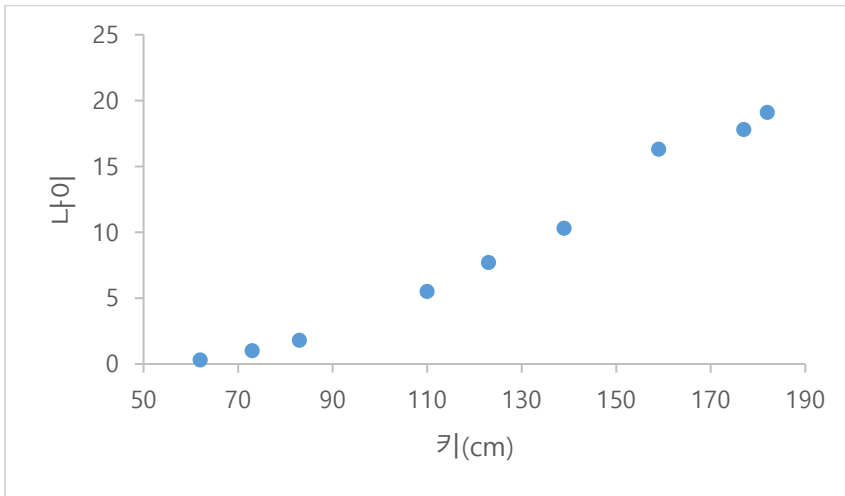


그림 114 데이터 그래프

이때 랜덤으로 K 개의 군집점 설정하고 각 군집점을 기준으로 가까운 데이터들을 묶습니다.

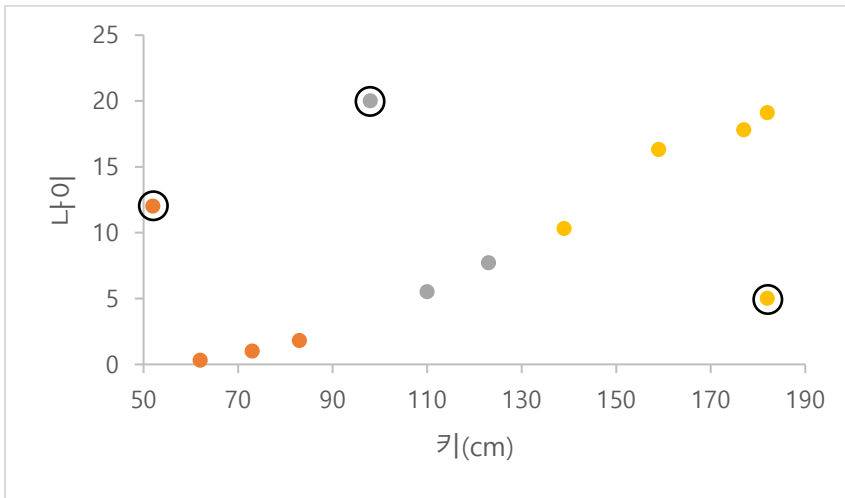


그림 115 데이터 그래프

묶인 데이터들의 중심점을 찾고 이 점을 군집점으로 재설정합니다.

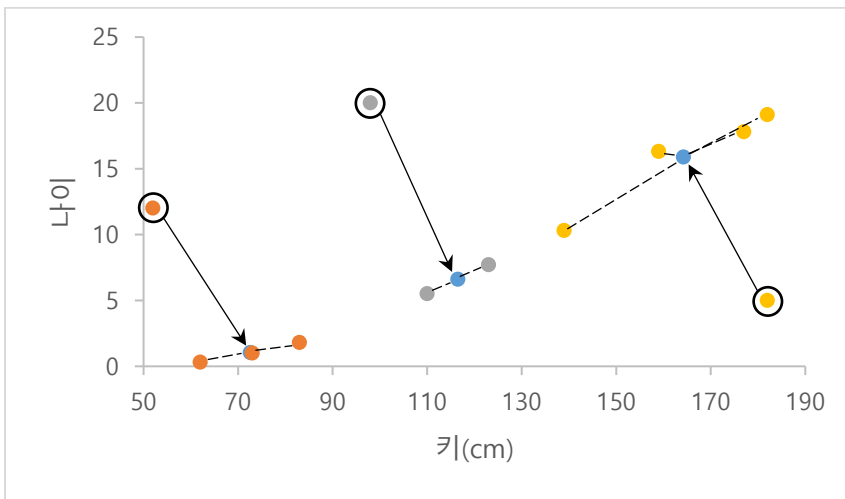


그림 116 데이터 그래프

이 과정을 군집점과 중심점의 오차가 최소가 될 때까지 반복합니다. 이를 최적화 *Optimize* 라고 합니다.

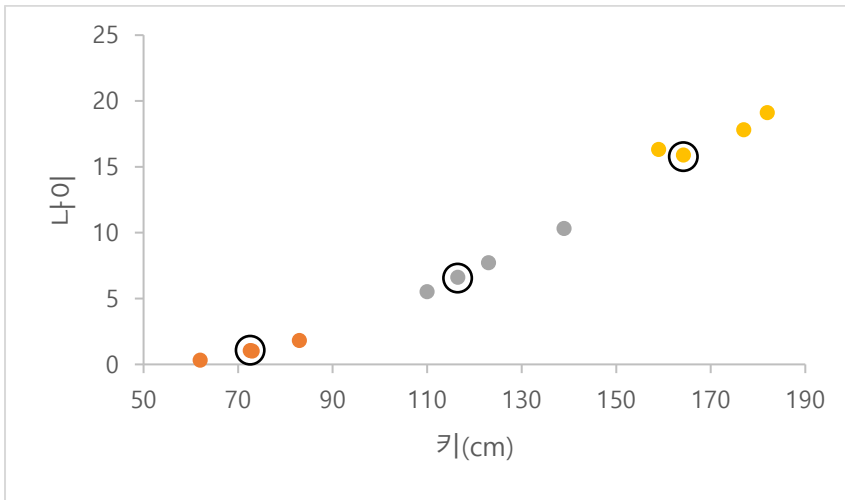


그림 117 데이터 그래프

최적화에 성공한 K-평균 군집화 모델은 새로운 키 데이터가 주어졌을 때 가장 가까운 군집에 포함시키고 필요에 따라 군집점을 재설정합니다.

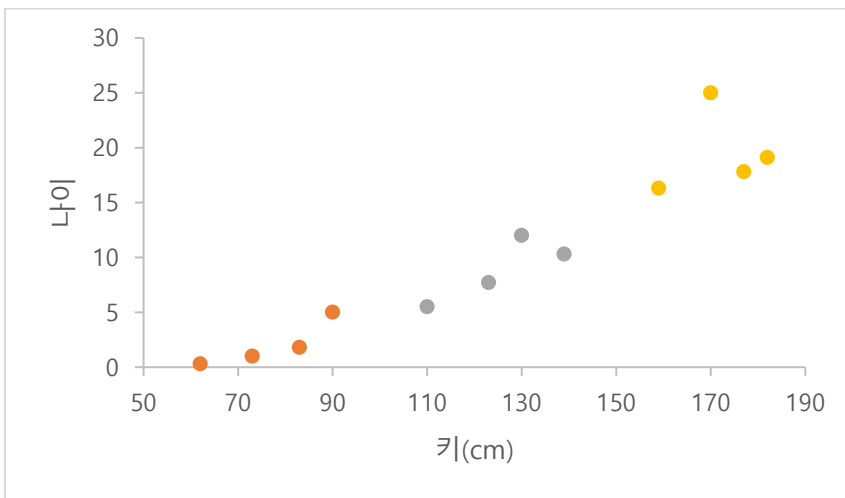


그림 118 데이터 그래프

내용정리

- 인공지능
 - 약인공지능 : 특정 분야에서 우수한 능력을 가진 인공지능
 - 강인공지능 : 모든 분야에서 인간과 비슷한 능력을 가진 인공지능
 - 초인공지능 : 모든 분야에서 인간을 초월한 능력을 가진 인공지능
- 머신러닝 : 컴퓨터 시스템이 데이터를 학습하는 과정
- 지도 학습 : 학습 결과가 어떻게 출력되어야 하는지 알려주고, 컴퓨터가 출력한 결과와 비교하여 머신러닝 모델을 최적화하는 방법
 - 분류 : 입력과 분류 결과가 주어지면, 분류 기준을 학습하고 새로운 데이터를 어떤 결과로 분류할지 선택하는 기법
 - 회귀 : 입력과 결과가 주어지면, 두 값의 수학적 관계를 학습하고 새로운 데이터가 어떤 결과를 출력할지 예측하는 기법
- 비지도 학습 : 입력 데이터만 주어지고, 컴퓨터가 입력 데이터의 특징을 추론하고 특징의 기준을 구체화하는 방법
 - 군집화 : 입력 데이터 값이 비슷한 것끼리 군집시켜 군집 기준을 구체화하고 새로운 데이터에 대한 군집 그룹을 정하는 기법
- 강화 학습 : 머신러닝 모델이 더 나은 결과를 출력할 때 보상을 주며 오차를 줄여나가는 학습 방법
- 머신러닝 기법 : 선형 회귀, 로지스틱 회귀, 소프트맥스 회귀, K-평균 군집화
 - 선형 회귀 : 입력과 결과의 선형 관계를 분석하는 머신러닝 모델

- 로지스틱 회귀 : 입력과 결과에 대해 이항 관계를 분석하는 머신러닝 모델
- 소프트맥스 회귀 : 입력과 결과에 대해 다항 관계를 분석하는 머신러닝 모델
- K-평균 군집화 : 가까운 입력 데이터끼리 군집시키는 방법으로 군집 기준을 추론하는 머신러닝 모델



그림 119 브라우저 가계도

1991

Tim Berners-Lee 의 GUI 브라우저 편집기는 NeXT 컴퓨터외에는 실행되지 않았습니다. 웹 팀은 빠른 배포를 위해 간단한 텍스트 전용 브라우저를 작성한 다음 개발자들에게 PC, Mac 및 UNIX 시스템에 필요한 GUI 브라우저를 작성하거나 적용하도록 요청했습니다. 8 명의 개발자들이 응답하여 UNIX, Mac 및 PC 에서 사용가능한 브라우저가 만들어졌습니다.



그림 120 JPG 데스크탑 아이콘

1992

1986 년 국제 표준기구 그룹이 JPEG (Joint Photographic Expert Group)를 분리하여 디지털 이미지에 대한 표준 세트를 작성했습니다. 1992 년 국제 표준기구 그룹은 jpeg (또는 .jpg) 형식의 규칙을 결정했습니다. jpeg 압축을 사용하면 사진 품질과 파일 크기간에 균형을 유지할 수 있습니다. jpeg 는 가장 널리 사용되는 이미지 형식 중 하나이며 디지털 카메라에서 가장 널리 사용되는 형식입니다.

연습문제

문제 35

다음 문장들을 읽고 빈 칸을 채워보세요.

- A. 특정 분야에서 우수한 능력을 가진 인공지능을 이라 한다.
- B. 컴퓨터 시스템이 주어진 데이터를 학습하는 과정을 이라 한다.
- C. 머신러닝은 학습 방식에 따라 , ,
으로 나눌 수 있다.
- D. 대표적 머신러닝 기법은 , ,
, 등이 있다.

문제 36

다음 코드는 Pop.AI 라이브러리를 이용하여 선형 회귀를 구현한 코드입니다. 질문을 읽고 답해보세요.

```
01: from pop import AI
02:
03: LR = AI.Linear_Regression()
04:
05: LR.X_data =  X
06: LR.Y_data =  Y
07:
08: LR.train(times=1000, print_every=100)
```

- A. 수식 $y = 2x$ 를 회귀할 수 있도록 빈 칸 X, Y에 들어갈 학습 데이터셋을 작성해보세요.
- B. A에서 작성한 데이터셋으로 손실을 0.001 이하까지 학습시키고, 100을 입력했을 때 출력을 작성하세요.
- C. 선형회귀 모델을 새롭게 생성하고, 다음과 같은 주가 그래프와 표가 주어졌을 때 선형 회귀하여 15시 30분 예측 값과 학습 횟수(Step)을 작성하세요.



9	9.5	10	10.5	11	11.5	12
48.7	48.45	48.45	48.6	48.7	48.9	49.05
12.5	13	13.5	14	14.5	15	
49.05	48.95	49.05	48.9	49.35	49.65	

문제 37

다음 코드는 Pop.AI 라이브러리를 이용하여 로지스틱 회귀를 구현한 코드입니다. 질문을 읽고 답해보세요.

```
01: from pop import AI
02:
03: LR = AI.Logistic_Regression()
04:
05: LR.X_data = 

|   |
|---|
| X |
|---|


06: LR.Y_data = 

|   |
|---|
| Y |
|---|


07:
08: LR.train()
09: LR.run()
```

- A. 양수면 1, 음수면 0 을 출력하도록 회귀하고자 할 때 빈 칸 X, Y 에 들어갈 학습 데이터셋을 작성하세요.
- B. 로지스틱 회귀 모델에 -1, 100, -0.01 를 입력했을 때의 출력을 확인해보고, -0.01 을 입력했을 때 오차가 0.1 이하인 모델을 만들어보고 모델의 손실율을 작성하세요.

문제 38

아래의 코드는 AIoT AutoCar 의 Cds 센서를 이용하여 밝기 값을 출력하는 코드입니다. 조도계를 사용하거나, 스마트폰에서 ‘조도계’ 애플리케이션을 다운 받아 다음 문제를 해결해보세요. (단, 조도계의 단위는 Lux 로 합니다.)

```
01: from pop import AI
```

```
02: from pop import Cds, delay
03:
04: cds = Cds(7)
05:
06: value = cds.readAverage()
07: print(value)
```

- A. 조도계를 옆에 두고 코드를 실행시킨 후 조도계 값과 출력 값의 차이를 확인해보세요.
- B. 빈 배열을 생성하고, Cds 값을 0.5 초 간격으로 10 회 이상 추가하는 코드를 작성하세요.
- C. 다음 코드를 이용해 빈 배열 2 개를 생성하고, Cds 값과 조도계 값을 동시에 측정하여 Cds 값 배열과 조도계 값 배열을 만들어보세요.

```
01: arr_cds = []
02: arr_lux = []
03:
04: for i in range(10):
05:     arr_cds.append(cds.readAverage())
06:     arr_lux.append(int(input("Lux: ")))
```

- D. Pop.AI 라이브러리의 Linear_Regression 클래스를 사용해 C 에서 만든 두 배열을 각각 X, Y 데이터로 하고 선형 회귀하는 코드를 작성하세요.
- E. 회귀 모델의 출력과 실제 조도계의 값을 비교해보고 데이터셋 추가 수집, 추가 학습 등 방법으로 ± 30 lux 미만의 오차 범위를 갖는 회귀 모델을 만들어보세요.

7.2. 딥러닝

딥러닝은 머신러닝 기법의 하나인 인공신경망 Artificial Neural Network, ANN 을 기반으로 설계된 기법들을 일컫는 말입니다. 인공신경망은 인간의 신경망을 모방하여 만들어진 알고리즘으로 퍼셉트론 Perceptron 에서 시작되었습니다. 이러한 인공신경망을 기반으로 만들어진 딥러닝 기법에는 심층신경망, 합성곱 신경망, 순환신경망 등이 있습니다.

7.2.1. 퍼셉트론(Perceptron)

퍼셉트론은 다수의 데이터로 하나의 결과를 출력하도록 하는 복합 논리 회로입니다. 하나 이상의 데이터 값을 입력 Input 받고 각 입력에 가중치 Weight 를 곱합니다. 이 값들을 모두 합해 기준값보다 크면 활성화(True)되고, 작으면 비활성화(False)됩니다. 복합적 요인 중 각 요인이 결과에 미치는 영향을 분석할 때 사용할 수 있는 방법입니다.

다음과 같이 키, 발 크기, 머리카락 길이와 성별에 관한 데이터가 주어졌을 때 복합적 요인에 따른 성별을 예측하기 위한 퍼셉트론 모델을 구하고자 합니다. 성별 값은 True(1)가 여성, False(0)가 남성일 경우로 설정합니다.

키(cm)	발 크기(mm)	머리카락 길이(cm)	성별
173	270	17	0
171	275	51	1
162	245	62	1
187	280	12	0
157	230	47	1
169	265	30	0
177	270	5	0
159	250	32	1
182	275	0	0

표 8 데이터셋

표에서의 튜플을 데이터 형태일 때는 데이터셋 Dataset 이라고 합니다. 주어진 데이터셋을 나열하고 각 데이터에 임의의 가중치 W_i 를 곱한 다음 모든 데이터를 합한 값 x 과 기준값 θ 을 비교합니다. 가중치 Weight 는 각 데이터 X_i 가 결과에 가지는 영향력을 의미합니다.

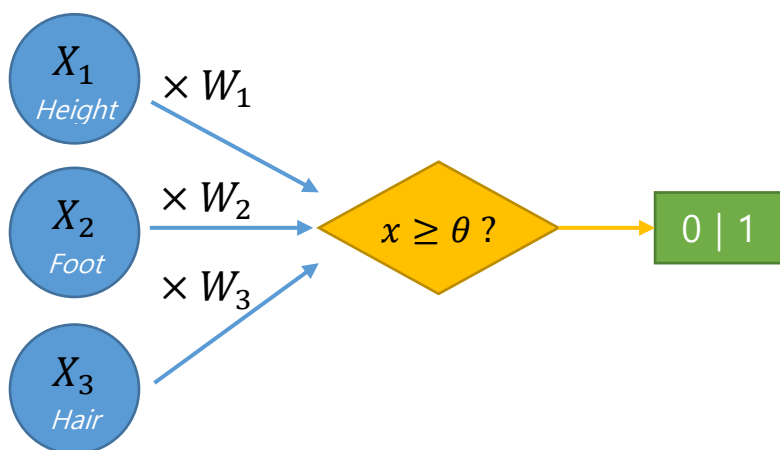


그림 121 퍼셉트론 모델

출력값과 실제값을 비교하고 가중치를 조절해 오차를 줄여나가도록 반복합니다. 이 과정을 최적화 Optimize 라 하고, 모델의 출력값과 실제값을 비교하는 함수를 손실 함수 Loss Function 또는 비용 함수 Cost Function 라고 합니다. 둘의 차이는 하나의 데이터셋에서 비교할 때는 손실 함수라 하고, 여러 데이터셋을 기준으로 통계를 내거나 성능을 평가할 때는 비용 함수라 합니다.

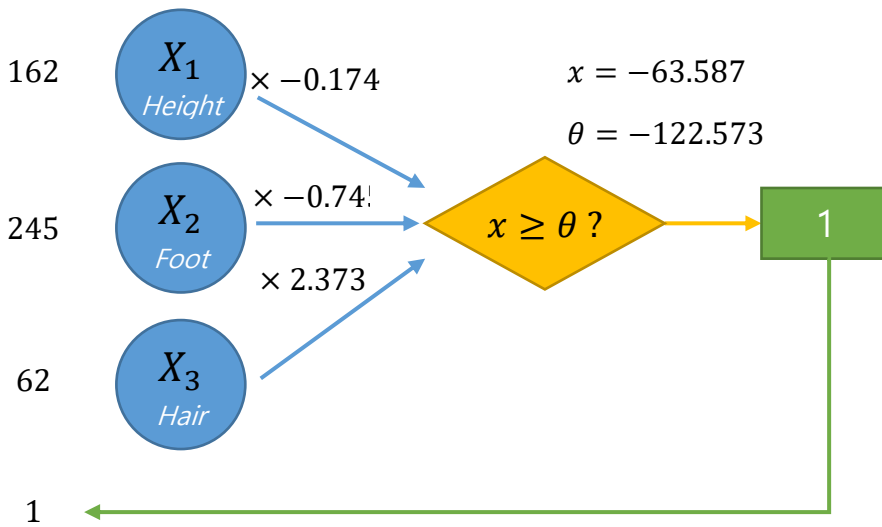
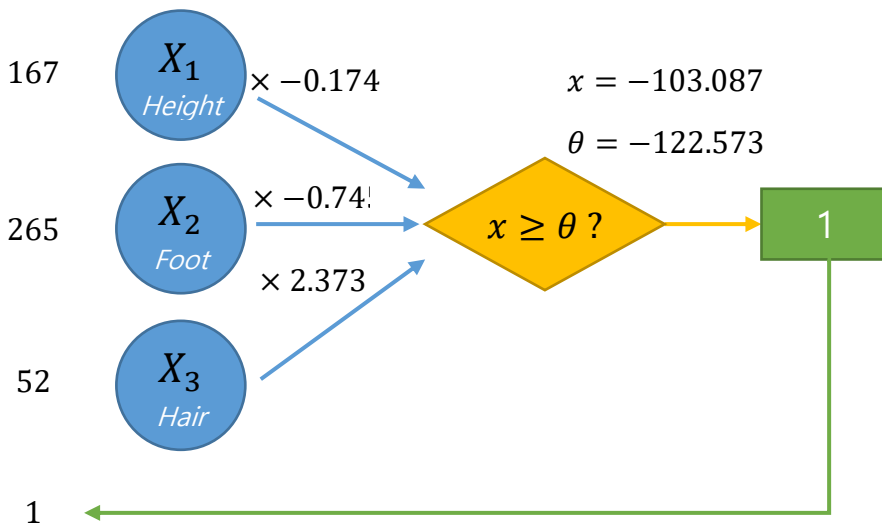


그림 122 퍼셉트론 모델

최적화에 성공한 퍼셉트론 모델은 새로운 복합 데이터가 주어졌을 때 여성인지 아닌지를 판단합니다. 또, 각 데이터에 대한 가중치를 조사해 성별과 관련 있는 데이터를 분석할 수 있습니다.



퍼셉트론 실습 과정은 Jupyter 환경에서 진행되며 Pop.AI 라이브러리를 사용합니다. Jupyter 환경에 대해서는 19 페이지를 참고할 수 있습니다. 키, 발 크기, 머리카락 길이, 성별 데이터를 입력받아 학습한 뒤 새로운 데이터를 입력하면 성별을 예측하는 모델을 실습합니다.

Perceptron 객체의 파라미터에는 input_size, output_size, restore, ckpt_name, softmax 가 있습니다.

- input_size: 입력 데이터의 크기를 의미하며 최하위 차원의 크기를 입력합니다.
 - 필수 입력
- output_size: 결과 데이터의 크기를 의미합니다. 예를 들어, 키를 이용해 성별과 발 크기를 구하고자 한다면 결과 데이터는 [[1, 270], [0, 245]]와 같은 형태이며 최하위 차원의 크기인 2 를 output_size 에 입력해야 합니다.
 - 기본값: 1
- restore: 최근 모델에 이어서 학습할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: False
- ckpt_name: 저장 및 불러올 모델 파일의 이름입니다.
 - 기본값: perceptron
- softmax: 결과 데이터의 크기가 2 이상일 때, 모델의 예측 결과에 소프트맥스 함수를 적용하여 총합이 1이 되도록 할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: True

input_size 는 사용자 입력이 필수이며 이를 제외한 각 기본값은 1, False, perceptron, True 로 설정되어 있습니다.

다음 소스를 작성해 Pop.AI 라이브러리를 가져오고, Perceptron 객체의 input_size 파라미터를 3 으로 설정하여 Perc 라는 변수에 생성합니다.

```
01: from pop import AI
02:
03: Perc = AI.Perceptron(input_size=3)
```

Perceptron 객체의 속성에도 X_data 와 Y_data 가 있습니다. 입력할 데이터들은 앞서 설명에 이용한 데이터들을 활용합니다.

```
04: Perc.X_data = [[173,270,17], [171,275,51], [162,245,62],
[187,280,12], [157,230,47], [169,265,30], [177,270,5], [159,250,32],
[182,275,0]]
05: Perc.Y_data = [[0],[1],[1],[0],[1],[0],[0],[1],[0]]
```

Perceptron 객체의 train() 메소드를 사용하면 퍼셉트론의 학습을 시작할 수 있습니다. train 메소드의 파라미터에는 times 와 print_every 가 있습니다. 또, 기본값은 100 과 10 으로 설정되어있습니다.

train 메소드를 실행하면 10 회마다 퍼셉트론 모델의 오차를 출력합니다.

```
06: Perc.train()
```

[출력]

```
10 step loss: 0.16669248
20 step loss: 0.052334767
30 step loss: 0.04746726
40 step loss: 0.04340801
50 step loss: 0.040468812
60 step loss: 0.038942944
70 step loss: 0.037969377
80 step loss: 0.037178453
90 step loss: 0.036465146
```

```
100 step loss: 0.035796117
Training is done.
Time spent: 0.8 s
Training speed: 130.8 step/s
```

Perceptron 객체의 run() 메소드를 사용하면 학습된 모델을 사용할 수 있습니다. 파라미터에는 inputs 가 있습니다. 또, 기본값은 X_data 를 사용하도록 설정되어 있습니다.

run 메소드를 실행하면 입력에 대한 퍼셉트론 모델의 성별 확률을 출력합니다.

```
07: Perc.run()
```

[출력]

```
run with : [[173, 270, 17], [171, 275, 51], [162, 245, 62], [187, 280, 12], [157, 230,
47], [169, 265, 30], [177, 270, 5], [159, 250, 32], [182, 275, 0]]
array([[0.33761472],
       [0.97953510],
       [0.99887633],
       [0.81049204],
       [0.97985792],
       [0.39180151],
       [0.25080442],
       [0.59181070],
       [0.73814782]], dtype=float32)
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import AI
02:
03: Perc = AI.Perceptron(input_size=3)
04:
05: Perc.X_data = [[173,270,17], [171,275,51], [162,245,62],
    [187,280,12], [157,230,47], [169,265,30], [177,270,5],
    [159,250,32], [182,275,0]]
06: Perc.Y_data = [[0],[1],[1],[0],[1],[0],[0],[1],[0]]
07:
08: Perc.train()
09: Perc.run()
```

100 회 학습한 퍼셉트론 모델의 실행한 결과가 머리카락 길이에 큰 비중을 두지만, 전체적으로 부정확한 것을 확인할 수 있습니다. 추가 학습을 위해 train 메소드의 times 파라미터를 1000 으로 설정하고 학습합니다.

```
10: Perc.train(times=1000, print_every=100)
```

[출력]

```
200 step loss: 0.02949865
300 step loss: 0.023657225
400 step loss: 0.018661896
500 step loss: 0.014682644
600 step loss: 0.011627553
700 step loss: 0.0093129575
800 step loss: 0.007558218
900 step loss: 0.006216968
1000 step loss: 0.005179543
1100 step loss: 0.0043661906
Training is done.
Time spent: 2.9 s
Training speed: 349.8 step/s
```

이전 학습 모델에 이어서 1,000 회 학습해 총 1,100 회를 학습했고, 100 회마다 학습 오차를 출력합니다. run 메소드를 이용해 학습 모델의 예측값을 출력해보겠습니다.

```
11: Perc.run()
```

[출력]

```
run with : [[173, 270, 17], [171, 275, 51], [162, 245, 62], [187, 280, 12], [157, 230, 47], [169, 265, 30], [177, 270, 5], [159, 250, 32], [182, 275, 0]]
array([[4.7683716e-07],
       [9.9999928e-01],
       [9.9999702e-01],
       [2.9504299e-06],
       [9.9999970e-01],
       [1.3818261e-01],
       [2.9504299e-06],
       [8.5786879e-01],
       [0.0000000e+00]], dtype=float32)
```

예측값이 설정된 결괏값과 비슷한 수준으로 출력되는 것을 확인할 수 있습니다. run 메소드의 파라미터로 새로운 데이터 [[174,265,6], [152,230,30], [162,255,10]]을 입력하여 어떻게 출력하는지 확인해봅시다.

```
12: Perc.run([[174,265,6], [152,230,30], [162,255,10]])
```

[출력]

```
array([[0.
       [0.84815747],
```

```
[0.      ], dtype=float32)
```

만약, 학습 횟수를 과도하게 늘려 100,000 회를 학습시켰을 때 어떤 결과를 출력하는지 확인합니다. 시간이 많이 소요될 수 있으므로 50,000 회 정도로 낮춰 학습시켜도 좋습니다.

```
13: Perc.train(times=100000, print_every=10000)
```

[출력]

```
11100 step loss: 5.569503e-06
21100 step loss: 3.6544726e-08
31100 step loss: 2.3947175e-10
41100 step loss: 6.90687e-12
51100 step loss: 9.614827e-12
61100 step loss: 7.5258315e-12
71100 step loss: 7.5258315e-12
81100 step loss: 7.493068e-12
91100 step loss: 9.29311e-12
101100 step loss: 8.991129e-12
Training is done.
Time spent: 276.1 s
Training speed: 362.2 step/s
```

run 메소드의 파라미터로 `[[152,230,28], [152,230,30]]`을 입력하여 어떻게 출력하는지 확인해봅니다.

```
14: Perc.run([[152,230,28], [152,230,30]])
```

[출력]

```
array([[0.      ],
```

```
[0.99999964]], dtype=float32)
```

두 표본 데이터의 차이는 2cm 정도의 머리카락 길이임에도 극단적인 결과를 보여줍니다. 이러한 현상이 소프트맥스 회귀에서 언급된 과적합 *overfitting* 현상입니다. 이렇듯 최적의 학습 모델이 무조건 학습 오차를 낮추는 것이 아니라 적절한 수준을 유지하는 것임을 알 수 있습니다.

부가적으로, Perceptron 객체 역시 `restore` 파라미터를 `True` 로 설정하면 최근 사용한 학습 모델을 불러와 다시 사용할 수 있습니다.

```
15: Perc = AI.Perceptron(input_size=3, restore=True, ckpt_name="model_1")
```

7.2.2. 인공신경망(Artificial Neural Network, ANN)

인공신경망은 퍼셉트론을 기반으로 고안되었으며 다수의 데이터로 하나 이상의 결과를 출력하도록 하는 알고리즘입니다. 하나 이상의 데이터 값을 입력 Input 받고 각 입력에 가중치 Weight 를 곱합니다. 이 값들을 모두 합해 활성화 함수 Activation Function 에 입력하고 0~1 사이값으로 결과를 출력합니다.

먼저, 퍼셉트론 형태를 기반으로 기초적인 형태의 인공신경망 모델을 설명하고자 합니다. 다음과 같이 키, 발 크기, 머리카락 길이와 성별에 관한 데이터가 주어졌을 때 복합적 요인에 따른 성별을 예측하기 위한 인공신경망 모델을 구하고자 합니다. 성별 값은 True(1)가 여성, False(0)가 남성일 경우로 설정합니다.

키(cm)	발 크기(mm)	머리카락 길이(cm)	성별
173	270	17	0
171	275	51	1
162	245	62	1
187	280	12	0
157	230	47	1
169	265	30	0
177	270	5	0
159	250	32	1
182	275	0	0

표 9 데이터셋

주어진 데이터셋을 나열하고 각 데이터에 임의의 가중치 W_i 를 곱한 다음 모든 데이터를 합한 값 x 을 활성화 함수 Activation Function 에 입력하면 0~1 사이값으로 출력합니다. 활성화 함수에는 주로 시그모이드 Sigmoid 함수와 렐루 ReLu 함수가 사용됩니다.

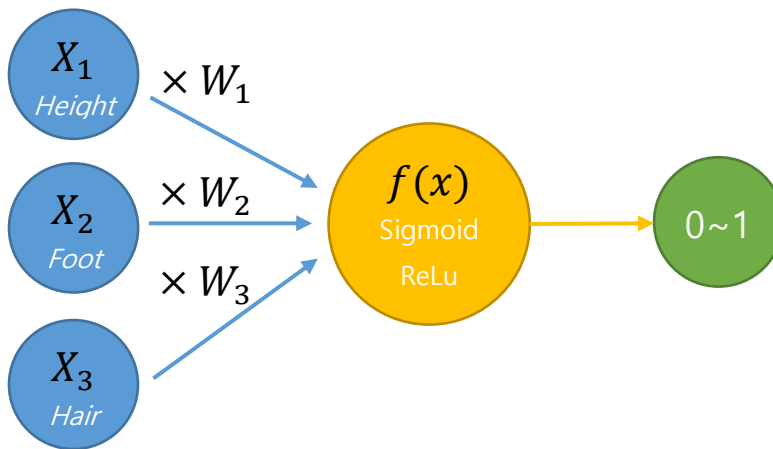


그림 124 기초 인공신경망 모델

활성화 함수의 출력값과 실제값을 비교하고 가중치를 조절하는 과정을 반복해 모델을 최적화 Optimize 합니다.

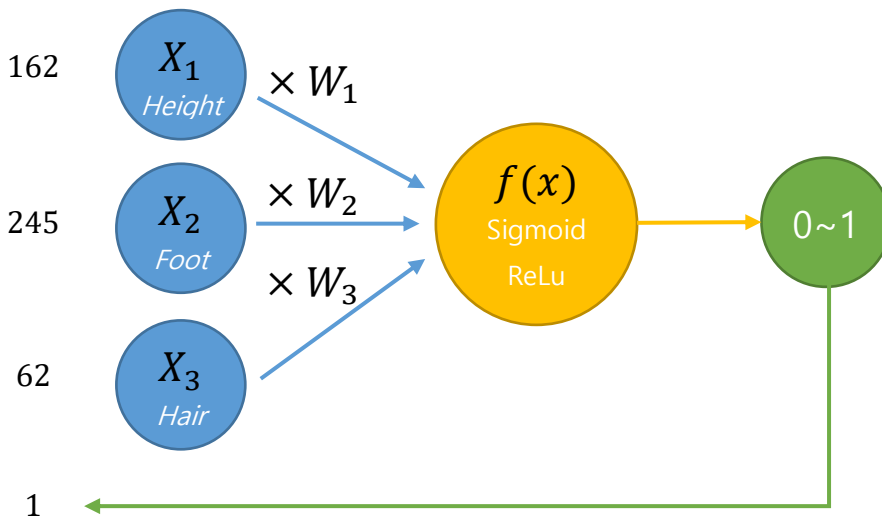


그림 125 기초 인공신경망 모델

앞서 설명한 기초 인공신경망 모델에서 역할에 따라 입력층, 은닉층, 출

력층으로 구분합니다. 각 계층에 속해있는 하나의 변수 또는 함수를 노드 Node 라고 합니다.

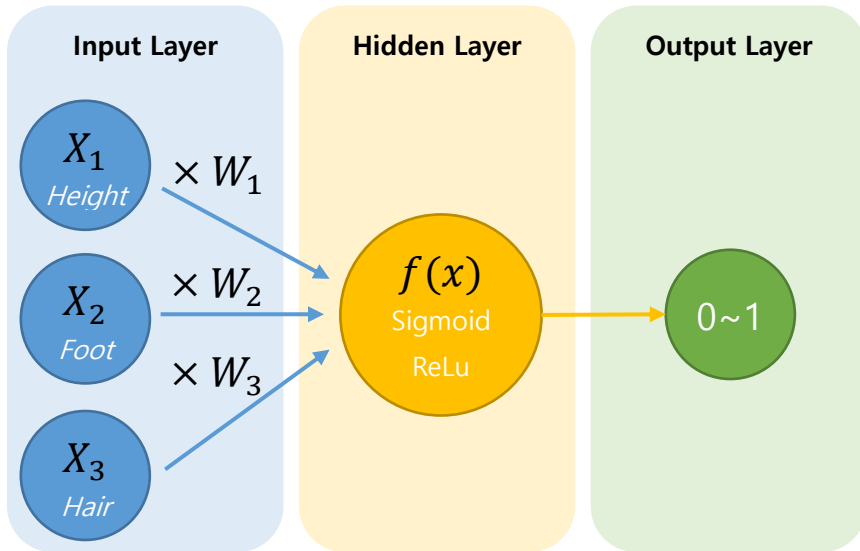


그림 126 인공지능망 모델 계층

이어서 인공지능망은 은닉층과 출력층 노드의 개수를 조절할 수 있습니다. 다수의 입력을 받아 다수의 출력을 할 수도 있고 다수의 활성화 함수를 뒤서 복잡한 학습이 가능하도록 할 수도 있습니다.

키, 발 크기, 머리카락 길이와 연령층 데이터가 주어졌을 때 복합적 요인에 따른 연령층을 예측하기 위한 인공지능망 모델을 구하고자 하면, 다음과 같이 설계할 수 있습니다.

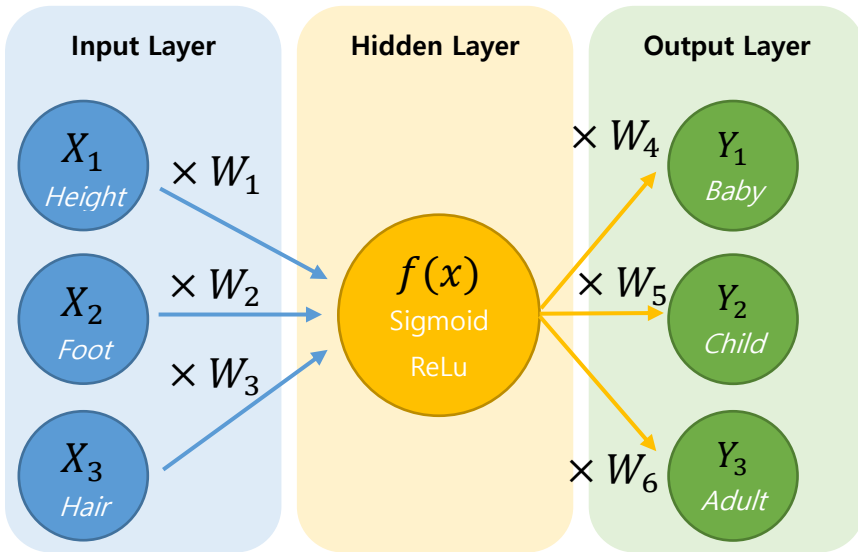


그림 127 인공신경망 모델

유아, 어린이, 성인에 대한 출력층 노드를 3 개로 늘립니다. 은닉 노드는 출력 노드들에 대해 가중치를 갖습니다. 만약 은닉 노드가 출력에 대한 가중치를 주지 않으면 모든 출력 노드가 같은 값을 갖게 됩니다.

그러나 은닉층에서 1 개 노드만으로 입력을 연산한다면 입력 데이터의 활용 한계가 낮고 잘못된 학습을 하게 될 가능성이 크므로 은닉층의 노드를 여러 개로 늘립니다.

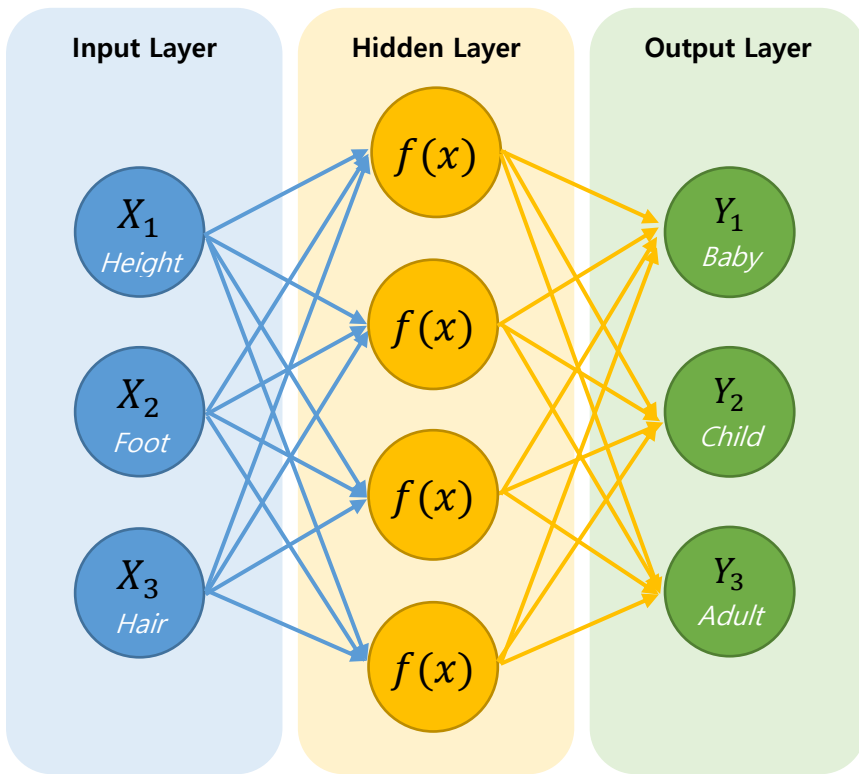


그림 128 인공신경망 모델

은닉층 노드가 여러 개일 경우 노드들이 각각 받아들이는 입력에 대한 가중치가 다르게 적용됩니다. 그러므로 출력에 대한 은닉 노드의 영향력을 분산합니다. 예컨대, 한 노드가 잘못된 학습을 하더라도 나머지 3 개의 노드에서 정상적인 값을 출력하기에 모델의 오차를 획기적으로 감소시킬 수 있습니다.

예시 모델에서는 3 개의 입·출력 노드에 대한 가중치를 4 개의 은닉 노드가 각각 다르게 갖게 되므로 입력-은닉 구간에서 12 개의 가중치와 은닉-출력 구간에서 12 개의 가중치를 이용하게 됩니다.

최종적으로 은닉층에서 나온 3 개의 출력은 실수 범위를 갖기 때문에 합산 범위를 알 수 없습니다. 결과를 활용하기 위해선 이를 정규화할 필요

가 있습니다. 그러므로 소프트맥스 함수를 이용해 출력 노드들의 합이 1이 되도록 조정합니다.

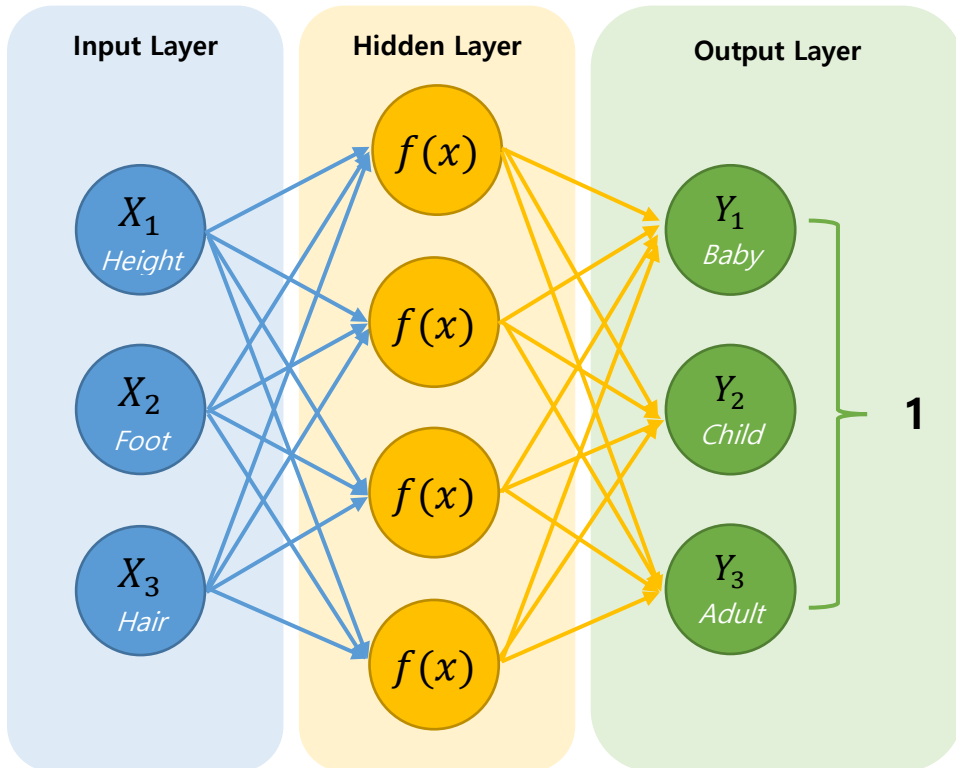


그림 129 인공지능망 모델

앞서 언급한 소프트맥스 회귀 역시 이러한 인공지능망을 이용해 구현할 수 있습니다.

인공지능망 실습 과정 역시 Jupyter 환경에서 진행되며 Pop.AI 라이브러리를 사용합니다. Jupyter 환경에 대해서는 19 페이지를 참고할 수 있습니다. 키, 발 크기, 머리카락 길이, 연령층에 대한 데이터를 입력받아 학습하고 새로운 데이터를 입력하면 연령층을 예측하는 모델을 실습합니다.

ANN 객체의 파라미터에는 `input_size`, `hidden_size`, `output_size`,

restore, ckpt_name, softmax 가 있습니다.

- input_size: 입력 데이터의 크기를 의미하며 최하위 차원의 크기를 입력합니다.
 - 필수 입력
- hidden_size: 은닉층의 노드 수를 의미합니다. hidden_size 를 조절하여 더 복잡한 학습이 가능하지만 크기가 커질수록 학습 속도는 느려집니다.
 - 기본값: 10
- output_size: 결과 데이터의 크기를 의미합니다. 최하위 차원의 크기를 입력합니다.
 - 기본값: 1
- restore: 최근 모델에 이어서 학습할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: False
- ckpt_name: 저장 및 불러올 모델 파일의 이름입니다.
 - 기본값: ANN
- softmax: 결과 데이터의 크기가 2 이상일 때, 모델의 예측 결과에 소프트맥스 함수를 적용하여 총합이 1이 되도록 할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: True

다음 소스를 작성해 Pop.AI 라이브러리를 가져오고, ANN 객체의 input_size 파라미터를 3, output_size 파라미터를 3 으로 설정하여 ANN 이라는 변수에 생성합니다.

```
01: from pop import AI
02:
03: ANN = AI.ANN(input_size=3, output_size=3)
```

ANN 객체의 속성에도 X_data 와 Y_data 가 있습니다. 입력할 데이터들은 유아, 어린이, 성인 각 분류별 3 개의 데이터를 사용합니다.

```
04: ANN.X_data = [[73,90,1], [62,70,0], [83,100,2], [110,150,7],
                  [139,220,15], [123,190,10], [177,275,7], [159,240,35],
                  [182,280,15]]
05: ANN.Y_data = [[1,0,0], [1,0,0], [1,0,0], [0,1,0], [0,1,0], [0,1,0],
                  [0,0,1], [0,0,1], [0,0,1]]
```

ANN 객체의 train() 메소드를 사용하면 인공신경망 학습을 시작할 수 있습니다. train 메소드의 파라미터에는 times 와 print_every 가 있습니다. 또, 기본값은 100 과 10 으로 설정되어있습니다.

train 메소드를 실행하면 10 회마다 인공신경망 모델의 오차를 출력합니다.

```
06: ANN.train()
```

[출력]

```
10 step loss: 1.0509754
20 step loss: 0.818166
30 step loss: 0.61001015
40 step loss: 0.51418316
50 step loss: 0.47884634
60 step loss: 0.46030685
70 step loss: 0.450966
80 step loss: 0.44335824
90 step loss: 0.43202797
100 step loss: 0.42544582
Training is done.
Time spent: 1.0 s
Training speed: 97.9 step/s
```

ANN 객체의 run() 메소드를 사용하면 학습된 모델을 사용할 수 있습니다. 파라미터에는 inputs 가 있습니다. 또, 기본값은 X_data 를 사용하도록 설정되어 있습니다.

run 메소드를 실행하면 입력에 대한 인공신경망 모델의 세대 분류 결과를 출력합니다.

```
07: ANN.run()
```

[출력]

```
run with : [[73, 90, 1], [62, 70, 0], [83, 100, 2], [110, 150, 7], [139, 220, 15], [123,
190, 10], [177, 275, 7], [159, 240, 35], [182, 280, 15]]
array([[9.87178683e-01, 1.03034209e-02, 2.51701847e-03],
       [9.98877823e-01, 9.37349454e-04, 1.84125151e-04],
       [9.91726816e-01, 7.00195367e-03, 1.26888242e-03],
       [1.32428184e-02, 7.19279289e-01, 2.67474025e-01],
       [1.00601485e-10, 2.86233485e-01, 7.13764250e-01],
       [2.92381884e-07, 4.10305083e-01, 5.89694619e-01],
       [8.54992038e-07, 4.43309069e-01, 5.56689978e-01],
       [6.84980717e-19, 2.27755815e-01, 7.72243202e-01],
       [2.85809126e-10, 3.88095438e-01, 6.11904621e-01]], dtype=float32)
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import AI
02:
03: ANN = AI.ANN(input_size=3, output_size=3)
04:
```

```
05: ANN.X_data = [[73,90,1], [62,70,0], [83,100,2], [110,150,7],  
[139,220,15], [123,190,10], [177,275,7], [159,240,35],  
[182,280,15]]  
06: ANN.Y_data = [[1,0,0], [1,0,0], [1,0,0], [0,1,0], [0,1,0], [0,1,0],  
[0,0,1], [0,0,1], [0,0,1]]  
07:  
08: ANN.train()  
09: ANN.run()
```

100 회 학습한 인공신경망 모델의 실행 결과가 비교적 구분이 쉬운 유아는 잘 구별하지만, 어린이와 성인에 대한 분류는 부정확한 것을 확인할 수 있습니다. 추가 학습을 위해 train 메소드의 times 파라미터를 1000으로 설정하고 학습합니다.

```
01: ANN.train(times=1000, print_every=100)
```

[출력]

```
200 step loss: 0.36396936  
300 step loss: 0.3103174  
400 step loss: 0.26555684  
500 step loss: 0.23622124  
600 step loss: 0.19967069  
700 step loss: 0.17223907  
800 step loss: 0.15140158  
900 step loss: 0.13391149  
1000 step loss: 0.11890842  
1100 step loss: 0.10637307  
Training is done.  
Time spent: 3.3 s  
Training speed: 304.6 step/s
```

이전 학습 모델에 이어서 1,000 회 학습해 총 1,100 회를 학습했고, 100 회마다 학습 오차를 출력합니다. run 메소드를 이용해 학습 모델의 예측값을 출력해보겠습니다.

```
02: ANN.run()
```

[출력]

```
run with : [[73, 90, 1], [62, 70, 0], [83, 100, 2], [110, 150, 7], [139, 220, 15], [123,
190, 10], [177, 275, 7], [159, 240, 35], [182, 280, 15]]
array([[9.9959713e-01, 3.9812620e-04, 2.3866107e-06],
       [9.9998844e-01, 7.6897259e-06, 2.2924256e-08],
       [9.9985343e-01, 1.4107778e-04, 2.3105406e-06],
       [3.4561017e-04, 9.0760958e-01, 9.2042603e-02],
       [3.1671339e-19, 7.1146411e-01, 2.8853193e-01],
       [1.8365446e-12, 8.8951766e-01, 1.1048082e-01],
       [2.2756497e-09, 1.8732229e-01, 8.1267077e-01],
       [3.7031554e-39, 1.0317733e-01, 8.9682251e-01],
       [7.8494800e-18, 8.2448967e-02, 9.1755110e-01]], dtype=float32)
```

예측값이 설정된 결괏값과 비슷한 수준으로 출력되는 것을 확인할 수 있습니다. run 메소드의 파라미터로 새로운 데이터 [[174,270,10], [57,70,1], [140,220,10]]을 입력하여 어떻게 출력하는지 확인해봅니다.

```
03: ANN.run([[174,270,10], [57,70,1], [140,220,10]])
```

[출력]

```
array([[1.1985912e-12, 1.9023199e-01, 8.0976802e-01],
       [9.8844868e-01, 1.1533152e-02, 1.8174847e-05],
       [3.0696471e-13, 7.4198306e-01, 2.5801697e-01]], dtype=float32)
```

만약, 학습 횟수를 과도하게 늘려 100,000 회를 학습시켰을 때 어떤 결과를 출력하는지 확인합니다. 시간이 많이 소요될 수 있으므로 50,000 회 정도로 낮춰 학습시켜도 좋습니다.

```
04: ANN.train(times=100000, print_every=10000)
```

[출력]

```
11100 step loss: 0.0009302711
21100 step loss: 0.00014373833
31100 step loss: 4.202472e-05
41100 step loss: 8.887607e-06
51100 step loss: 2.980216e-06
61100 step loss: 1.3775269e-06
71100 step loss: 1.059638e-07
81100 step loss: 3.7087312e-07
91100 step loss: 9.2718324e-08
101100 step loss: 8.8744514e-07
Training is done.
Time spent: 320.3 s
Training speed: 312.2 step/s
```

run 메소드의 파라미터로 `[[140,220,10], [140,200,11]]`을 입력하여 어떻게 출력하는지 확인해봅니다.

```
05: ANN.run([[140,220,10],[140,200,11]])
```

[출력]

```
array([[0.0000000e+00, 9.9997711e-01, 2.2873061e-05],
       [0.0000000e+00, 3.7771206e-02, 9.6222883e-01]], dtype=float32)
```

두 표본 데이터의 차이는 1cm의 머리카락 길이임에도 극단적인 결과를 보여줍니다.

부가적으로, ANN 객체 역시 `restore` 파라미터를 `True`로 설정하면 최근 사용한 학습 모델을 불러와 다시 사용할 수 있습니다.

```
06: ANN = AI.ANN(input_size=3, output_size=3, restore=True,
    ckpt_name="model_1")
```

7.2.3. 딥러닝 알고리즘

인공신경망의 응용 범위는 다양하고 각 알고리즘이 적용된 인공신경망마다 활용 가치가 매우 뛰어납니다. 은닉층을 늘려 비선형적 관계도 파악할 수 있는 심층신경망, 입력을 압축해 방대한 데이터를 빠르게 학습할 수 있는 합성곱 신경망, 데이터 위조 신경망 모델과 위조 판별 신경망 모델을 경쟁시켜 고수준의 위조 데이터를 생성하는 생산적 적대 신경망, 데이터들의 순서와 각 데이터 간 상호관계를 학습하는 순환신경망까지 설명하고자 합니다.

7.2.3.1. 심층신경망(Deep Neural Network, DNN)

심층신경망은 더욱 더 복잡한 모델링을 위해 여러 개의 은닉층을 가지고 있는 인공신경망입니다. 많은 은닉 노드를 거치기 때문에 더 많은 클래스를 출력할 수 있고, 입력 데이터와 출력 데이터 사이에서 쉽게 드러나지 않는 비선형적 관계까지 파악할 수 있습니다.

키, 발 크기, 머리카락 길이와 성별, 연령층, 손 크기에 관한 데이터가 주어졌을 때 복합적 요인에 따른 성별, 연령층, 손 크기를 예측하기 위한 심층신경망 모델을 구하고자 합니다. 성별 값은 True(1)가 여성, False(0)가 남성일 경우로 설정합니다.

인공신경망과 비슷한 구조로 출력에 성별과 손 크기만 추가되어 있습니다. 성별 값은 0~1 사이로 출력되도록 시그모이드 또는 ReLu 함수를 사용하고, 연령층 값들은 그 합이 1이 되도록 소프트맥스 함수를 사용합니다.

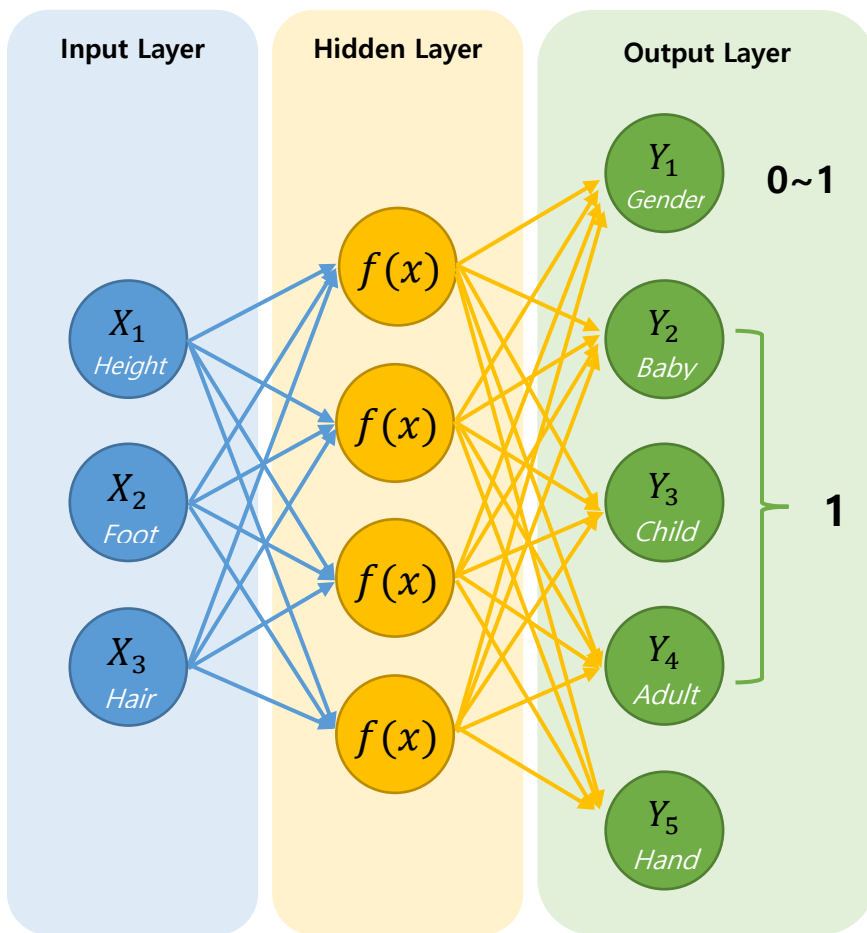


그림 130 인공신경망 모델

이런 형태의 인공신경망 모델은 하나의 은닉층에서 복잡한 모델링을 하기엔 무리가 있습니다. 은닉층을 여러 개로 늘려 연결하면 은닉층 간 다양한 네트워크를 구성하기 때문에 복잡한 모델링을 하기에 유리합니다.

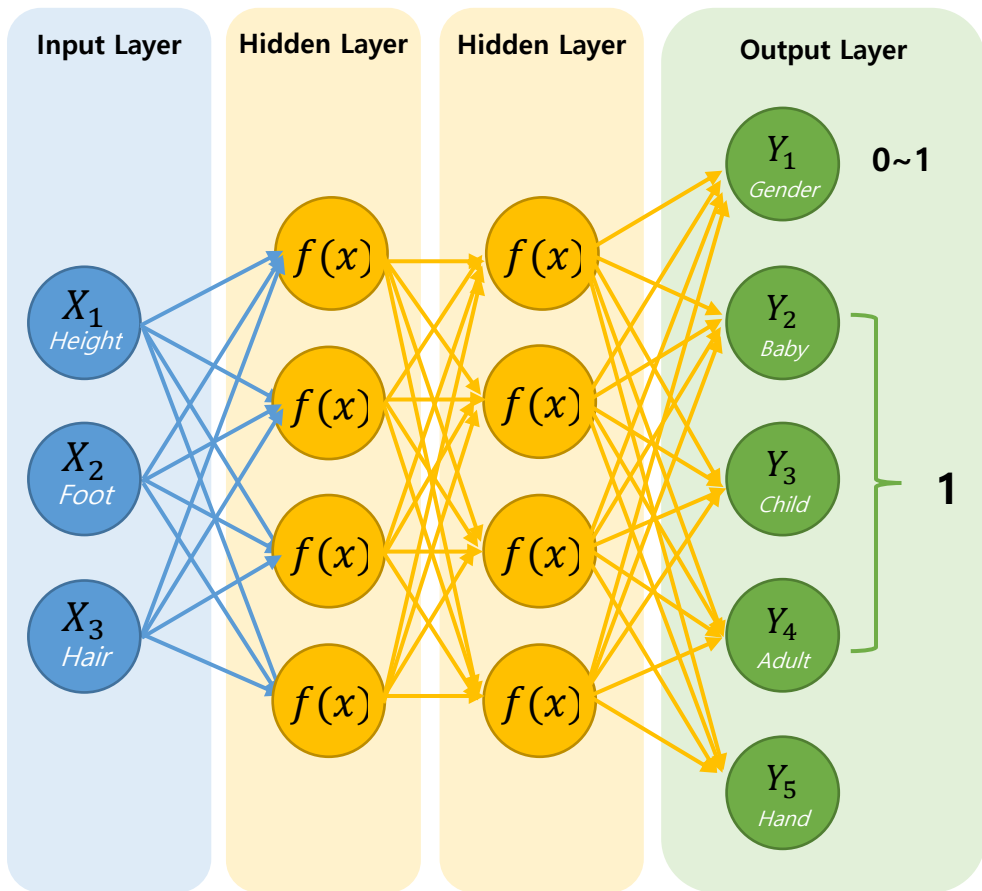


그림 131 심층신경망 모델

최적화에 성공한 심층신경망 모델은 새로운 복합 데이터가 주어졌을 때 여성일 확률, 연령층, 손 크기를 예측합니다.

심층신경망 실습 과정 역시 Jupyter 환경에서 진행되며 Pop.AI 라이브러리를 사용합니다. Jupyter 환경에 대해서는 19 페이지를 참고할 수 있습니다. 이번 실습에는 쉬운 이해를 위해 앞서 설명에 사용된 복잡한 데이터보다는 퍼셉트론 실습의 데이터를 활용합니다. 그러므로 퍼셉트론과 비교해 얼마나 적은 시간으로 더 효율적인 학습을 하는지에 대해 초점을 맞춥니다.

DNN 객체의 파라미터에는 `input_size`, `hidden_size`, `output_size`, `layer_level`, `restore`, `ckpt_name`, `softmax` 가 있습니다.

- `input_size`: 입력 데이터의 크기를 의미하며 최하위 차원의 크기를 입력합니다.
 - 필수 입력
- `hidden_size`: 은닉층의 노드 수를 의미합니다. `hidden_size` 를 조절하여 더 복잡한 학습이 가능하지만 크기가 커질수록 학습 속도는 느려집니다.
 - 기본값: 10
- `output_size`: 결과 데이터의 크기를 의미합니다. 최하위 차원의 크기를 입력합니다.
 - 기본값: 1
- `layer_level`: 은닉층의 수를 의미합니다. `layer_level` 을 조절하여 더 깊은 신경망을 만들어 복잡한 학습이 가능하지만, 은닉층 차원이 커질수록 학습 속도는 느려지고 과적합 현상이 쉽게 발생할 가능성이 커집니다.
 - 기본값: 3
- `restore`: 최근 모델에 이어서 학습할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: False
- `ckpt_name`: 저장 및 불러올 모델 파일의 이름입니다.
 - 기본값: DNN
- `softmax`: 결과 데이터의 크기가 2 이상일 때, 모델의 예측 결과에 소프트맥스 함수를 적용하여 총합이 1이 되도록 할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: True

다음 소스를 작성해 Pop.AI 라이브러리를 가져오고, DNN 객체의 `input_size` 파라미터를 3, `output_size` 파라미터를 2, `layer_level` 파라미터를 5로 설정하여 DNN 이라는 변수에 생성합니다.

```
01: from pop import AI
02:
03: DNN=AI.DNN(input_size=3, output_size=2, layer_level=5)
```


DNN 객체의 속성에도 X_data 와 Y_data 가 있습니다. 입력할 데이터들은 키, 발 크기, 머리카락 길이 데이터를 사용하고 결과 데이터로는 성별 데이터를 사용합니다. 성별 데이터는 퍼셉트론 실습과 다르게 [1, 0] 이면 여성, [0, 1] 이면 남성으로 설정합니다. 즉, 리스트의 첫 번째 값이 여성일 확률, 두 번째 값이 남성일 확률을 의미합니다.

```
04: DNN.X_data = [[173,270,17], [171,275,27], [162,245,42], [187,280,12],  
                  [157,230,47], [169,265,30], [177,270,5],  
                  [159,250,32], [182,275,0]]  
05: DNN.Y_data = [[0,1],[1,0],[1,0],[0,1],[1,0],[0,1],[0,1],[1,0],[0,1]]
```

DNN 객체의 train() 메소드를 사용하면 심층신경망 학습을 시작할 수 있습니다. train 메소드의 파라미터에는 times 와 print_every 가 있습니다. 또, 기본값은 100 과 10 으로 설정되어있습니다.

train 메소드를 실행하면 10 회마다 심층신경망 모델의 오차를 출력합니다.

```
06: DNN.train()
```

[출력]

```
10 step loss: 932.4021  
20 step loss: 120.65367  
30 step loss: 229.3342  
40 step loss: 53.072456  
50 step loss: 52.751225  
60 step loss: 51.598797  
70 step loss: 50.434517  
80 step loss: 49.640434
```

```
90 step loss: 48.07495
100 step loss: 47.787678
Training is done.
Time spent: 1.3 s
Training speed: 78.1 step/s
```

DNN 객체의 `run()` 메소드를 사용하면 학습된 모델을 사용할 수 있습니다. 파라미터에는 `inputs` 가 있습니다. 또, 기본값은 `X_data` 를 사용하도록 설정되어 있습니다.

`run` 메소드를 실행하면 입력에 대한 심층신경망 모델의 성별 분류 결과를 출력합니다.

```
07: DNN.run()
```

[출력]

```
run with : [[173, 270, 17], [171, 275, 27], [162, 245, 42], [187, 280, 12], [157, 230,
47], [169, 265, 30], [177, 270, 5], [159, 250, 32], [182, 275, 0]]
array([[0.000000e+00, 9.999962e-01],
       [0.000000e+00, 9.999962e-01],
       [9.999962e-01, 0.000000e+00],
       [0.000000e+00, 9.999962e-01],
       [9.999962e-01, 0.000000e+00],
       [9.999962e-01, 2.101948e-43],
       [0.000000e+00, 9.999962e-01],
       [9.999962e-01, 0.000000e+00],
       [0.000000e+00, 1.000000e+00]], dtype=float32)
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import AI
02:
03: DNN=AI.DNN(input_size=3, output_size=2, layer_level=5)
04: DNN.X_data = [[173,270,17], [171,275,27], [162,245,42], [187,280,12],
    [157,230,47], [169,265,30], [177,270,5], [159,250,32], [182,275,0]]
05: DNN.Y_data = [[0,1],[1,0],[1,0],[0,1],[1,0],[0,1],[0,1],[1,0],[0,1]]
06: DNN.train()
07: DNN.run()
```

단 100 회만 학습한 심층신경망 모델의 실행 결과가 퍼셉트론, 인공신경망과 비교해 더 이상 학습이 필요하지 않을 정도로 준수한 결과를 출력하는 것을 확인할 수 있습니다.

run 메소드의 파라미터로 새로운 데이터 [[174,265,6], [152,230,30], [162,255,10]]을 입력하여 어떻게 출력하는지 확인해봅시다.

```
08: DNN.run([[174,265,6], [152,230,30], [162,255,10]])
```

[출력]

```
array([[0.00431357, 0.9956864 ],
       [0.81640583, 0.18359424],
       [0.01818254, 0.9818174 ]], dtype=float32)
```

7.2.3.2. 합성곱 신경망(Convolutional Neural Network, CNN)

합성곱 신경망은 음성과 이미지 데이터처럼 입력 데이터의 양이 많을 때 입력 데이터를 압축하여 모델링하는 인공신경망입니다. 필요한 수준으로 압축하여 성능과 속도를 모두 확보할 수 있습니다.

예를 들어, 텐서 데이터가 28x28 사이즈 일 때 데이터의 크기는 784 가 됩니다. 이를 심층신경망에 입력하려면 적절한 학습을 위해 은닉층과 노드도 늘려야 하므로 연산량이 급격히 늘어나게 되어 학습 속도가 매우 느려집니다. 또, 성공적인 학습이 되어도 응용 과정에서 매우 느린 처리 속도를 보입니다. 그렇기에 대부분의 이미지 인식 인공신경망은 합성곱 신경망을 사용합니다.

합성곱 신경망은 입력 데이터를 압축하는 과정에서 슬라이드 윈도우 Slide Window 방식을 사용합니다. 슬라이드 윈도우 방식은 입력 데이터보다 작은 사이즈의 배열을 순서대로 옮겨가며 연산하는 방식입니다. 이때 옮겨가며 연산하는 과정을 필터링 Filtering 또는 마스킹 Masking 이라고 하며, 필터링에 사용되는 배열을 커널 Kernel 또는 윈도우 Window 라고 합니다.

다음과 같이 4x4 텐서 데이터가 주어졌을 때 임의로 2x2 가중치 커널을 생성합니다.

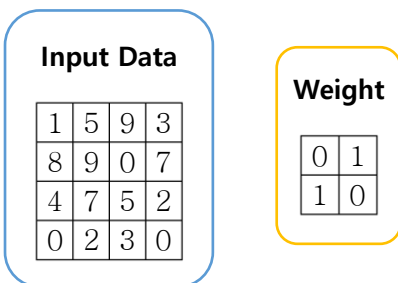


그림 132 4x4 텐서 데이터(좌), 2x2 가중치 커널(우)

커널을 이용해 데이터의 왼쪽 위부터 값을 곱한 후 모두 합해 1 개 값으

로 반환하고 옆으로 한 칸 옮겨 반복합니다. 이 과정을 필터링 Filtering 또는 마스킹 Masking 이라 합니다.

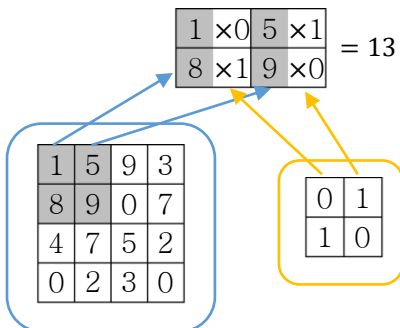


그림 133 데이터 필터링

필터링 중 중간 과정을 그림으로 표현하면 다음과 같습니다.

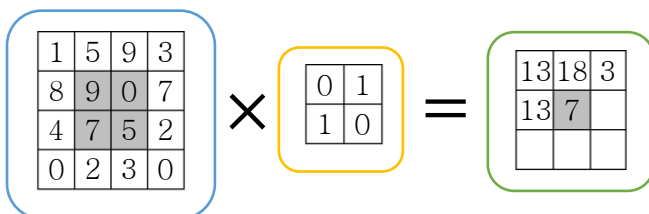


그림 134 데이터 필터링 진행 과정

필터링이 완료되면 3x3 사이즈의 압축 데이터가 반환되고, 이 데이터를 심층신경망에 연결하면 보다 빠른 속도를 기대할 수 있습니다.

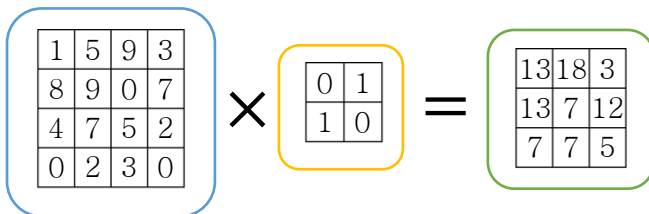


그림 135 압축된 데이터

커널이 슬라이드 할 때, 한 번에 넘어갈 칸 수를 스트라이드 Stride 라고 합니다. 만약 스트라이드를 2로 설정하면 2칸씩 건너뛰어 필터링합니다.

$$\begin{array}{|c|c|c|c|} \hline 1 & 5 & 9 & 3 \\ \hline 8 & 9 & 0 & 7 \\ \hline 4 & 7 & 5 & 2 \\ \hline 0 & 2 & 3 & 0 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline 0 & 1 \\ \hline 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 13 & 3 \\ \hline 7 & 5 \\ \hline \end{array}$$

그림 136 스트라이드가 2인 압축 데이터

이 내용을 이용해 0 또는 1을 그린 4x4 이미지를 분류하는 합성곱 신경망 모델을 구하고자 합니다. 그림 데이터에서 흰색을 0, 검은색을 1로 설정하고 스트라이드가 2인 2x2 커널을 이용해 압축 데이터를 구합니다.

$$\begin{array}{|c|c|c|c|} \hline \text{Input Data} & & & \\ \hline 0 & 1 & 1 & 0 \\ \hline 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline 0 & 1 & 1 & 0 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline \text{Weight} & & \\ \hline 0 & 1 \\ \hline 1 & 0 \\ \hline \end{array} = \begin{array}{|c|c|} \hline \text{Return} & & \\ \hline 2 & 0 \\ \hline 0 & 2 \\ \hline \end{array}$$

그림 137 압축된 0 데이터

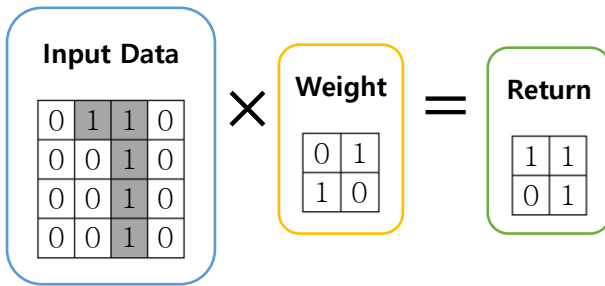


그림 138 압축된 1 데이터

반환된 값을 나열하여 심층신경망에 입력합니다. 여기서 압축 데이터를 나열하는 과정을 평탄화 Flattening 라 하고 압축 데이터와 심층신경망을 연결하는 은닉층을 완전 연결 계층 Fully-connected layer 라고 합니다.

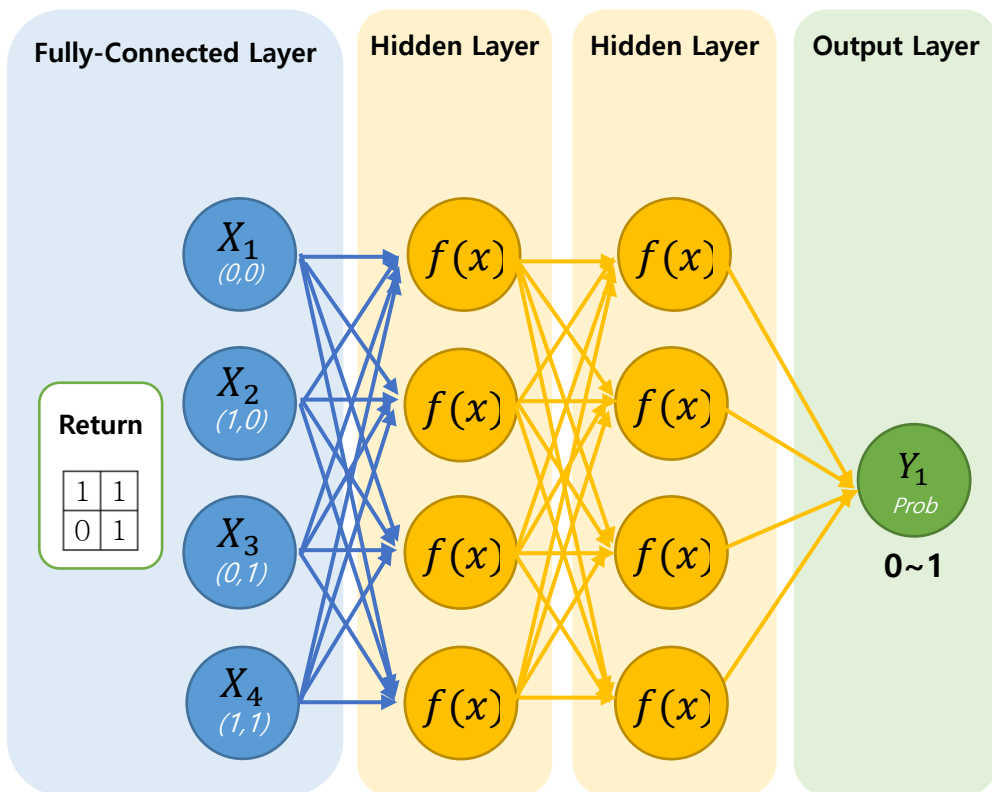


그림 139 합성곱 신경망 모델

합성곱 신경망 실습 과정 역시 Jupyter 환경에서 진행되며 Pop.AI 라이브러리를 사용합니다. Jupyter 환경에 대해서는 19 페이지를 참고할 수 있습니다. 손글씨로 쓴 숫자 이미지를 입력받아 학습하고 새로운 이미지 데이터를 입력하면 어떤 숫자인지 식별하는 모델을 실습합니다.

CNN 객체의 파라미터에는 `input_size`, `input_level`, `kernel_size`, `kernel_level`, `strides`, `hidden_size`, `output_size`, `conv_level`, `layer_level`, `restore`, `ckpt_name`, `softmax` 가 있습니다.

- `input_size`: 이전 신경망 객체와 다르게 2 차원 데이터를 입력 데이터로 사용하기에 [28, 28]과 같이 리스트로 입력합니다.
 - 기본값: [28, 28]
- `input_level`: 색상에 관련된 입력으로 RGB 이미지인 경우 3, 흑백 이미지인 경우 1 을 입력합니다.
 - 기본값: 1
- `kernel_size`: 합성곱 계층에서 사용할 커널의 크기이며 [3, 3]과 같이 리스트로 입력합니다.
 - 기본값: [3, 3]
- `kernel_count`: 하나의 입력 데이터에 사용할 가중치 커널의 개수입니다. 커널의 개수가 많아지면 더 다양한 특징을 찾아내지만, 속도는 급격히 느려집니다.
 - 기본값: 32
- `strides`: 커널의 스트라이드를 설정하고, [3, 3]과 같이 리스트로 입력합니다.
 - 기본값: [1, 1]
- `hidden_size`: 은닉층의 노드 수를 의미합니다. `hidden_size` 를 조절하여 더 복잡한 학습이 가능하지만 크기가 커질수록 학습 속도는 느려집니다.
 - 기본값: 128
- `output_size`: 결과 데이터의 크기를 의미합니다. 최하위 차원의 크기를 입력합니다.
 - 기본값: 1
- `conv_level`: 합성곱 계층의 개수를 설정합니다. 이 값이 커질수록 합성곱 계층은 깊어집니다.
 - 기본값: 2

- `layer_level`: 은닉층의 수를 의미합니다. `layer_level` 을 조절하여 더 깊은 신경망을 만들어 복잡한 학습이 가능하지만, 은닉층 차원이 커질수록 학습 속도는 느려지고 과적합 현상이 쉽게 발생할 가능성이 커집니다.
 - 기본값: 1
- `restore`: 최근 모델에 이어서 학습할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: False
- `ckpt_name`: 저장 및 불러올 모델 파일의 이름입니다.
 - 기본값: CNN
- `softmax`: 결과 데이터의 크기가 2 이상일 때, 모델의 예측 결과에 소프트맥스 함수를 적용하여 총합이 1이 되도록 할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: True

다음 소스를 작성해 Pop.AI 라이브러리를 가져오고, CNN 객체의 `output_size` 파라미터를 10 으로 설정하여 CNN 이라는 변수에 생성합니다.

```
01: from pop import AI
02:
03: CNN = AI.CNN(output_size=10)
```

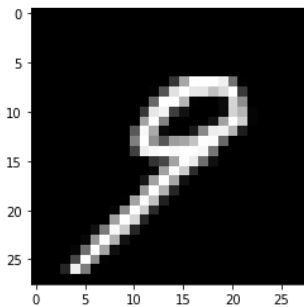
CNN 객체의 속성에도 `X_data` 와 `Y_data` 가 있지만, 이미지 데이터는 직접 입력하기에 무리가 있습니다. 그러므로 MNIST 데이터셋을 다운로드 받아 사용합니다. MNIST 는 손으로 쓴 숫자 이미지 데이터 베이스입니다. CNN 객체의 `load_MNIST()` 메소드를 호출하면 자동으로 데이터셋을 불러와 `X_data` 와 `Y_data` 에 입력합니다. 만약 장치에 데이터셋이 없다면 자동으로 다운로드 합니다. 단, 이 경우 데이터셋을 다운로드해야하므로 인터넷 연결이 필요합니다.

```
04: CNN.load_MNIST()
```

또, CNN 객체의 `show_img` 메소드를 사용하면 이미지를 출력해볼 수 있습니다. `show_img` 의 파라미터는 `input` 이 있으며 `x`, `y`, `color` 를 담는 3 차원 배열을 입력합니다. 이미지 사이즈는 CNN 객체 선언이 사용된 `input_size` 로 지정합니다.

```
05: CNN.show_img(CNN.X_data[501])
```

[출력]



CNN 객체의 `train()` 메소드를 사용하면 합성곱 신경망 학습을 시작할 수 있습니다. `train` 메소드의 `times` 와 `print_every` 가 100 과 10 으로 설정되어 있습니다. 학습 데이터가 매우 큰 편이라 시간이 오래 걸리므로 `times` 와 `print_every` 를 각각 10, 1 로 설정하여 진행해도 좋습니다.

`train` 메소드를 실행하면 10 회마다 합성곱 신경망 모델의 오차를 출력합니다.

```
06: CNN.train()
```

[출력]

```
10 step loss: 45.40264
```

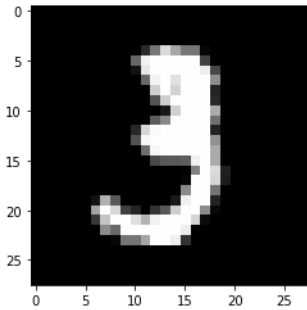
```
20 step loss: 2.0439079
30 step loss: 2.0274675
40 step loss: 1.6290655
50 step loss: 1.4405715
60 step loss: 1.2389251
70 step loss: 0.95969236
80 step loss: 0.76282966
90 step loss: 0.57149863
100 step loss: 0.70514756
Training is done.
Time spent: 223.8 s
Training speed: 0.4 step/s
```

CNN 객체의 `run()` 메소드를 사용하면 학습된 모델을 사용할 수 있습니다. 파라미터에는 `inputs` 가 있습니다. 또, 기본값은 `X_data` 를 사용하도록 설정되어있습니다. 그러나 `X_data` 의 크기가 매우 크므로 `X_data` 의 일부를 입력해야 합니다.

`run` 메소드를 실행하면 입력에 사용된 이미지를 표시하고, 입력에 대한 합성곱 신경망 모델의 숫자 분류 결과를 출력 배열의 순서대로 0~9 일 확률을 출력합니다.

```
07: X = [CNN.X_data[10]]
08: CNN.run(X)
```

[출력]



```
array([[4.5709689e-03, 2.5395965e-03, 4.2006695e-03, 9.2817014e-01,  
       8.2698063e-04, 4.0422164e-02, 1.2632059e-04, 9.4046563e-06,  
       1.9098978e-02, 3.4786102e-05]], dtype=float32)
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import AI  
02:  
03: CNN = AI.CNN(output_size=10)  
04: CNN.load_MNIST()  
05:  
06: CNN.train()  
07:  
08: X = [CNN.X_data[10]]  
09: CNN.run(X)
```

부가적으로, CNN 객체 역시 `restore` 파라미터를 `True` 로 설정하면 최근 사용한 학습 모델을 불러와 다시 사용할 수 있습니다.

```
01: CNN=AI.CNN(output_size=10, restore=True)
```


7.2.3.3. 생산적 적대 신경망(Generative Adversarial Network, GAN)

생산적 적대 신경망은 특정 데이터의 진위를 구별하는 판별 모델과 가짜 데이터를 생성하는 위조 모델을 두고 경쟁시켜 두 모델 모두 학습시켜 고 수준의 가짜 데이터를 모델링하는 인공지능망입니다.

데이터를 생성하는 인공지능망 모델을 생성자 Generator, 데이터를 판별하는 인공지능망 모델을 판별자 Discriminator 로 칭합니다.

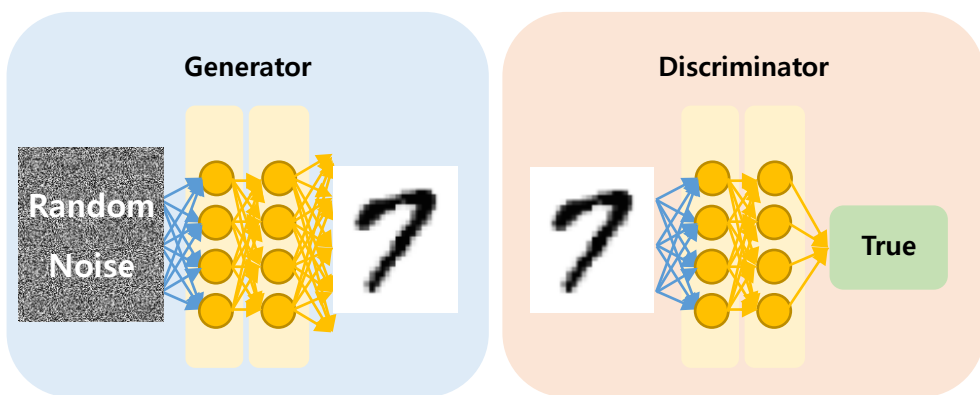


그림 140 생성자 모델(좌)과 판별자 모델(우)

생성자 모델은 랜덤 노이즈를 생성하고 가중치를 조절해 위조 데이터를 만들고, 판별자 모델은 훈련 데이터셋과 위조 데이터셋을 입력받아 위조 여부를 학습합니다. 예컨대, 시대가 발전함에 따라 범죄와 수사 수준이 동시에 발전된 것처럼 위조와 판별을 반복하며 두 모델을 계속해서 발전 시킵니다.

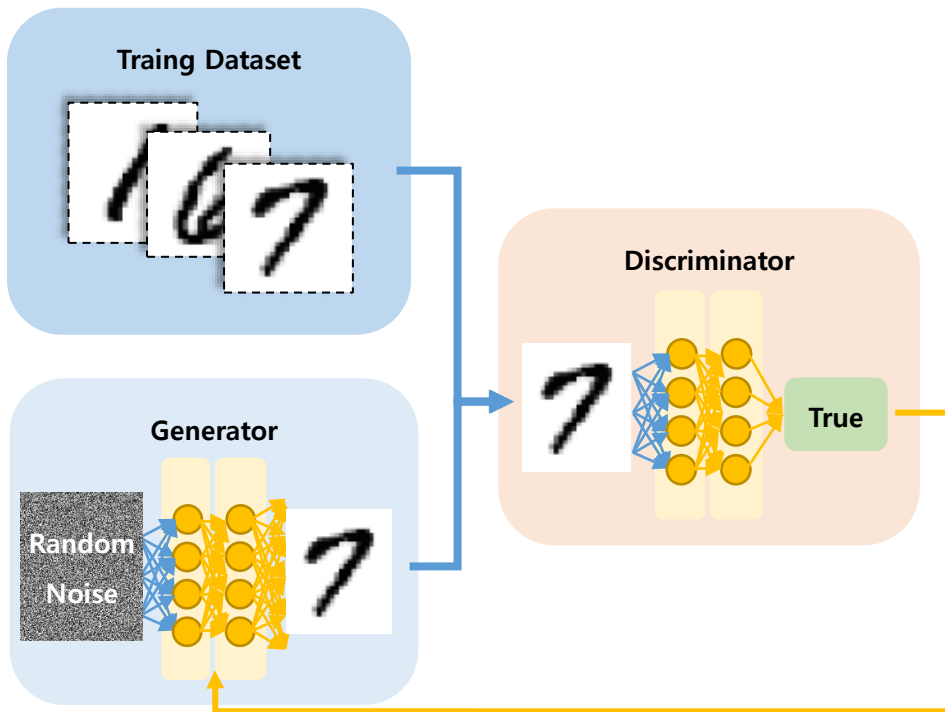


그림 141 생산적 적대 신경망 모델

성공적으로 생산적 적대 신경망 모델을 학습시키기 위해서는 두 모델의 수준이 비슷하도록 학습률 *Learning Rate* 를 조절해 유지해야 합니다. 만약 생성자 모델이 너무 뛰어나면 판별자 모델은 진짜 데이터를 가짜라고 판별하게 학습할 가능성이 크고, 반대로 판별자 모델이 너무 뛰어나면 생성자 모델은 False 피드백만 받기 때문에 진짜 같은 데이터를 생성하기 위한 가중치를 찾을 수 없게 됩니다.

7.2.3.4. 순환신경망(Recurrent Neural Network, RNN)

순환신경망은 은닉층의 출력을 다음 스텝에서 입력 데이터와 함께 다시 입력합니다. 즉, 과거 입력 데이터를 이후 처리에도 반영하여 사용하는 인공신경망입니다. 기존 딥러닝 알고리즘들의 최대 결점은 시간 개념이 없다는 것입니다. 다시 말해, 여러 데이터셋들 간의 관계나 순서를 반영하지 않습니다. 예를 들어, 합성곱 신경망으로 이미지를 인식한다고 했을 때 사물, 사람 등을 식별할 뿐 상황을 추측하진 못합니다. 또, 어떤 문장을 인식하고자 할 때 심층신경망은 각 단어의 뜻을 식별할 수 있을 진 몰라도 단어 배치 순서와 관계를 파악해 문장 전체의 의미를 추려내지는 못합니다. 순환신경망은 과거 데이터값을 요약하여 갖고 있기에 문장의 전후 관계, 이미지의 사물 관계를 파악할 수 있습니다.

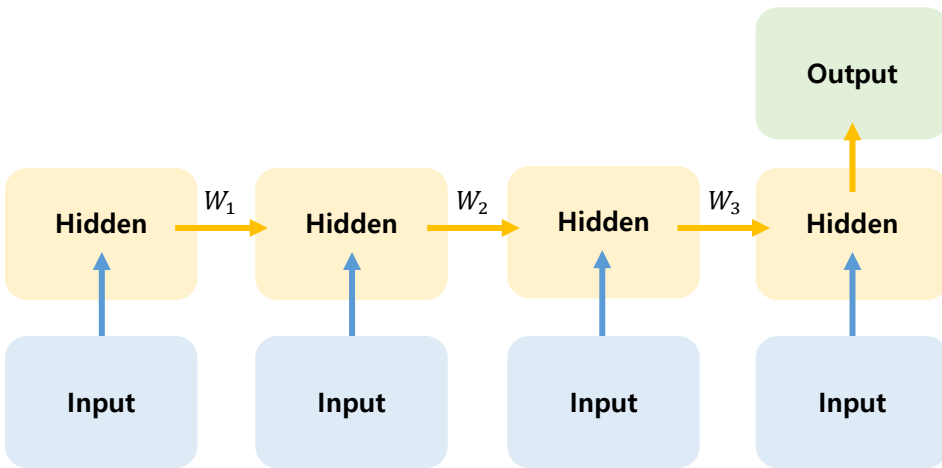


그림 142 순환신경망

다음과 같은 두 데이터셋이 입력으로 주어졌을 때 ‘Like’의 동사와 전치사를 구분하는 순환신경망 모델과 심층신경망 모델을 비교하며 설명하고자 합니다. 전치사를 True(1), 동사를 False(0)이라고 설정합니다.

문장	구분
I like you.	0

I am like you.	1
----------------	---

표 10 문장 데이터셋

두 데이터셋의 구분 핵심은 'I' 다음 'like'가 붙어 동사로써 사용되는지, 'am' 다음 'like'가 붙어 전치사로써 사용되는 지입니다.

심층신경망 모델에서는 문장 전체를 입력으로 받아 처리합니다.

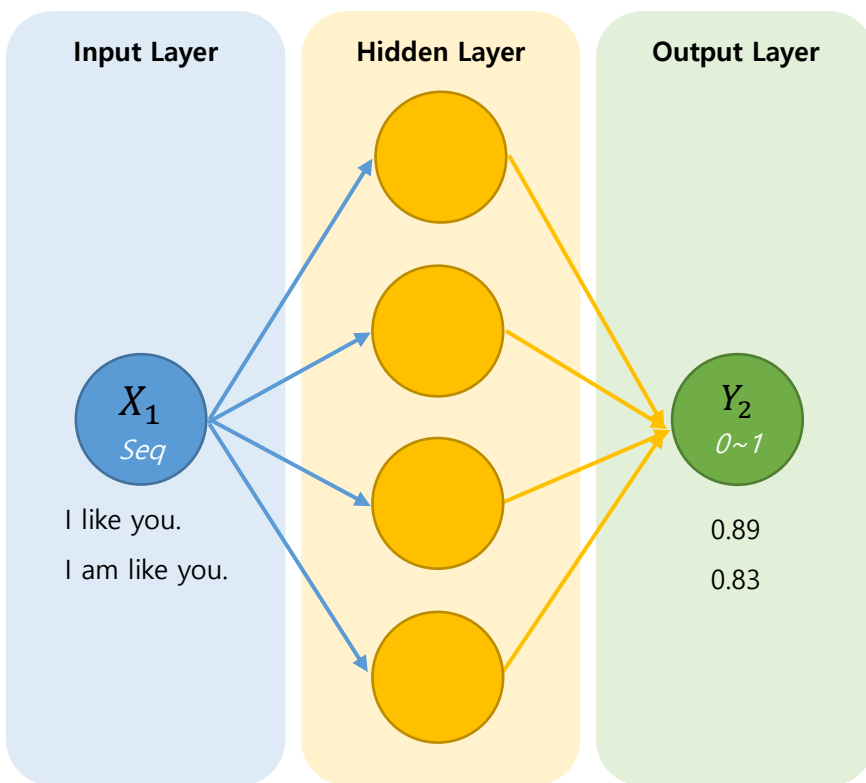


그림 143 심층신경망의 문장 구분

각 단어의 순서와 상호 관계를 알 수 없어 전체적인 형태가 매우 비슷한 두 데이터셋을 똑같이 구분할 것입니다.

반면 순환신경망은 문장에 포함된 단어를 순서대로 입력받고 다음 처리에 연결하기 때문에 'am'의 문장 결정력을 구분할 수 있습니다.

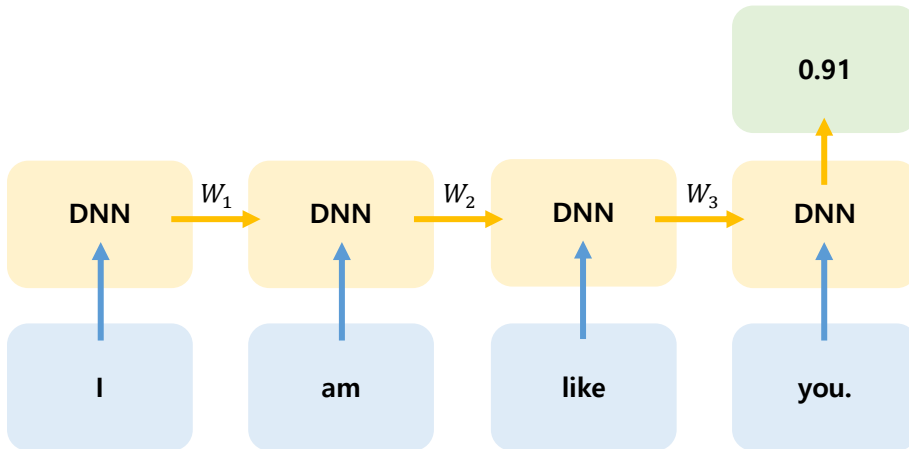


그림 144 순환신경망의 'I am like you' 구분

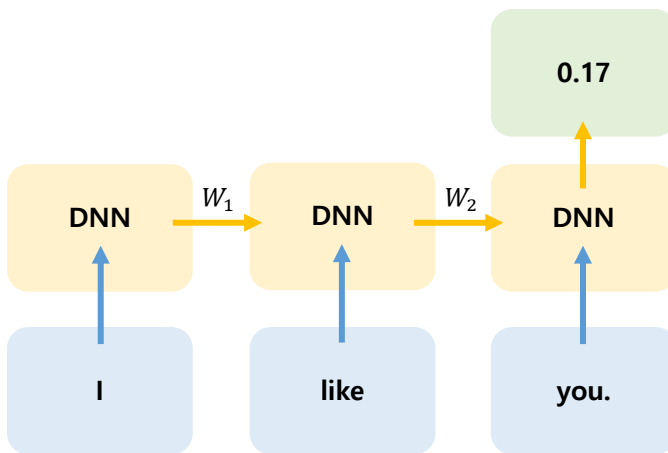


그림 145 순환신경망의 'I like you' 구분

내용정리

- 딥러닝 : 인공신경망 이론을 기반으로 설계된 머신러닝 기법
- 퍼셉트론 : 다수의 데이터로 하나의 결과를 출력하는 복합 논리 회로
 - 각 요인이 결과에 미치는 영향을 분석할 때 사용
- 인공신경망 : 퍼셉트론을 기반으로 고안되었으며, 다수의 데이터로 하나 이상의 결과를 출력하는 알고리즘
- 심층신경망 : 입력과 결과의 복잡한 관계를 분석하기 위해 여러 은닉층을 가지고 있는 인공신경망
- 합성곱 신경망 : 대량의 입력 데이터를 압축하여 분석하는 인공신경망
- 생산적 적대 신경망 : 데이터의 진위를 구별하는 판별 모델과 가짜 데이터를 생성하는 위조 모델을 경쟁시켜 모방 데이터를 생성하도록 모델링하는 인공신경망
- 순환신경망 : 이전 입력을 이후 처리에도 반영하여 데이터의 연속 관계를 분석하는 인공신경망



그림 146 자바 로고

1995

Java 1.0 은 Sun Microsystems 에서 도입했습니다. Java 플랫폼의 "Write Once, Run Anywhere"기능을 사용하면 모든 시스템에서 프로그램을 실행할 수 있으므로 Microsoft 또는 Apple 과 같은 기존의 대규모 소프트웨어 공급 업체와 독립성을 유지할 수 있습니다.



그림 147 Apple Airport Wi-Fi base

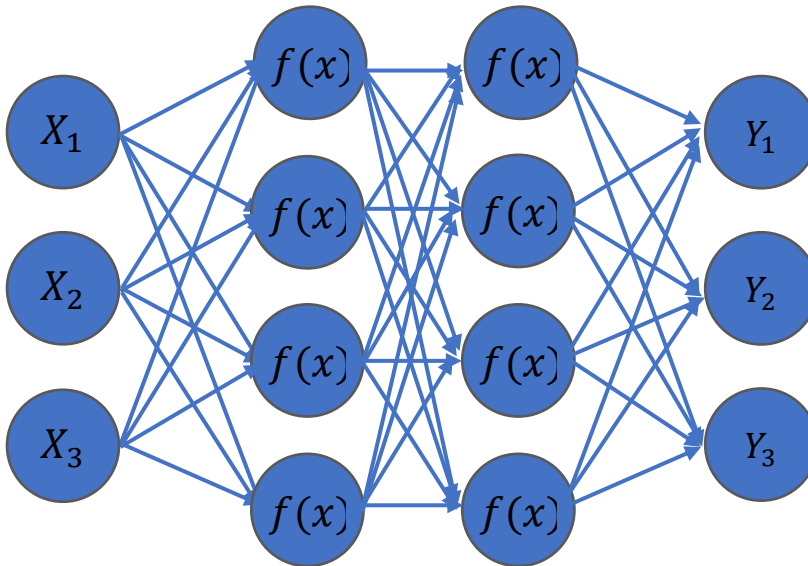
1999

1999 년, 증가하는 IEEE 802.11b 근거리 무선 네트워킹 표준은 Wi-Fi Alliance 에서 "Wi-Fi"로 이름이 변경되었습니다. 같은 해에 애플은 "공향"Wi-Fi 라우터를 출시하고 새로운 Mac 에 Wi-Fi 연결을 구축합니다. 이 제품과 다른 소비자 제품은 직장, 카페 및 가정에서 wifi 를 대중화하는 데 도움이됩니다.

연습문제

문제 39

다음 신경망 그림을 보고 질문에 답해보세요.



A. 신경망의 입력과 출력의 크기를 답해보세요.

B. 신경망의 은닉층 수를 답해보세요.

문제 40

다음 문장들을 읽고 빈 칸을 채워보세요.

A. 딥러닝은 을 기반으로 설계된 머신러닝 기법들이다.

B. 퍼셉트론은 를 로 출력하는 복합 논리 회로이다.

- C. 심층신경망은 여러개의 층을 가지고 있다.
- D. 합성곱 신경망은 방식으로 대용량 데이터를 압축한다.
- E. 합성곱 신경망의 데이터 압축 과정 중 필터링에 사용되는 배열을 이라 하고, 이것이 한 번에 이동할 칸 수를 라고 한다.
- F. 생산적 적대 신경망은 와 를 경쟁시키는 방법으로 학습하여 위조 데이터를 생성한다.

문제 41

다음 그림의 배열(좌)과 커널(우)을 이용해 질문에 답해보세요.

0	1	0	0	0
1	0	0	0	0
0	0	1	0	0
0	1	0	0	1
0	0	0	1	0

0	1
1	0

- A. 스트라이드가 3 (또는 $[3, 3]$) 일 때 출력을 작성하세요.
- B. A의 출력이 완전 연결 계층에 입력될 때 완전 연결 계층의 노드 수를 답하세요.

문제 42

다음 코드는 Pop.AI 라이브러리를 이용하여 2 개의 입력을 받아 2 개의 출력을 하는 심층신경망을 구현한 코드입니다. 질문을 읽고 답해보세요.

```
01: from pop import AI
02:
03: DNN = AI.DNN(input_size=2, output_size=2, softmax=True)
04:
05: DNN.X_data = 

|   |
|---|
| X |
|---|


06: DNN.Y_data = 

|   |
|---|
| Y |
|---|


07:
08: DNN.train(times=1000, print_every=100)
09: DNN.run()
```

- A. 두 입력에 대해 크기를 비교하여 큰 쪽을 1, 작은 쪽을 0 으로 출력할 수 있도록 빈 칸 X, Y 에 들어갈 학습 데이터셋을 작성해보세요.
- B. A 에서 작성한 데이터셋으로 학습시키고, [1,0]과 [-1,0]을 입력했을 때 출력을 작성하세요.

문제 43

다음 코드는 AIoT AutoCar 의 카메라 영상을 BGR 값으로 받아 이미지로 출력하는 코드입니다. 이 코드를 응용해 다음 문제들을 해결해보세요.

```
01: from pop import Camera, Util, delay
02:
03: cam = Camera(width=50, height=50)
04:
05: value = cam.value
06: Util.imshow("Title", value)
```


A. 다음 코드를 실행해 카메라 BGR 데이터의 형태를 확인해보세요.

```
07: print(value.shape)
```

B. 다음 사진처럼 손바닥을 카메라에 비출 때 BGR 값을 받아 이미지로 확인해보세요.



- C. 빈 배열을 생성하고, B와 같은 방법으로 다양한 손바닥 데이터를 20개 이상 추가하는 코드를 작성하고 실행해보세요.
- D. C의 배열에 이어서, 손바닥이 없는 카메라 데이터를 20개 이상 추가해보세요.
- E. 빈 배열을 생성하고, 손바닥이 있는 사진과 없는 사진을 각각 1과 0으로 하여 C에서 추가한 개수만큼 1을 추가하고 D에서 추가한 개수만큼 0을 추가하세요.
- F. Pop.AI 라이브러리의 CNN 클래스를 사용해 D에서 완성된 배열을 X 데이터, E에서 완성된 배열을 Y 데이터로 하는 합성곱 신경망 코드를 작성하세요.

G. F에서 작성한 합성곱 신경망 모델을 학습시켜 손실율을 최소화해보세요.

H. G에서 학습한 모델에 새로운 카메라 BGR 데이터를 입력하여 결과를 확인해보세요.

7.3. 강화 학습

강화 학습은 학습 초기에는 우연에 의존하여 학습하다가 점차 보상을 최대화하도록 학습합니다. 그렇기에 학습 초기 오차는 매우 높은 편이며 시간이 지날수록 급격히 오차가 줄어듭니다. 학습 데이터 가공이 필요 없고 필요한 선택만 하도록 학습하기 때문에 고성능 환경에서 짧은 시간에 많은 학습을 할 수 있습니다. 그러나 학습 초기 과정은 우연히 이루어지기 때문에 초기 학습 모델에 따라 최종 학습 모델이 달라질 수 있습니다. 학습 특징은 비지도 학습과 매우 유사하지만, 그 적용 대상이 다릅니다.

어떤 상황에 어떤 선택이 가장 합리적인지를 목적으로 학습하기 때문에 데이터 식별보다는 데이터 시뮬레이션 용도에 적합합니다. 예컨대 비지도 학습은 과일의 종류를 구별하도록 학습하는 반면, 강화 학습은 어떤 과일이 당도가 높은지 학습합니다. 대표적인 강화 학습의 예시로 구글의 알파고, 보스턴 다이내믹스의 보행 로봇 등이 있습니다.

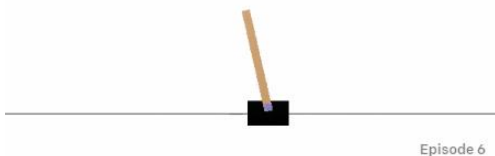


그림 148 카트를 좌우로 움직여 막대의 중심을 잡는 강화 학습

7.3.1. Deep Q-Network

딥러닝 알고리즘을 그대로 가져와 강화학습에 적용하면 학습이 전혀 되지 않습니다. 그 이유는 역전파 Back Propagation 에 있습니다. 역전파는 신경망

모델이 출력한 결과와 정답 데이터를 비교하고 어떤 가중치 변수를 얼마나 조절할지 결정하여 해당 가중치 변수에 전달하는 과정입니다.

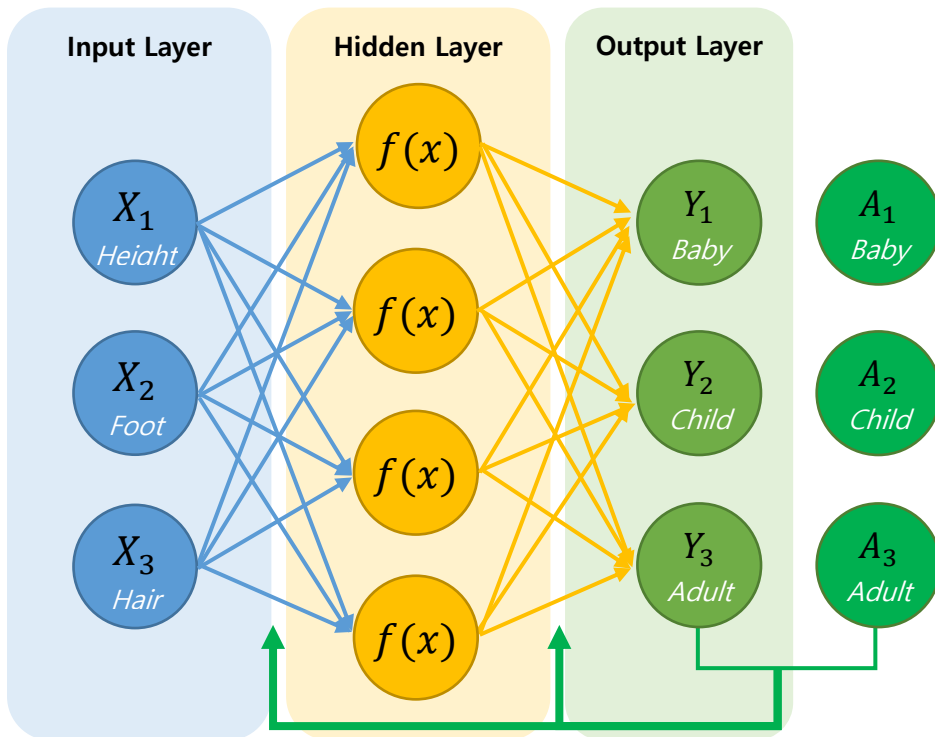


그림 149 인공지능망의 역전파 과정

역전파를 하기 위해서는 모델의 출력 데이터와 비교할 정답 데이터가 같은 의미여야 합니다. 신경망은 가중치를 조절해 결과를 출력하는데 출력한 결과가 정답 데이터와 다른 의미를 가진다면 가중치를 아무리 조절해도 원하는 결과가 나오지 않습니다. 요컨대 의미 있는 학습을 위해서는 가중치가 정답 데이터와 연관이 있어야 합니다.

그러면 강화 학습 이론을 살펴보겠습니다. 벽돌 깨기 게임을 학습한다고 가정했을 때, 게임에 대한 보상을 입력해 학습하지만 게임을 진행하기 위해서는 현재 게임의 상태를 입력해 행동합니다. 즉, 학습하는 시점에서는 ‘보상’을 입력으로 하고, 행동하는 시점에서는 ‘상태’를 입력으로 해야 합

니다. 모델이 행동할 때의 입력과 학습할 때의 입력이 전혀 다르므로 입력에 대한 가중치는 하나로 정해질 수 없습니다.

결론적으로 ‘상태’에 집중한 모델은 ‘보상’을 전혀 학습하지 못할 것이고 ‘보상’에 집중한 모델은 ‘상태’에 대한 행동을 전혀 하지 못할 것입니다. 요컨대 신경망의 핵심 중 하나인 역전파 과정이 불가능한 학습이 됩니다.

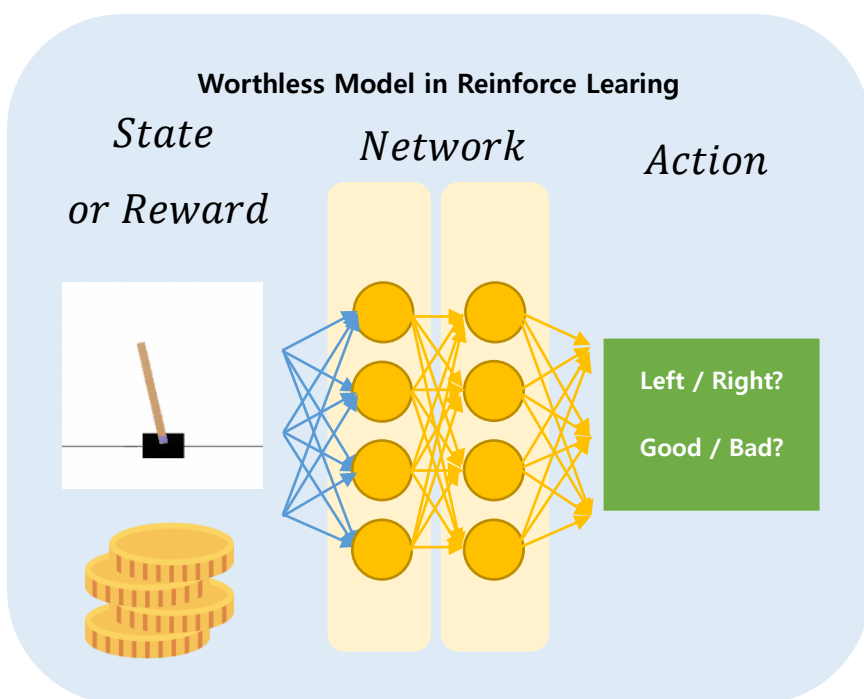


그림 150 학습 혼동 모델

이러한 문제점이 강화 학습을 실현하는 데 걸림돌이 되어왔습니다. 그러나 구글의 DeepMind 에서 DQN Deep Q-Network 을 발표하며 새로운 지평을 열게 되었습니다.

DQN 알고리즘은 게임을 진행하는 모델과 보상에 대해 학습하는 모델을 분리하고 리플레이 Replay 라는 개념을 추가해 문제를 해결했습니다. 여기

서 게임을 플레이하는 행동 모델을 에이전트 Agent 라 합니다.

리플레이는 이전에 진행했던 게임의 상태 기록, 에이전트 행동 기록, 보상 기록을 의미합니다. 에이전트는 게임이 끝날 때까지 게임의 상태를 입력받아 어떤 행동할지 출력하고 게임에서는 행동에 대한 보상을 주게 됩니다. 즉, 에이전트의 행동에 대하여 게임은 어떤 보상을 줬는지 기록합니다.

에이전트는 다음 그림과 같이 게임의 상태 S 가 주어졌을 때 신경망 N 에 입력되어 결과 데이터로 행동 A 가 출력됩니다.

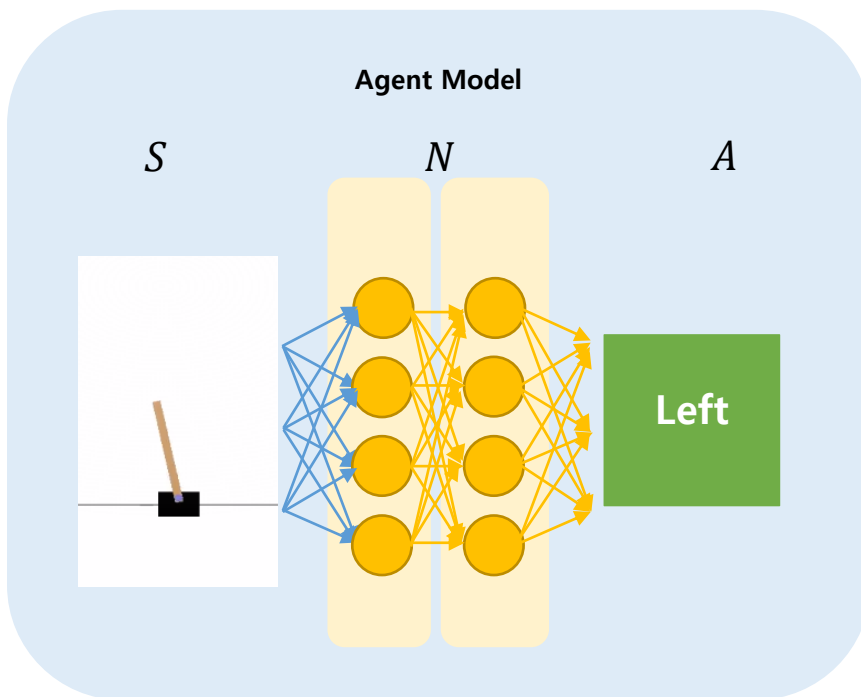


그림 151 강화 학습의 에이전트

학습 모델은 보상 R 이 주어졌을 때 리플레이 데이터를 불러와 에이전트에 입력하고 출력된 행동 데이터와 보상 R 을 비교해 에이전트의 행동이

좋은 결과를 가져왔는지, 나쁜 결과를 가져왔는지를 판단하여 에이전트의 가중치를 조절합니다.

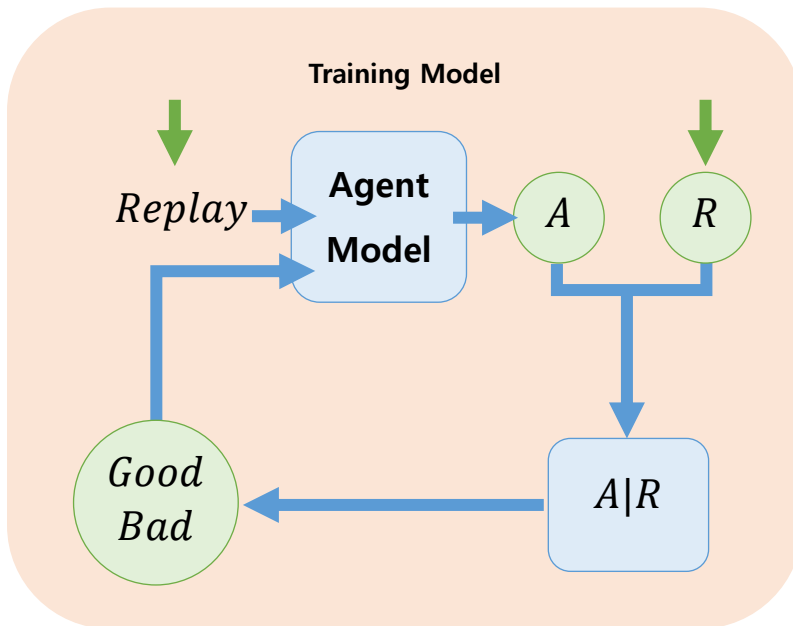


그림 152 강화 학습의 학습 모델

이러한 방법으로 에이전트는 어떤 상태에 어떤 행동을 해야 할지 알아갈 수 있습니다.

강화 학습에 유용한 라이브러리인 OpenAI Gym 을 사용하여 DQN 을 실습합니다. OpenAI Gym 은 강화 학습을 위해 그래픽 요소, 행동과 보상 등을 제공하여 알고리즘 구현에 집중할 수 있는 환경을 만들어 줍니다. 단, 알고리즘 적용을 위한 환경이므로 OpenAI Gym 그 자체에서 강화 학습을 지원하지는 않습니다. 그러므로 Pop.AI 라이브러리를 이용해 DQN 을 구현합니다.

7.3.2. OpenAI Gym

OpenAI 는 모든 인류의 이익을 위해 인공지능을 연구하는 비영리 연구 기업이며 강화학습에 대해 쉽게 배우고 개발할 수 있도록 Gym 패키지를 만들어 배포하고 있습니다. Gym 패키지에는 중심 잡기, 펜던럼 등 간단한 예제부터 아타리의 고전 게임을 학습해 클리어하는 예제까지 다양한 학습 활동이 있습니다. 이 책에서는 Gym 의 CartPole 예제를 통해 DQN 에 대한 이해를 돕고자 합니다.

실습에 활용할 Gym 패키지는 다음 명령으로 설치하는데, Soda OS 에는 미리 설치되어 있습니다.

- `pip3 install gym`
- `pip3 install gym[classic_control]`

설명할 OpenAI Gym 예제는 Jupyter 환경에서 프로그래밍합니다. 그러나 OpenAI Gym 그래픽 창을 띄울 수 없는 Jupyter 에서는 아무것도 표시되지 않습니다. 이를 해결하기 위해 Pop.Util 라이브러리에서 `imshow()` 메소드를 사용합니다. `imshow()` 메소드의 인자는 다음과 같습니다.

- `Title`: 각 출력을 구분하기 위한 창의 제목
- `Image`: 표시할 이미지 데이터
- `width`: 표시할 창의 너비
- `height`: 표시할 창의 높이
- `mode`: 이미지 데이터의 표현 방식
 - RGB, BGR

```
01: from pop import Util
02: Util.imshow("Title", Image, width=300, height=300, mode='RGB')
```

실습에 들어가기 앞서, Gym 이 어떤 식으로 환경을 제공하는지부터 살펴 보겠습니다. 다음 소스 코드를 입력해 CartPole 예제를 실행해봅니다.

```
01: import gym
02: from pop import Util
03:
04: env = gym.make('CartPole-v1')
05: env.reset()
06:
07: for _ in range(1000):
08:     img = env.render(mode = 'rgb_array')
09:     Util.imshow("CartPole", img, mode='RGB')
10:     env.step(env.action_space.sample())
11: env.close()
```

[출력]



Gym 에서 간단한 물리 엔진을 포함한 게임을 표시하고 카트가 빠른 속도로 오른쪽으로 움직여 사라지는 것을 확인할 수 있습니다. 아무런 게임 규칙이 적용되지 않아 리셋되지 않습니다. 다음 소스 코드를 입력해 게임의 규칙을 적용합니다. 게임의 규칙은 아래와 같습니다.

- 게임 종료 : 막대가 15 도 이상 기울거나 500 스텝 이상 진행될 경우 게임이 끝납니다.

- 보상 : 막대가 15 도 이하로 직립한 스텝에서 1의 보상이 주어집니다.
- 행동 : env.step 메소드에 True 또는 False 를 입력해 좌우로 움직일 수 있습니다.
- 결과 : env.step 메소드를 실행하면 결과값들이 반환됩니다.
 - state : 행동으로 인해 변화된 상태입니다. 리스트로 반환되며 차례대로 카트의 위치, 카트의 속도, 막대의 각도, 막대의 속도가 들어있습니다.
 - reward : 해당 스텝의 보상입니다. 0 또는 1 이 반환됩니다.
 - done : 게임이 종료되었는지 여부입니다.

```

01: import gym
02: from pop import Util
03:
04: env = gym.make('CartPole-v1')
05: for i_episode in range(20):
06:     state = env.reset()
07:     for t in range(100):
08:         img = env.render(mode = 'rgb_array')
09:         Util.imshow("CartPole", img, mode='RGB')
10:
11:         action = env.action_space.sample()
12:         state, reward, done, _ = env.step(action)
13:         if done:
14:             print("Episode finished after {} timesteps".format(t+1))
15:             break
16: env.close()

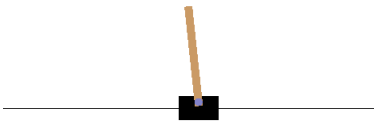
```

[출력]

```

[-0.04912004  0.20994516 -0.04768523 -0.2696619 ] 1.0 False
[-0.04492114  0.01553499 -0.05307846  0.00760773] 1.0 False
...
[-0.04038291  0.40721118 -0.05895308 -0.61023233] 1.0 False

```



행동이 랜덤으로 입력되고 그 행동으로 인한 상태, 보상, 종료 여부가 출력됩니다. 게임에서 막대가 15도 이상 기울자 게임이 리셋되는 것을 확인할 수 있습니다.

이렇게 Gym은 물리 엔진, 게임 구성 등 부가 요소에 신경 쓰지 않고 강화 학습 구현에만 집중할 수 있도록 합니다.

다음은 Pop.AI 라이브러리를 사용한 실습입니다. Jupyter 환경에서 실습이 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다.

DQN 객체의 파라미터에는 `state_size`, `hidden_size`, `output_size`, `layer_level`, `restore`, `ckpt_name`, 가 있습니다.

- `state_size`: 상태 데이터의 크기를 의미하며 최하위 차원의 크기를 입력합니다.
 - 필수 입력
- `hidden_size`: 은닉층의 노드 수를 의미합니다. `hidden_size`를 조절하여 더 복잡한 학습이 가능하지만 크기가 커질수록 학습 속도는 느려집니다.
 - 기본값: 5
- `output_size`: 결과 데이터의 크기를 의미합니다. 최하위 차원의 크기를 입력합니다.
 - 기본값: 1
- `layer_level`: 은닉층의 수를 의미합니다. `layer_level`을 조절하여 더 깊은 신경망을 만들어 복잡한 학습이 가능하지만, 은닉층 차원이 커질수록 학습 속도는 느려지고 과적합 현상이 쉽게 발생할 가능성이 커집니다.
 - 기본값: 1

- restore: 최근 모델에 이어서 학습할지에 대한 여부를 Boolean 으로 입력합니다.
 - 기본값: False
- ckpt_name: 저장 및 불러올 모델 파일의 이름입니다.
 - 기본값: DQN

다음 소스를 작성해 Gym 과 Pop.AI 라이브러리를 가져오고, AI 모듈에서 DQN 객체의 state_size 파라미터를 4 로 설정하여 DQN 이라는 변수에 생성합니다. 그리고 gym 의 CartPole 환경을 생성하여 env 변수에 저장합니다.

```
01: import gym
02: from pop import AI, Util
03:
04: DQN=AI.DQN(state_size=4)
05:
06: env = gym.make('CartPole-v1')
```

이어서, 게임을 1000 번 플레이하도록 for 루프를 만들고 env 객체의 reset 메소드로 환경을 초기화합니다. 그리고 step 과 total_reward 를 0 으로 초기화하고 리플레이 구현을 위해 상태, 보상, 행동을 기록할 리스트 생성합니다.

```
07: for i_episode in range(1000):
08:     state = env.reset()
09:     step = 0
10:     total_reward = 0
11:
12:     states, rewards, actions=[], [], []
```

한 게임을 진행할 때 게임이 종료될 때까지 행동하도록 무한 루프를 만들고 env 객체의 render 메소드로 현재 상황을 그래픽으로 출력합니다. DQN 객체의 run 메소드에 현재 상태를 담고 있는 state 를 입력하면 에이전트의 행동 값이 출력됩니다. 이 행동 값을 env.step 에 입력하면 CartPole 환경에서 행동으로 인해 변화된 상태와 보상을 출력합니다. 상태, 보상, 행동 기록을 리플레이 리스트에 추가하고 총 보상과 스텝을 갱신합니다.

```
13:     while True:
14:         img = env.render(mode = 'rgb_array')
15:         Util.imshow("CartPole", img, mode='RGB')
16:
17:         action=DQN.run([state])
18:
19:         state, reward, done, _ = env.step(action)
20:
21:         states.append(state)
22:         rewards.append(reward)
23:         actions.append(action)
24:
25:         total_reward+=reward
26:         step+=1
```

done 변수가 True 라면 한 번의 게임이 끝난 것입니다. 게임이 끝나면 이번 게임에서 몇 스텝까지 진행했는지 출력하고, DQN 객체의 train 메소드에 리플레이 리스트들을 입력하여 에이전트 가중치를 조절합니다. train 메소드의 파라미터에는 states, rewards, actions 가 있습니다.

```
27:         if done:
28:             print("Done after {} steps".format(step+1))
29:
30:             loss=DQN.train(states, rewards, actions)
```

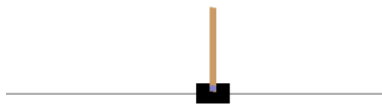
```
31:             print('episode  ' + str(i_episode  +  1)+"  
    reward : ", total_reward, ", loss : ",loss)  
32:             break
```

게임이 모두 끝나면 CartPole 환경을 종료합니다.

```
33: env.close()
```

[출력]

```
Done after 10 steps  
episode 1 reward :  9.0 , loss :  52.105938  
Done after 10 steps  
episode 2 reward :  9.0 , loss :  49.986176  
...  
Done after 396 steps  
episode 18 reward : 395.0 , loss :  1.0100955
```



처음에는 중심을 잘 못 잡고 쓰러지지만, 시간이 지날수록 점점 중심을 잘 잡는 것을 확인할 수 있습니다. 단, 강화 학습의 초반에는 우연에 의존한 학습을 하므로 학습 초반에 보상을 더 받는 방법을 찾지 못한다면 낮은 수준에만 고착되고 끝날 수 있습니다. 이럴 경우 학습을 처음부터 재시도해야 합니다.

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: import gym
02: from pop import AI, Util
03:
04: DQN=AI.DQN(state_size=4)
05:
06: env = gym.make('CartPole-v1')
07: for i_episode in range(1000):
08:     state = env.reset()
09:     step = 0
10:     total_reward = 0
11:
12:     states, rewards, actions=[], [], []
13:     while True:
14:         img = env.render(mode = 'rgb_array')
15:         Util.imshow("CartPole", img, mode='RGB')
16:
17:         action=DQN.run([state])
18:
19:         state, reward, done, _ = env.step(action)
20:
21:         states.append(state)
22:         rewards.append(reward)
23:         actions.append(action)
24:
25:         total_reward+=reward
26:         step+=1
27:
28:         if done:
29:             print("Done after {} steps".format(step+1))
30:
31:             loss=DQN.train(states, rewards, actions)
32:             print('episode ' + str(i_episode + 1)+" reward :
    ", total_reward, ", loss : ",loss)
33:             break
```

```
34:  
35: env.close()
```

내용정리

- 강화 학습 : 상황에 대한 데이터가 주어지고, 어떤 행동을 할지 출력하면 그에 따른 보상 및 벌이 발생했을 때 상황, 행동, 보상의 관계를 분석해 머신러닝 모델을 최적화 하는 방법
- DQN 알고리즘 : 게임을 진행하는 모델과 보상을 학습하는 모델을 분리하고 진행했던 게임 기록으로 보상이 발생한 행동을 학습하는 기법
- 에이전트 : 게임을 플레이하는 행동 모델
- 리플레이 : 이전에 진행했던 게임의 상태 기록, 에이전트 행동 기록, 보상 기록
- OpenAI Gym : 강화학습에 대해 쉽게 배우고 개발할 수 있도록 그래픽 환경을 제공하는 패키지



그림 153 SanDisk 4GB Cruzer flash drive

2000

USB 플래시 드라이브가 도입되었습니다. USB 인터페이스가있는 소형 폼 팩터 컨테이너에 포함 된 플래시 메모리로 구성되어 있으며 데이터 저장 및 다양한 장치 간의 파일 백업 및 전송에 사용될 수 있습니다. 이전 스토리지 미디어보다 속도가 빠르고 데이터 용량이 더 컸습니다. 또한 플로피 디스크와 달리 광학 디스크처럼 긁히지 않고 자기 소거에 탄력적이었습니다.



그림 154 2000 년 1 월 3 일에 1900 년을 잘못 표시하는 전자 기호

2000

1990 년대 후반, (Y2K) 버그는 2000 년의 시작이 통신, 금융 부문 및 기타 중요한 인프라를 무너 뜨릴 것이라는 소식을 전합니다. 이 문제는 대부분 이전에 작성된 소프트웨어의 날짜 스탬프가 연도 정보를 나타내는 데 두 자리 숫자만 사용했다는 사실에 기인합니다. 이는 일부 컴퓨터가 1900 년을 2000 년과 구별하지 못할 수 있음을 의미했습니다. 2000 년 새해 첫날에는 약간의 결함이 있었지만, 일부 비즈니스, 정부 및 산업계의 노력으로 인해 큰 문제는 발생하지 않았습니다.

연습문제

문제 44

다음 문장들을 읽고 빈 칸을 채워보세요.

- A. 신경망 모델이 출력한 결과를 비교해 가중치를 조절하는 과정을 라 한다.
- B. 강화 학습은 을 최대화하도록 학습한다.
- C. DQN 에서 게임을 플레이하는 모델을 라 한다.
- D. DQN 은 상태, 행동, 보상 기록을 통해 학습하고, 이 기록들을 라 한다.

문제 45

다음 코드는 Gym 라이브러리를 이용하여 강화 학습 환경을 구현한 코드입니다. 질문을 읽고 답해보세요.

```
01: import gym
02: from pop import Util
03:
04: env = gym.make('CartPole-v1')
05: for i_episode in range(20):
06:     state = env.reset()
07:     for t in range(100):
08:         img = env.render(mode = 'rgb_array')
09:         Util.imshow("CartPole", img, mode='RGB')
10:
11:         action = env.action_space.sample()
```

```
12:         state, reward, done, _ = env.step(action)
13:         if done:
14:             print("Episode finished after {} timesteps".format(t+1))
15:             break
16: env.close()
```

- A. 코드를 실행해보고 게임 20 회의 최대 보상값과 평균 보상값을 답해보세요.
- B. Pop.AI 라이브러리의 DQN 클래스를 사용해 보상을 최대화하는 코드를 작성해보세요.
- C. B에서 작성한 코드를 실행해 최대 보상값과 평균 보상값을 답해보세요.

7.4. Tensorflow

텐서플로우는 구글에서 제공하는 머신러닝 프레임워크입니다. ReLu, Sigmoid 등 가벼운 활성화 함수부터 경사하강법, Adam Optimizer, 합성곱 계층 구현 등 어려운 이론을 사용하기 쉬운 API 로 제공합니다. 이번 장에서는 Tensorflow 의 기본 구성을 알아보고 이를 이용해 직접 회귀와 딥러닝 알고리즘들을 구현해봅니다. 실습은 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다.

Tensorflow 1 은 Tensorflow Core 를 기반으로 동작하지만, Tensorflow 2 는 주로 Keras 를 기반으로 동작합니다. 즉, Tensorflow 1 과 2 를 각각 Tensorflow 와 Keras 로 구분할 수 있습니다. Keras 에 대해서는 다음 장에서 다루도록 하고, Tensorflow 에 대한 설명과 실습을 위해 Tensorflow 2 를 비활성화하고 Tensorflow 1 을 활성화합니다.

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
```

7.4.1. 상수, 변수와 플레이스 홀더

Tensorflow 는 특별한 연산 방법을 사용합니다. 우선 직접 눈으로 확인해본 뒤 자세히 설명하겠습니다.

다음과 같이 소스 코드를 입력해 Tensorflow 라이브러리를 불러오고 상수와 변수를 생성한 뒤 출력해봅니다.

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: C = tf.constant(10)
```

```
05: V = tf.Variable(5)
06:
07: print(C)
08: print(V)
```

[출력]

```
Tensor("Const:0", shape=(), dtype=int32)
<tf.Variable 'Variable:0' shape=() dtype=int32>
```

파이썬의 상수, 변수와 다르게 값이 출력되지 않고 객체가 출력됩니다. 그렇다면 다음 소스 코드를 추가해 실행시켜봅시다.

```
09: sess = tf.Session()
10: sess.run(tf.global_variables_initializer())
11:
12: print(sess.run(C))
13: print(sess.run(V))
```

[출력]

```
10
5
```

이제야 상수, 변수의 값이 출력되는 것을 볼 수 있습니다. 이렇듯 세션 Session이라는 개념이 있고, 세션에서 연산을 시작해야 결과가 출력되는 것을 알 수 있습니다. 즉, 텐서플로우는 프로그램의 흐름과 또 다른 연산 흐름인 ‘세션’에서 연산하도록 설계되어 있습니다. 또, 변수의 경우 tf.global_variables_initializer() 메소드로 변수를 초기화해야만 연산이 가능합니다.

세션이라는 개념을 배제한다면 텐서플로우는 상수, 변수는 파이썬의 상수, 변수와 같은 역할을 한다고 볼 수 있습니다. 상수는 처음 선언에서

절대 변하지 않으며, 변수는 사용자나 세션에 의해 변할 수 있습니다.

텐서플로우에서는 상수와 변수 외에 플레이스 홀더 `Placeholder` 라는 개념이 있습니다. 플레이스 홀더는 단어 그대로 세션에 연산에 사용될 값의 자리를 미리 잡아놓는 역할을 합니다. 풀어서 설명하면, 세션의 흐름과 프로그램의 흐름이 따로 존재한다고 가정합니다. 또, 프로그램에서 세션의 흐름을 정의해야 합니다. 그러나 프로그램은 사용자가 어떤 입력을 할지 알 수 없습니다. 그래서 프로그램은 세션의 사용자 입력 공간에 값을 특정할 수 없기에 빈 공간으로 생성해놓는 것입니다.

이해를 위해 직접 사용해보겠습니다.

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: C = tf.constant(10)
05: V = tf.Variable(5)
06: X = tf.placeholder(tf.int32)
07:
08: sess = tf.Session()
09: sess.run(tf.global_variables_initializer())
10:
11: F1 = C * X
12: F2 = V * X
13:
14: R1 = sess.run(F1, feed_dict = {X : 2})
15: R2 = sess.run(F2, feed_dict = {X : 5})
16:
17:
18: print(R1)
19: print(R2)
```

[출력]

20

25

이렇게 세션에서 `feed_dict` 파라미터를 이용해 프로그램의 입력을 세션의 플레이스 홀더로 전달하는 것을 확인할 수 있습니다.

7.4.2. 그래프와 세션

텐서플로우에는 그래프 `Graph` 와 세션 `Session` 이라는 개념이 있습니다. 그래프란 하나의 텐서 연산 묶음이라 표현할 수 있고 세션은 앞서 설명한 것처럼 하나의 연산 흐름이라 표현할 수 있습니다. 앞의 소스 코드에서 상수, 변수, 플레이스 홀더 등을 선언하고 이를 연산자를 이용해 연산 과정을 표현하는 것까지를 그래프라 합니다. 이 그래프를 연산하는 것이 바로 세션입니다. 즉, 앞의 소스 코드에서는 2 개의 그래프를 사용했습니다.

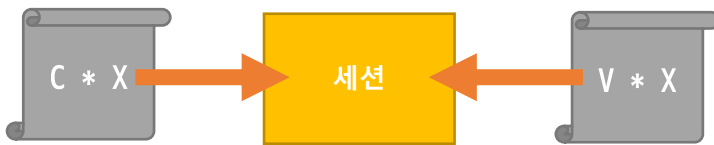


그림 155 그래프와 세션

7.4.3. 최적화 함수

결과를 비교해 가중치를 조절하는 최적화 과정은 이론만 접해본 상태에서는 구현하기 매우 까다로워 보입니다. 그러나 텐서플로우에서는 이 과정을 간단한 객체와 메소드로 제공하여 신경망 구성에 집중할 수 있도록 했습니다.

대표적인 최적화 방법인 경사하강법, Adam, NAdam 등 다양한 최적화를 제공합니다. 다음 소스 코드는 경사하강법의 예시이며, `optimizer` 변수에

GradientDescentOptimizer 객체를 생성하고 minimize 메소드를 이용해 최적화를 실행합니다. 최적화 함수에 대한 실습은 선형 회귀부터 직접 활용해보도록 합니다.

```
01: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001)
02: train_op = optimizer.minimize(loss)
```

7.4.4. 선형 회귀

선형 회귀는 가설 H 을 변수 W 와 B 를 조절하여 모든 데이터에 가장 최적으로 만들어가는 방법을 사용합니다. 선형 회귀의 기반은 1차 함수로 다음과 같은 수식으로 표현할 수 있습니다.

$$Y = W \cdot X + B$$

이 수식에서 입력되는 값은 X , 입력에 대한 결과값은 Y 이며 회귀 모델은 W 와 B 만을 조절할 수 있습니다. 모델은 W 와 B 를 조절해 입력 X 에 대한 출력과 실제 결과값 Y 를 비교해 최적의 W 와 V 를 찾아냅니다.

다음 소스 코드와 같이 텐서플로우를 불러오고, 변수와 플레이스 홀더를 생성합니다.

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: W = tf.Variable(1.)
05: B = tf.Variable(0.)
06: X = tf.placeholder(tf.float32)
07: Y = tf.placeholder(tf.float32)
```

가설 모델 H 를 생성하고 모델 H 가 출력하는 결과와 실제 결과와의 오차를 계산하는 그래프 $Loss$ 를 생성합니다. $Loss$ 에서 사용된 `reduce_mean` 메소드는 최하위 차원의 값들의 평균을 구해 차원을 줄여나가는 메소드입니다. 예를 들어 1차원 텐서 $[1, 5, 9]$ 를 `reduce_mean` 메소드에 입력하면 0차원 텐서 5가 반환됩니다.

```
08: H = W * X + B
09: Loss = tf.reduce_mean(tf.reduce_mean(tf.abs(Y - H)))
```

이어서, 텐서플로우에서 제공하는 최적화 함수를 이용해 $Loss$ 를 최소화하는 방법으로 W 와 B 를 조절합니다. 경사하강법 객체 `GradientDescentOptimizer`를 생성하고 최소화할 대상을 $Loss$ 로 지정합니다.

```
10: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001)
11: train_op = optimizer.minimize(Loss)
```

세션을 생성하고 텐서플로우 변수를 초기화합니다.

```
12: sess = tf.Session()
13: sess.run(tf.global_variables_initializer())
```

간단한 데이터셋을 만들고 `train_op`에 생성된 최적화 함수를 100회 반복 실행합니다. 또, 최적화 함수가 끝날 때마다 변화된 오차 $Loss$ 를 출력합니다.

```
14: X_data = [[0],[1],[2],[3],[4]]
15: Y_data = [[0],[2],[4],[6],[8]]
16:
17: for i in range(100):
18:     sess.run(train_op, feed_dict={X : X_data, Y : Y_data})
19:     loss = sess.run(Loss, feed_dict={X : X_data, Y : Y_data})
20:     print(loss)
```

[출력]

```
1.99552
1.9911602
...
0.9622122
```

최적화가 끝난 가설 모델 H에 임의의 데이터 n을 입력하여 결과를 확인해봅니다.

```
21: n = [[7], [-10], [358]]
22: result = sess.run(H, feed_dict={X : n})
23: print(result)
```

[출력]

```
[[ 13.96547]
 [-19.8851 ]
 [712.8801 ]]
```

입력 데이터와 출력 데이터를 학습해 $2 \times X + 0$ 에 근접하게 예측한 것을 확인할 수 있습니다.

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```

01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: W = tf.Variable(1.)
05: B = tf.Variable(0.)
06: X = tf.placeholder(tf.float32)
07: Y = tf.placeholder(tf.float32)
08:
09: H = W * X + B
10: Loss = tf.reduce_mean(tf.reduce_mean(tf.abs(Y - H)))
11:
12: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001)
13: train_op = optimizer.minimize(Loss)
14:
15: sess = tf.Session()
16: sess.run(tf.global_variables_initializer())
17:
18: X_data = [[0],[1],[2],[3],[4]]
19: Y_data = [[0],[2],[4],[6],[8]]
20:
21: for i in range(1000):
22:     sess.run(train_op, feed_dict={X : X_data, Y : Y_data})
23:     loss = sess.run(Loss, feed_dict={X : X_data, Y : Y_data})
24:     print(loss)
25:
26: n = [[7], [-10], [358]]
27: result = sess.run(H, feed_dict={X : n})
28: print(result)

```

7.4.5. 인공신경망

인공신경망은 입력 데이터에 가중치를 곱하는 은닉층과 은닉층 데이터에 다시 가중치를 곱하여 최종 결괏값을 내는 출력층이 있습니다. 이를 구현

하는 과정은 행렬곱 메소드 `tf.matmul` 을 사용합니다.

다음 소스 코드와 같이 플레이스 홀더를 생성하고 2 차원 텐서로 가중치를 생성합니다. 입력 가중치 텐서의 형태 `Shape` 는 `[input size, hidden size]`로 지정해야 합니다. 또, 은닉층 가중치 텐서의 형태는 `[hidden size, output size]`로 지정해야 합니다. 예를 들어 입력 데이터의 수가 1, 은닉층 노드 수가 10, 출력이 1 이라면 다음과 같이 생성합니다.

`random_uniform` 메소드는 특정한 형태로 랜덤 값을 생성해줍니다.

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: X = tf.placeholder(tf.float32)
05: Y = tf.placeholder(tf.float32)
06: W1 = tf.Variable(tf.random_uniform([1, 10], -1., 1.))
07: W2 = tf.Variable(tf.random_uniform([10, 1], -1., 1.))
```

입력층 → 은닉층, 은닉층 → 출력층 구간의 행렬곱 메소드 `tf.matmul` 을 이용해 수식을 만들어줍니다. 또, `tf.nn.relu` 메소드를 이용해 ReLu 활성화 함수를 적용합니다. 단, 은닉층 → 출력층 구간은 최종값이 나오는 구간 이므로 이 신경망의 모델이 됩니다. 각각 `L`, `model` 이라는 이름으로 생성합니다.

```
08: L = tf.matmul(X, W1) #input - hidden
09: L = tf.nn.relu(L)
10: model = tf.matmul(L, W2) #hidden - output
11: Loss = tf.reduce_mean(tf.reduce_mean(tf.square(Y - model)))
```

이어서, 최적화 함수로 `W1` 과 `W2` 를 조절합니다. 경사하강법 객체 `GradientDescentOptimizer` 를 생성하고 최소화할 대상을 `Loss` 로 지정합니다.

```
12: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001)
13: train_op = optimizer.minimize(Loss)
```

세션을 생성하고 텐서플로우 변수를 초기화합니다.

```
14: sess = tf.Session()
15: sess.run(tf.global_variables_initializer())
```

간단한 데이터셋을 만들고 train_op 에 생성된 최적화 함수를 500 회 반복 실행합니다. 또, 최적화 함수가 끝날 때마다 변화된 오차 Loss 를 출력합니다.

```
16: X_data = [[-4],[-3],[-2],[-1],[0],[1],[2],[3],[4]]
17: Y_data = [[-8],[-6],[-4],[-2],[0],[2],[4],[6],[8]]
18:
19: for i in range(500):
20:     sess.run(train_op, feed_dict={X : X_data, Y : Y_data})
21:     loss = sess.run(Loss, feed_dict={X : X_data, Y : Y_data})
22:     print(loss)
```

[출력]

4.3924932

4.244338

...

1.3439047e-10

최적화가 끝난 신경망 모델에 임의의 데이터 n 을 입력하여 결과를 확인 해봅니다.

```
23: n = [[7], [-10], [358]]
```

```
24: result = sess.run(model, feed_dict={X : n})
25: print(result)
```

[출력]

```
[[ 13.999958]
 [-19.999979]
 [715.99786  ]]
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: X = tf.placeholder(tf.float32)
05: Y = tf.placeholder(tf.float32)
06: W1 = tf.Variable(tf.random_uniform([1, 10], -1., 1.))
07: W2 = tf.Variable(tf.random_uniform([10, 1], -1., 1.))
08:
09: L = tf.matmul(X, W1) #input - hidden
10: L = tf.nn.relu(L)
11: model = tf.matmul(L, W2) #hidden - output
12: Loss = tf.reduce_mean(tf.reduce_mean(tf.square(Y - model),axis=1))
13:
14: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001)
15: train_op = optimizer.minimize(Loss)
16:
17: sess = tf.Session()
18: sess.run(tf.global_variables_initializer())
19:
20: X_data = [[-4],[-3],[-2],[-1],[0],[1],[2],[3],[4]]
21: Y_data = [[-8],[-6],[-4],[-2],[0],[2],[4],[6],[8]]
22:
```

```

23: for i in range(500):
24:     sess.run(train_op, feed_dict={X : X_data, Y : Y_data})
25:     loss = sess.run(Loss, feed_dict={X : X_data, Y : Y_data})
26:     print(loss)
27:
28: n = [[7], [-10], [358]]
29: result = sess.run(model, feed_dict={X : n})
30: print(result)

```

7.4.6. 심층신경망

심층신경망은 인공신경망에서 은닉층의 수를 늘려 구현할 수 있습니다.
 입력층 → 은닉층, 은닉층 → 출력층 구간 외에 은닉층 → 은닉층 구간을
 중간에 추가하는 것입니다.

다음 소스 코드와 같이 플레이스 홀더를 생성하고 2차원 텐서로 가중치
 를 생성합니다. 입력층 → 은닉층 구간의 가중치는 W1, 은닉층 → 은닉
 층 구간의 가중치는 W2, 은닉층 → 출력층 구간의 가중치는 W3 입니다.

```

01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: X = tf.placeholder(tf.float32)
05: Y = tf.placeholder(tf.float32)
06: W1 = tf.Variable(tf.random_uniform([1, 10], -1., 1.))
07: W2 = tf.Variable(tf.random_uniform([10, 10], -1., 1.))
08: W3 = tf.Variable(tf.random_uniform([10, 1], -1., 1.))

```

행렬곱을 이용해 각 구간의 수식을 만들어줍니다. 단, 은닉층 → 출력층
 구간은 최종값이 나오는 구간이므로 이 신경망의 모델이 됩니다. 각각

L1, L2, model 이라는 이름으로 생성합니다.

```
09: L1 = tf.matmul(X, W1) #input - hidden
10: L1 = tf.nn.relu(L1)
11: L2 = tf.matmul(L1, W2) #hidden - hidden
12: L2 = tf.nn.relu(L2)
13: model = tf.matmul(L2, W3) #hidden - output
14: Loss = tf.reduce_mean(tf.reduce_mean(tf.sqaure(Y - model)))
```

이어서, 최적화 함수로 W1, W2, W3 를 조절합니다. 경사하강법 객체 GradientDescentOptimizer 를 생성하고 최소화할 대상을 Loss 로 지정합니다.

```
15: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001)
16: train_op = optimizer.minimize(Loss)
```

세션을 생성하고 텐서플로우 변수를 초기화합니다.

```
17: sess = tf.Session()
18: sess.run(tf.global_variables_initializer())
```

간단한 데이터셋을 만들고 train_op 에 생성된 최적화 함수를 500 회 반복 실행합니다. 또, 최적화 함수가 끝날 때마다 변화된 오차 Loss 를 출력합니다.

```
19: X_data = [[0],[1],[2],[3],[4]]
20: Y_data = [[0],[2],[4],[6],[8]]
21:
22: for i in range(500):
```

```
23: sess.run(train_op, feed_dict={X : X_data, Y : Y_data})
24: loss = sess.run(Loss, feed_dict={X : X_data, Y : Y_data})
25: print(loss)
```

[출력]

```
15.552911
13.894471
...
1.10986775e-11
```

최적화가 끝난 신경망 모델에 임의의 데이터 n 을 입력하여 결과를 확인 해봅니다.

```
26: n = [[7], [-10], [358]]
27: result = sess.run(model, feed_dict={X : n})
28: print(result)
```

[출력]

```
[[ 14.000001]
 [-19.99998 ]
 [716.      ]]
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: X = tf.placeholder(tf.float32)
05: Y = tf.placeholder(tf.float32)
06: W1 = tf.Variable(tf.random_uniform([1, 10], -1., 1.))
```

```

07: W2 = tf.Variable(tf.random_uniform([10, 10], -1., 1.))
08: W3 = tf.Variable(tf.random_uniform([10, 1], -1., 1.))
09:
10: L1 = tf.matmul(X, W1) #input - hidden
11: L1 = tf.nn.relu(L1)
12: L2 = tf.matmul(L1, W2) #hidden - hidden
13: L2 = tf.nn.relu(L2)
14: model = tf.matmul(L2, W3) #hidden - output
15: Loss = tf.reduce_mean(tf.reduce_mean(tf.sqaure(Y - model)))
16: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001)
17: train_op = optimizer.minimize(Loss)
18:
19: sess = tf.Session()
20: sess.run(tf.global_variables_initializer())
21:
22: X_data = [[-4],[-3],[-2],[-1],[0],[1],[2],[3],[4]]
23: Y_data = [[-8],[-6],[-4],[-2],[0],[2],[4],[6],[8]]
24:
25: for i in range(500):
26:     sess.run(train_op, feed_dict={X : X_data, Y : Y_data})
27:     loss = sess.run(Loss, feed_dict={X : X_data, Y : Y_data})
28:     print(loss)
29:
30: n = [[7], [-10], [358]]
31: result = sess.run(model, feed_dict={X : n})
32: print(result)

```

내용정리

- 텐서플로우(Tensorflow) : 구글에서 제공하는 머신러닝 프레임워크
 - 프로그램의 흐름과 또 다른 흐름인 '세션'에서 연산
- 플레이스 홀더 : 세션 연산에 사용될 미지수의 공간을 예약
- 그래프 : 하나의 텐서 연산 묶음. 연산자를 이용해 연산 과정을 표현
- 최적화 : 인공신경망이 올바른 출력을 위해 표본(정답) 데이터와 출력 데이터를 비교해 가중치를 조절하는 과정
- 최적화 함수 : 인공신경망의 복잡한 가중치 최적화 과정을 간단하게 구현한 함수



그림 156 스탠포드 자율 주행 차량

2005

스탠포드 레이싱 팀의 자율 주행 차량 “Stanley”가 라스 베이거스 근처에서 개최 된 2005 DARPA“Grand Challenge”에서 우승했습니다. 폭스바겐 투아렉 R5는 오프로드, 175 마일 길이의 사막 코스에서 자율적으로 운전하면서 주어진 10 시간의 제한 시간 중 사람의 개입없이 7 시간 이내에 문제를 해결했습니다. 2004 년에 처음 도입 된 DARPA 는 자율 주행 자동차 분야에서 관심을 끌고 혁신을 일으키기 위해 시작되었습니다.



그림 157 클라우드 프로그램

2006

2000 년대 중반부터 컴퓨터 유틸리티 모델은 “클라우드” 라는 이름으로 유행하기 시작했으며 네트워킹과 컴퓨팅의 주요 트렌드였습니다. Amazon 의 2006 Elastic Compute Cloud 는 아이디어를 대중화하는데 도움이되었습니다. 오늘날 클라우드 기반 회사는 개인용 컴퓨터나 회사의 IT 부서에서 운영하는 대형 컴퓨터에서 수행할 수 있는 대부분의 소프트웨어 또는 서비스 (데이터 스토리지 포함)를 제공합니다.

연습문제

문제 46

다음은 코드 조각들입니다. Tensorflow 상수와 변수 값을 출력하고자 할 때 코드를 순서에 맞게 나열해보세요.

A	<pre>01: print(C) 02: print(V)</pre>
B	<pre>01: sess = tf.Session() 02: sess.run(tf.global_variables_initializer())</pre>
C	<pre>01: import tensorflow.compat.v1 as tf 02: tf.disable_v2_behavior()</pre>
D	<pre>01: C = tf.constant(13) 02: V = tf.Variable(1)</pre>

[출력]

13

1

문제 47

다음은 Tensorflow 플레이스 홀더를 사용해 1 차 함수 값을 출력하는 코드입니다. 정상적으로 출력되도록 빈 칸에 들어갈 코드를 작성해보세요.

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
```

```

04: C = tf.constant(13)
05: V = tf.Variable(1)
06: X = tf.placeholder(tf.int32)
07:
08: sess = tf.Session()
09: sess.run(tf.global_variables_initializer())
10:
11: F1 = C * X
12: F2 = V * X
13:
14: 

|   |
|---|
| A |
|---|


15: R2 = sess.run(F2, 

|   |
|---|
| B |
|---|

)
16:
17: print(R1)
18: print(R2)

```

[출력]

26

5

문제 48

다음 수식과 코드는 선형 회귀 목표 가설과 Tensorflow 로 선형 회귀 모델을 구현한 코드입니다. 질문을 읽고 답해보세요.

$$y = 2x + 1$$

```

01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: W = tf.Variable(1.)
05: B = tf.Variable(1.)
06: X = tf.placeholder(tf.float32)

```

```

07: Y = tf.placeholder(tf.float32)
08:
09: H = W * X + B
10: Loss = tf.reduce_mean(tf.reduce_mean(tf.abs(Y - H)))
11:
12: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.00001)
13: train_op = optimizer.minimize(Loss)
14:
15: sess = tf.Session()
16: sess.run(tf.global_variables_initializer())
17:
18: X_data = 

|   |
|---|
| X |
|---|


19: Y_data = 

|   |
|---|
| Y |
|---|


20:
21: for i in range(1000):
22:     sess.run(train_op, feed_dict={X : X_data, Y : Y_data})
23:     loss = sess.run(Loss, feed_dict={X : X_data, Y : Y_data})
24:     print(loss)

```

- A. 수식을 회귀할 수 있도록 빈 칸 X, Y에 들어갈 학습 데이터셋을 작성해보세요.
- B. 선형 회귀 모델에 100, 1000, 10000를 입력했을 때의 출력을 작성해보세요.
- C. 10000를 입력했을 때 오차가 ± 0.01 이하인 모델을 만들고 손실율을 작성해보세요.

문제 49

다음 코드들은 각각 Tensorflow를 이용한 선형 회귀 모델을 구현한 코드와 Cds 센서를 이용하여 밝기 값을 출력하는 코드입니다. 조도계를 사

용하거나, 스마트폰에서 ‘조도계’ 애플리케이션을 다운 받아 다음 문제를 해결해보세요. (단, 조도계의 단위는 Lux 로 합니다.)

```
01: import tensorflow.compat.v1 as tf
02: tf.disable_v2_behavior()
03:
04: W = tf.Variable(1.)
05: B = tf.Variable(0.)
06: X = tf.placeholder(tf.float32)
07: Y = tf.placeholder(tf.float32)
08:
09: H = W * X + B
10: Loss = tf.reduce_mean(tf.reduce_mean(tf.abs(Y - H)))
11:
12: optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.001)
13: train_op = optimizer.minimize(Loss)
14:
15: sess = tf.Session()
16: sess.run(tf.global_variables_initializer())
```

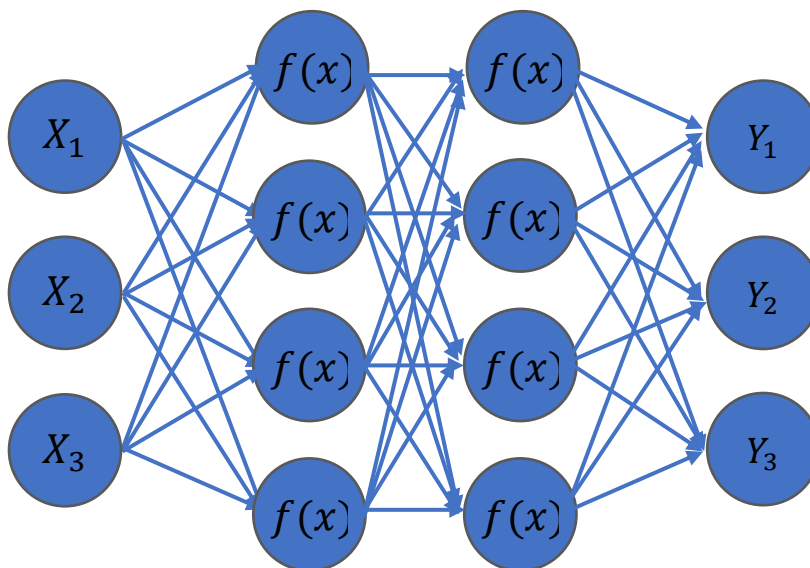
```
01: from pop import Cds
02:
03: cds = Cds(7)
04:
05: value = cds.readAverage()
06: print(value)
```

- A. 빈 배열 2 개를 생성하고, Cds 값과 조도계 값을 동시에 측정하여 Cds 값 배열과 조도계 값 배열을 만들어보세요.
- B. Tensorflow 를 이용하여 A 에서 만든 두 배열을 선형 회귀하는 코드를 작성하세요.

- C. 회귀 모델의 출력과 실제 조도계의 값을 비교해보고 데이터셋 추가 수집, 추가 학습 등 방법으로 ± 30 lux 미만의 오차 범위를 갖는 회귀 모델을 만들어보세요.

문제 50

다음 신경망 그림을 Tensorflow 로 구현해보세요.



7.5. Keras

Keras 는 딥러닝 구현에 특화된 고수준 신경망 API 이며 구글의 Tensorflow 2 프레임워크와 함께 사용할 수 있습니다. CNN, RNN 등 복잡한 신경망을 쉬운 API 로 제공하고 직관적인 구조로 되어 있어, 다양한 신경망을 쉽게 설계할 수 있습니다. 이번 장에서는 Tensorflow2 와 Keras 에 대해 알아보고 이를 이용해 직접 회귀와 딥러닝 알고리즘들을 구현해봅니다. 실습은 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다.

7.5.1. 모델과 레이어

모델은 입력층, 은닉층, 출력층 등 레이어가 연결되어 이루는 하나의 레이어 연결체입니다. 또, 레이어는 모델을 구성하는 하나의 연산 계층으로 가중치 조절하여 모델의 학습이 이루어지도록 합니다. Keras 를 이용하면 직관적으로 레이어를 연결해 학습 모델을 만들 수 있습니다. Keras 의 Sequential 메소드로 순차 Sequential 모델을 생성할 수 있고, 순차 모델에 add 메소드로 레이어를 추가하기만 하면 알아서 가중치를 생성해 레이어를 연결합니다.

다음과 같이 소스 코드를 입력해 Tensorflow 에서 Keras 라이브러리를 불러오고 간단한 모델을 만들어 임의 값을 입력하고 출력을 확인해봅니다.

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(1,)))
06: model.add(keras.layers.Dense(50))
07: model.add(keras.layers.Dense(1))
```

```
08:
09: value = model.predict([1])
10: print(value)
```

[출력]

```
[[ -0.15168795]]
```

입력의 값 10 개가 출력되는 것을 확인할 수 있습니다. 이어서, 어떤 구조로 10 개의 데이터가 출력된 것인지 알고자 합니다. 모델의 구조를 보려면 summary 메소드를 사용할 수 있습니다. 다음과 같이 summary 메소드를 통해 모델의 구조를 확인해봅니다.

```
11: model.summary()
```

[출력]

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 50)	100
dense_1 (Dense)	(None, 10)	510

Total params: 610
Trainable params: 610
Non-trainable params: 0

출력에는 입력층은 표시되지 않고 은닉층과 출력층만 표시되며 각 구간별 가중치의 수를 Parameter 에 표시합니다.

7.5.2. 컴파일

Keras 에서 어떤 방식과 손실 함수로 모델을 최적화할지 학습 과정을 명시하는것을 컴파일이라 합니다. model 객체의 compile 메소드로 컴파일 할 수 있으며 손실 함수, 최적화 함수 등을 선택해 모델의 학습 방향을 지정하는 역할을 합니다.

다음의 코드처럼 손실 함수로 평균 제곱 오차 MSE, 최적화 함수로 경사하강법 SGD 을 사용해 컴파일할 수 있습니다.

```
12: model.compile(loss='MSE', optimizer='SGD')
```

대표적으로 많이 사용하는 손실함수는 MSE(평균 제곱 오차), binary_crossentropy(이진 교차 엔트로피), categorical_crossentropy(다항 교차 엔트로피) 등이며, 최적화 함수에는 SGD(경사하강법), Adam, Nadam, RMSProp 등이 있습니다.

7.5.3. 선형 회귀

Keras 에서 선형 회귀는 Tensorflow 에 비해 매우 간단한 방법으로 만들 수 있습니다. 입력층과 출력층의 노드를 1 개로 하고 평균 제곱 오차와 경사하강법으로 컴파일합니다. 이어서 model 의 fit 메소드를 사용해 컴파일한대로 모델을 학습시킵니다. 기본적인 선형 회귀 모델을 목표로 X 와 Y 데이터를 입력하고 epochs 에 학습 횟수를 입력하여 fit 메소드를 호출합니다.

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
```

```
05: model.add(keras.layers.Input(shape=(1,)))
06: model.add(keras.layers.Dense(1))
07:
08: model.compile(loss='MSE', optimizer='SGD')
09:
10: X=[[0],[1],[2],[3],[4],[5]]
11: Y=[[0],[2],[4],[6],[8],[10]]
12:
13: model.fit(X, Y, epochs=100)
```

[출력]

Train on 6 samples

Epoch 1/100

6/6 [=====] - 2s 398ms/sample - loss: 32.8144

Epoch 2/100

6/6 [=====] - 0s 5ms/sample - loss: 2.7552

Epoch 3/100

6/6 [=====] - 0s 4ms/sample - loss: 0.1038

...

Epoch 100/100

6/6 [=====] - 0s 4ms/sample - loss: 7.2125e-04

학습이 끝나면 model 의 predict 메소드를 이용해 모델의 출력을 확인해
봅니다.

```
14: value = model.predict([[7], [-10], [358]])
15: print(value)
```

[출력]

[[13.955667]

[-19.82356]

[711.3973]]

선형 회귀 모델이 $y = 2x$ 를 회귀한 것을 확인할 수 있습니다.

전체 소스 코드

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(1,)))
06: model.add(keras.layers.Dense(1))
07:
08: model.compile(loss='MSE', optimizer='SGD')
09:
10: X=[[0],[1],[2],[3],[4],[5]]
11: Y=[[0],[2],[4],[6],[8],[10]]
12:
13: model.fit(X, Y, epochs=100)
14:
15: value = model.predict([[7], [-10], [358]])
16: print(value)
```

7.5.4. 로지스틱 회귀

로지스틱 회귀는 입력에 대해 0 ~ 1 을 출력하는 회귀 모델입니다. 기본적인 구성은 선형 회귀 모델과 같고 활성화 함수, 손실 함수, 최적화 함수를 조절해 구현할 수 있습니다. 출력층에 활성화 함수를 Sigmoid 로 지정합니다. 이어서 손실 함수를 binary_crossentropy 로, 최적화 함수를 Adam 으로 하여 컴파일 하고 학습시킵니다. 이번 모델은 음수와 양수를 구분하는 것을 목표로 X와 Y 데이터를 입력하고 epochs 에 학습 횟수를 입력하여 fit 메소드를 호출합니다.

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(1,)))
06: model.add(keras.layers.Dense(1, activation='sigmoid'))
07:
08: model.compile(loss='binary_crossentropy', optimizer='Adam')
09:
10: X=[[-3],[-2],[-1],[1],[2],[3]]
11: Y=[[0],[0],[0],[1],[1],[1]]
12:
13: model.fit(X, Y, epochs=100)
```

[출력]

Train on 6 samples

Epoch 1/100

6/6 [=====] - 2s 282ms/sample - loss: 0.7602

Epoch 2/100

6/6 [=====] - 0s 4ms/sample - loss: 0.7422

Epoch 3/100

6/6 [=====] - 0s 5ms/sample - loss: 0.7245

...

Epoch 100/100

6/6 [=====] - 0s 4ms/sample - loss: 0.1368

학습이 끝나면 model 의 predict 메소드를 이용해 모델의 출력을 확인해
봅니다.

```
14: value = model.predict([[7], [-10], [358]])
15: print(value)
```

[출력]

```
[[9.992175e-01]
 [3.652672e-05]
 [1.000000e+00]]
```

양수일 확률을 출력하는 모델이 완성된 것을 확인할 수 있습니다.

전체 소스 코드

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(1,)))
06: model.add(keras.layers.Dense(1, activation='sigmoid'))
07:
08: model.compile(loss='binary_crossentropy', optimizer='Adam')
09:
10: X=[[-3],[-2],[-1],[1],[2],[3]]
11: Y=[[0],[0],[0],[1],[1],[1]]
12:
13: model.fit(X, Y, epochs=100)
14:
15: value = model.predict([[7], [-10], [358]])
16: print(value)
```

7.5.5. 심층신경망

심층신경망은 다수의 은닉층을 추가해 구현할 수 있습니다.

심층신경망의 학습 능력을 실험해보기 위해 비교적 복잡한 데이터셋을 준비합니다. 3 개의 입력이 주어졌을 때 1 번째 원소와 2 번째 원소의 크기

를 비교한 결과를 1 번째 출력, 2 번째 원소와 3 번째 원소의 크기를 비교한 결과를 2 번째 출력으로 하는 모델을 만들어봅니다. 각 출력은 좌향이 크면 0, 우향이 크면 1 을 출력합니다.

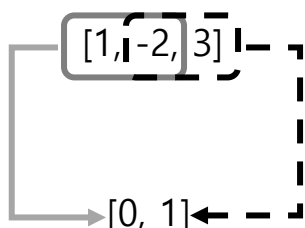


그림 158 입력에 대한 결과 출력 예시

입력을 3 개, 출력을 2 개로 하는 모델에 노드를 100 개씩 갖는 은닉층 레이어를 4 개 추가하고 활성화 함수를 Relu 로 합니다. 또, 출력층의 활성화 함수를 sigmoid 로 하고 손실 함수를 binary_crossentropy 로 컴파일 합니다. 또, 심층신경망이 조금만 학습해도 충분한 학습이 이루어지는지 실험하기 위해 20 회만 학습해봅니다.

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(3,)))
06: model.add(keras.layers.Dense(100, activation='relu'))
07: model.add(keras.layers.Dense(100, activation='relu'))
08: model.add(keras.layers.Dense(100, activation='relu'))
09: model.add(keras.layers.Dense(100, activation='relu'))
10: model.add(keras.layers.Dense(2, activation='sigmoid'))
11:
12: model.compile(loss='binary_crossentropy', optimizer='Adam')
13:
```

```
14: X=[[1,-2,3],[2,-3,4],[3,5,4],[4,-5,6],[7,-5,3], [1,6,8], [3,8,1]]
15: Y=[[0, 1],[0, 1],[1, 0],[0, 1],[1, 1],[0, 1],[1, 1],[1, 0]]
16:
17: model.fit(X, Y, epochs=20)
```

[출력]

Train on 7 samples

Epoch 1/20

7/7 [=====] - 2s 331ms/sample - loss: 0.6853

Epoch 2/20

7/7 [=====] - 0s 4ms/sample - loss: 0.5920

Epoch 3/20

7/7 [=====] - 0s 4ms/sample - loss: 0.5200

...

Epoch 20/20

7/7 [=====] - 0s 5ms/sample - loss: 0.0601

학습이 끝나면 model 의 predict 메소드를 이용해 모델의 출력을 확인해
봅니다.

```
18: value = model.predict([[10,-5,3], [-1,6,-8], [3,5,8]])
19: print(value)
```

[출력]

```
[[0.00180283 0.999791 ]
```

```
 [0.94741815 0.07349858]
```

```
 [0.9574717  0.85406953]]
```

입력 3 개의 크기를 비교해 2 개의 출력을 내는 모델이 완성된 것을 확인
할 수 있습니다.

전체 소스 코드

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(3,)))
06: model.add(keras.layers.Dense(100, activation='relu'))
07: model.add(keras.layers.Dense(100, activation='relu'))
08: model.add(keras.layers.Dense(100, activation='relu'))
09: model.add(keras.layers.Dense(100, activation='relu'))
10: model.add(keras.layers.Dense(2, activation='sigmoid'))
11:
12: model.compile(loss='binary_crossentropy', optimizer='Adam')
13:
14: X=[[1,-2,3],[2,-3,4],[3,5,4],[4,-5,6],[7,-5,3], [1,6,8], [3,8,1]]
15: Y=[[0, 1],[0, 1],[1, 0],[0, 1],[1, 1],[0, 1],[1, 1],[1, 0]]
16:
17: model.fit(X, Y, epochs=20)
18:
19: value = model.predict([[10,-5,3], [-1,6,-8], [3,5,8]])
20: print(value)
```


내용정리

- 케라스(Keras) : 딥러닝에 특화된 고수준 신경망 API 라이브러리
 - 프로그램의 흐름과 또 다른 흐름인 '세션'에서 연산
- 모델 : 입력층, 은닉층, 출력층이 연결된 하나의 레이어 연결체
- 레이어 : 모델을 구성하는 하나의 연산 계층
- 컴파일 : 모델이 어떻게 학습할지 명시하는 과정. 손실 함수, 최적화 함수 등을 설정.



그림 159 IBM 1311 Disk Storage Drive

1962

1962년 10월 11일에 발표된 IBM 1311은 이동식 디스크 팩으로 만든 최초의 디스크 드라이브입니다. 각 팩의 무게는 약 10파운드이고 6개의 디스크를 보유하고 있으며 용량은 2백만 character입니다. 디스크는 1,500rpm으로 회전했으며 디스크당 하나의 헤드가 있는 유압 액추에이터로 액세스했습니다.

Dec	Hex	Name	Char	Ctrl-char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	Null	NUL	CTRL-@	32	20	Space	64	40	@	96	60	
1	1	Start of heading	SOM	CTRL-A	33	21	!	65	41	A	97	61	a
2	2	Start of text	STX	CTRL-B	34	22	"	66	42	B	98	62	b
3	3	End of text	ETX	CTRL-C	35	23	#	67	43	C	99	63	c
4	4	End of limit	EOT	CTRL-D	36	24	\$	68	44	D	100	64	d
5	5	Enquiry	ENQ	CTRL-E	37	25	%	69	45	E	101	65	e
6	6	Acknowledge	AOK	CTRL-F	38	26	&	70	46	F	102	66	f
7	7	Bell	BEL	CTRL-G	39	27	'	71	47	G	103	67	g
8	8	Backspace	BS	CTRL-H	40	28	(	72	48	H	104	68	h
9	9	Horizontal tab	HT	CTRL-I	41	29	)	73	49	I	105	69	i
10	0A	Line feed	LF	CTRL-J	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	VT	CTRL-K	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	FF	CTRL-L	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage feed	CR	CTRL-M	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	SO	CTRL-N	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	SI	CTRL-O	47	2F	/	79	4F	O	111	6F	o
16	10	Delta line escape	DLE	CTRL-P	48	30	0	80	50	P	112	70	p
17	11	Device control 1	DC1	CTRL-Q	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	DC2	CTRL-R	50	32	2	82	52	R	114	72	r
19	13	Device control 3	DC3	CTRL-S	51	33	3	83	53	S	115	73	s
20	14	Device control 4	DC4	CTRL-T	52	34	4	84	54	T	116	74	t
21	15	Neg acknowledge	NAK	CTRL-U	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	SYN	CTRL-V	54	36	6	86	56	V	118	76	v
23	17	End of unit block	ETB	CTRL-W	55	37	7	87	57	W	119	77	w
24	18	Cancel	CAN	CTRL-X	56	38	8	88	58	X	120	78	x
25	19	End of medium	EM	CTRL-Y	57	39	9	89	59	Y	121	79	y
26	1A	Substitute	SUB	CTRL-Z	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	ESC	CTRL-[	59	3B	;	91	5B	[	123	7B	{

그림 160 ASCII 코드표

1963

ASCII(American Standard Code for Information Interchange) 코드는 1과 0의 128개의 고유한 문자열로 구성되었습니다. 각 시퀀스는 영어 알파벳의 문자, 아라비아 숫자, 문장 부호 및 기호의 분류 또는 캐리지 리턴과 같은 기능을 나타냅니다. ASCII는 최대 256개의 기호만 나타낼 수 있으며, 이로 인해 많은 다른 언어가 유니 코드를 통해 더 잘 지원되므로 100,000개 이상의 기호를 나타낼 수 있습니다.

연습문제

문제 51

다음은 코드 조각들입니다. Keras 로 3 개의 입력에 대해 4 개의 출력을 하는 모델을 작성하고자 할 때 코드를 순서에 맞게 나열하세요.

A	<pre>01: model = keras.models.Sequential()</pre>
B	<pre>01: model.add(keras.layers.Input(shape=(3,)))</pre>
C	<pre>01: from tensorflow import keras</pre>
D	<pre>01: model.add(keras.layers.Dense(4))</pre>

문제 52

어떤 모델 A 를 평균 제곱 오차로 손실율을 계산하고, 경사하강법으로 최적화하는 모델로 컴파일하려고 할 때 빈 칸에 들어갈 내용을 작성하세요.

```
01: A.compile([ ])
```

문제 53

다음 코드는 Keras 로 로지스틱 회귀 모델을 구현한 코드입니다. 질문을 읽고 답하세요.

```

01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(1,)))
06: model.add(keras.layers.Dense(1, activation= 'sigmoid'))
07:
08: model.compile(loss='binary_crossentropy',
09:               optimizer='Adam')
10:
11: X=
12: Y=
13:
14: model.fit(X, Y, epochs=100)

```

- A. 양수면 1, 음수면 0 을 출력하도록 회귀하고자 할 때 빈 칸 X, Y 에 들어갈 학습 데이터셋을 작성하세요.
- B. 로지스틱 회귀 모델에 -1, 1001, -10000 를 입력했을 때의 출력을 작성해보세요.
- C. 0.1 을 입력했을 때 오차가 ± 0.01 이하인 모델을 만들어보고 모델의 손실율을 작성해보세요.

문제 54

다음 코드들은 각각 Keras 를 이용한 선형 회귀 모델을 구현한 코드와 Cds 센서를 이용하여 밝기 값을 출력하는 코드입니다. 조도계를 사용하거나, 스마트폰에서 ‘조도계’ 애플리케이션을 다운 받아 다음 문제를 해결하세요. (단, 조도계의 단위는 Lux 로 합니다.)

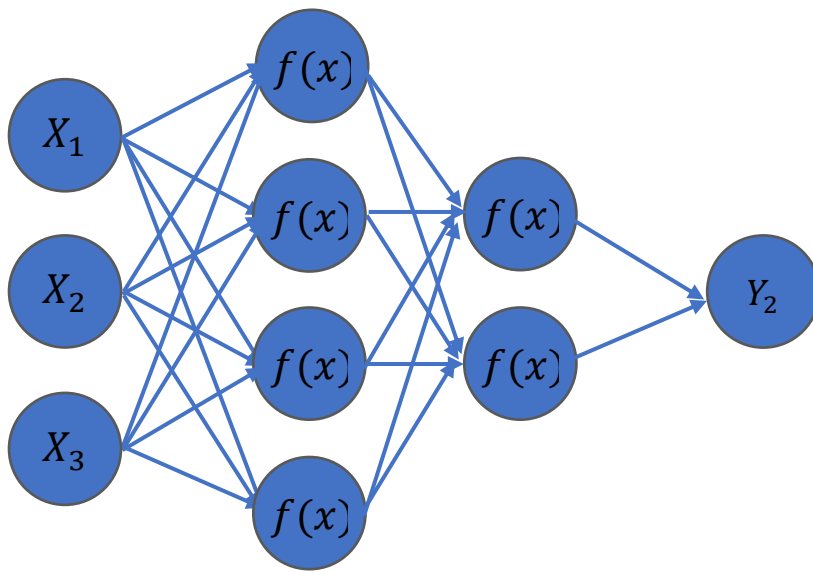

```
01: from tensorflow import keras
02:
03: model = keras.models.Sequential()
04:
05: model.add(keras.layers.Input(shape=(1,)))
06: model.add(keras.layers.Dense(1))
07:
08: model.compile(loss='MAE', optimizer='Adam')
```

```
01: from pop import Cds
02:
03: cds = Cds(7)
04:
05: value = cds.readAverage()
```

- A. 빈 배열 2 개를 생성하고, Cds 값과 조도계 값을 동시에 측정하여 Cds 값 배열과 조도계 값 배열을 만들어보세요.
- B. Keras 를 이용하여 A 에서 만든 두 배열을 선형 회귀하는 코드를 작성하세요.
- C. 회귀 모델의 출력과 실제 조도계의 값을 비교해보고 데이터셋 추가 수집, 추가 학습 등 방법으로 ± 30 lux 미만의 오차 범위를 갖는 회귀 모델을 만들어보세요.

문제 55

다음 신경망 그림을 Keras 로 구현해보세요.



8. 온디바이스 AI

인공지능은 데이터가 매우 방대할 뿐만 아니라 고성능 연산 능력을 요구하기에 이전까지는 엣지 디바이스는 데이터만 수집하고 인공지능 서버에 전송하여 처리하는 방식을 이용해왔습니다. 그런데 이 방식은 보안 문제, 네트워크 지연 문제와 더불어 기본적인 환경을 구성하는 비용이 매우 비싸고 장비 설치에 넓은 공간을 차지하는 등 문제가 있었습니다.

기술이 발전함에 따라 인공지능 구현에 필요한 성능 수준이 낮아졌고, 연산 장치의 성능은 좋아지면서 가격은 저렴해졌습니다. 이에 따라 엣지 디바이스에 인공지능을 내장시키는 방법으로 보안 문제, 네트워크 지연 문제를 해결하고 환경 구성 비용이 매우 저렴해졌으며 장치의 크기는 손바닥만큼 작아졌습니다. 이렇게 엣지 디바이스에 내장되는 인공지능을 온디바이스 AI 라고 합니다.

온디바이스 AI 기술을 통해 작은 공간에서도 많은 장치에 인공지능을 내장할 수 있게 되었습니다. 이는 곧 사물인터넷 Internet of Things 환경에 인공지능을 접목할 수 있는 기회가 되었고, 인공지능과 사물인터넷을 결합한 AIoT 실현이 가능케 했습니다.

대표적으로 온디바이스 AI 가 적용된 사례로 TV, 냉장고, 에어컨 등 가전 제품이 있습니다.

TV 의 경우, 사용자의 시청 패턴을 분석해 채널 또는 영화를 추천하거나 시청 빈도가 낮은 시간대에는 전원을 차단하고 시청 빈도가 높은 시간대에는 슬립 모드로 전환해 빠른 반응을 준비하는 등의 기능을 할 수 있습니다.

냉장고에 온디바이스 AI 를 적용하여 많이 열리는 시간대에는 냉각 성능을 높이고, 그렇지 않은 시간대에는 냉각 성능을 낮춰 전기를 절약하는 기능과 칸마다 들어있는 음식들을 인식하고 최적 온도로 맞춰주는 등의

기능을 합니다.

에어컨에 적용된 경우 사람을 인식하여 바람을 사람한테만 보내는 기능, 사용자의 공조 패턴을 분석해 개인이 선호하는 최적 온도를 맞춰주는 등의 기능을 합니다.

더 나아가 카메라, 초음파 등 데이터를 이용해 인공지능이 차량을 제어하는 기술을 자율주행이라고 합니다. 여러가지 센서 중 가장 중요한 역할을 하는 것이 카메라입니다. 이를 이용해 차선을 유지하거나 주변 교통 상황을 파악하는 등 인공지능을 구현할 수 있습니다.

AIoT AutoCar 에서 데이터를 수집해 온디바이스 AI 를 구현해보고 카메라를 이용해 장애물을 회피하는 인공지능, 사람 또는 사물을 따라다니는 인공지능, 차선을 인식해 트랙을 주행하는 인공지능 등을 구현합니다. 다음 과정에 따라 차근차근 구현해보도록 합니다.

앞으로 설명할 예제 중 장애물 회피, 목표물 추적, 트랙 주행 예제에서는 Nvidia 의 Jetbot 예제를 재구성하여 제작된 Pop.Pilot 라이브러리를 사용합니다. 알고리즘 설계 시간을 절약하고 쉬운 이해와 빠른 이론 적용을 위해 한백전자에서 제작된 라이브러리입니다. 복잡한 개념과 어려운 로직을 구현할 필요없이 직관적인 GUI 와 짧은 코드로 실습 예제를 직접 구현해볼 수 있도록 구성하였습니다.

8.1. 휠 얼라인먼트

AIoT AutoCar 는 앞서 언급한 바와 같이 Pop.Pilot 라이브러리의 AutoCar 클래스를 이용해 쉽게 제어할 수 있습니다. 이를 기초로 하여 온디바이스 AI 를 구현해봅니다.

오랫동안 운영된 자동차에서는 휠 얼라인먼트 오차 현상이 나타납니다. 이 현상은 핸들을 중앙에 놓았음에도 차량 진행 방향이 한 쪽으로 치우치

는 현상입니다. 이 문제를 해결하기 위해 휠 얼라인먼트의 오차를 보정합니다.

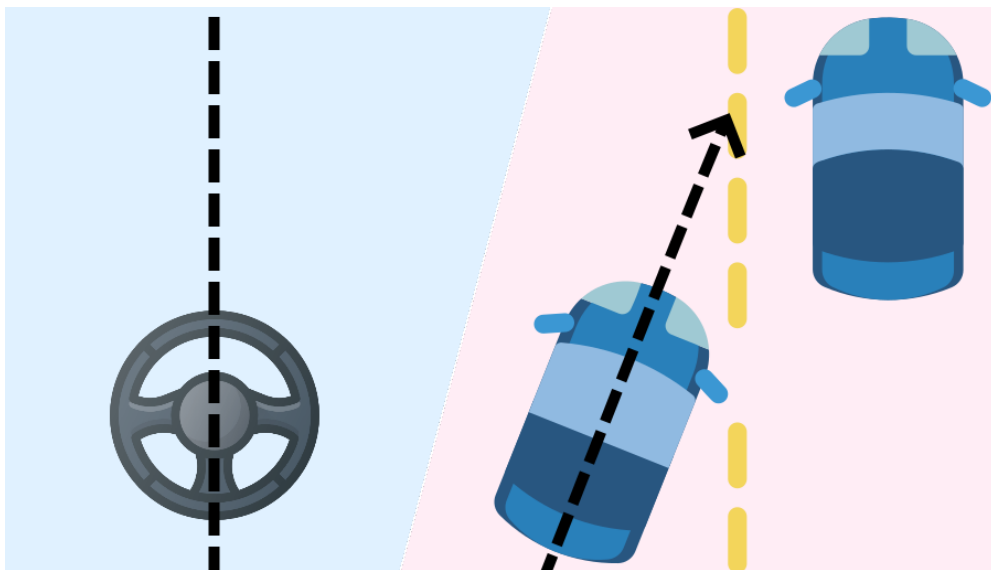


그림 161 휠 얼라인먼트 오차 현상

이와 비슷하게 AutoCar의 조향 모터에도 오차가 존재하므로 중앙 각도로 제어했음에도 곧게 전진하지 않을 수 있습니다. 이러한 문제 역시 오차를 보정하여 개선할 수 있습니다. 하지만 사람이 직접 오차를 보정하려면 수 많은 값들을 대입해보며 근사값을 찾아내므로 프로그램에 비해 정밀하게 보정되지 않습니다. 즉, 정확하게 문제를 해결할 수 없습니다.

이번 실습에서는 6축 센서를 응용하여 조향 모터의 오차를 보정해주는 인공지능을 만들어 봅니다. 이 실습 과정은 실제 차량의 휠 얼라인먼트를 보정하는 과정과 비슷하게 진행됩니다.

이 예제는 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다.

먼저, 학습을 위해 데이터를 수집해야 합니다. 파이썬 조향 제어값과 6 축 센서의 값을 데이터셋으로 구성합니다. 데이터셋은 다양한 각도로 조향해 보고 이에 따른 6 축 센서 값을 수집해야 합니다.

먼저, 차량 제어를 위한 Pop.Pilot 라이브러리와 Numpy, Time 라이브러리를 불러오고 AutoCar 객체를 생성합니다.

```
01: from pop import Pilot
02: import numpy as np
03: import time
04:
05: Car = Pilot.AutoCar()
```

이어서 여러 각도로 조향하고, 6 축 센서의 차이로 값 중 Z축 값을 읽어 저장합니다. 조향 제어값은 좌회전을 음수(-), 우회전을 양수(+)로 하여 저장하며 센서의 측정 시간을 고려해 조향 후 0.5 초가 지난 시점에 6 축 센서의 값을 읽어옵니다. 마지막으로 다음 데이터 수집을 위해 전진한 만큼 후진하여 제자리로 복귀 시킵니다.

이 과정을 여러 번 반복하여 데이터셋을 많이 수집할수록 정확한 오차 보정이 가능해집니다.

```
06: dataset={ 'gyro' : [], 'steer' : [] }
07:
08: for n in np.arange(-1, 1.1, 0.2):
09:     n = round(n,1)
10:
11:     Car.steering = n
12:     Car.forward()
13:
14:     time.sleep(0.5)
15:
```

```
16:     m = Car.getGyro('z')
17:
18:     time.sleep(0.5)
19:
20:     Car.backward()
21:
22:     time.sleep(1)
23:
24:     Car.stop()
25:
26:     dataset['gyro'].append(m)
27:     dataset['steer'].append(n)
28:
29:     print({ 'gyro' : m , 'steer' : n })
```

[출력]

```
{'gyro': 4312, 'steer': -1.0}
{'gyro': 3188, 'steer': -0.8}
{'gyro': 2224, 'steer': -0.6}
{'gyro': 1416, 'steer': -0.4}
{'gyro': -203, 'steer': -0.2}
{'gyro': -790, 'steer': -0.0}
{'gyro': -1777, 'steer': 0.2}
{'gyro': -2779, 'steer': 0.4}
{'gyro': -2844, 'steer': 0.6}
{'gyro': -4348, 'steer': 0.8}
{'gyro': -4667, 'steer': 1.0}
```

다음으로 선형 회귀를 위해 Pop.AI 라이브러리를 불러오고 Linear_Regression 객체를 생성한 뒤, Linear_Regression 의 X_data 와 Y_data 에 각각 gyro 와 steer 를 지정합니다. 충분한 학습을 위해 train 메소드의 파라미터에 times 를 5000 으로 설정하고 데이터를 학습시킵니다.

```
30: from pop import AI
31:
32: LR=AI.Linear_Regression(input_size=1, output_size=1)
33: LR.X_data=dataset['gyro']
34: LR.Y_data=dataset['steer']
35:
36: LR.train(times=5000, print_every=100)
```

[출력]

```
100 step loss: 4782.306
200 step loss: 0.10436215
300 step loss: 0.09105281
400 step loss: 0.08754397
500 step loss: 0.08369036
600 step loss: 0.07959534
...
5000 step loss: 0.024287213
Training is done.
Time spent: 32.5 s
Training speed: 154.0 step/s
```

Z 축 차이로 값이 0 에 가까울 때의 조향 값이 정중앙일 것이므로 학습된 모델의 run 메소드에 0 을 입력하여 실행시켜봅니다.

```
37: value = LR.run([0])
38: print(value)
```

[출력]

```
[-0.1206646]
```

정중앙으로 주행하기 위한 조향 제어값을 알 수 있고, 이 값을 입력하여

조향해봅시다.

```
39: Car.steering = value[0]  
40: Car.forward()
```

차량이 비교적 정확하게 직진하는 것을 확인할 수 있습니다. 단, 전륜 축 베어링에는 미세한 틈이 존재하기 때문에 좌향 상태에서 중앙으로 정렬한 각도와 우향 상태에서 정렬한 각도에 미세한 차이가 있으므로 완벽한 보정은 불가능합니다.

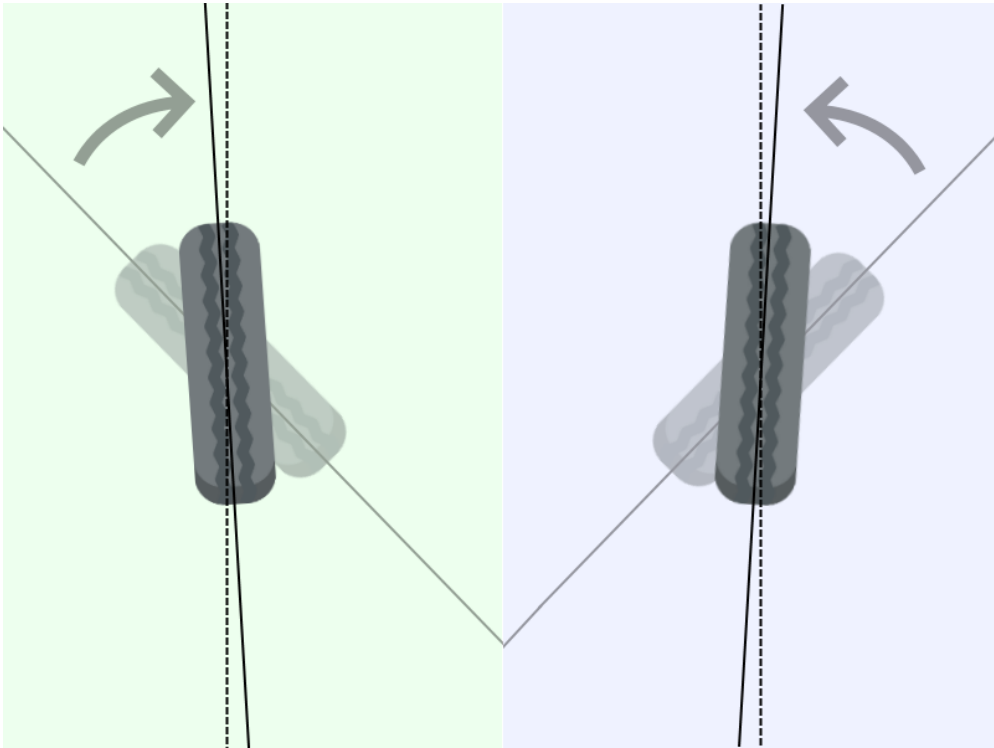


그림 162 베어링으로 인한 오차 발생

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import Pilot
02: import numpy as np
03: import time
04:
05: Car = Pilot.AutoCar()
06:
07: dataset={ 'gyro' : [], 'steer' : [] }
08:
09: for n in np.arange(-1, 1.1, 0.2):
10:     n = round(n,1)
11:
12:     Car.steering = n
13:     Car.forward()
14:
15:     time.sleep(0.5)
16:
17:     m = Car.getGyro('z')
18:
19:     time.sleep(0.5)
20:
21:     Car.backward()
22:
23:     time.sleep(1)
24:
25:     Car.stop()
26:
27:     dataset['gyro'].append(m)
28:     dataset['steer'].append(n)
29:
30:     print({ 'gyro' : m , 'steer' : n })
31:
32: from pop import AI
33:
34: LR=AI.Linear_Regression(input_size=1, output_size=1)
```

```
35: LR.X_data=dataset['gyro']
36: LR.Y_data=dataset['steer']
37:
38: LR.train(times=5000, print_every=100)
39:
40: Car.steering = value[0]
41: Car.forward()
```

8.2. 차량 자세 제어

실제 차량의 상황에서는 바람, 돌 등 외부 요인에 의해 예상치 못한 방향으로 핸들이 움직이는 경우가 발생합니다. 이 때 사람은 자연스럽게 차량의 자세를 유지하는 동작을 하게됩니다. AutoCar 에서도 이와 비슷한 상황이 발생할 수 있습니다.

이번 실습에서는 6 축 센서를 응용하여 자세를 스스로 제어하며 주행하는 인공지능을 만들어 봅니다. 6 축 센서에서 좌회전이 감지되면 차량을 우회전으로 제어하여 자세를 유지하도록 하는 방식으로 구현합니다.

이 예제는 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다.

학습 데이터는 직전 예제와 비슷한 방법으로 수집합니다. 단, 직전 예제를 통해 조향 모터의 오차를 보정한 상황에서 진행합니다.

먼저, 차량 제어를 위한 Pop.Pilot 라이브러리와 Numpy, Time 라이브러리를 불러오고 AutoCar 객체를 생성합니다. 또, correctError 메소드를 이용해 직전 예제에서 얻은 오차 보정값을 적용합니다.

```
01: from pop import Pilot
02: import numpy as np
03: import time
04:
05: Car = Pilot.AutoCar()
06: Car.correctError( -0.1206646 )
```

이어서 여러 각도로 조향하고, 6 축 센서의 차이로 값 중 Z 축 값을 읽어 저장합니다. 조향 제어값을 중앙 값(0)을 제외하고 저장하며 센서의 측정 시간을 고려해 조향 후 0.5 초가 지난 시점에 6 축 센서의 값을 읽어옵니다. 마지막으로 다음 데이터 수집을 위해 전진한 만큼 후진하여 제자리로

복귀 시킵니다.

단, 유의해야할 점은 조향 제어값을 저장할 때 실제와 반대로 좌회전을 양수(+), 우회전을 음수(-)로 저장합니다. 값을 반전하여 저장하는 이유는 6 축 센서에서 우회전이 감지되면 차량을 좌회전시켜 자세를 유지하도록 만들기 위함입니다.

이 과정을 여러 번 반복하여 데이터셋을 많이 수집할수록 정확한 오차 보정이 가능해집니다.

```
07: dataset={ 'gyro' : [], 'steer' : [] }
08:
09: for n in np.arange(-1, 1.1, 0.2):
10:     n = round(n,1)
11:
12:     if n == 0.0 :
13:         continue
14:
15:     Car.steering = n
16:     Car.forward()
17:
18:     time.sleep(0.5)
19:
20:     m = Car.getGyro('z')
21:
22:     time.sleep(0.5)
23:
24:     Car.backward()
25:
26:     time.sleep(1)
27:
28:     Car.stop()
29:
30:     n = -n #조향 제어값을 반전하여 저장
31:
```

```
32:     dataset['gyro'].append( m )
33:     dataset['steer'].append( n )
34:
35:     print({ 'gyro' : m , 'steer' : n })
```

[출력]

```
{'gyro': 4497, 'steer': 1.0}
{'gyro': 3504, 'steer': 0.8}
{'gyro': 3076, 'steer': 0.6}
{'gyro': 1598, 'steer': 0.4}
{'gyro': 585, 'steer': 0.2}
{'gyro': -1251, 'steer': -0.2}
{'gyro': -2458, 'steer': -0.4}
{'gyro': -3276, 'steer': -0.6}
{'gyro': -4104, 'steer': -0.8}
{'gyro': -4621, 'steer': -1.0}
```

다음으로 Linear_Regression 학습을 위해 Pop.AI 라이브러리를 불러오고 Linear_Regression 객체를 생성한 뒤, Linear_Regression 의 X_data 와 Y_data 에 각각 gyro 와 steer 를 지정합니다. 충분한 학습을 위해 train 메소드의 파라미터에 times 를 5000 으로 설정하고 데이터를 학습시킵니다.

```
36: from pop import AI
37:
38: LR=AI.Linear_Regression(input_size=1, output_size=1)
39: LR.X_data=dataset['gyro']
40: LR.Y_data=dataset['steer']
41:
42: LR.train(times=5000, print_every=100)
```

[출력]

```
100 step loss: 34.657368
200 step loss: 0.041308325
300 step loss: 0.036143642
400 step loss: 0.030838653
500 step loss: 0.02569146
600 step loss: 0.020954141
...
5000 step loss: 0.0022562856
Training is done.
Time spent: 32.4 s
Training speed: 154.1 step/s
```

Z 축 자이로 값을 입력으로 선형 회귀 모델을 실행시키고 반환 값을 steering 에 입력하여 차량을 제어하도록 합니다. 원활한 제어 값 송수신을 위해 0.1 초 간격으로 반복 실행합니다. 만약 간격을 0.1 초 미만으로 할 경우 반응성이 좋아지지만 모터 제어를 위한 통신이 불안정해질 수 있습니다.

만약 자세 복구를 위해 조향 값을 줄이지 않고 그대로 제어하면 학습 모델은 다시 그 반대 방향으로 복구해야 한다고 판단해 좌우로 크게 왔다갔다 합니다. 이러한 현상을 방지하기 위해 모델의 반환 값을 3 으로 나눕니다.

```
43: Car.forward()
44:
45: while True:
46:     err = Car.getGyro('z')
47:     value = LR.run([err])
48:
49:     Car.steering = value / 3
50:     time.sleep(0.1)
```

차량이 전진할 때 경로를 이탈시키기 위해 강제로 밀어보면 차량이 원 상태를 복구하려는 것을 확인할 수 있습니다.

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import Pilot
02: import numpy as np
03: import time
04:
05: Car = Pilot.AutoCar()
06: Car.correctError( -0.1206646 ) #보정값 입력
07:
08: dataset={ 'gyro' : [], 'steer' : [] }
09:
10: for n in np.arange(-1, 1.1, 0.2):
11:     n = round(n,1)
12:
13:     if n == 0.0 :
14:         continue
15:
16:     Car.steering = n
17:     Car.forward()
18:
19:     time.sleep(0.5)
20:
21:     m = Car.getGyro('z')
22:
23:     time.sleep(0.5)
24:
25:     Car.backward()
26:
27:     time.sleep(1)
28:
29:     Car.stop()
```

```
30:
31:     n = -n #조향 제어값을 반전하여 저장
32:
33:     dataset['gyro'].append( m )
34:     dataset['steer'].append( n )
35:
36:     print({ 'gyro' : m , 'steer' : n })
37:
38: from pop import AI
39:
40: LR=AI.Linear_Regression(input_size=1, output_size=1)
41: LR.X_data=dataset['gyro']
42: LR.Y_data=dataset['steer']
43:
44: LR.train(times=5000, print_every=100)
45:
46: Car.forward()
47:
48: while True:
49:     err = Car.getGyro('z')
50:     value = LR.run([err])
51:
52:     Car.steering = value / 3
53:     time.sleep(0.1)
```

8.3. 사진 분류

이번 실습에서는 카메라를 응용하여 특정 사물을 구분하는 사물 인식 인공지능을 만들어 봅니다.

이 예제는 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다.

먼저, 휴대전화 카메라나 장비의 카메라를 이용해 사진 데이터를 수집해야 합니다. 이 예제에서는 마우스와 마우스가 아닌 것을 구분하도록 기준을 잡았습니다. 다양한 각도, 다양한 배경, 다양한 조명에서 찍힌 마우스 사진과 다른 사물 사진을 찍습니다. 정확한 학습을 위해서는 각 클래스별 사진 500 장 이상이 적절합니다. 그러나 이 예제에서는 간단한 설명과 빠른 진행을 위해 30 장 내외의 사진으로 학습시켜봅니다. 또, 사진을 일일이 배열로 하드코딩 하기에는 무리가 있으므로 OpenCV 와 Numpy 를 활용합니다.



그림 163 마우스 사진과 기타 사진들

사진 수집이 끝났다면 Samba, USB 등을 이용해 이미지를 장비로 옮기고

파일명을 다음과 같이 숫자로 수정합니다. 파일명을 숫자로 지정하는 이유는 for 문을 이용해 쉽게 접근하기 위함입니다.

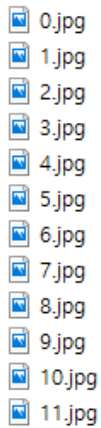


그림 164 정형화된 파일명

다음으로 각 사진에 관한 결과 데이터를 만들어야 합니다. 다음과 같이 텍스트 파일을 만들어 데이터를 만드는데 마우스가 아닌 사진이라면 첫 번째 숫자가 1, 마우스 사진이라면 두 번째 숫자가 1로 지정하고 두 숫자는 ‘ ’(스페이스 바)로 구분합니다. 또, 텍스트 파일의 이름은 각 사진의 이름과 일치하도록 합니다. 이 또한 for 문 사용의 편의를 위함입니다. 이 과정을 라벨링 Labeling 이라고 합니다.

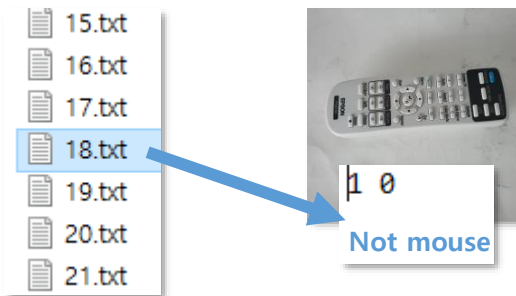
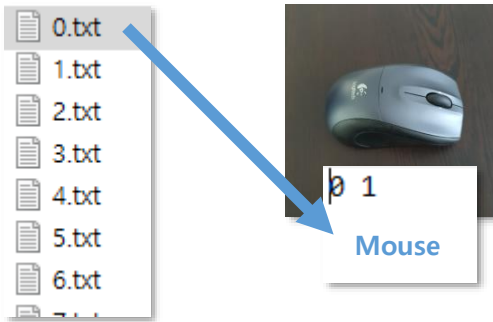


그림 165 사진 라벨링

라벨링 과정까지 끝냈다면 본격적인 프로그래밍을 시작할 수 있습니다.

사용해야 할 패키지를 미리 불러오고, 입력 데이터와 결과 데이터를 리스트로 선언합니다. 이어서, for 문을 사진 개수만큼 반복하는 루프를 생성합니다. cv2.imread 메소드로 사진을 읽은 후 cv2.cvtColor 메소드를 이용해 흑백사진으로 변환합니다. cv2.resize 메소드로 이미지의 사이즈를 50x50 으로 축소하고 X_data 리스트에 추가합니다. 라벨링 했던 텍스트 파일을 numpy 의 loadtxt 메소드로 읽어 Y_data 리스트에 추가합니다.

이 과정은 사진 개수에 따라 수 분이 걸릴 수 있습니다.

```
01: from pop import AI, Camera
```

```
02: import numpy as np
03: import cv2
04:
05:
06: X_data=[]
07: Y_data=[]
08:
09: for i in range(33):
10:     img=cv2.imread('./img/'+str(i)+'.jpg')
11:     img=cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
12:     img=cv2.resize(img, (50,50), interpolation=cv2.INTER_AREA)
13:     X_data.append(img.reshape(50,50,1).astype(float))
14:     y=np.loadtxt('./img/'+str(i)+'.txt')
15:     Y_data.append(y)
```

CNN 객체의 파라미터에 input_size 를 [50, 50], output_size 를 2 로 주고
생성한 뒤, CNN 의 X_data 와 Y_data 지정합니다. 충분한 학습을 위해
train 메소드의 파라미터에 times 를 500 으로 설정하고 데이터를 학습시
킵니다.

```
16: CNN=AI.CNN(input_size=[50,50], output_size=2)
17: CNN.X_data=X_data
18: CNN.Y_data=Y_data
19:
20: CNN.train(times=500)
```

[출력]

```
10 step loss: 6.756868
20 step loss: 0.0
30 step loss: 0.54854834
40 step loss: 0.19673054
50 step loss: 0.47202036
60 step loss: 0.18123364
```

...

500 step loss: 0.0054680733

Training is done.

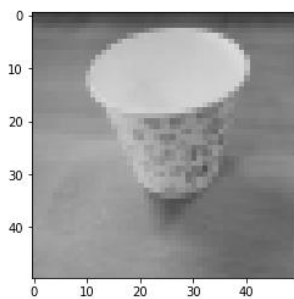
Time spent: 165.5 s

Training speed: 3.0 step/s

학습된 모델의 run 메소드에 CNN의 X_data 중 임의 데이터를 입력하여 실행시켜봅니다.

```
21: CNN.run([CNN.X_data[20]])
```

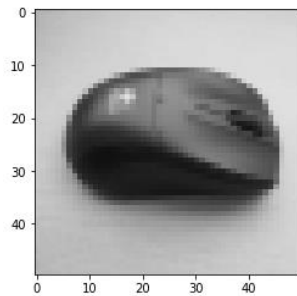
[출력]



array([[0.97286326, 0.02713668]], dtype=float32)

```
22: CNN.run([CNN.X_data[7]])
```

[출력]



```
array([[0.06768391, 0.93231606]], dtype=float32)
```

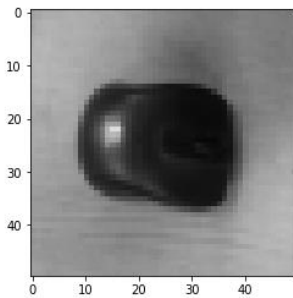
인식률이 괜찮은 모델이 완성되었습니다. 그러면 이제 카메라의 이미지를 읽어와 인식시켜보도록 합니다. 다음과 같이 Pop 라이브러리의 Camera 객체를 생성합니다.

```
23: cam = Camera()
```

Camera 객체에서 카메라의 장면을 읽어오고 cvtColor 메소드로 흑백으로 변환한 뒤 resize 메소드로 50x50 사이즈로 축소합니다. 카메라 이미지를 run 메소드에 입력하고 결과를 확인합니다.

```
24: frame = cam.value
25: frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
26: frame = cv2.resize(frame, (50, 50), interpolation=cv2.INTER_AREA)
27: CNN.run([frame.reshape(50,50,1)])
```

[출력]



```
array([[0.10028321, 0.89971685]], dtype=float32)
```

카메라로 새롭게 찍은 이미지에서 마우스를 인식하는 것을 확인할 수 있습니다. 그러나 데이터셋이 부족하고 이미지 사이즈가 작은 편 이기에 마우스가 아닌 사물을 마우스로 오인식하는 경우가 발생할 수 있습니다. 더 완벽한 모델을 학습시키고자 한다면 마우스 사진 500 장, 기타 사진 500 장 이상을 수집하고 이미지 사이즈를 128x128 정도로 늘려야 합니다.

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import AI, Camera
02: import numpy as np
03: import cv2
04:
05: X_data=[]
06: Y_data=[]
07:
08: for i in range(33):
09:     img=cv2.imread('./img/'+str(i)+'.jpg')
10:     img=cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
11:     img=cv2.resize(img, (50,50), interpolation=cv2.INTER_AREA)
12:     X_data.append(img.reshape(50,50,1).astype(float))
13:     y=np.loadtxt('./img/'+str(i)+'.txt')
14:     Y_data.append(y)
```

```
15:
16: CNN=AI.CNN(input_size=[50,50], output_size=2)
17: CNN.X_data=X_data
18: CNN.Y_data=Y_data
19:
20: CNN.train(times=500)
21:
22: cam = Camera()
23:
24: frame = cam.value
25: frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
26: frame = cv2.resize(frame, (50, 50), interpolation=cv2.INTER_AREA)
27: CNN.run([frame.reshape(50,50,1)])
```

8.4. 음성 분류

이번 실습에서는 마이크를 응용하여 0 부터 9 사이 숫자 음성을 인식하는 인공지능을 만들어 봅니다.

이 예제는 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다.

먼저, 0 부터 9 까지의 음성 데이터를 수집해야 합니다. 다양한 사람, 다양한 톤, 다양한 크기(dB)로 녹음합니다. 적절한 학습을 위해서는 각 숫자별 음성 데이터셋 200 개 이상이 필요합니다. 그러나 데이터를 수집하기에는 시간이 충분치 않을 때, 오픈 데이터셋 사이트인 Kaggle 에서 데이터셋을 구할 수 있습니다. 아래 링크는 Kaggle 에 공개된 SDD(Spoken Digit Dataset) 프로젝트의 데이터셋만 추출한 파일입니다. 단, 영어로 녹음된 파일이며 4 명의 목소리만 있기 때문에 실제 사용했을 때 다른 사람의 목소리를 제대로 인식 못할 가능성이 있습니다. 그러므로 해당 데이터셋에 본인 또는 다른 사람의 목소리를 더 추가하여 사용하면 좋습니다. 녹음을 할 때는 다른 파일들과 비슷한 길이인 1 초 내외로 해야합니다.

- https://github.com/hanback-docs/Spoken_Digit_Dataset



수월한 음성 데이터 수집을 위해 한백전자에서 제공하는 Pop.Dataset 라이브러리를 사용할 수 있습니다. 다음과 같이 Pop 의 Dataset 라이브러리에서 Collector 메소드에 “Audio” 인자를 입력하여 오디오 컬렉터를 실행합니다.

```
01: from pop import Dataset
02:
03: Dataset.Collector("Audio")
```

[출력]

REC 버튼을 클릭하면 1 초 뒤 녹음이 시작되며, 녹음 결과를 플레이어로 듣고 Save 버튼을 통해 ‘Label_Name_녹음시간.wav’ 이름의 파일로 저장할 수 있습니다. Save 버튼으로 저장하지 않으면 언제든지 삭제될 수 있습니다. 각 개체에 대한 설명은 다음과 같습니다.

- Duration (숫자 텍스트): 녹음 시간, 초 단위
- REC (버튼): 녹음 시작
- Path (텍스트): 데이터셋을 저장할 경로
- Label (숫자 텍스트): 학습 과정의 Y 값에 해당하는 라벨
- Name (텍스트): 파일 식별을 위한 이름
- Save (버튼): 저장

데이터 수집이 끝났다면 사용해야 할 패키지를 미리 불러오고, 입력 데이터와 결과 데이터를 리스트로 선언합니다. 이어서, os 모듈을 사용해 데이터셋 리스트를 수집하고, 음성 데이터를 MFCC 데이터로 변환하는 과정을 데이터셋 개수만큼 for 문으로 반복합니다. 또, 파일명에서 Label 을 추출해 One Hot 데이터로 변환합니다.

MFCC 는 음성의 특징을 추출하는 알고리즘입니다. 자세한 설명은 부록 521 쪽을 참고할 수 있습니다.

Util 의 toMFCC 메소드를 사용할 때는 파일명과 재생 시간을 파라미터로 입력합니다. 해당 파일의 실제 재생 시간이 파라미터 입력 보다 짧으면 부족한 부분을 0 으로 채워 길이를 맞추고, 길면 나머지 부분을 잘라 길

이를 맞추므로 최대한 길이를 맞춰 녹음하도록합니다.

이 과정은 데이터셋 개수에 따라 수 분이 걸릴 수 있습니다.

```
04: from pop import AI
05: from pop import Util
06: import os
07:
08: X_data=[]
09: Y_data=[]
10:
11: datalist = os.listdir("audio_datasets")
12:
13: for data in datalist:
14:     feat = Util.toMFCC("audio_datasets/" + data,
                        duration=1)
15:     label = int(data.split("_")[0])
16:     label = Util.one_hot(label,10)
17:
18:     X_data.append(feat)
19:     Y_data.append(label)
20:
21: X_data=np.array(X_data)
22: Y_data=np.array(Y_data)
```

CNN 객체의 파라미터에 input_size 를 데이터셋 하나의 크기, ouput_size 를 10 으로 주고 생성한 뒤, CNN 의 X_data 와 Y_data 지정합니다. 충분한 학습을 위해 train 메소드의 파라미터에 times 를 100 으로 설정하고 데이터를 학습시킵니다.

```
23: dataset_size = X_data.shape[1:3]
24: CNN=AI.CNN(input_size=dataset_size, output_size=10)
25:
26: CNN.X_data=X_data
```

```
27: CNN.Y_data=Y_data
28:
29: CNN.train(times=100)
```

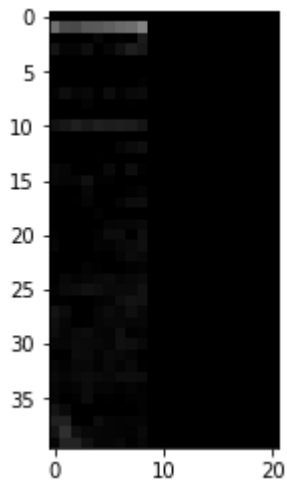
[출력]

```
10 step loss: 50.1121940612793
20 step loss: 8.384210586547852
30 step loss: 3.475531578063965
40 step loss: 1.4908788204193115
50 step loss: 0.977618396282196
...
100 step loss: 0.3892776370048523
Training is done.
Time spent: 31.5 s
Training speed: 3.2 step/s
```

학습된 모델의 run 메소드에 CNN의 X_data 중 임의 데이터를 입력하여 실행시켜봅니다. 또, 결과 비교를 위해 Y_data를 함께 출력합니다.

```
30: Y=CNN.Y_data[20]
31: R=CNN.run([CNN.X_data[20]])
32:
33: print(Y)
34: print(R)
```

[출력]



```
[0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
```

```
[[1.3247161e-07 1.5562894e-07 1.1709916e-09 4.3990020e-12 4.5990603e-07  
 1.3456181e-04 5.3105353e-11 9.9986219e-01 3.6276246e-14 2.5419445e-06]]
```

```
35: Y=CNN.Y_data[7]  
36: R=CNN.run([CNN.X_data[7]])  
37:  
38: print(Y)  
39: print(R)
```

[출력]



```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
```

```
[[1.5442951e-03 3.8058080e-03 4.4027012e-05 3.1492839e-09 1.4033595e-06  
2.3970984e-04 2.7837310e-10 4.6601548e-05 3.3552367e-16 9.9431819e-01]]
```

인식률이 꽤 좋은 모델이 완성되었습니다. 그러면 이제 마이크에서 음성을 읽어와 인식시켜보도록 합니다. 앞서 데이터 수집 과정과 같이 Pop.Dataset 의 Collector 메소드를 활용합니다.

```
40: Dataset.Collector("Audio")
```

[출력]

Time Ready to record.
Duration (sec)

● REC

▶ 0:00 / 0:00 🔊 ⋮

Path
Label
Name

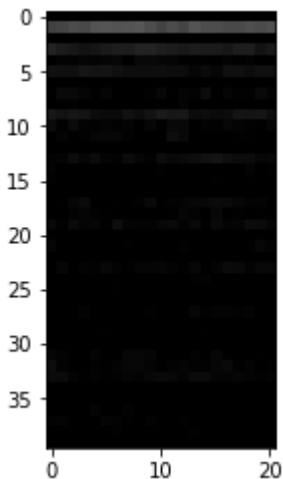
💾 Save

오디오 컬렉터로 숫자 음성을 녹음하고, 경로를 지정해 파일을 저장합니다. 파일명은 'Label_Name_녹음시간.wav' 형식으로 저장됩니다.

저장된 녹음 파일을 Util 의 toMFCC 메소드를 이용해 MFCC 데이터로 변환하고 학습된 모델에 입력합니다.

```
41: data=Util.toMFCC("audio_datasets/9_identifier_2004132053280.wav",  
                    duration=1)  
42: CNN.run([data])
```

[출력]



```
array([[9.4436213e-02, 2.5679762e-03, 8.0499412e-06, 3.2427909e-08,  
        2.4467956e-06, 1.9633518e-04, 6.0831361e-11, 4.9063664e-07,  
        8.1591374e-13, 9.0278840e-01]], dtype=float32)
```

새롭게 녹음한 음성이 9로 인식되는 것을 확인할 수 있습니다.

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
43: from pop import Dataset
44:
45: Dataset.Collector("Audio")
46:
47: from pop import AI
48: from pop import Util
49: import os
50:
51: X_data=[]
52: Y_data=[]
53:
54: datalist = os.listdir("audio_datasets")
55:
56: for data in datalist:
57:     feat = Util.toMFCC("audio_datasets/" + data,
                        duration=1)
58:     label = int(data.split("_")[0])
59:     label = Util.one_hot(label,10)
60:
61:     X_data.append(feat)
62:     Y_data.append(label)
63:
64: X_data=np.array(X_data)
65: Y_data=np.array(Y_data)
66:
67: dataset_size = X_data.shape[1:3]
68: CNN=AI.CNN(input_size=dataset_size, output_size=10)
69:
70: CNN.X_data=X_data
71: CNN.Y_data=Y_data
72:
73: CNN.train(times=100)
74:
75: Y=CNN.Y_data[20]
76: R=CNN.run([CNN.X_data[20]])
77:
78: print(Y)
```

```
79: print(R)
80:
81: Dataset.Collector("Audio")
82:
83: data=Util.toMFCC("audio_datasets/9_identifier_2004132053280.wav",
                    duration=1)
84: CNN.run([data])
```

8.5. 장애물 회피

장애물 회피는 합성곱 신경망을 이용해 카메라 전방에 근접한 사물이 있는지 학습하여 장애물을 감지하는 모델입니다. 여러 장애물 데이터셋을 수집하고 데이터량과 다양성에 따라 어떤 모델이 만들어지는지 확인해봅니다. 또, 이에 따라 후진 또는 좌우 조향을 이용해 차량이 장애물을 회피하도록 실습합니다.

실습은 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다. 또, 쉬운 이해와 빠른 실습을 위해 Pop.Pilot 라이브러리를 사용합니다.

8.5.1. 데이터 수집

먼저, 장애물에 대한 학습을 위해 이미지 데이터를 수집해야 합니다. 이미지 데이터는 Pop.Pilot 라이브러리의 Data_Collector 클래스로 편리하게 수집할 수 있습니다. 또, OpenCV 코드 작성으로 인한 불편 해소와 핵심 내용 집중을 위해 Pop.Pilot 라이브러리에 카메라를 쉽게 읽을 수 있는 Camera 클래스를 미리 구현해두었습니다.

Pop 에서 Pilot 라이브러리를 불러오고 Camera 클래스를 width 와 height 파라미터에 300 을 입력해 300 x 300 사이즈로 생성합니다. Camera 클래스의 show() 메소드를 이용해 실시간 영상을 확인해볼 수 있습니다.

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04: cam.show()
```

[출력]



stop() 메소드를 호출하면 영상을 정지할 수 있습니다. 단, 실제 촬영은 백그라운드에서 지속되고 있고 보여지는 영상만 정지되는 것입니다.

```
05: cam.stop()
```

Pilot 라이브러리에서 Data_Collector 클래스를 생성합니다. 생성자에 Collision_Avoid 데이터 수집이 목적임을 명시하기 위해 Pilot.Collision_Avoid 를 입력합니다. 또, 카메라 영상을 이용하기 위해 camera 파라미터에 앞서 생성한 카메라 클래스를 입력합니다.

```
06: dataCollector=Pilot.Data_Collector(Pilot.Collision_Avoid,  
    camera=cam)
```

Data_Collector 클래스의 show() 메소드를 사용하면 데이터 수집을 위한 GUI 환경이 표시됩니다.

```
07: dataCollector.show()
```

[출력]



데이터 수집을 위해 실시간 영상과 2 개의 버튼 그리고 조이스틱이 표시 됩니다. 조이스틱을 이용해 차량을 움직일 수 있고 add free 버튼을 클릭 하면 현재 표시되고 있는 장면을 장애물이 없는 이미지로 분류하고, add blocked 버튼을 클릭하면 장애물 이미지로 분류합니다.

다음과 같이 조이스틱을 이용해 차량을 움직이며 데이터를 수집합니다. 데이터는 각각 300 장 이상이 적절하며 실내, 실외, 타일 등 최대한 다양 하고 많은 데이터를 수집해야 양질의 모델을 기대할 수 있습니다. 사진

데이터는 현재 경로의 collision_dataset 디렉토리에서 free 와 blocked 디렉토리로 구분하여 저장됩니다.

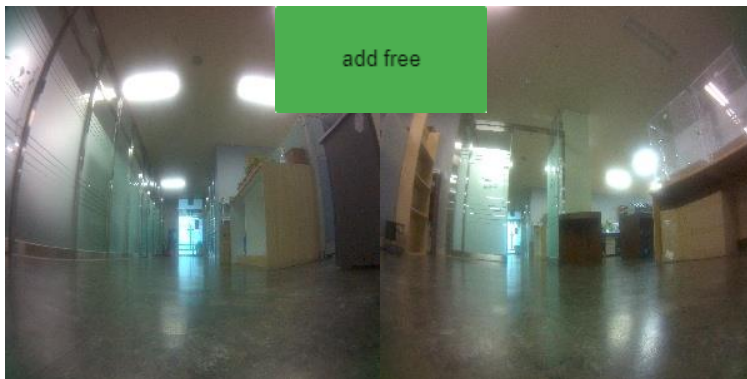


그림 166 blocked 데이터와 free 데이터

8.5.2. 데이터 딥러닝

장애물 학습을 위해 Collision_Avoid 클래스를 생성합니다. 카메라 영상을 이용해야하므로 앞서 생성한 camera 클래스를 객체 생성자에 입력하여 생성합니다.

Collision_Avoid 클래스는 장애물 인식을 위한 합성곱 신경망이 설계되어 있는 신경망 클래스입니다. 효율적인 학습으로 학습 시간을 절약하고 빠

르게 연산하여 실시간 사용이 가능하도록 설계된 신경망입니다.

이어서, Collision_Avoid 클래스의 load_datasets() 메소드를 이용해 앞서 수집한 데이터셋들을 로드합니다. 데이터셋을 로드할 때, 현재 경로에 있는 collision_dataset 디렉토리에서 가져옵니다.

```
08: CA=Pilot.Collision_Avoid(cam)
09: CA.load_datasets()
```

데이터셋 로드가 완료되면 train() 메소드를 이용해 학습을 시작합니다. train 메소드의 파라미터인 times 에 숫자를 입력해 학습할 횟수를 조절할 수 있습니다. 우선 10 회만 학습시켜보도록 합니다.

train 메소드를 통해 학습이 진행되면 매 스텝마다 자동으로 학습 모델이 저장됩니다. 만약 자동 저장을 비활성화 하고싶다면 autosave 파라미터를 False 로 지정하면 됩니다.

```
10: CA.train(times=10)
```

[출력]

```
0 step accuracy : 0.800000
1 step accuracy : 0.920000
2 step accuracy : 0.960000
3 step accuracy : 1.000000
4 step accuracy : 1.000000
5 step accuracy : 1.000000
6 step accuracy : 0.980000
7 step accuracy : 1.000000
8 step accuracy : 1.000000
9 step accuracy : 1.000000
```


정확도가 1에 가까울수록 오차가 작은 모델입니다. 적절히 잘 학습된 모델이 나왔다면 run() 메소드를 이용해 모델의 예측값을 출력해볼 수 있습니다. 다음과 같이 run() 메소드를 실행하고 반환값을 출력해봅니다.

```
11: value=CA.run()  
12: print(value)
```

[출력]

1.00

카메라를 이용해 전방에 장애물이 있을 확률을 0~1의 범위로 출력합니다. 그러나 텍스트 출력만으로는 어떤 장면으로 어떻게 판단했는지 알기 어렵습니다. 이 때, show() 메소드를 사용할 수 있습니다. show() 메소드를 실행하면 현재 카메라 영상과 run()메소드의 결과를 직관적으로 표시합니다. show() 메소드는 run() 메소드와 동기화 되어있기 때문에 run() 메소드와 별개로 한 번만 실행하면 계속해서 표시됩니다.

```
13: CA.show()
```

[출력]



run() 메소드에는 Callback 기능을 사용할 수 있도록 만들어져 있습니다. 다음과 같이 메소드를 선언하고 run() 메소드의 callback 파라미터로 입력하여 호출하면, run() 메소드의 반환값을 Callback 메소드의 인자로 넘겨 호출합니다.

Collision_Avoid의 run() 메소드는 입력된 영상이 장애물일 확률을 0에서 1 사이 실수로 반환합니다.

```
14: def is_blocked(value):
15:     print(value>0.5)
16:
17: value=CA.run(callback=is_blocked)
18: print(value)
```

[출력]

True

1.00000

장애물을 잘 인식한다면 이 모델을 이용해 차량을 제어해봅니다. 차량 제어를 위해 Pop.Pilot 라이브러리에서 AutoCar 클래스를 생성합니다. Callback 메소드는 모델 예측값이 0.5를 초과하면 우측으로 후진하고 0.5 이하면 전진하도록 구성합니다.

```
19: Car=Pilot.AutoCar()
20: Car.setSpeed(50)
21:
22: def drive(value):
23:     if value<=0.5:
24:         Car.steering=0
25:         Car.forward()
26:     else:
27:         Car.steering=1
```

```
28:         Car.backward()
29:
30: while True:
31:     CA.run(callback=drive)
```

차량이 주행하며 장애물이 인식되면 후진하고, 장애물이 없으면 전진하는 것을 확인할 수 있습니다. 그러나 데이터셋에는 없는 조명, 바닥 등 새로운 환경에서는 제대로 주행하지 않습니다. 이는 데이터셋의 다양성이 부족하기 때문입니다. 최대한 다양한 환경에서 데이터를 수집하여 학습한다면 매우 정확한 회피 모델을 얻을 수 있습니다.

부가적으로, 저장된 학습 모델을 로드하여 이어서 학습하고자 한다면 `load_model(path)` 메소드를 사용할 수 있습니다.

```
01: CA.load_model(path = "collision_avoid_model.pth")
```

또, `save_model(path)` 메소드를 사용하면 원하는 경로에 수동으로 모델을 저장할 수 있습니다.

```
01: CA.save_model(path = "collision_avoid_model.pth")
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

소스 코드 : 데이터 수집

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
```

```
04:
05: dataCollector=Pilot.Data_Collector(Pilot.Collision_Avoid,
    camera=cam)
06: dataCollector.show()
```

소스 코드 : 딥러닝

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04:
05: CA=Pilot.Collision_Avoid(cam)
06: CA.load_datasets()
07:
08: CA.train(times=10)
09:
10: CA.show()
11:
12: Car=Pilot.AutoCar()
13: Car.setSpeed(50)
14:
15: def drive(value):
16:     if value<=0.5:
17:         Car.steering=0
18:         Car.forward()
19:     else:
20:         Car.steering=1
21:         Car.backward()
22:
23: while True:
24:     CA.run(callback=drive)
```

8.6. 목표물 추적

목표물 추적은 사전 학습된 사물 인식 모델을 이용하여 여러 사물들을 인식하고 특정 사물을 지정하여 차량이 따라가도록 합니다.

실습은 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다. 또, 쉬운 이해와 빠른 실습을 위해 Pop.Pilot 라이브러리를 사용합니다.

8.6.1. 사전 학습 모델 다운로드

이 실습 예제에서는 사물들에 대한 데이터 수집이 어려운 점과 상세한 데이터 라벨링, 고성능의 학습 알고리즘이 필요하기 때문에 데이터 수집과 학습은 하지 않습니다. 대신 공개된 사물 인식 모델을 사용합니다. 사물 인식 모델은 다음 주소에서 받을 수 있으며 인식 가능한 사물 목록도 확인할 수 있습니다.

- https://github.com/hanback-docs/ssd_mobilenet_v2_coco_engine



위 주소에서 `ssd_mobilenet_v2_coco.engine` 파일을 다운로드하고 다음과 같이 Jupyter 경로에 업로드합니다.

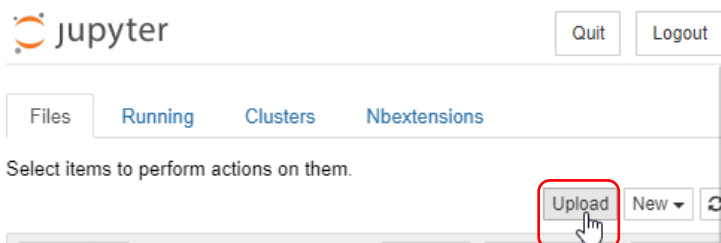


그림 167 Jupyter 업로드

8.6.2. 학습 모델 활용

카메라 클래스를 생성하고 Object_Follow 클래스를 생성합니다.

Object_Follow 클래스에는 최적의 사물 인식을 위한 합성곱 신경망이 구현되어 있습니다.

생성된 Object_Follow 클래스의 load_model(path) 메소드를 이용해 다운로드 받은 모델 파일을 로드합니다. 모델 파일을 로드할 때 path가 입력되지 않은 경우 현재 경로에서 가져옵니다.

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04: ac=Pilot.AutoCar()
05: OF=Pilot.Object_Follow(cam)
06: OF.load_model()
```

모델 로드가 완료되면 detect() 메소드를 이용해 사물을 인식하고 결과를 반환합니다. index 파라미터에 1 또는 'person'이라고 입력하고 실행하면 사람을 인식하고 index가 입력되지 않으면 인식할 수 있는 모든 사물에 대한 인식 결과를 반환합니다. index에 관한 입력과 인식가능한 목록은 다음 주소를 통해 확인할 수 있습니다.

- https://github.com/hanback-docs/ssd_mobilenet_v2_coco_engine



detect 메소드의 파라미터들은 다음과 같습니다.

- image: 직접 이미지 데이터를 입력하여 인식하도록 합니다. 현재 클래스에 지정된 카메라 영상이 아닌 다른 데이터를 인식해보고자 할 때 사용합니다.
 - 기본값: Camera.value

- index: 어떤 사물을 인식할 지 지정할 수 있습니다. Index 를 입력하지 않을 경우 인식가능한 모든 사물에 대한 결과를 반환합니다.
 - 기본값: All
- show: 실행 결과를 그래픽 요소로 표현할 지에 대한 여부입니다.
 - 기본값: True

먼저, index 에 ‘person’을 입력하여 사람을 구분하면 어떤 값을 반환하는지 확인해보도록 합니다.

```
07: v=OF.detect(index='person')
08: print(v)
```

[출력]

```
{'x':    0.02924691140651703,    'y':    0.26995646953582764,    'size_rate':
0.18161612565622676}
```

인식된 ‘person’ 개체들 중에 가장 사이즈가 큰 개체에 대한 결과를 딕셔너리 형태로 반환합니다. x 와 y 는 인식된 개체의 상대적 좌표이며 화면 정중앙을 0 으로 하고 -1 ~ 1 의 범위를 갖습니다. size_rate 는 개체의 크기입니다. 0 ~ 1 사이값으로 표현되며, 1 에 가까울수록 인식된 개체의 거리가 가깝다는 의미로 해석할 수 있습니다.

이어서, 이번에는 index 입력없이 모든 사물에 대한 데이터를 확인해보입니다.

```
09: v=OF.detect()
10: print(v)
```

[출력]

```
[[{'label': 76, 'confidence': 0.9231778383255005, 'bbox': [0.143454909324646,
0.062126800417900085, 0.8638043403625488, 0.3971424102783203]}],
{'label': 74, 'confidence': 0.6338636875152588, 'bbox': [0.6076321601867676,
0.5661253333091736, 0.7583435773849487, 0.8835135102272034]}],
{'label': 77, 'confidence': 0.6270381212234497, 'bbox': [0.259377121925354,
0.4745787978172302, 0.4690379500389099, 0.9588668942451477]}]]]
```

Index 가 76 인 ‘Keyboard’와 index 가 74 인 ‘mouse’ 그리고 index 가 77 인 ‘cell phone’이 인식된 것을 확인할 수 있습니다. 다음과 같이 show() 메소드를 이용하면 인식 결과를 시각적으로 확인할 수 있습니다. 인식 결과를 표시하는 show() 메소드는 detect() 메소드와 동기화되어있기 때문에 detect() 메소드와 별개로 한 번만 실행하면 계속해서 표시됩니다.

```
11: OF.show()
```

[출력]

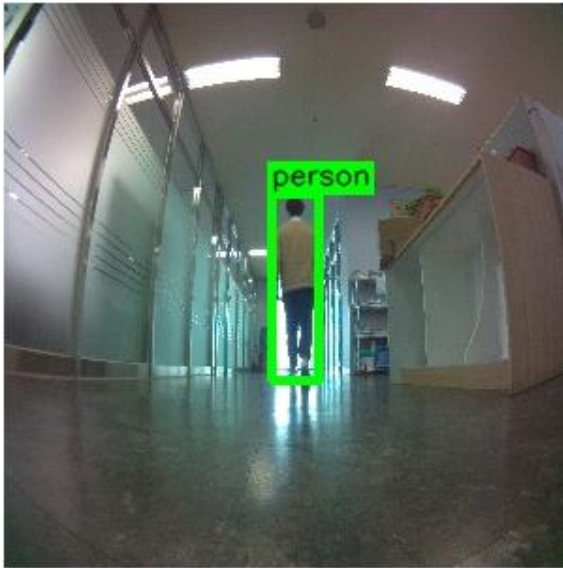


detect() 메소드의 index 파라미터에 1 또는 ‘person’을 입력해 사람을 인식하도록 하고 반환된 결과값을 이용해 사람이 좌측에 있으면 좌회전, 우측에 있으면 우회전 하도록 합니다. 또, size_rate 값을 이용해 화면 대

비 크기가 20% 이상이 되면 정지하고 그 미만일 경우에만 전진하도록 하여 충돌을 방지합니다. steer 값에 4를 곱하는 것은 조향 각도를 더 민감하게 하기 위함입니다.

```
12: while True:
13:     v=OF.detect(index='person')
14:
15:     if v is not None:
16:         steer=v['x']*4
17:
18:         if steer > 1:
19:             steer=1
20:         elif steer < -1:
21:             steer=-1
22:
23:         ac.steering=steer
24:
25:         if v['size_rate']<0.20:
26:             ac.forward(50)
27:         else:
28:             ac.stop()
29:     else:
30:         ac.stop()
```

[출력]



표시된 화면에서 초록색 박스 표시가 된 사람을 따라가며 근접하거나 아무것도 인식되지 않으면 정지합니다. detect() 메소드의 index 파라미터를 수정하면 의자, 컵, 휴대폰 등 더 다양한 사물을 따라가도록 설계할 수 있습니다.

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

전체 소스 코드

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04: ac=Pilot.AutoCar()
05: OF=Pilot.Object_Follow(cam)
06: OF.load_model()
07:
08: while True:
09:     v=OF.detect(index='person')
```

```
10:
11:     if v is not None:
12:         steer=v['x']*4
13:
14:         if steer > 1:
15:             steer=1
16:         elif steer < -1:
17:             steer=-1
18:
19:         ac.steering=steer
20:
21:         if v['size_rate']<0.20:
22:             ac.forward(50)
23:         else:
24:             ac.stop()
25:     else:
26:         ac.stop()
```

8.7. 트랙 주행

트랙 주행은 벽 Wall, 차선 Lane, 콘 Cone 등을 이용해 트랙을 만들고 차량이 주행할 때 발생하는 여러 상황을 수집하여 학습시킵니다. 상황 데이터셋의 양과 다양성에 따라 어떤 모델이 만들어지는지 확인해보고 체계적인 데이터셋 수집을 통해 안정적으로 주행할 수 있도록 합니다.

트랙은 콘 < 벽 < 차선 순으로 차선이 가장 높은 인식률을 보입니다. 차선을 그릴 때는 바닥색과 대비되는 색의 종이 테이프를 사용하면 좋습니다.

실습은 Jupyter 환경에서 진행되며 이에 대해서는 19 페이지를 참고할 수 있습니다. 또, 쉬운 이해와 빠른 실습을 위해 Pop.Pilot 라이브러리를 사용합니다.

8.7.1. 데이터 수집

먼저, 트랙과 차선 등 주행에 필요한 정보를 학습하기 위해 이미지 데이터를 수집해야 합니다. 이미지 데이터는 Pop.Pilot 라이브러리의 Data_Collector 클래스로 편리하게 수집할 수 있습니다. 또, OpenCV 코드 작성으로 인한 불편 해소와 핵심 내용 집중을 위해 Pop.Pilot 라이브러리에 카메라를 쉽게 읽을 수 있는 Camera 클래스를 미리 구현해두었습니다.

Pop에서 Pilot 라이브러리를 불러오고 Camera 클래스를 width와 height 파라미터에 300을 입력해 300 x 300 사이즈로 생성합니다. Camera 클래스의 show() 메소드를 이용해 실시간 영상을 확인해볼 수 있습니다.

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
```

```
04: cam.show()
```

[출력]



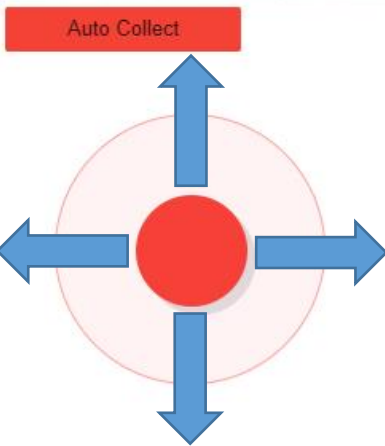
Pilot 라이브러리에서 Data_Collector 클래스를 생성합니다. 생성자에 Track_Follow 데이터 수집이 목적임을 명시하기 위해 Pilot.Track_Follow 를 입력합니다. 또, 카메라 영상을 이용하기 위해 camera 파라미터에 앞서 생성한 카메라 클래스를 입력합니다.

```
05: dataCollector=Pilot.Data_Collector(Pilot.Track_Follow, camera=cam)
```

Data_Collector 클래스의 show() 메소드를 사용하면 데이터 수집을 위한 GUI 환경이 표시됩니다.

```
06: dataCollector.show()
```

[출력]



데이터 수집을 위해 실시간 영상과 Auto Collect 버튼, 조이스틱이 표시됩니다. Track Follow 데이터는 조이스틱과 클릭 두 가지 방법으로 데이터를 수집할 수 있습니다. 데이터가 수집되면 오른쪽 장면에 그 결과를 표시합니다.

첫 번째 데이터 수집 방법은 조이스틱을 이용한 데이터 수집입니다. 조이스틱으로 차량을 제어하면 자동으로 영상 데이터와 조이스틱 제어 데이터를 초당 5회 수집합니다. 단, Auto Collect 버튼을 클릭해 초록색이 되어야 해당 기능이 활성화됩니다. 조이스틱으로 제어하는 Auto Collect 기능은 빠르게 대량의 데이터를 수집할 수 있지만 제어 숙련 정도에 따라 정

확한 데이터 수집이 힘들 수 있습니다. 또, 차선에 근접하거나 벗어나는 등 예외적인 상황에 대한 데이터를 수집하기 힘듭니다.

Auto Collect



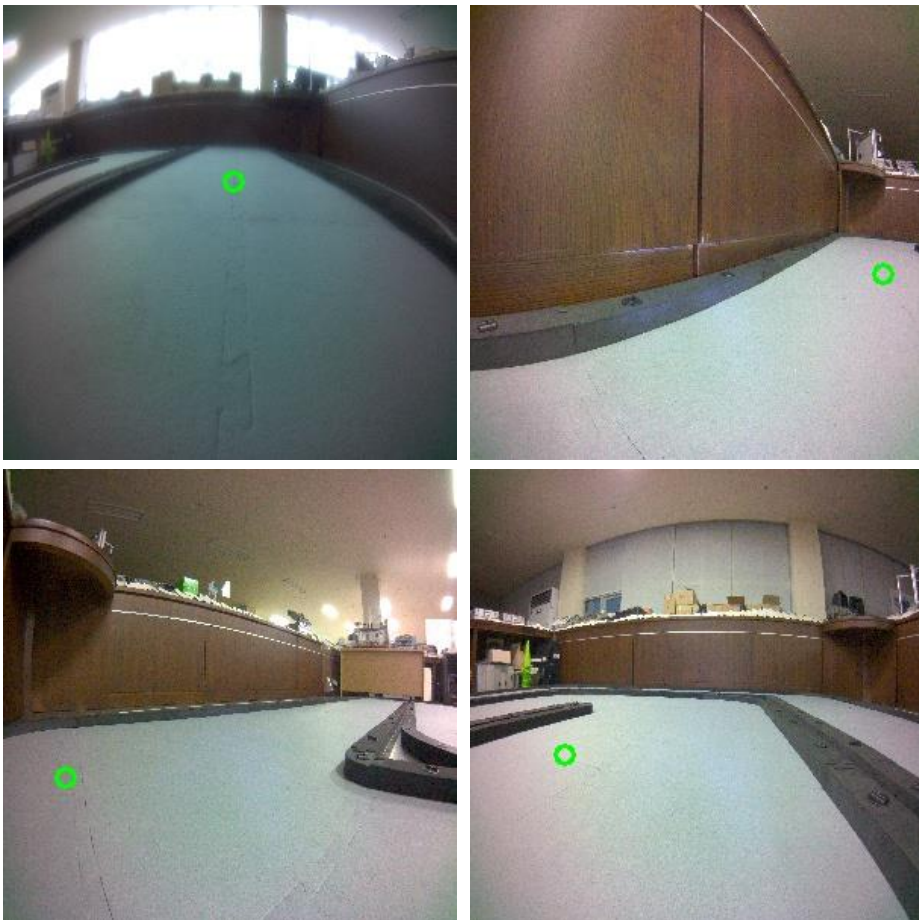
그림 168 가상 조이스틱을 통한 데이터 수집

두 번째 방법은 영상 클릭을 이용한 데이터 수집입니다. 실시간 영상을 클릭하면 해당 장면과 클릭 좌표를 데이터로 수집할 수 있고, 앞으로 진행되어야 할 지점을 클릭하여 수집해야 합니다. 원하는 장면에 원하는 좌표를 지정할 수 있으므로 정확한 데이터 수집이 가능하지만 데이터 수집이 반자동으로 이루어지기에 수집 속도가 느립니다.



이 글에서는 클릭을 통해 데이터를 수집하는 방법을 설명합니다.

Auto Collect 기능이 비활성화된 상태에서 조이스틱을 이용해 조종하거나 직접 옮겨다니며 데이터를 수집합니다. 데이터는 최소한 500 장 이상이 필요하며 정상적인 주행 상황을 비롯해 좌우측 차선에 치우친 상황, 과도한 코너링 상황 등 최대한 다양하고 많은 데이터를 수집해야 양질의 모델을 기대할 수 있습니다. 사진 데이터는 현재 경로에서 track_dataset 디렉토리에 저장됩니다.



8.7.2. 데이터 딥러닝

트랙 학습을 위해 `Track_Follow` 클래스를 생성합니다. 카메라 영상을 이용해야하므로 생성 파라미터에 앞서 생성한 `camera` 클래스를 입력하여 생성합니다.

`Track_Follow` 클래스는 트랙 인식을 위한 합성곱 신경망이 설계되어 있는 신경망 클래스입니다.

이어서, `Track_Follow` 클래스의 `load_datasets()` 메소드를 이용해 앞서 수집한 데이터셋들을 로드합니다. 데이터셋을 로드할 때 현재 경로에 있는 `track_dataset` 디렉토리에서 가져옵니다.

```
07: TF=Pilot.Track_Follow(camera=cam)
08: TF.load_datasets()
```

데이터셋 로드가 완료되면 `train()` 메소드를 이용해 학습을 시작합니다. `train` 메소드의 파라미터인 `times`에 숫자를 입력해 학습할 횟수를 조절할 수 있습니다. 우선 5 회만 학습시켜보도록 합니다.

`train` 메소드를 통해 학습이 진행되면 매 스텝마다 자동으로 학습 모델이 저장됩니다. 만약 자동 저장을 비활성화 하고싶다면 `autosave` 파라미터를 `False`로 지정하면 됩니다.

```
09: TF.train(times=5)
```

[출력]

```
0 step loss : 0.025837895073226893
1 step loss : 0.014605101206779094
2 step loss : 0.007547952315493215
3 step loss : 0.006795135795176357
4 step loss : 0.004984153598415068
```

Loss 가 0 에 가까울수록 오차가 작은 모델입니다. 적절히 잘 학습된 모델이 나왔다면 run() 메소드를 이용해 모델의 예측값을 출력해볼 수 있습니다. 다음과 같이 run() 메소드를 실행하고 반환값을 출력해봅니다.

```
10: value=TF.run()
11: print(value)
```

[출력]

```
{'x': -0.05498507, 'y': -0.029995378}
```

카메라를 이용해 앞으로 진행할 지점을 예측합니다. x 와 y 는 진행 지점의 상대적 좌표이며 화면 정중앙을 0 으로 하고 -1 ~ 1 의 범위를 갖습니다. 그러나 텍스트로된 출력만으로는 어떤 장면으로 어떻게 판단했는지 알기 어렵습니다. 이 때, show() 메소드를 사용할 수 있습니다. show() 메소드를 실행하면 현재 카메라 영상과 run()메소드의 결과를 직관적으로 표시합니다. show() 메소드는 run() 메소드와 동기화 되어있기 때문에 run() 메소드와 별개로 한 번만 실행하면 계속해서 표시됩니다.

```
12: TF.show()
```

[출력]



run() 메소드에는 Callback 기능을 사용할 수 있도록 만들어져 있습니다. 다음과 같이 메소드를 선언하고 run() 메소드의 callback 파라미터로 입력하여 호출하면, run() 메소드의 반환값을 Callback 메소드의 인자로 넘겨 호출합니다.

```
13: def is_Left(value):  
14:     print(value['x']<0)  
15:  
16: value=TF.run(callback=is_Left)  
17: print(value)
```

[출력]

True

{'x': -0.05498507, 'y': -0.029995378}

트랙을 잘 인식한다면 이 모델을 이용해 차량을 제어해봅니다. 차량 제어

를 위해 Pop.Pilot 라이브러리에서 AutoCar 클래스를 생성합니다. Callback 메소드는 차량을 전진시키고 x 값을 이용해 좌우 조향하도록 구성합니다.

```
18: Car=Pop.Pilot.AutoCar()
19: Car.setSpeed(50)
20:
21: def drive(value):
22:     Car.forward()
23:     steer=value['x']
24:
25:     if steer > 1:
26:         steer=1
27:     elif steer < -1:
28:         steer=-1
29:
30:     Car.steering=steer*1.5
31:
32: while True:
33:     TF.run(callback=drive)
```

차량이 주행하며 트랙 내부에서 주행하기 위해 조향하는 것을 확인할 수 있습니다. 그러나 데이터셋에는 없는 조명, 바닥 등 새로운 환경에서는 제대로 주행하지 않습니다. 이는 데이터셋의 다양성이 부족하기 때문입니다. 최대한 다양한 환경에서 데이터를 수집하여 학습한다면 매우 정확한 주행 모델을 얻을 수 있습니다.

부가적으로, 저장된 학습 모델을 로드하여 이어서 학습하고자 한다면 load_model(path) 메소드를 사용할 수 있습니다.

```
34: TF.load_model(path = "track_follow_model.pth")
```

또, save_model(path) 메소드를 사용하면 원하는 경로에 수동으로 모델을 저장할 수 있습니다.

```
35: TF.save_model(path = "track_follow_model.pth")
```

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

소스 코드 : 데이터 수집

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04:
05: dataCollector=Pilot.Data_Collector(Pilot.Track_Follow, camera=cam)
06: dataCollector.show()
```

소스 코드 : 딥러닝

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04:
05: TF=Pilot.Track_Follow(camera=cam)
06: TF.load_datasets()
07:
08: TF.train(times=5)
09:
10: TF.show()
11:
12: Car=Pilot.AutoCar()
13: Car.setSpeed(50)
14:
```

```
15: def drive(value):
16:     Car.forward()
17:     steer=value['x']
18:
19:     if steer > 1:
20:         steer=1
21:     elif steer < -1:
22:         steer=-1
23:
24:     Car.steering=steer*1.5
25:
26: while True:
27:     TF.run(callback=drive)
```

8.8. 트래픽 콘 주행

8.8.1. 데이터 수집

주행에 필요한 트래픽 콘 정보를 학습하기 위해 이미지 데이터를 수집합니다.

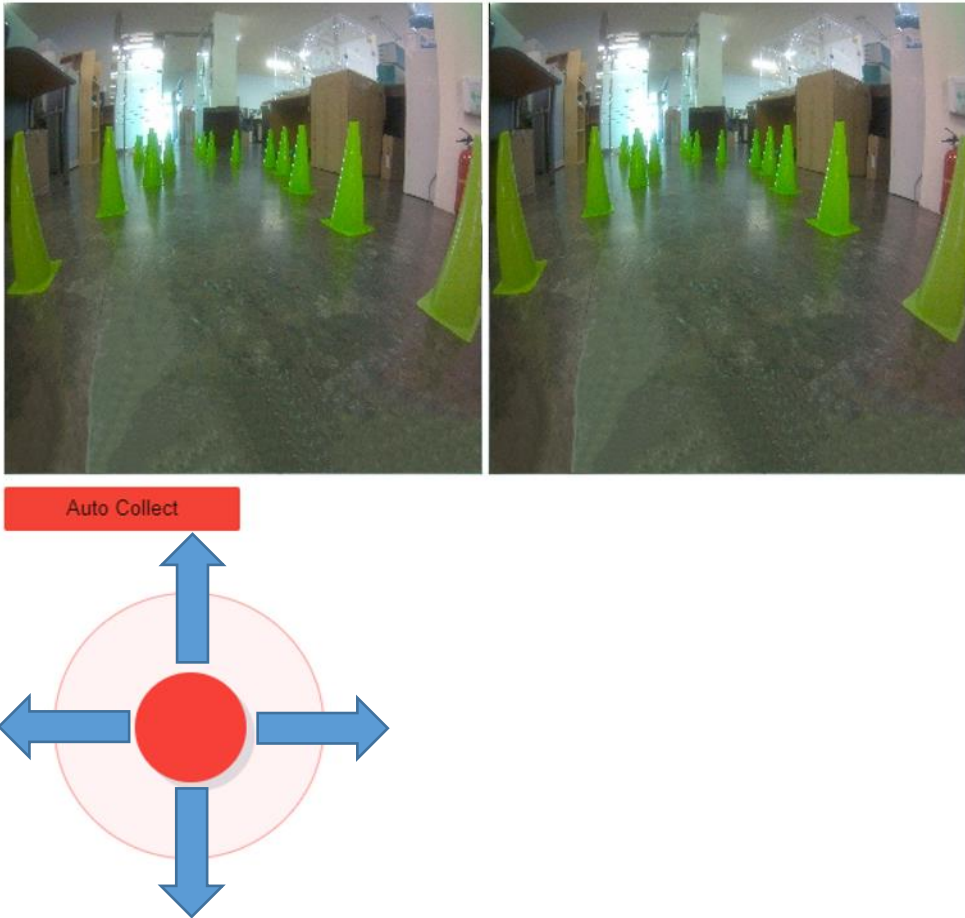
다음과 같이 카메라를 생성하고, Pilot 라이브러리에서 Data_Collector 클래스를 생성합니다. 생성 파라미터로 Pilot.Track_Follow 와 camera 파라미터에 앞서 생성한 카메라 클래스를 입력합니다.

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04: cam.show()
05: dataCollector=Pilot.Data_Collector(Pilot.Track_Follow, camera=cam)
```

Data_Collector 클래스의 show() 메소드를 사용해 GUI 환경을 표시합니다.

```
06: dataCollector.show()
```

[출력]



데이터 수집을 위해 실시간 영상과 Auto Collect 버튼, 조이스틱이 표시됩니다. 이 예제는 이미지를 클릭하여 데이터셋을 수집하는 방법으로 설명합니다.

Auto Collect 기능이 비활성화된 상태에서 조이스틱을 이용해 조종하거나 직접 옮겨다니며 데이터를 수집합니다. 데이터는 최소한 500 장 이상이 필요하며 정상적인 주행 상황을 비롯해 트래픽 콘에 근접한 상황, 트래픽 콘에 부딪힌 상황 등 최대한 다양하고 많은 데이터를 수집해야 양질의 모델을 기대할 수 있습니다. 사진 데이터는 현재 경로에서 track_dataset 디렉토리에 저장됩니다.

니다.

만약 앞서 수집한 사진 데이터를 보존시키고 싶다면 기존에 있던 track_dataset 디렉토리의 이름을 변경하거나 위치를 옮기면 됩니다.



그림 171 데이터셋 예시

8.8.2. 데이터 딥러닝

트랙 인식을 위한 합성곱 신경망이 사전 구성된 Track_Follow 클래스를 생성합니다. 카메라 영상을 이용해야하므로 생성 파라미터에 앞서 생성한 camera 클래스를 입력하여 생성합니다.

이어서, Track_Follow 클래스의 load_datasets() 메소드를 이용해 앞서 수집한 데이터셋들을 로드합니다. 데이터셋을 로드할 때 현재 경로에 있는 track_dataset 디렉토리에서 가져옵니다.

```
07: CF=Pilot.Track_Follow(camera=cam)
08: CF.load_datasets()
```

데이터셋 로드가 완료되면 train() 메소드를 이용해 학습을 시작합니다. 우선, 5 회만 학습시켜보도록 합니다.

```
09: CF.train(times=5)
```

[출력]

```
0 step loss : 0.018639845439853767
1 step loss : 0.003417585875528554
2 step loss : 0.002770397887798026
3 step loss : 0.0018943121530658876
4 step loss : 0.002116210151386137
```

적절히 잘 학습된 모델이 나왔다면 run() 메소드를 이용해 모델의 예측값을 출력해볼 수 있습니다. 다음과 같이 run() 메소드를 실행하고 반환값을 출력해봅니다.

```
10: value=CF.run()
```

```
11: print(value)
```

[출력]

```
{'x': -0.05498507, 'y': -0.029995378}
```

run() 메소드와 동기화 된 show() 메소드를 호출합니다.

```
12: CF.show()
```

[출력]



트래픽 콘으로 이루어진 트랙을 잘 인식한다면 AutoCar 클래스를 생성하고 Callback 메소드를 작성합니다. Callback 메소드는 차량을 전진시키고 x 값을 이용해 좌우 조향하도록 구성합니다. 이어서, 모델의 run() 메소드에 callback 파라미터로 입력하여 실행합니다.

```
13: Car=Pilot.AutoCar()
14: Car.setSpeed(50)
15:
16: def drive(value):
17:     Car.forward()
18:     steer=value['x']
19:
20:     if steer > 1:
21:         steer=1
22:     elif steer < -1:
23:         steer=-1
24:
25:     Car.steering=steer*1.5
26:
27: while True:
28:     CF.run(callback=drive)
```

차량이 주행하며 트래픽 콘을 따라 주행하기 위해 조향하는 것을 확인할 수 있습니다. 그러나 다른 색의 트래픽 콘, 다른 바닥 등 새로운 환경에서는 제대로 주행하지 않습니다. 이는 데이터셋의 다양성이 부족하기 때문입니다. 최대한 다양한 환경에서 데이터를 수집하여 학습한다면 매우 정확한 주행 모델을 얻을 수 있습니다.

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

소스 코드 : 데이터 수집

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04:
05: dataCollector=Pilot.Data_Collector(Pilot.Track_Follow, camera=cam)
06: dataCollector.show()
```

소스 코드 : 딥러닝

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04:
05: CF=Pilot.Track_Follow(camera=cam)
06: CF.load_datasets()
07:
08: CF.train(times=5)
09:
10: CF.show()
11:
12: Car=Pilot.AutoCar()
13: Car.setSpeed(50)
14:
15: def drive(value):
16:     Car.forward()
17:     steer=value['x']
18:
19:     if steer > 1:
20:         steer=1
21:     elif steer < -1:
22:         steer=-1
23:
24:     Car.steering=steer*1.5
25:
26: while True:
27:     CF.run(callback=drive)
```

8.9. 라인 트레이서

8.9.1. 데이터 수집

주행에 필요한 라인 정보를 학습하기 위해 이미지 데이터를 수집합니다.

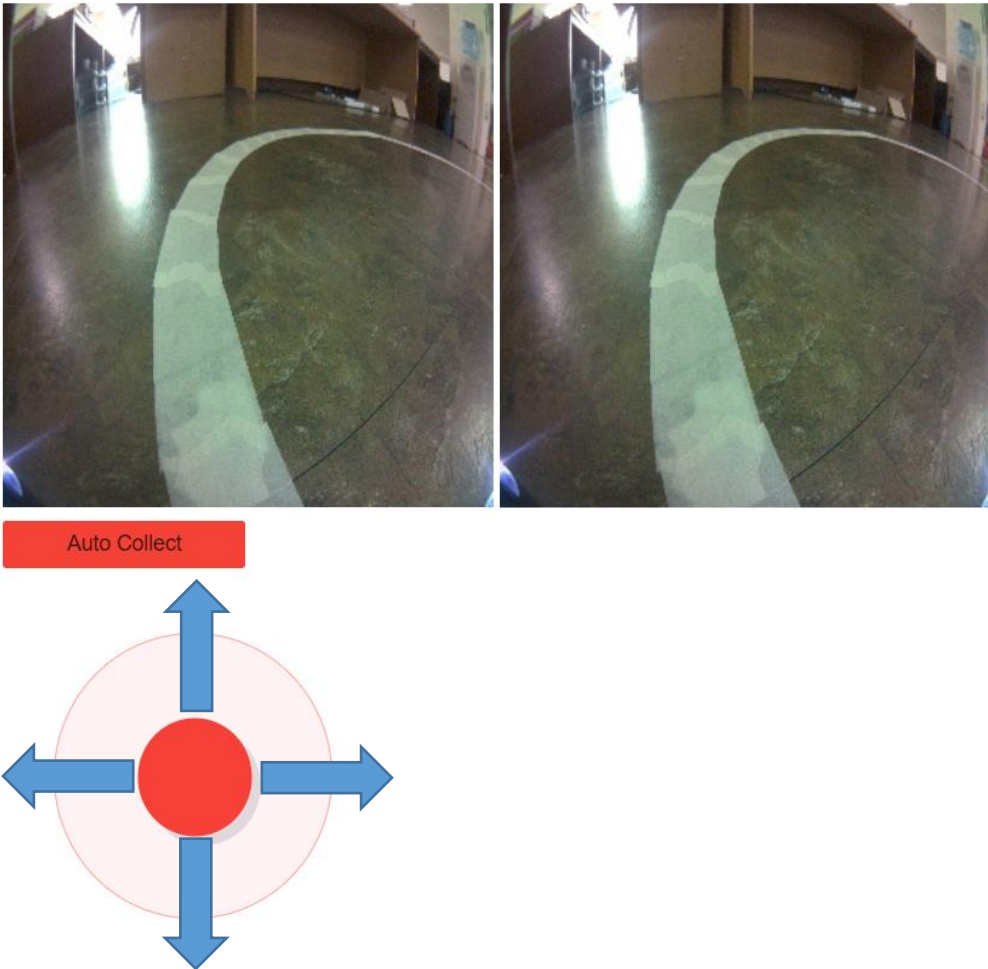
다음과 같이 카메라를 생성하고, Pilot 라이브러리에서 Data_Collector 클래스를 생성합니다. 생성 파라미터로 Pilot.Track_Follow 와 camera 파라미터에 앞서 생성한 카메라 클래스를 입력합니다.

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04: cam.show()
05: dataCollector=Pilot.Data_Collector(Pilot.Track_Follow, camera=cam)
```

Data_Collector 클래스의 show() 메소드를 사용해 GUI 환경을 표시합니다.

```
06: dataCollector.show()
```

[출력]



데이터 수집을 위해 실시간 영상과 Auto Collect 버튼, 조이스틱이 표시됩니다. 이 예제는 이미지를 클릭하여 데이터셋을 수집하는 방법으로 설명합니다.

Auto Collect 기능이 비활성화된 상태에서 조이스틱을 이용해 조종하거나 직접 옮겨다니며 데이터를 수집합니다. 데이터는 최소한 500 장 이상 필요하며 정상적인 주행 상황을 비롯해 라인을 벗어난 상황, 과도한 코너링 상황 등 최대한 다양하고 많은 데이터를 수집해야 양질의 모델을 기대할 수 있습니다.

니다. 사진 데이터는 현재 경로에서 track_dataset 디렉토리에 저장됩니다.



그림 172 데이터셋 예시

8.9.2. 데이터 딥러닝

라인 인식을 위한 합성곱 신경망이 사전 구성된 Track_Follow 클래스를 생성합니다. 카메라 영상을 이용해야하므로 생성 파라미터에 앞서 생성한 camera 클래스를 입력하여 생성합니다.

이어서, Track_Follow 클래스의 load_datasets() 메소드를 이용해 앞서 수집한 데이터셋들을 로드합니다. 데이터셋을 로드할 때 현재 경로에 있는 track_dataset 디렉토리에서 가져옵니다.

```
07: LF=Pilot.Track_Follow(camera=cam)
08: LF.load_datasets()
```

데이터셋 로드가 완료되면 train() 메소드를 이용해 학습을 시작합니다. 우선, 5 회만 학습시켜보도록 합니다.

train 메소드를 통해 학습이 진행되면 매 스텝마다 자동으로 학습 모델이 저장됩니다. 만약 자동 저장을 비활성화 하고싶다면 autosave 파라미터를 False 로 지정하면 됩니다.

```
09: LF.train(times=5)
```

[출력]

```
0 step loss : 0.006127875036639218
1 step loss : 0.004890237900769981
2 step loss : 0.004016204625288233
3 step loss : 0.00333160844720032
4 step loss : 0.0028236951989210274
```

Loss 가 0 에 가까울수록 오차가 작은 모델입니다. 적절히 잘 학습된 모델이 나왔다면 run() 메소드를 이용해 모델의 예측값을 출력해볼 수 있습니다. 다음과 같이 run() 메소드를 실행하고 반환값을 출력해봅니다.

```
10: value=LF.run()
11: print(value)
```

[출력]

```
{'x': -0.2776801, 'y': 0.345897}
```

run() 메소드와 동기화 된 show() 메소드를 호출합니다.

```
12: LF.show()
```

[출력]



라인을 잘 인식한다면 AutoCar 클래스를 생성하고 Callback 메소드를 작성합니다. Callback 메소드는 차량을 전진시키고 x 값을 이용해 좌우 조향하도록 구성합니다. 이어서, 모델의 run() 메소드에 callback 파라미터로 입력하여 실행합니다.

```
13: Car=Pilot.AutoCar()  
14: Car.setSpeed(50)
```

```
15:
16: def drive(value):
17:     Car.forward()
18:     steer=value['x']
19:
20:     if steer > 1:
21:         steer=1
22:     elif steer < -1:
23:         steer=-1
24:
25:     Car.steering=steer*1.5
26:
27: while True:
28:     LF.run(callback=drive)
```

차량이 주행하며 라인 위에서 주행하기 위해 조향하는 것을 확인할 수 있습니다. 그러나 다른 바닥, 다른 라인 색 등 새로운 환경에서는 제대로 주행하지 않습니다. 이는 데이터셋의 다양성이 부족하기 때문입니다. 최대한 다양한 환경에서 데이터를 수집하여 학습한다면 매우 정확한 주행 모델을 얻을 수 있습니다.

앞의 조각 코드들의 전체 소스 코드는 다음과 같습니다.

소스 코드 : 데이터 수집

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04:
05: dataCollector=Pilot.Data_Collector(Pilot.Track_Follow, camera=cam)
06: dataCollector.show()
```

소스 코드 : 딥러닝

```
01: from pop import Pilot
02:
03: cam=Pilot.Camera(width=300, height=300)
04:
05: LF=Pilot.Track_Follow(camera=cam)
06: LF.load_datasets()
07:
08: LF.train(times=5)
09:
10: LF.show()
11:
12: Car=Pilot.AutoCar()
13: Car.setSpeed(50)
14:
15: def drive(value):
16:     Car.forward()
17:     steer=value['x']
18:
19:     if steer > 1:
20:         steer=1
21:     elif steer < -1:
22:         steer=-1
23:
24:     Car.steering=steer*1.5
25:
26: while True:
27:     LF.run(callback=drive)
```

부록

A. OpenCV 설치

이 과정은 아래 링크를 통해 웹에서 확인할 수도 있습니다.

- <https://github.com/hanback-docs/OpenCV-Installation>



I. 의존성 라이브러리 설치

```
sudo apt install -y build-essential cmake libavcodec-dev
sudo apt install -y libavformat-dev libavutil-dev libeigen3-dev
sudo apt install -y libglew-dev libgtk2.0-dev libgtk-3-dev
sudo apt install -y libjpeg-dev libpng-dev libpostproc-dev
sudo apt install -y libswscale-dev libtbb-dev libtiff5-dev
sudo apt install -y libv4l-dev libxvidcore-dev libx264-dev
sudo apt install -y zlib1g-dev libgstreamer1.0-dev pkg-config
sudo apt install -y libgstreamer-plugins-base1.0-dev libatlas-base-dev
sudo apt install -y libtbb2 mesa-utils libgl1-mesa-dri gfortran
```

II. 설치 파일 소스 다운로드

```
wget -O opencv.zip https://github.com/opencv/opencv/archive/4.2.0.zip
wget -O opencv_contrib.zip https://github.com/opencv/opencv\_contrib/archive/4.2.0.zip
unzip opencv.zip
```

```
unzip opencv_contrib.zip
```

III. 빌드 환경 설정

```
cd opencv-4.2.0
mkdir build
cd build
make -D CMAKE_BUILD_TYPE=RELEASE -D CMAKE_INSTALL_PREFIX=/usr/local -D
WITH_CUDA=ON -D OPENCV_DNN_CUDA=ON -D CUDA_FAST_MATH=ON -D
CUDA_ARCH_BIN=${ARCH_BIN} -D CUDA_ARCH_PTX="" -D ENABLE_FAST_MATH=ON
-D WITH_CUBLAS=ON -D WITH_CUFFT=ON -D WITH_NVCUVID=ON -D
WITH_FFMPEG=ON -D WITH_LIBV4L=ON -D WITH_V4L=ON -D WITH_GSTREAMER=ON
-D WITH_GSTREAMER_0_10=OFF -D WITH_GTK=ON -D WITH_OPENGL=ON -D
BUILD_JASPER=ON -D ENABLE_FAST_MATH=ON -D OPENCV_ENABLE_NONFREE=ON
-D BUILD_TESTS=OFF -D BUILD_PERF_TESTS=OFF -D
BUILD_opencv_python_tests=OFF -D INSTALL_TESTS=OFF -D
INSTALL_C_EXAMPLES=OFF -D INSTALL_PYTHON_EXAMPLES=YES -D
BUILD_NEW_PYTHON_SUPPORT=ON -D BUILD_opencv_python3=ON
-D OPENCV_GENERATE_PKGCONFIG=ON -D
OPENCV_EXTRA_MODULES_PATH=../../opencv_contrib/modules
-D BUILD_EXAMPLES=OFF -D WITH_TBB=ON -D BUILD_TBB=ON ../
```

IV. 빌드 및 설치

```
make -j4
sudo make install
```

B. OpenCV 비디오 속성

속성명(Attribute)	설명(Details)
cv2.CAP_PROP_POS_MSEC	비디오의 현재 위치. 밀리초 또는 타임 스탬프
cv2.CAP_PROP_POS_FRAMES	다음 캡처/디코딩할 프레임 인덱스
cv2.CAP_PROP_FRAME_WIDTH	비디오 스트림의 프레임 폭
cv2.CAP_PROP_FRAME_HEIGHT	비디오 스트림의 프레임 높이
cv2.CAP_PROP_FPS	프레임 속도
cv2.CAP_PROP_FOURCC	코덱의 4 문자 코드
cv2.CAP_PROP_FRAME_COUNT	비디오 파일의 프레임 수
cv2.CAP_PROP_BRIGHTNESS	카메라 밝기
cv2.CAP_PROP_CONTRAST	카메라 대비
cv2.CAP_PROP_SATURATION	카메라 채도
cv2.CAP_PROP_HUE	카메라 색조
cv2.CAP_PROP_GAIN	카메라 게인
cv2.CAP_PROP_EXPOSURE	카메라 노출
cv2.CAP_PROP_CONVERT_RGB	True 이면 이미지를 RGB 로 변환

표 11 비디오 속성

C. MFCC

음성 데이터 전처리의 핵심은 잡음 섞인 데이터로부터 유효한 데이터를 구분하는 것입니다. 노이즈는 대체로 주파수 및 진폭이 일정한 수준과 패턴을 가지고 있고 음성은 다양한 파형과 불규칙한 패턴으로 이루어져 있습니다. 그러므로 음성이 가지고 있는 특징이 잘 드러나도록 전처리 `preprocess` 한다면 노이즈를 최소화하고 쉽게 음성을 분석할 수 있는 최적의 데이터가 될 것입니다.

음성의 특징 추출할 때 대표적으로 MFCC Mel-Frequency Cepstral Coefficient 를 사용합니다. 사람의 청각 기관은 고주파 High Frequency 보다 저주파 Low Frequency 에서 더 민감하게 반응합니다. MFCC 는 이를 적용하여 저주파수를 집중적으로 특징을 추출하는 방법입니다. MFCC 처리 과정은 크게 4 단계로 다음과 같습니다.

- | 1 | 프레임화 : 음성 데이터를 0.02~0.04 초 간격으로 분할
- | 2 | 고속 푸리에 변환 FFT, Fast Fourier Transform : 주파수 밀집 그래프 방식으로 변환

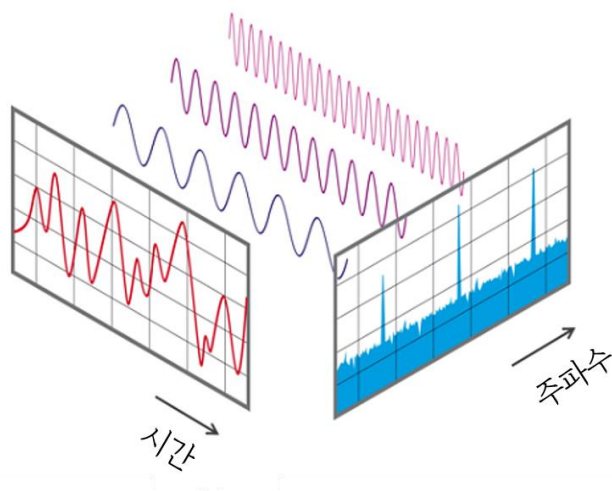


그림 173 고속 푸리에 변환 과정

- | 3 | 멜 변환 Mel Scale : 저주파수에서 작은 필터를 사용, 고주파수로 갈수록 큰 필터 사용
큰 필터를 사용하면 많은 데이터를 하나로 합축시키기에 한 프레임 당 영향력이 감소

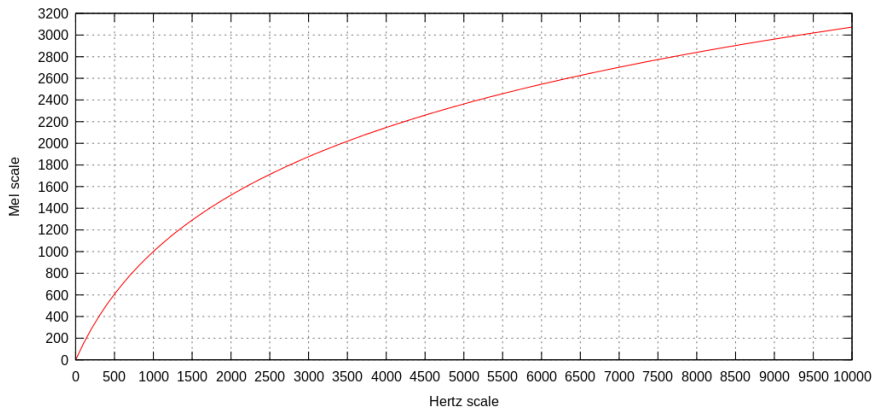


그림 174 주파수 별 멜 필터 크기

| 4 | 켈스트럴 분석 Cepstral Analysis : 밀집도가 높은 주파수를 추출

다음 사진은 원본 음성의 그래프와 MFCC 처리된 음성 사진입니다.

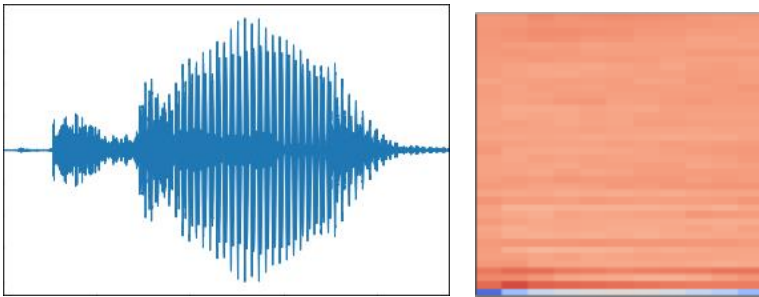


그림 175 음성 그래프(좌)와 MFCC 처리된 음성(우)

D.One Hot

One Hot 은 데이터 표현 기법 중 하나로 소프트맥스, 자연어 처리 등에 자주 사용됩니다. 표현하고자 하는 값 N 이 있다면, 배열의 N 번째 인덱스

를 1로 하고 나머지를 모두 0으로 표현하는 방법입니다. 예를 들어, 길이가 5인 배열에 2, 3, 0을 표현하고자 한다면 다음과 같이 표현할 수 있습니다.

2	3	0
[0 0 1 0 0]	[0 0 1 0 0]	[0 0 1 0 0]

One Hot 인코딩은 데이터를 정형화할 수 있고, 모든 원소들의 합이 항상 1이므로 소프트맥스 활용에 부합하다는 특징이 있습니다.

AIoT AutoCar 로 배우는

온디바이스 AI 프로그래밍 (v1.1.4)

지은이 **한백전자 기술연구소**

펴낸이 **진수춘**

펴낸곳 **도서출판 한백** 전화 **042-610-1111** 팩스 **042-610-1199**

주소 **대전광역시 유성구 유성대로 518**

가격 **35,000 원**

초판 발행 **2020 년 04 월 28 일**

ISBN **978-89-6633-229-8**

홈페이지 **www.hanback.com**

이 책의 저작권은 저작권자와의 독점 계약으로 도서출판 한백이 소유합니다.

신 저작권법에 의해 보호를 받는 저작물이므로 무단 전재와 복제를 금합니다.

이 책의 내용에 대한 추가 지원과 문의는 한백전자 홈페이지 www.hanback.com 이나
전화 042-610-1111 를 이용해 주세요.